

An underwater photograph showing a large, dense school of small, silvery fish swimming in clear blue water. In the lower-left foreground, a larger, darker fish, possibly a stingray, is visible. The overall scene is bright and vibrant, with sunlight filtering through the water.

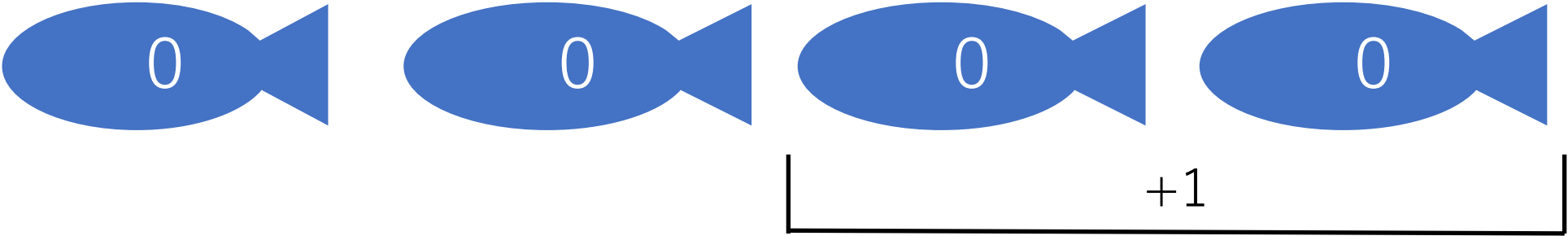
Fish 3

Editorial : blackyuki

問題概要

- N 匹の魚のIQをそれぞれ決まった値まで増やしたい
- 操作A：魚 k のIQを D だけ増やす
- 操作B：魚 $k, k + 1, \dots, N$ のIQを1ずつ増やす
- 操作Aの回数の最小値は？
- Q 個の区間クエリに答える

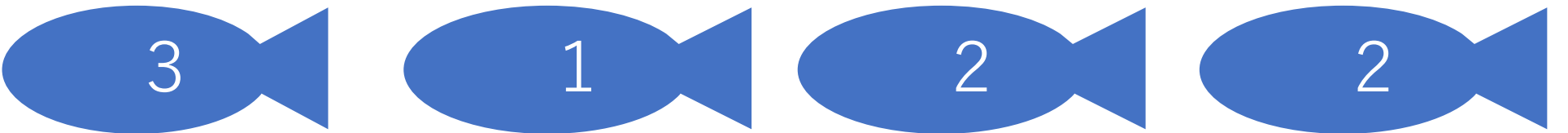
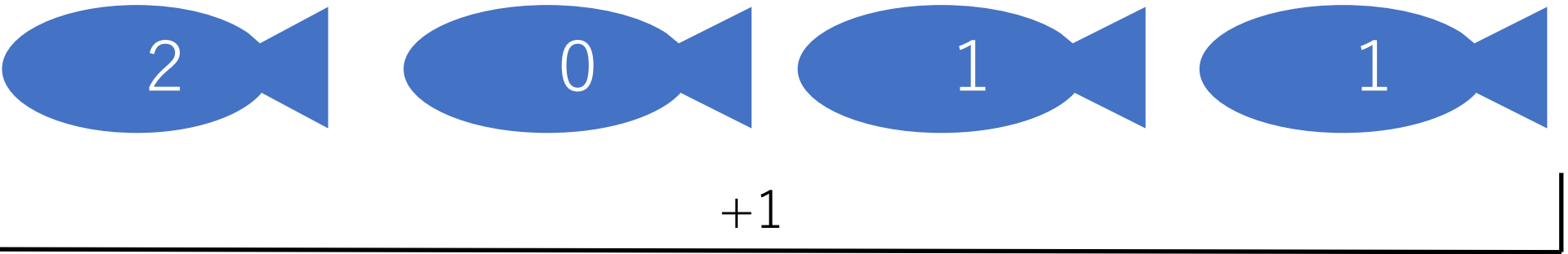
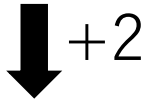
入力例 1



$D=2$

$C=[3,1,2,1]$

$(L,R) = (1,3)$



問題の性質



クエリ



最適化

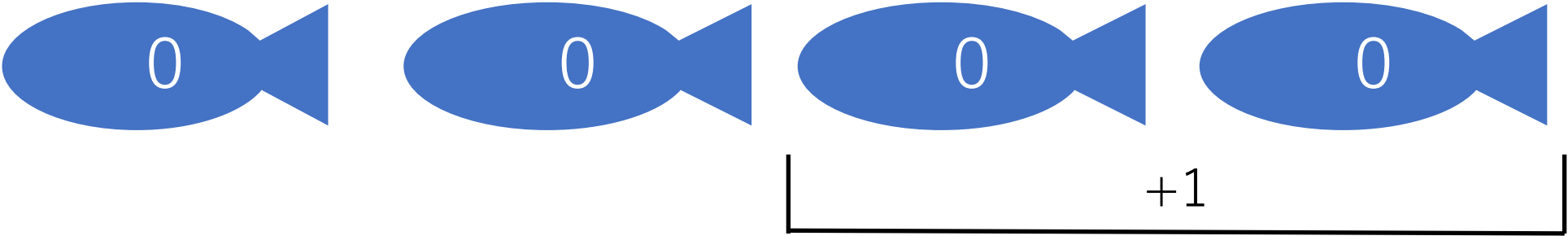
小課題 1

$O(NQ)$

気づき 1

- ・クエリの区間外の魚は無視してよい

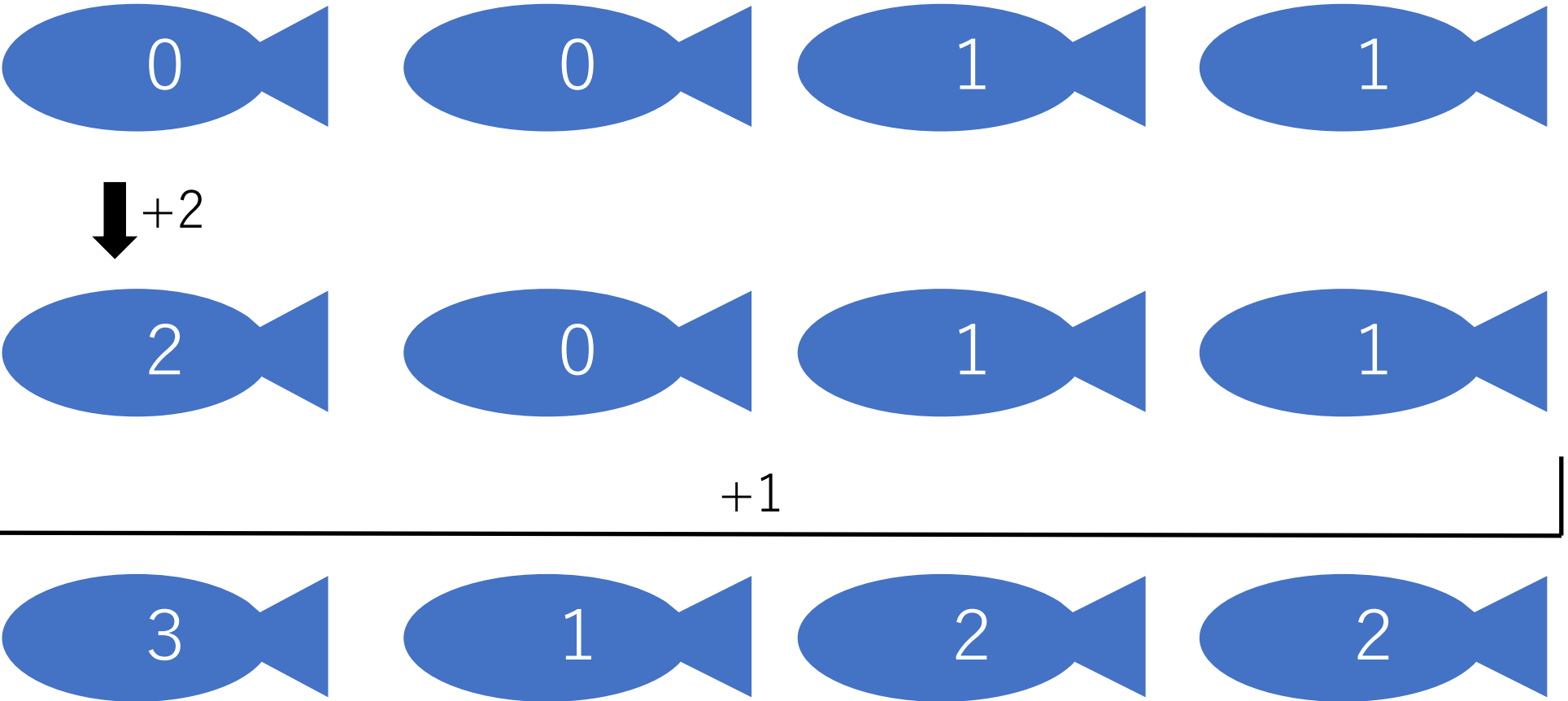
入力例 1



D=2

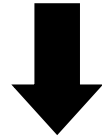
C=[3,1,2,1]

↓ +2
(L,R) = (1,3)



気づき 1

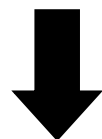
- クエリの区間外の魚は無視してよい



- $Q=1, (L,R)=(1,N)$ の場合が $O(N)$ で解ければ、全体 $O(NQ)$ で解ける

問題の緩和

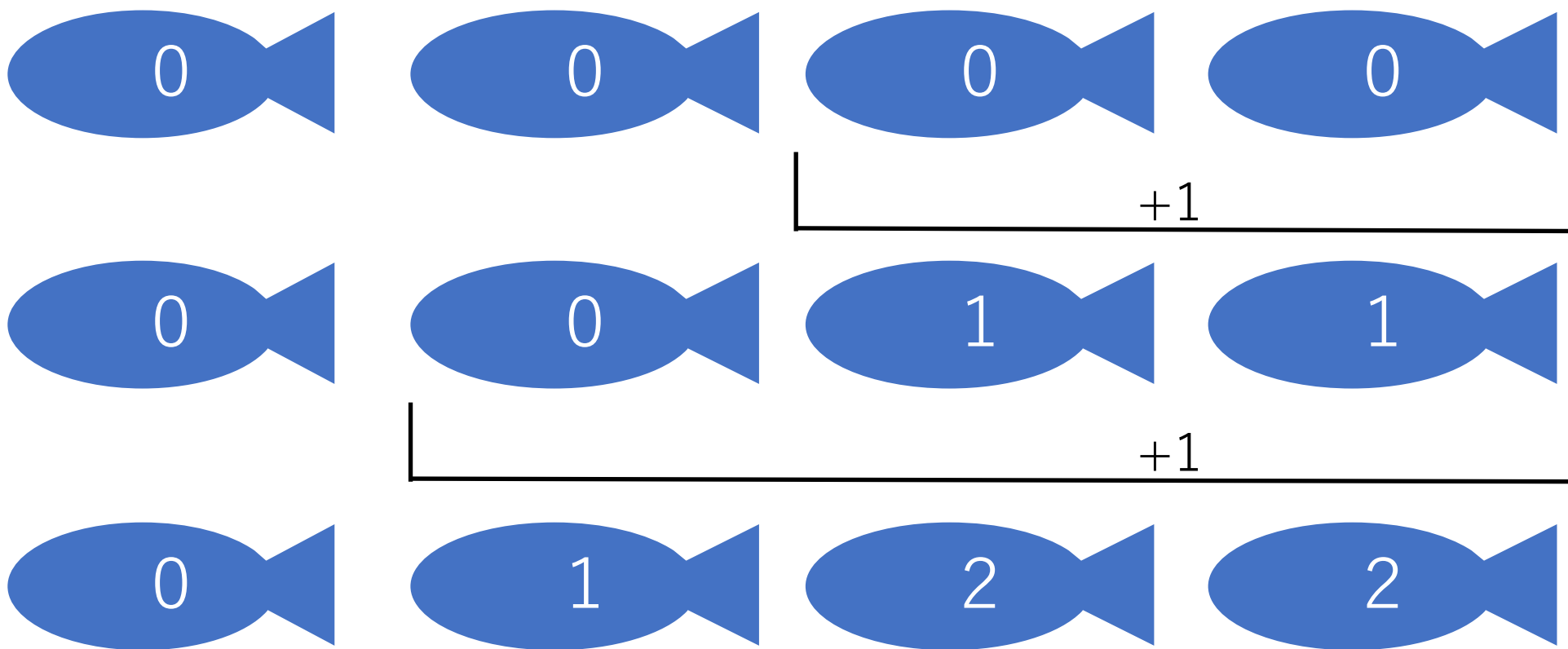
- 操作Aの回数の最小値を求めよ



- 操作Aの回数の最小値が0回かどうか判定せよ

問題の緩和

- 操作Bのみで作れる魚のIQ列を考える



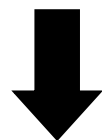
問題の緩和

- 操作Bのみで作れる魚のIQ列
 - • • 任意の広義単調増加列

(証明の概略：差分をとる)

問題の緩和

- 操作Aの回数の最小値が0回かどうか判定せよ

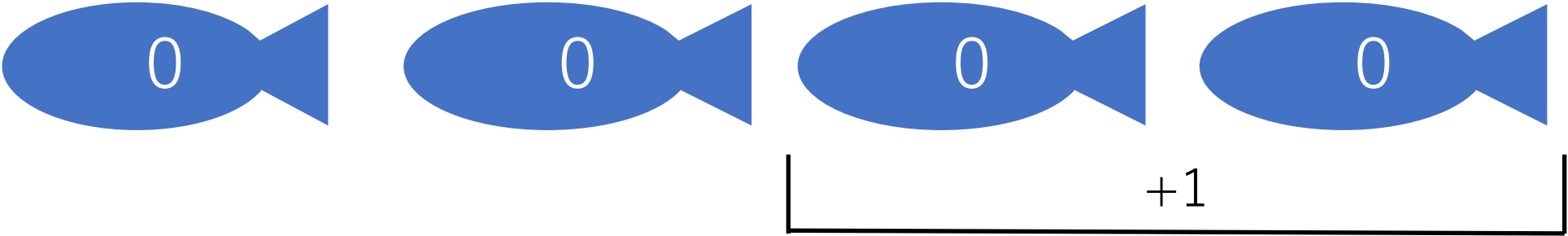


- 数列が広義単調増加か判定すればよい

気づき 2

- 操作の順番は関係ない
- 必要な情報はどの魚にどの餌を何回あげたか

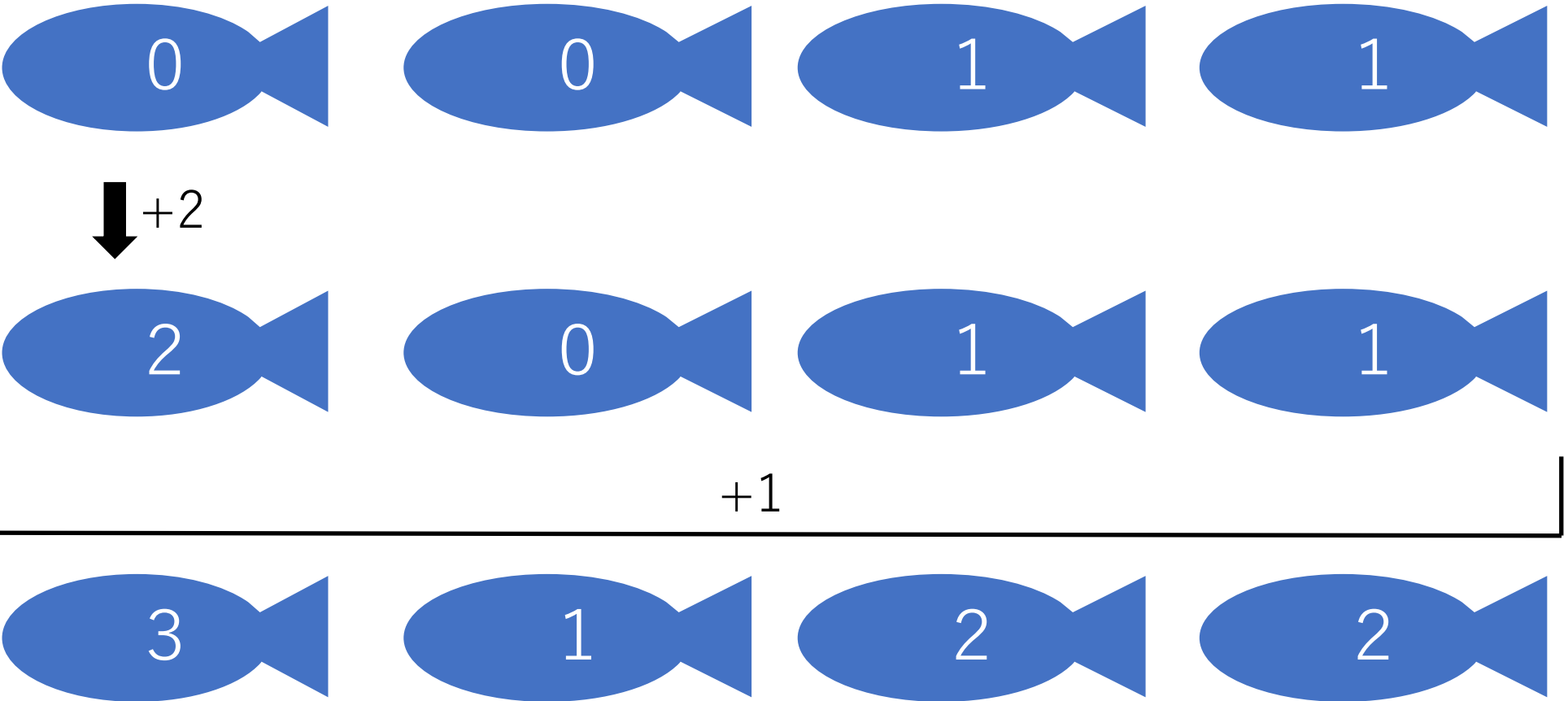
入力例 1



D=2

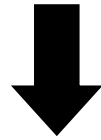
C=[3,1,2,1]

↓ +2
(L,R) = (1,3)



気づき 2

- 操作の順番は関係ない
- 必要な情報はどの魚にどの餌を何回あげたか

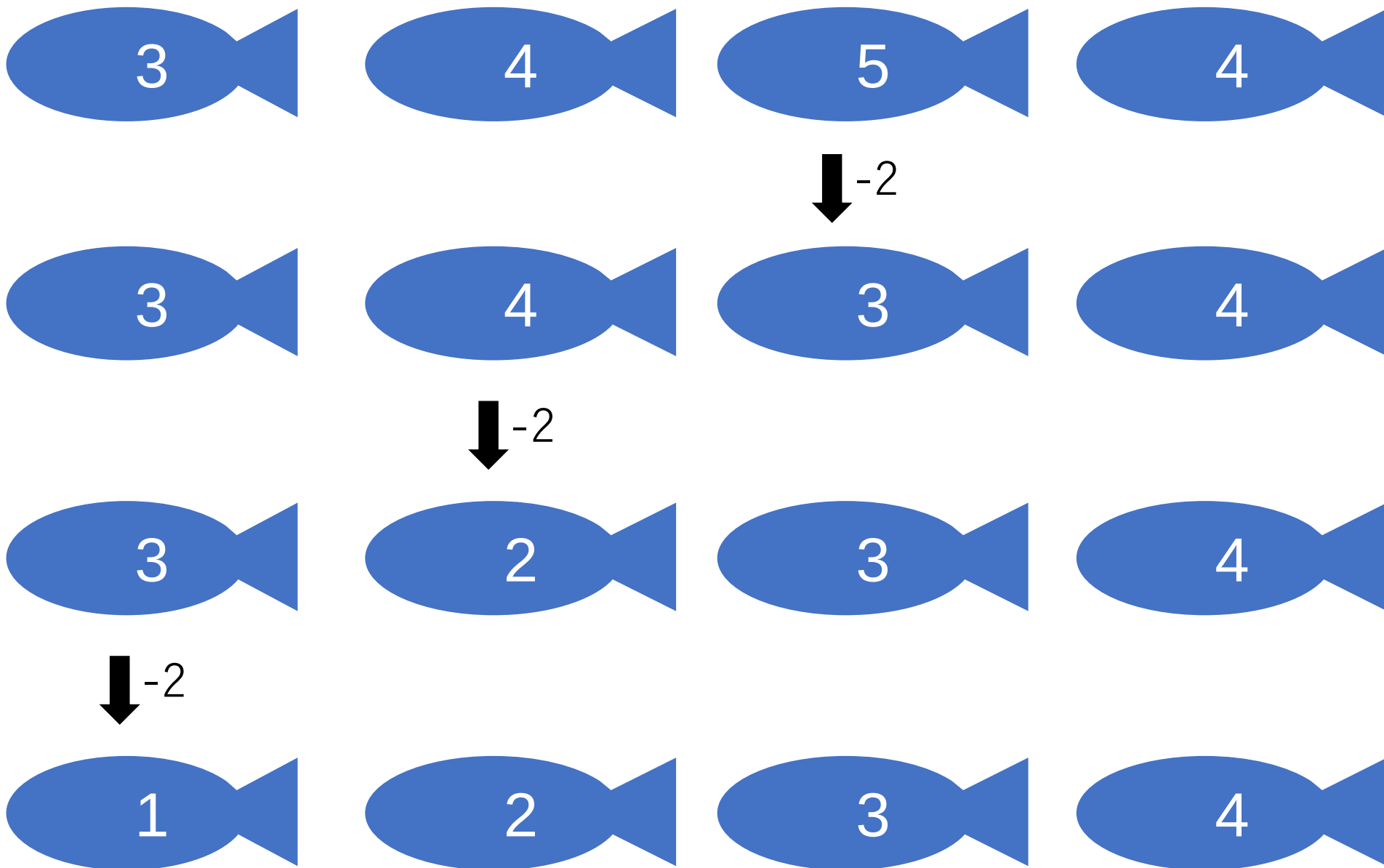


- 例えば初めに操作Bをまとめて行ってから操作Aをまとめて行う、としてよい

気づき 3

- 操作を逆順に見た方が分かりやすい
- 最初魚のIQは C_i
- ある魚を選んで D 減らす、という操作をできるだけ少ない回数行って広義単調増加列にしたい

D=2



気づき 4

- 右の魚から貪欲にIQを削っていけばよい
- 左端の魚のIQが負になれば不可能

小課題 1 が通る (9点)

```
11 sol(11 n,11 d,vi v){
    11 ans=0,n=v.size();
    for(11 i=n-2;i>=0;i--)if(v[i]>v[i+1]){
        11 k=(v[i]-v[i+1]+d-1)/d;
        ans+=k;
        v[i]-=k*d;
    }
    if(v[0]<0)return -1;
    return ans;
}
```

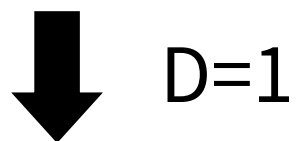
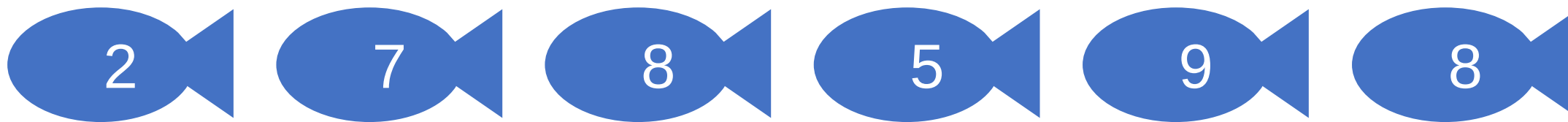
小課題 2

$$C_i \leq 1$$

累積和と二分探索で頑張る

小課題 3

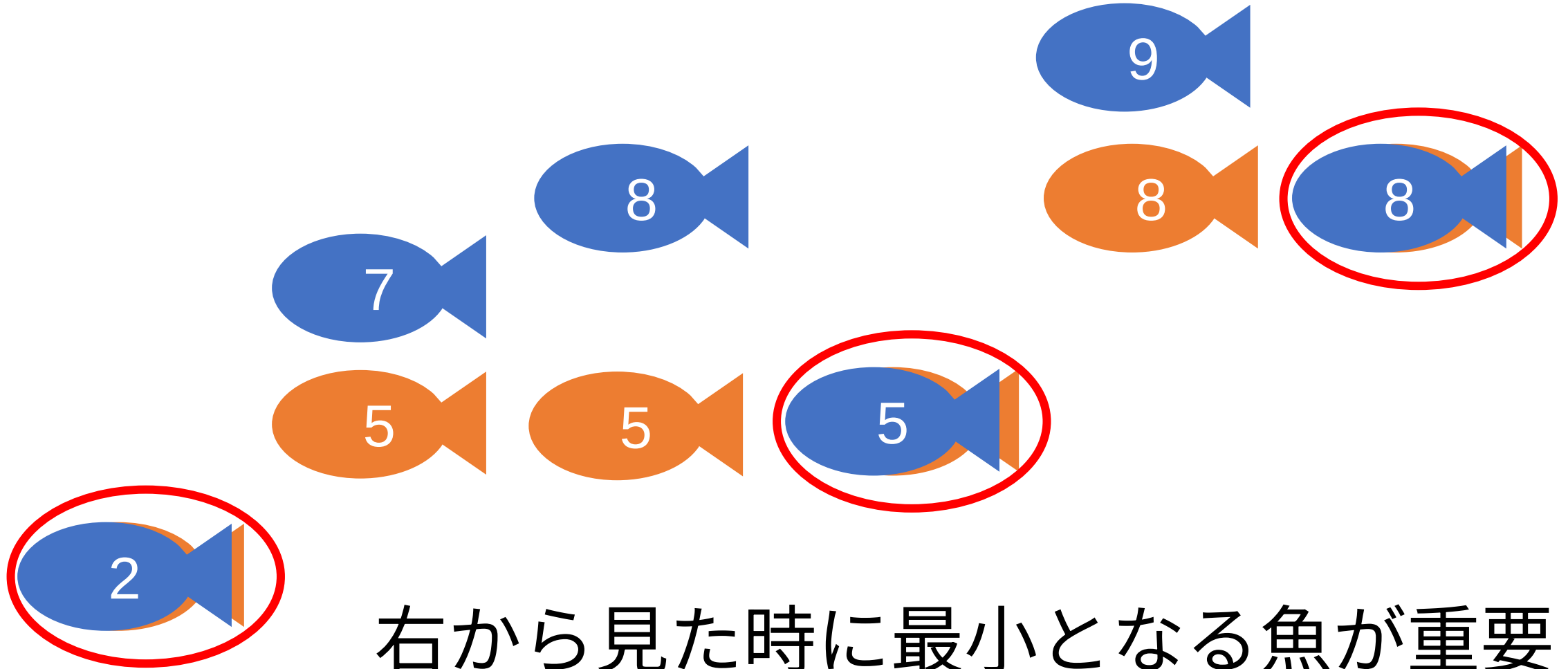
$D=1$

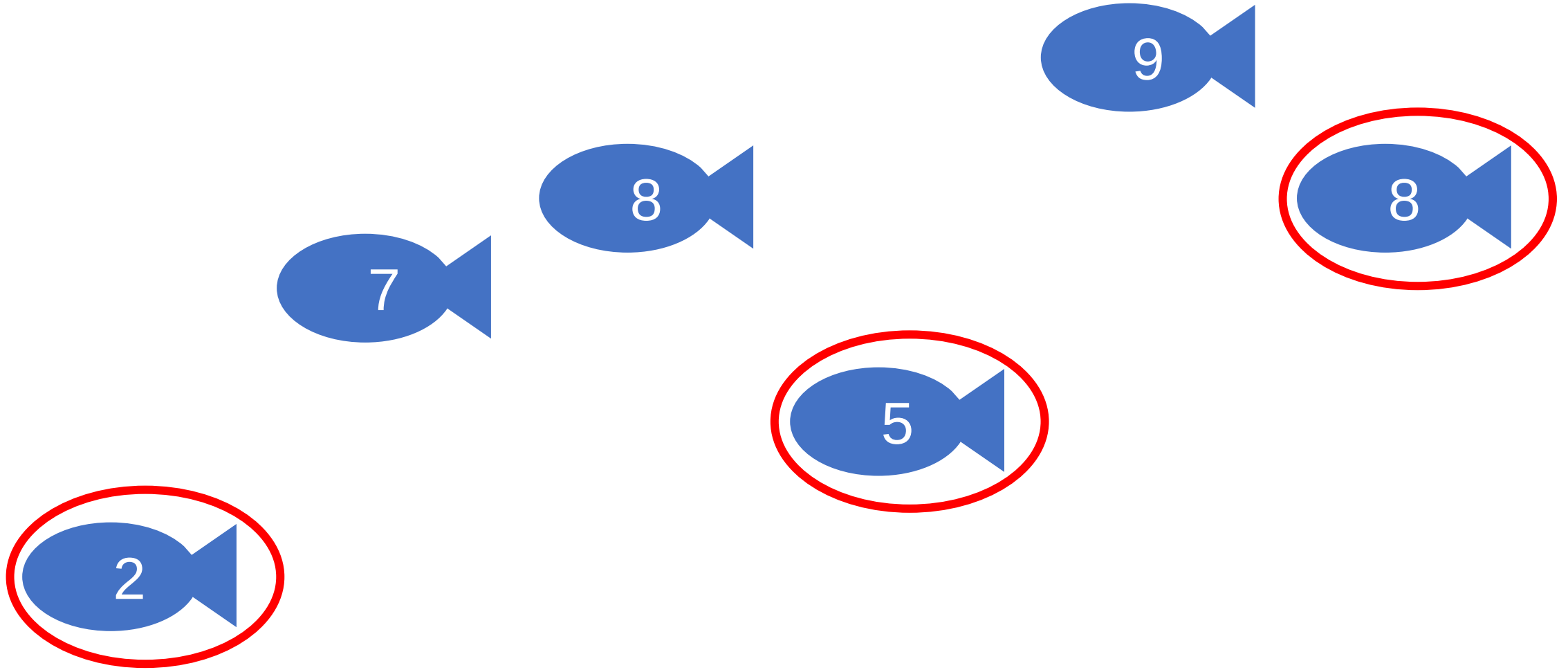


始状態と終状態の総和の差から操作回数が計算できる

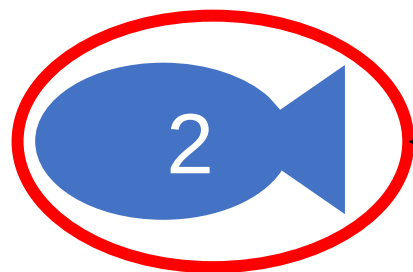
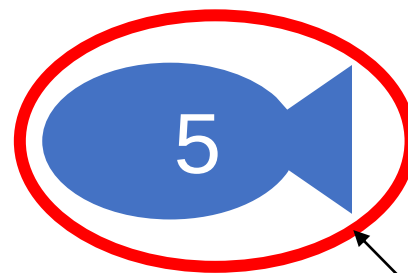
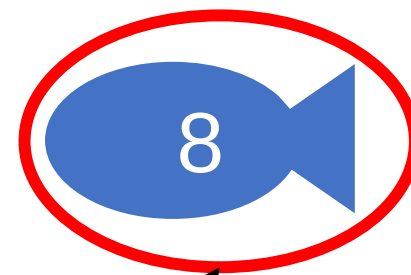
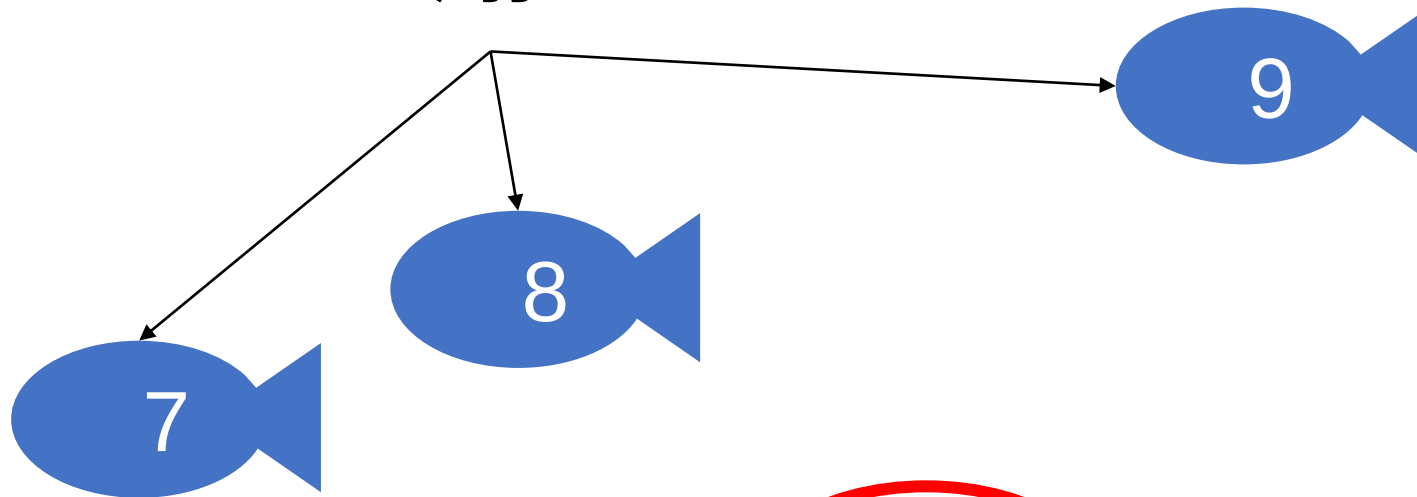
→終状態を求めたい

2次元平面にプロットします

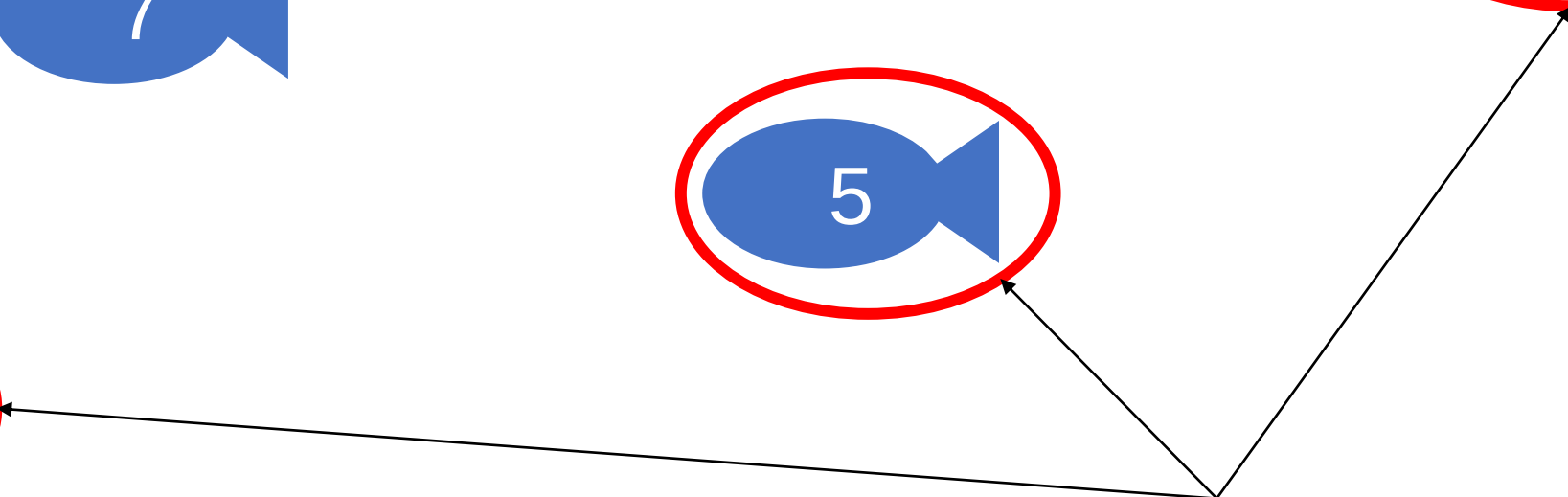




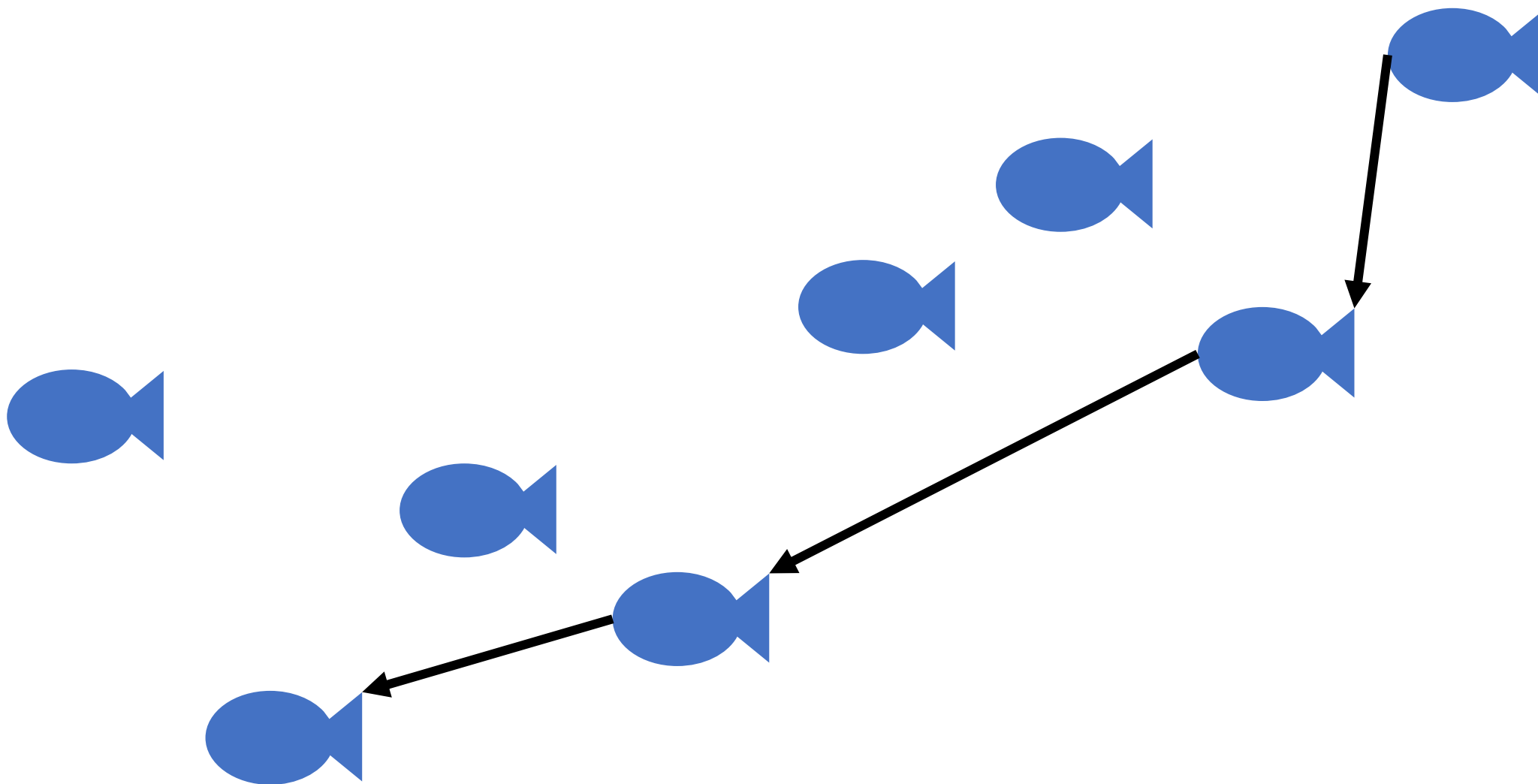
鰯



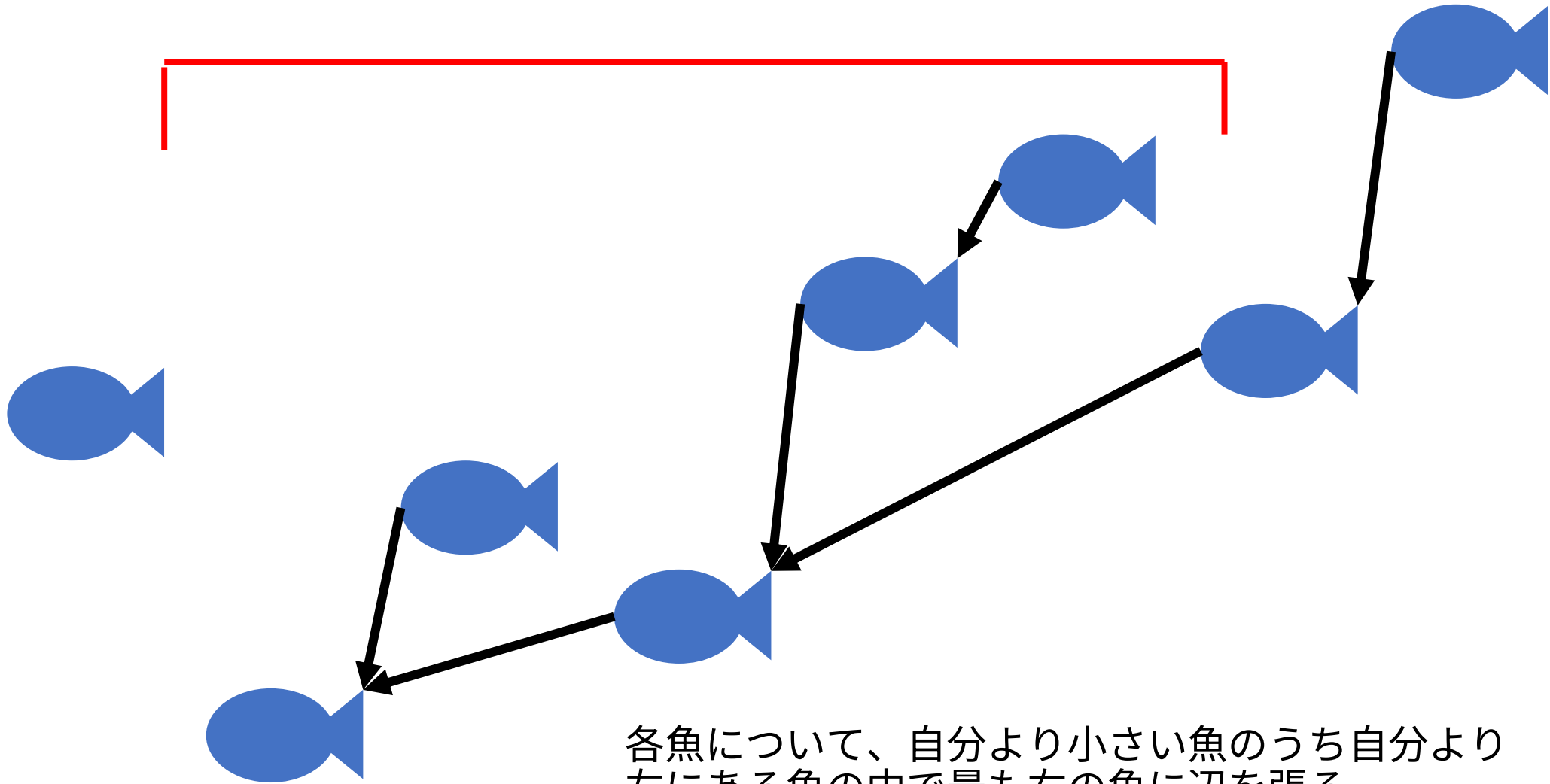
秋刀魚



秋刀魚だけを取り出したい！

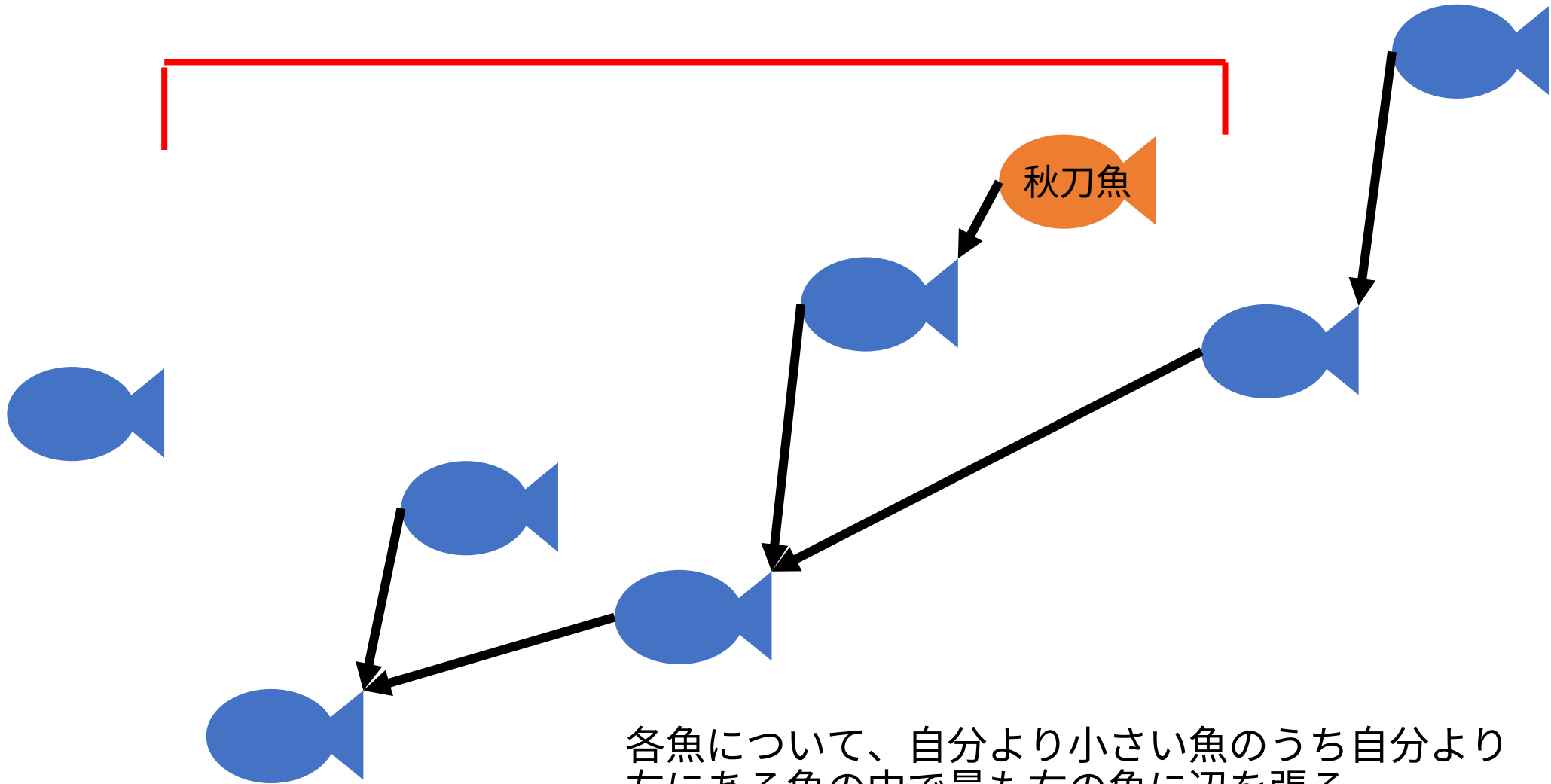


任意の区間クエリで秋刀魚だけを取り出
したい！

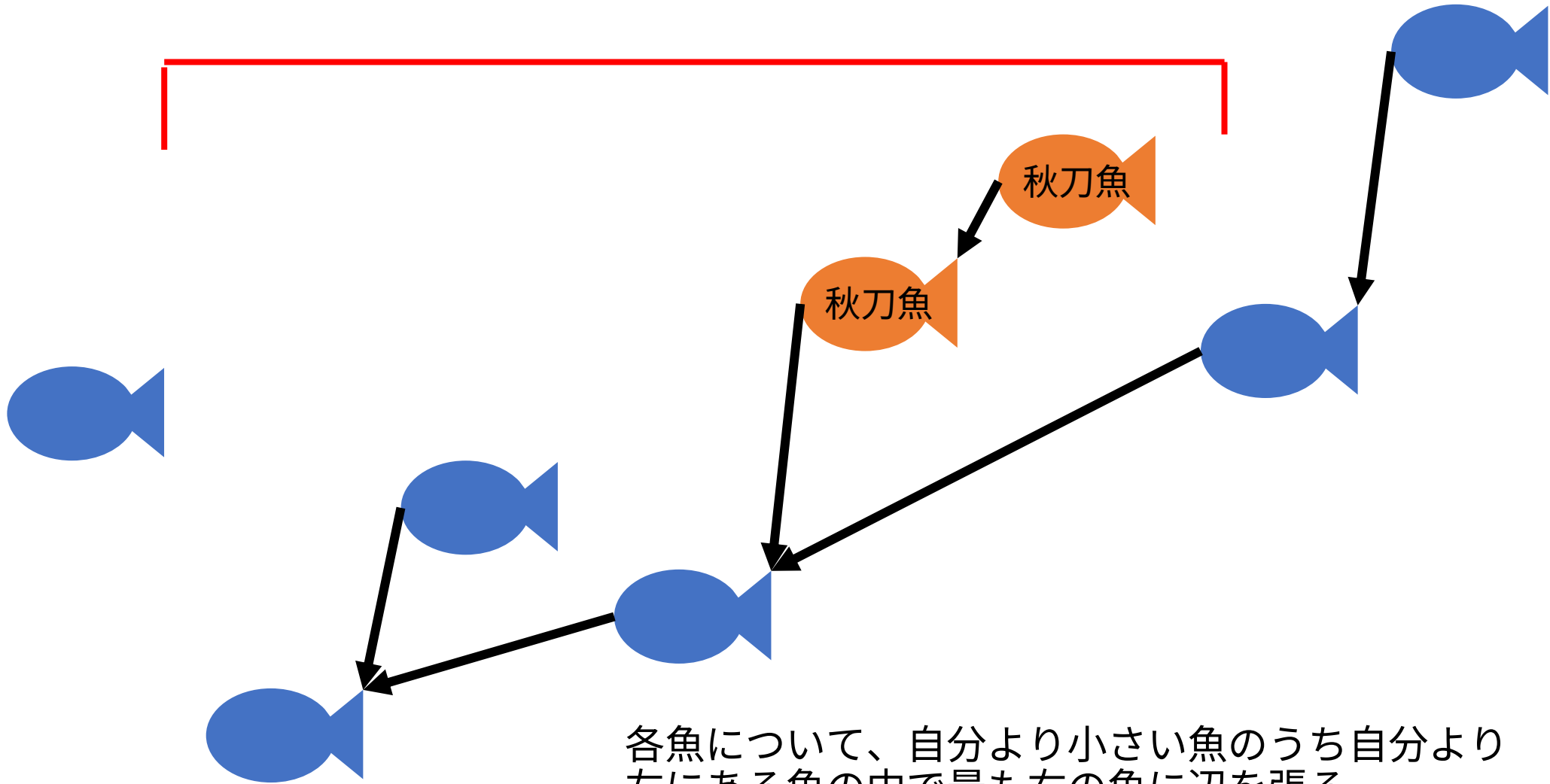


各魚について、自分より小さい魚のうち自分より左にある魚の中で最も右の魚に辺を張る

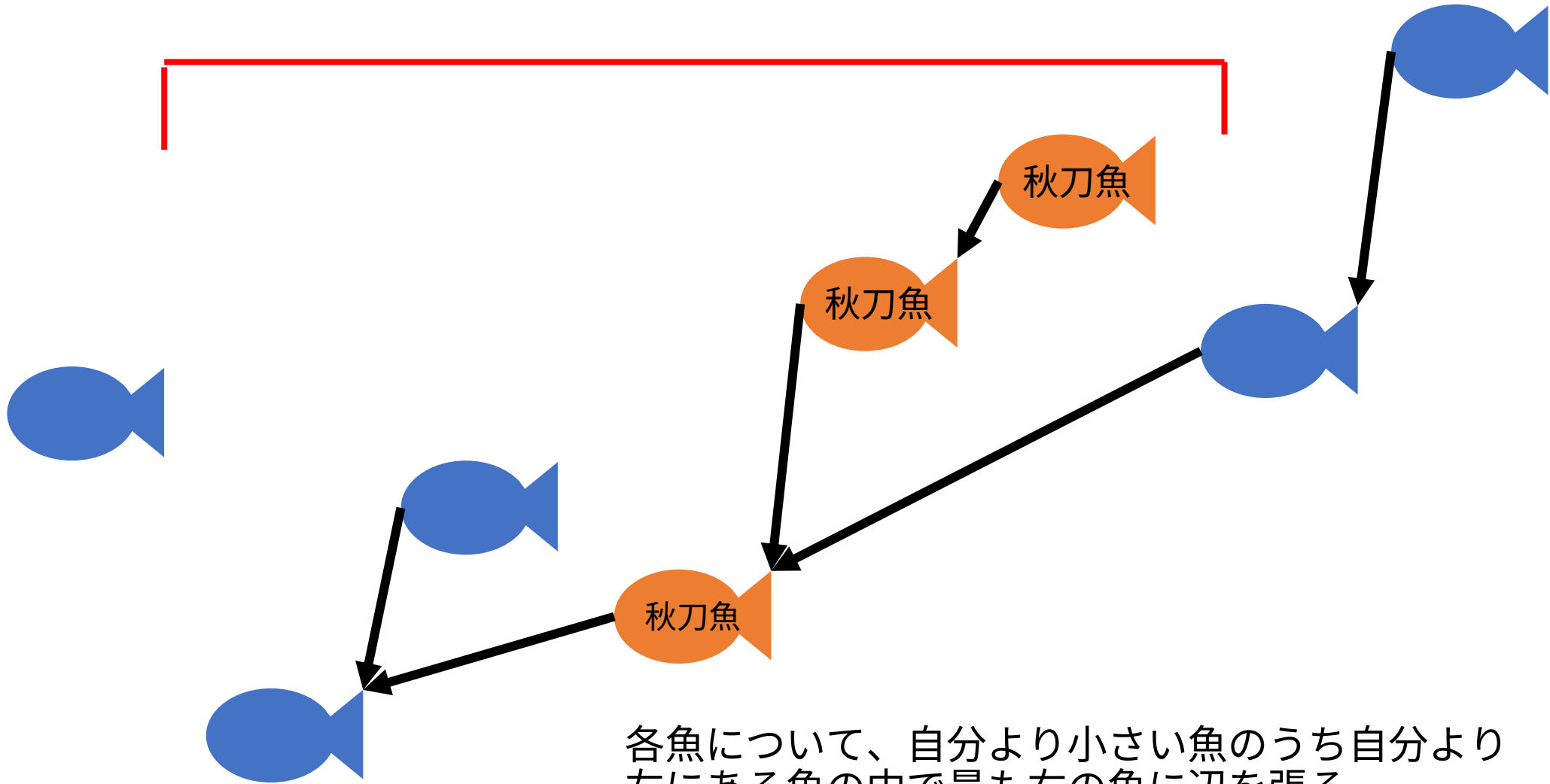
→有向森



→有向森

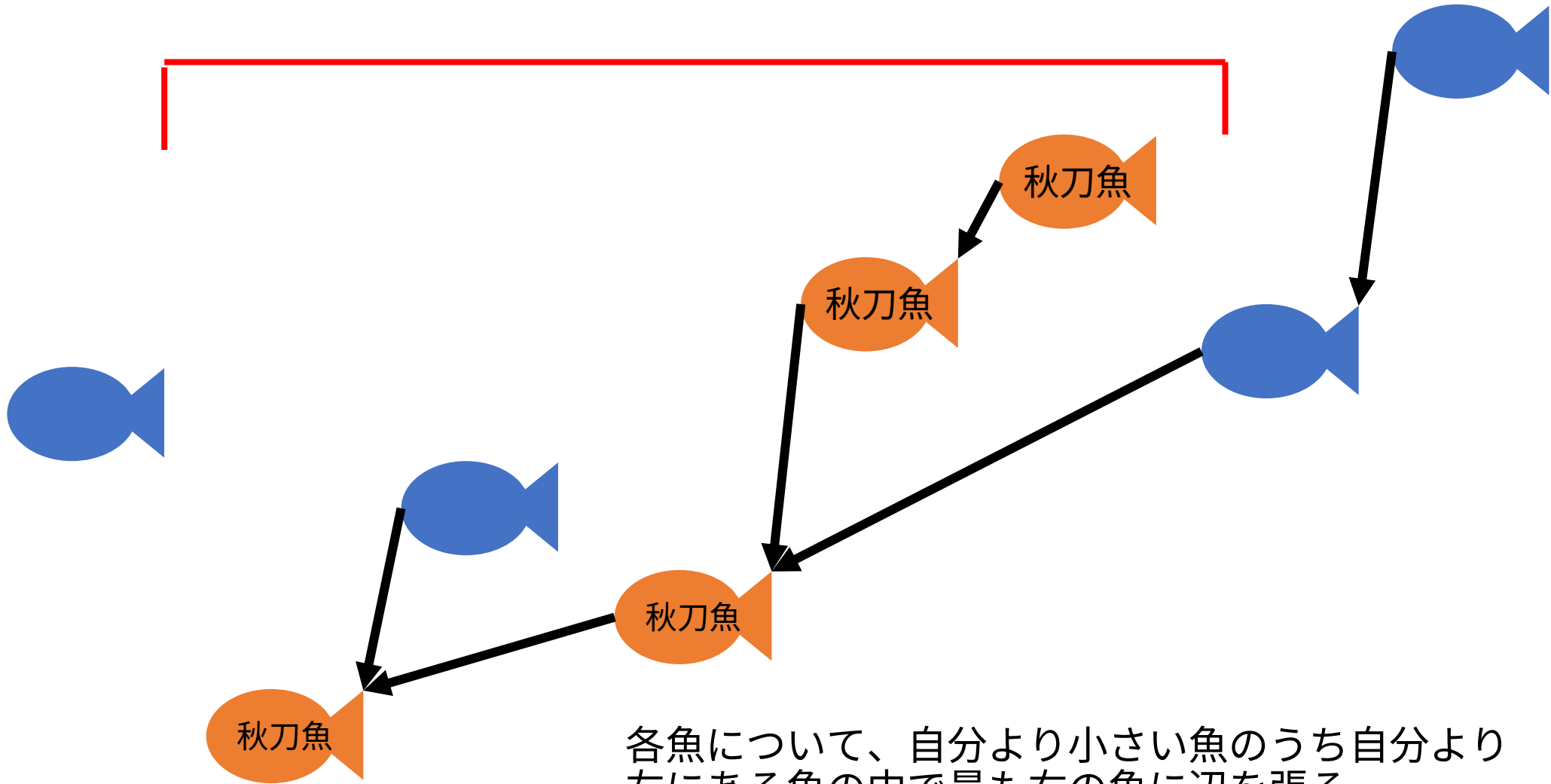


→有向森



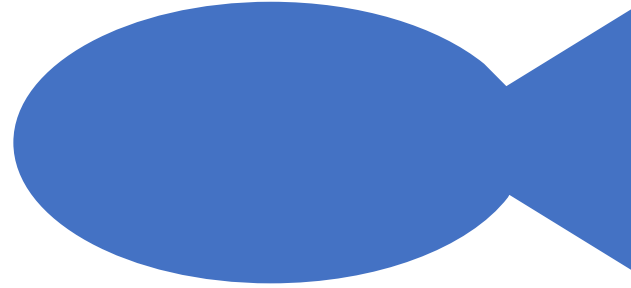
各魚について、自分より小さい魚のうち自分より
左にある魚の中で最も右の魚に辺を張る

→有向森



→有向森

愚直に辺をたどっていくとクエリあたり $O(N)$ にかかってしまう



出次数が1以下のグラフで高速に辺をたどる時の典型テク

・・・ダブリング

$O((N+Q)\log N)$ で小課題3が通る

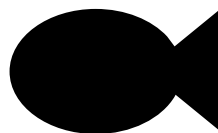
(別解)

データ構造を使わない選択肢

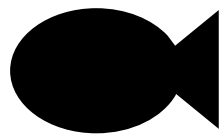
クエリは任意の順番で処理してよい

Rの昇順でクエリを処理するとどうなるか？

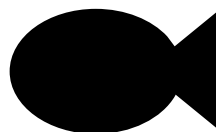




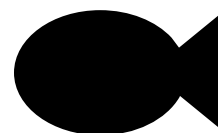


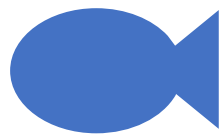


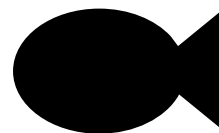
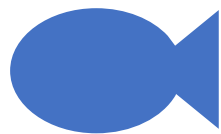




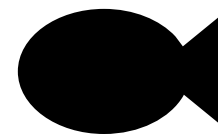
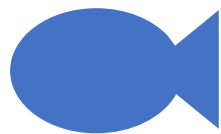




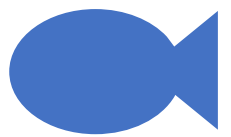
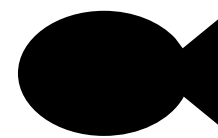














Rを右に移動させつつ、秋刀魚の一覧をスタックで管理しておく

クエリは適宜処理

秋刀魚の管理は合計で $O(N)$

スタックに累積和の情報も持たせておくと
クエリの処理は二分探索で各 $O(\log N)$

```

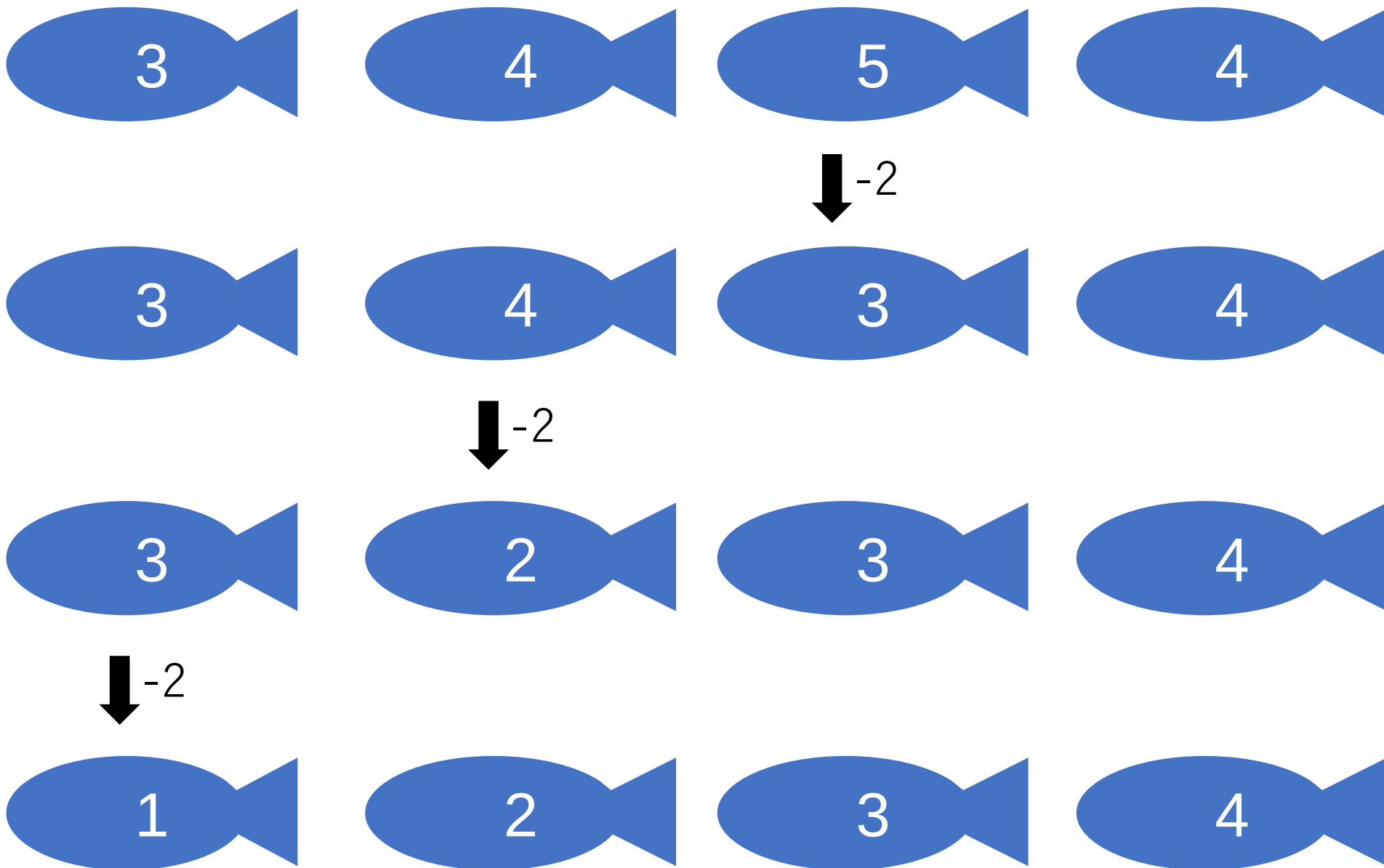
vi sol(ll n,ll q,vi v,vvp query){
    vi ans(q),rui(n+1);
    rep(i,n)rui[i+1]=rui[i]+v[i];
    vp st;st.pb(-1,0);
    rep(i,n){
        while(st.size()>1&&v[st.back().fi]>=v[i])st.pop_back();
        st.pb(i,st.back().se+v[i]*(i-st.back().fi));
        for(auto x:query[i]){
            ll idx=lb(st,P(x.fi,-inf));
            ans[x.se]=st.back().se-st[idx-1].se-v[st[idx].fi]*(x.fi-1-st[idx-1].fi);
            ans[x.se]=rui[i+1]-rui[x.fi]-ans[x.se];
        }
    }
    return ans;
}

```

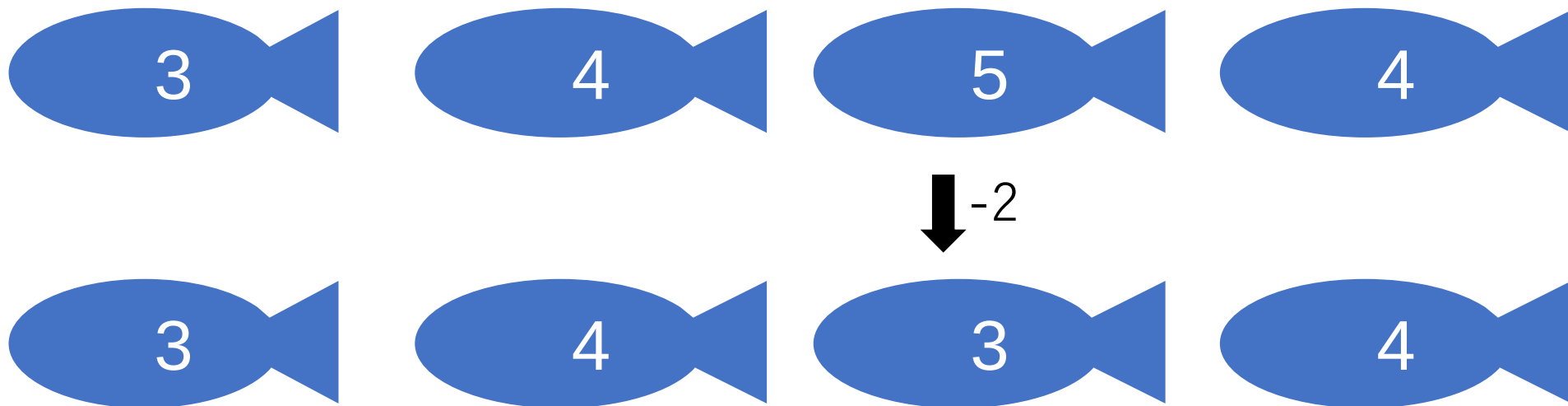
小課題 4

Cが単調減少

D=2



D=2

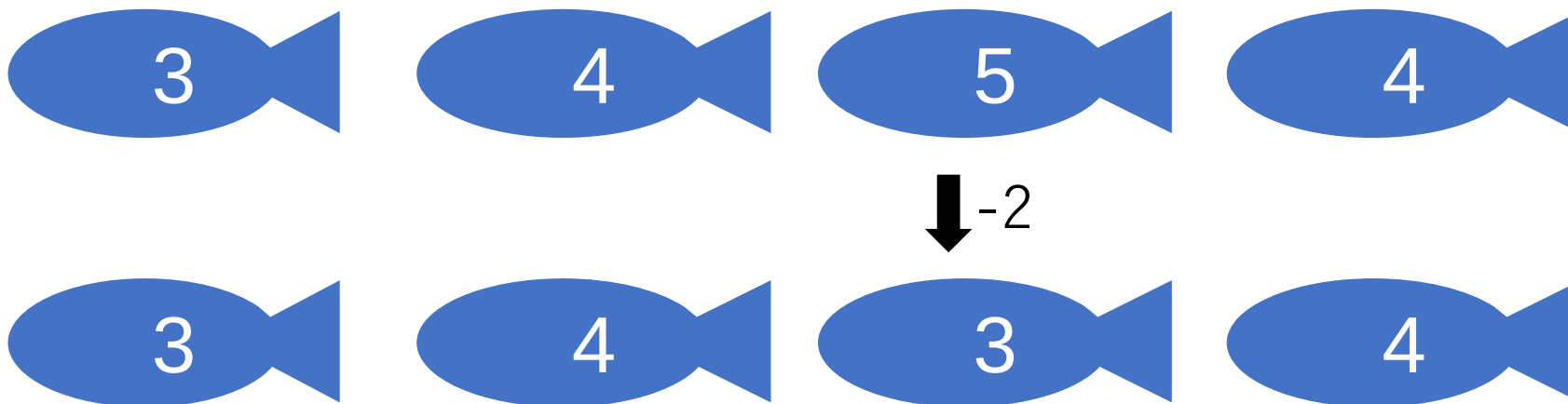


D=1の時にはなかった余計なIQの減少

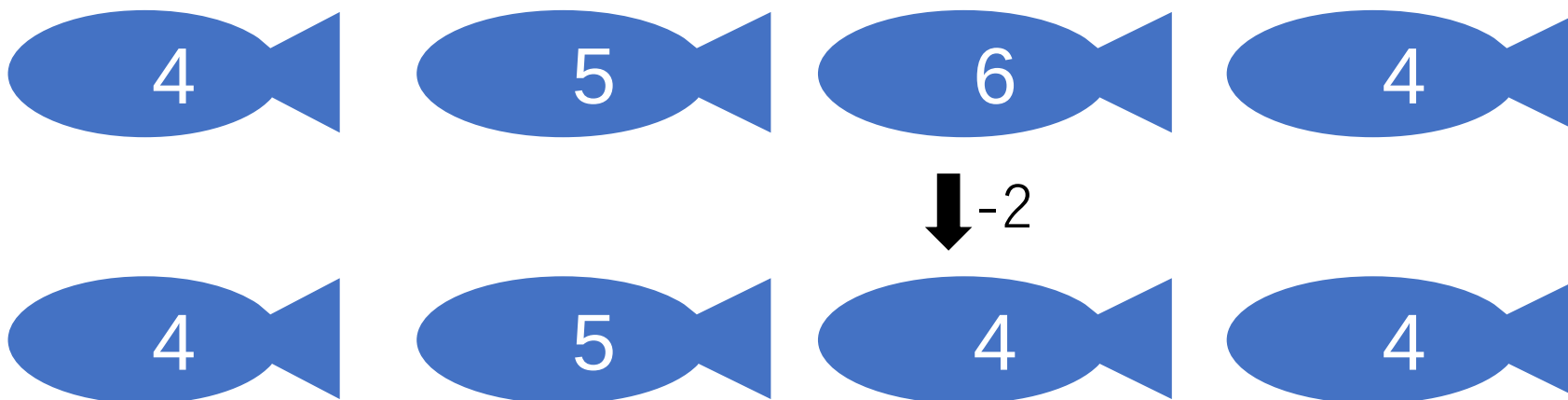
→なんとかしたい

とりあえず不可能なケースは後回し

D=2



元の数列を同値変形



全部の値がDの倍数になるように同値変形できる！

D=2



最後に全部 D で割れば $D=1$ の場合に帰着される

小課題4では元の数列が単調減少だから同値変形後も単調減少

操作後はすべての魚のIQが右端の魚のIQに一致するため、簡単に操作回数が求まる

左端の魚に対する操作回数が分かるため、同値変形前の数列を用いて不可能かどうか判定できる

```
for(ll i=n-2;i>=0;i--){  
    if(a[i]>=a[i+1])v[i]=v[i+1]+(a[i]-a[i+1]+d-1)/d;  
    else v[i]=v[i+1]-(a[i+1]-a[i])/d;  
}
```


小課題 5

追加の制約なし

小課題3と小課題4の解法を組み合わせます

AC !

(注意点)

魚のIQの最大値は 10^{12} なのでオーバーフローに気を付けましょう

得点分布

100点：9人

64点：3人

48点：1人

44点：1人

36点：1人

29点：2人

20点：1人

16点：5人

9点：4人

0点：0人

A blue, multi-pointed starburst or badge-like shape with a white border, centered on a white background. It contains the text '0点:0人'.

0点:0人



JOI 2024-sp Day1

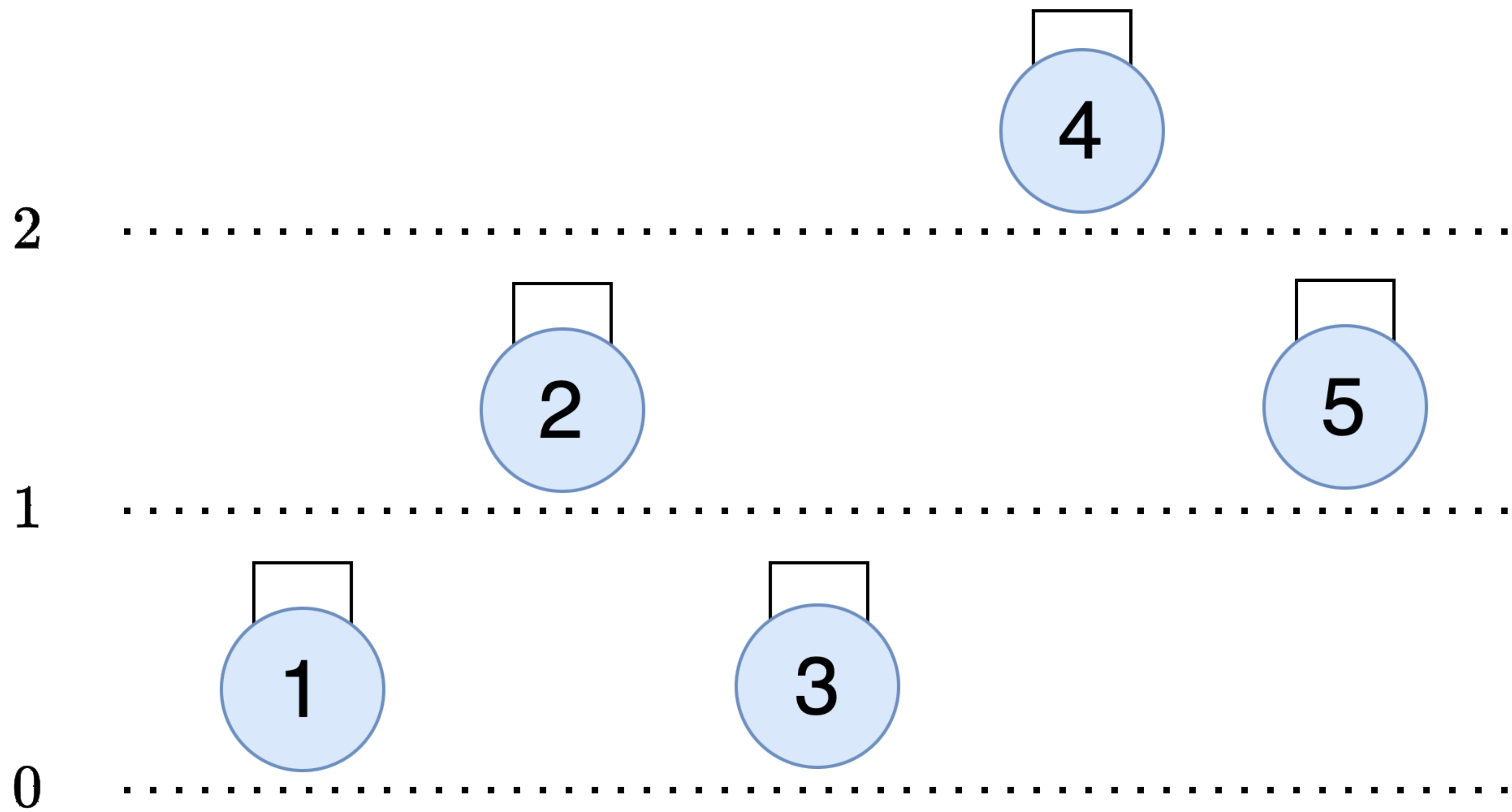
Ski 2 解説

渡邊 雄斗 (yuto1115)

問題概要

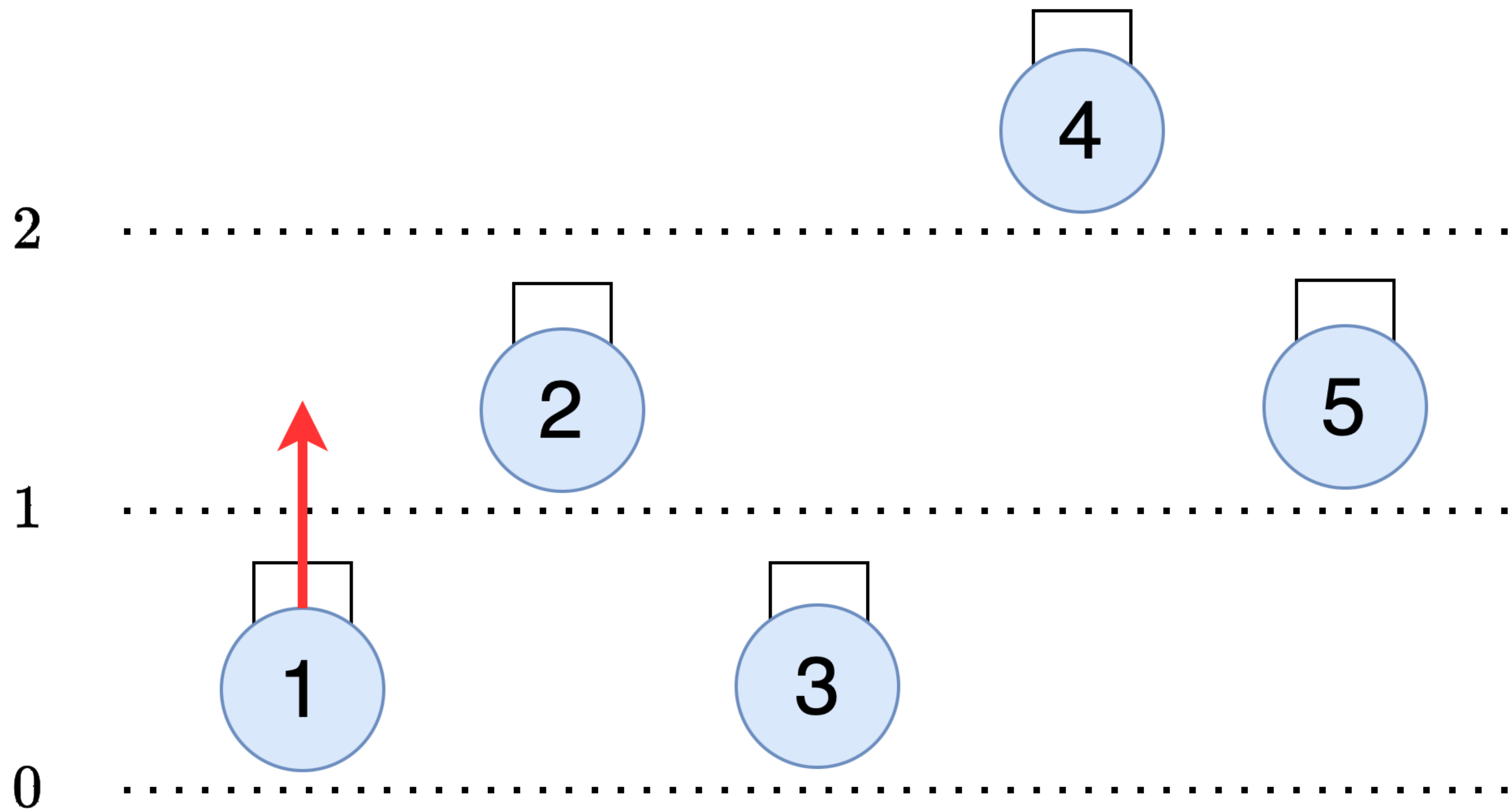
- N 個の地点があり、標高（初期値 H_i ）と接続設備の個数（初期値 1）が定まっている
- 以下の工事を好きな回数行う：
 - 盛土工事： K 払って、任意の地点の標高を 1 上げる
 - 拡張工事： C_i 払って、地点 i の接続設備の個数を 1 増やす
- どこかの地点 x にホテルを建てる
- x 以外のそれぞれの地点 i について、 i より標高が真に低い地点 j を選んで、 j の接続設備を一つ使って $i \rightarrow j$ のコースを作る
- コストの総和の最小値は？

$K = 2, S = 0$



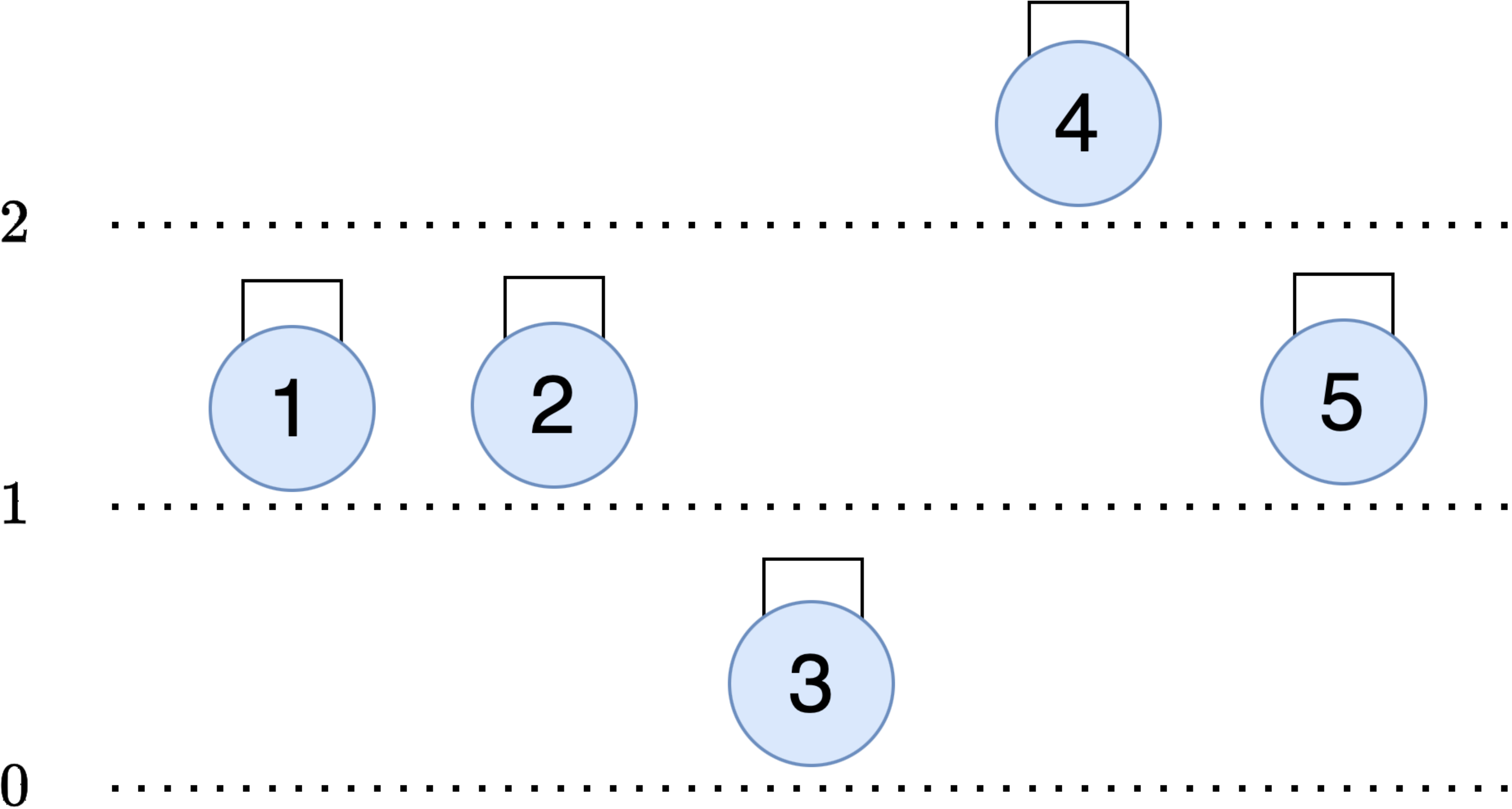
$C_i =$ 6 1 5 1 2

$K = 2, S = 0$



$C_i =$ 6 1 5 1 2

$K = 2, S = 2$



$C_i =$

6

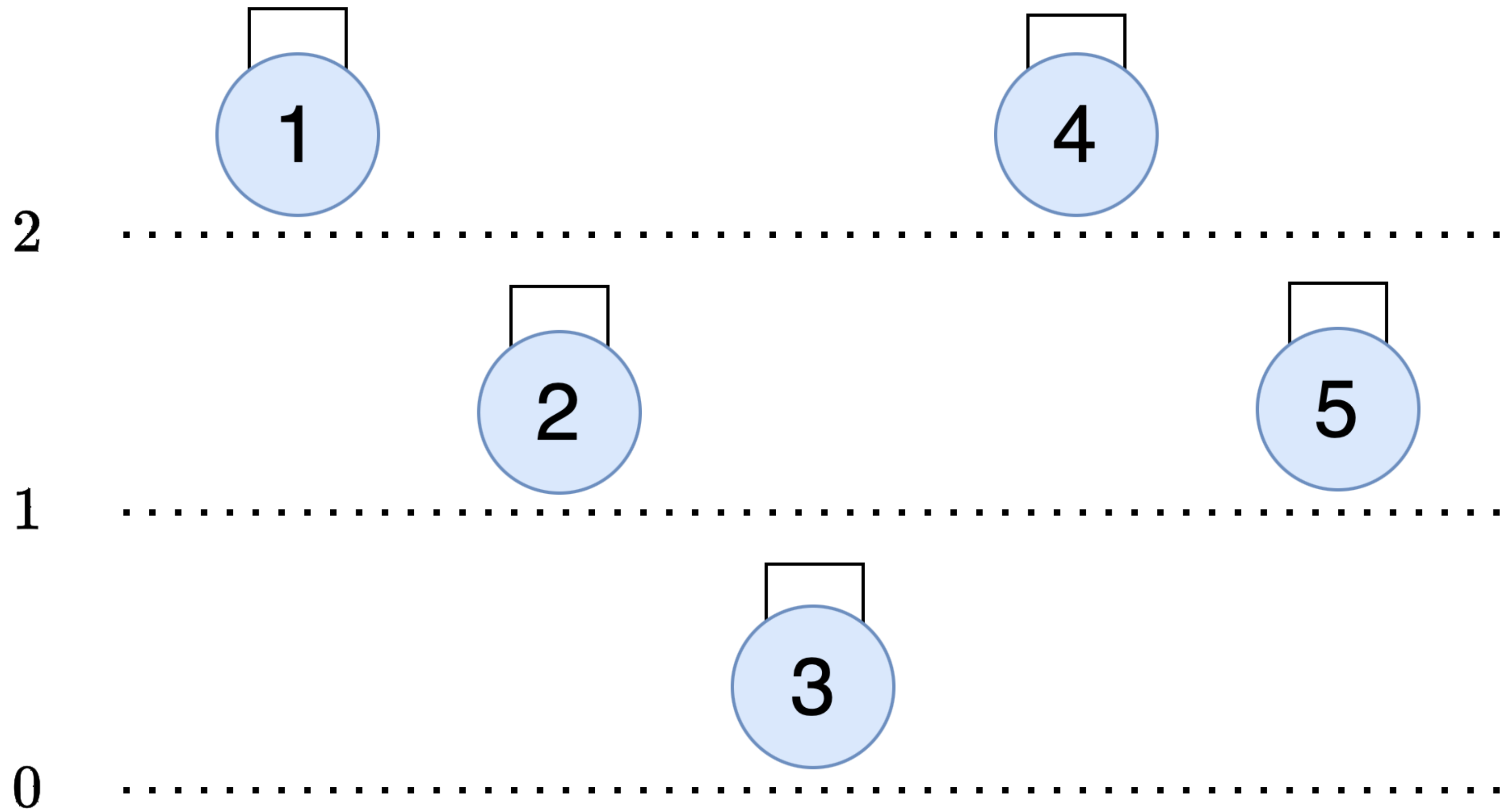
1

5

1

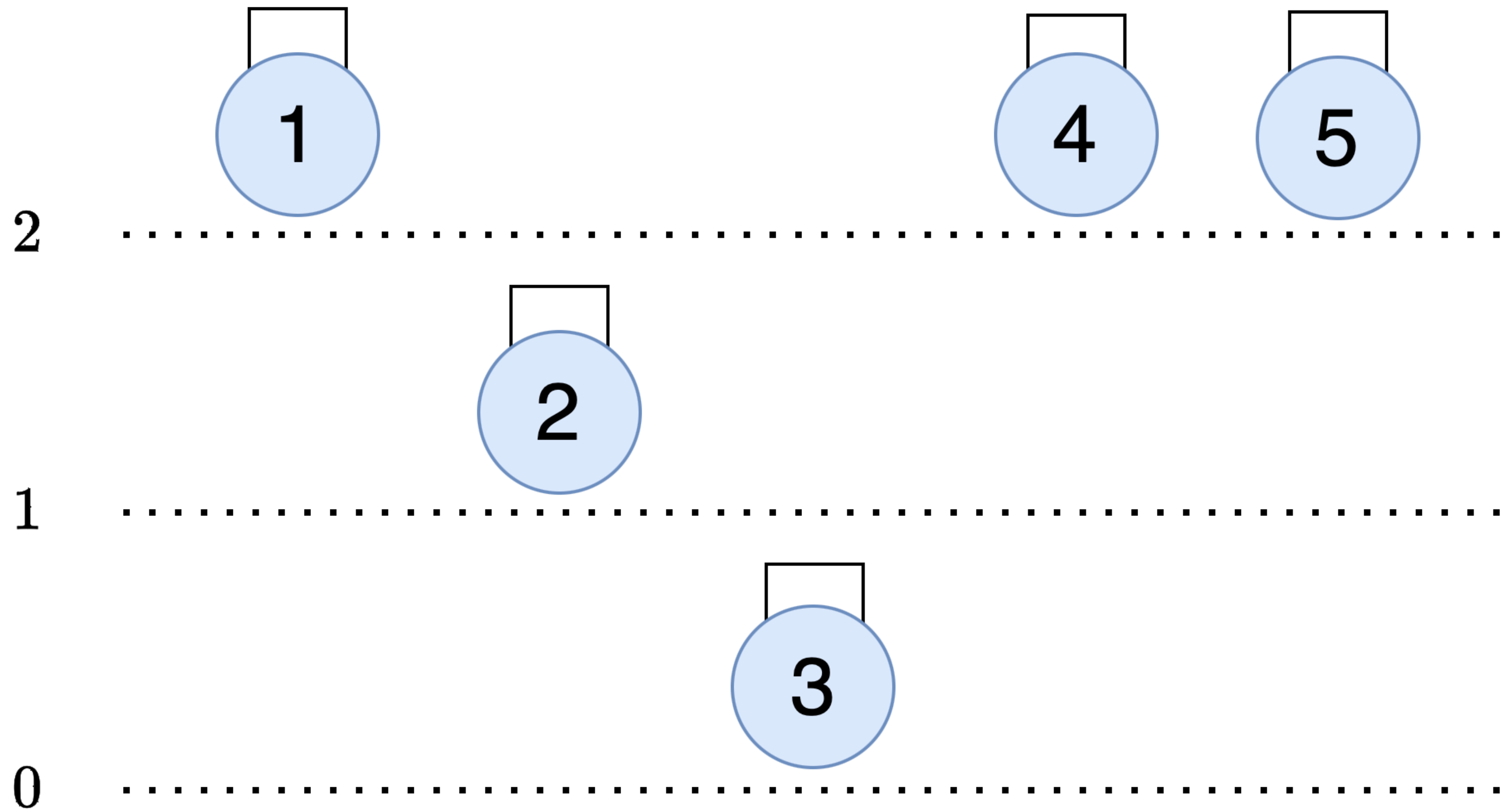
2

$K = 2, S = 4$



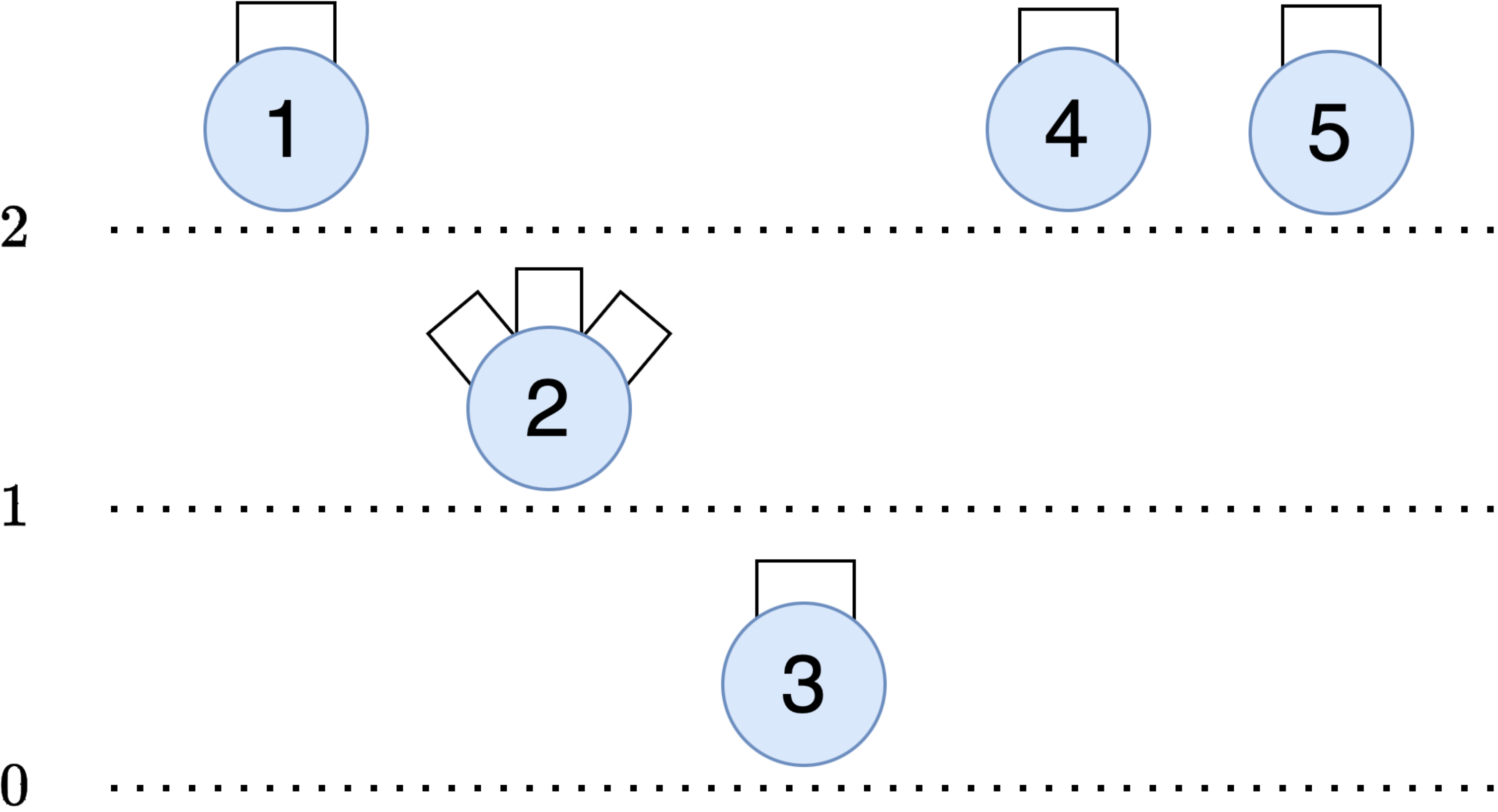
$C_i =$ 6 1 5 1 2

$K = 2, S = 6$



$C_i =$ 6 1 5 1 2

$K = 2, S = 8$



$C_i =$

6

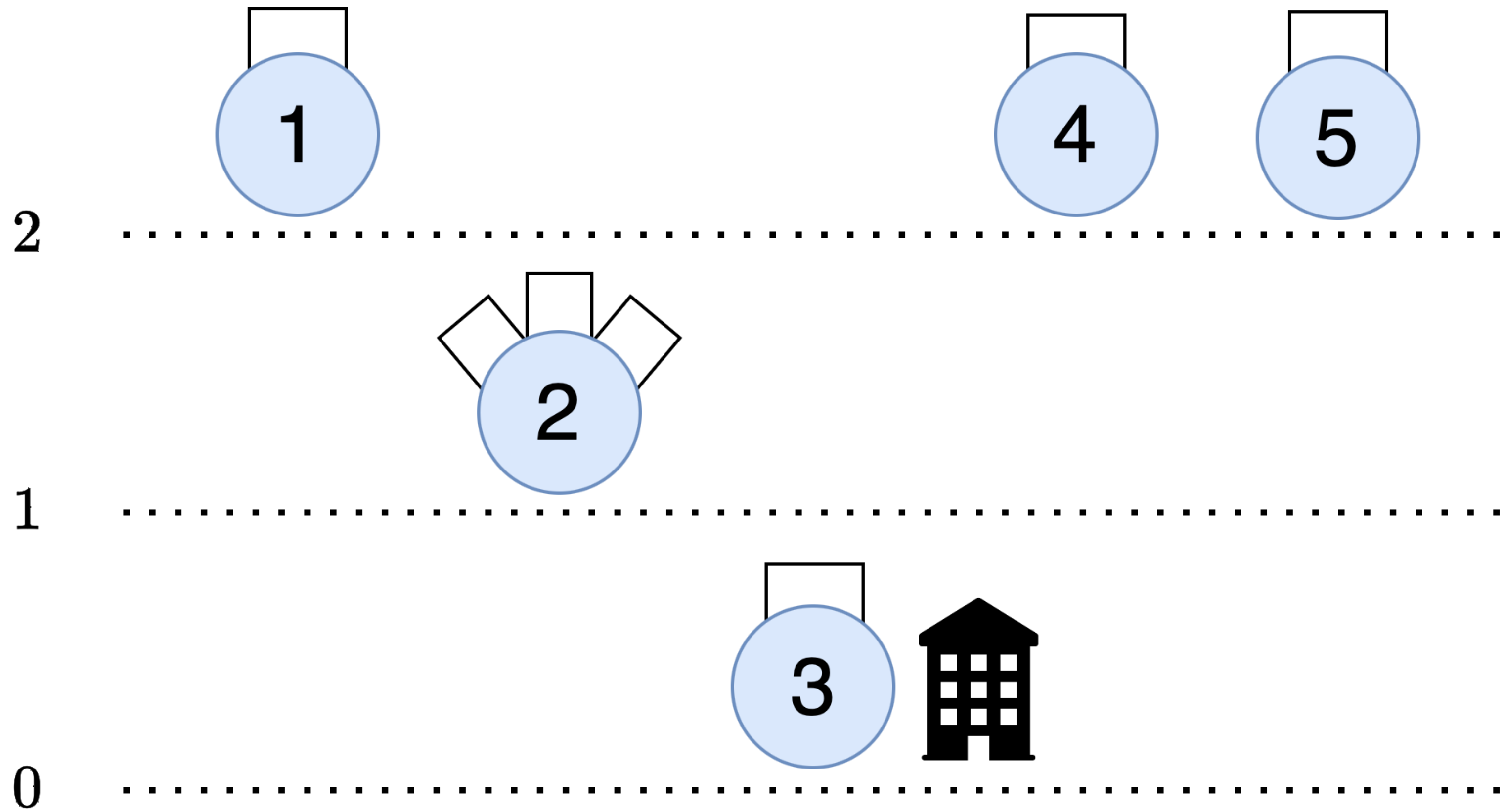
1

5

1

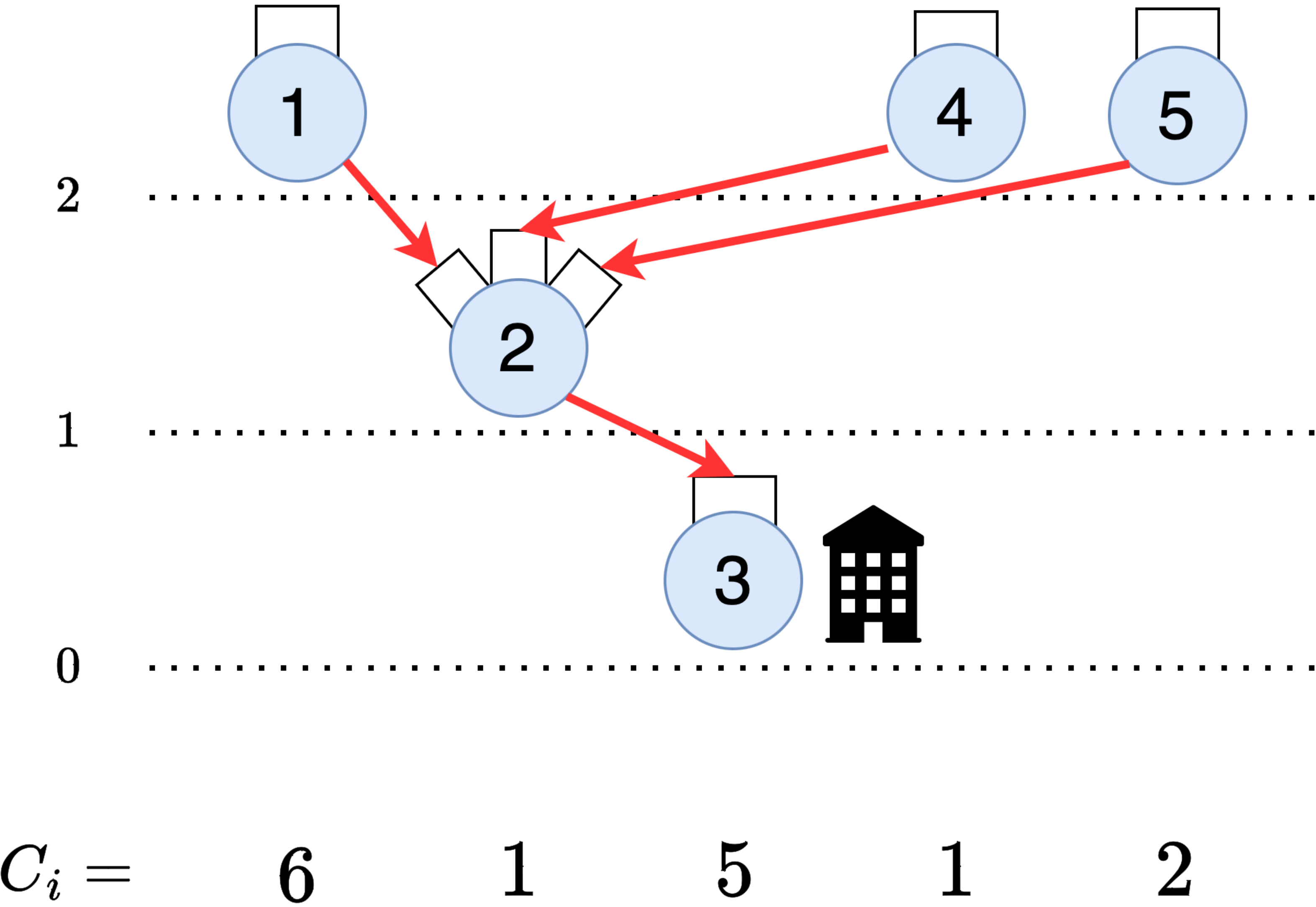
2

$K = 2, S = 8$



$C_i = \quad 6 \quad 1 \quad 5 \quad 1 \quad 2$

$K = 2, S = 8$

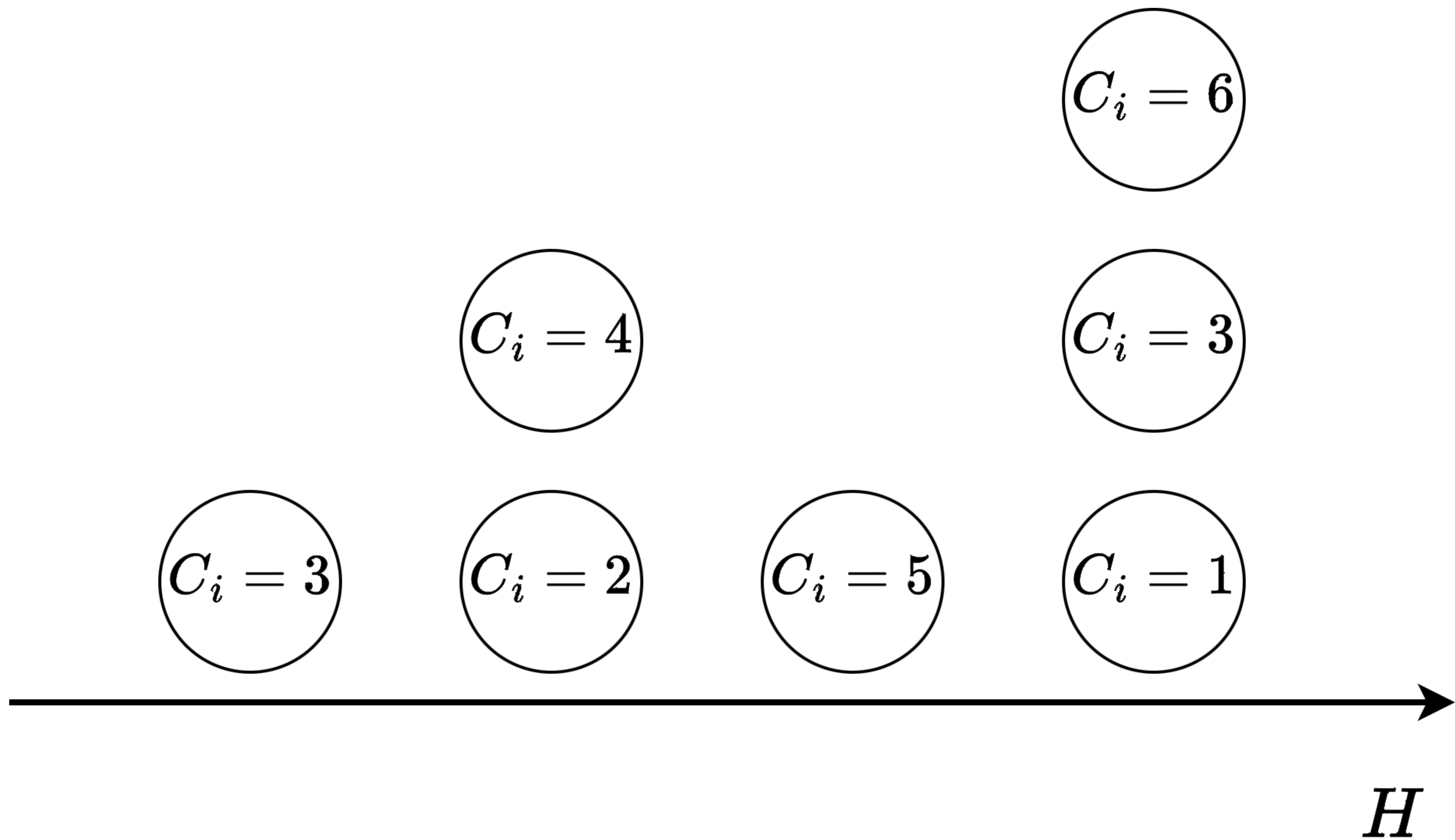


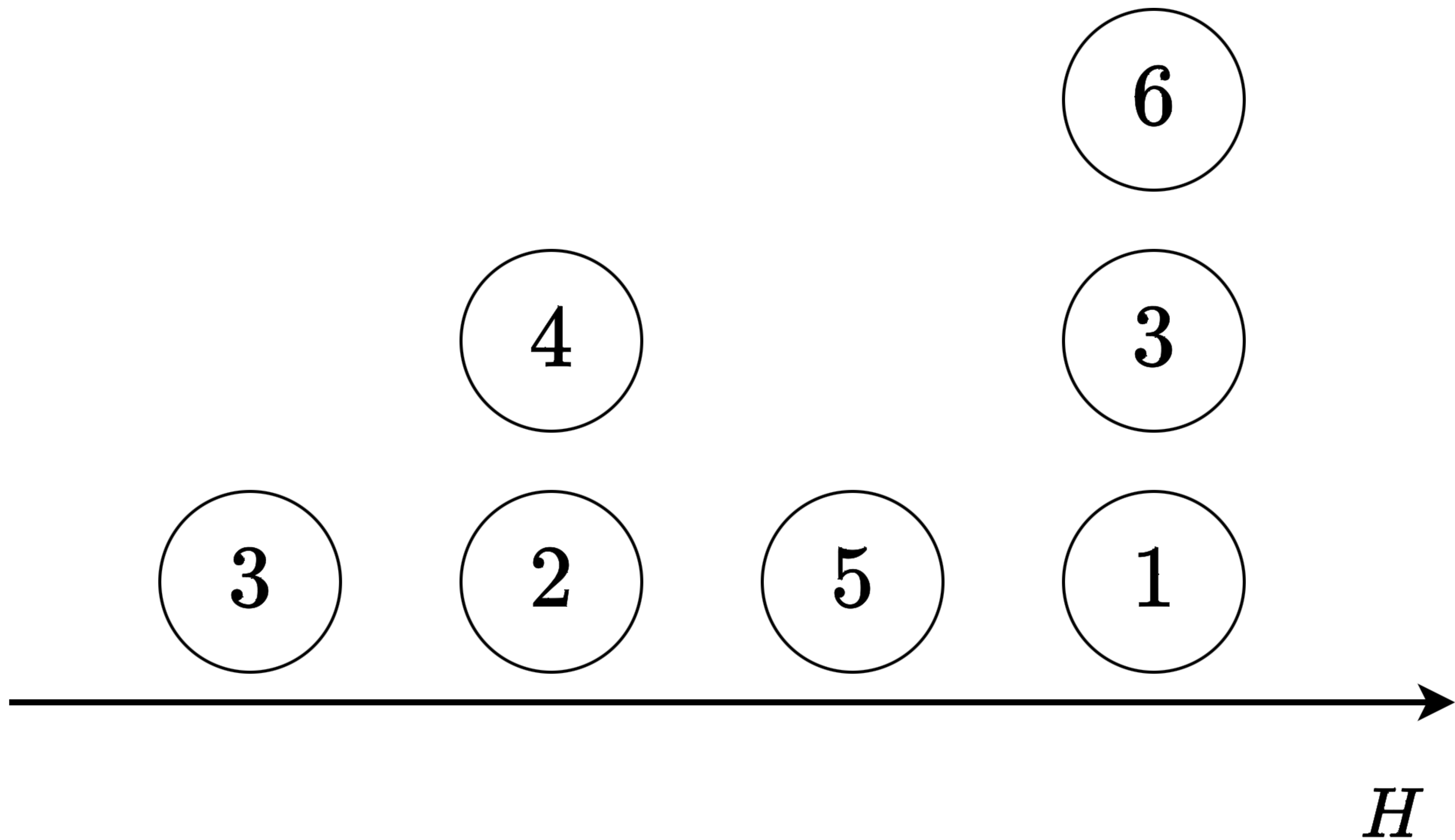
自明な考察

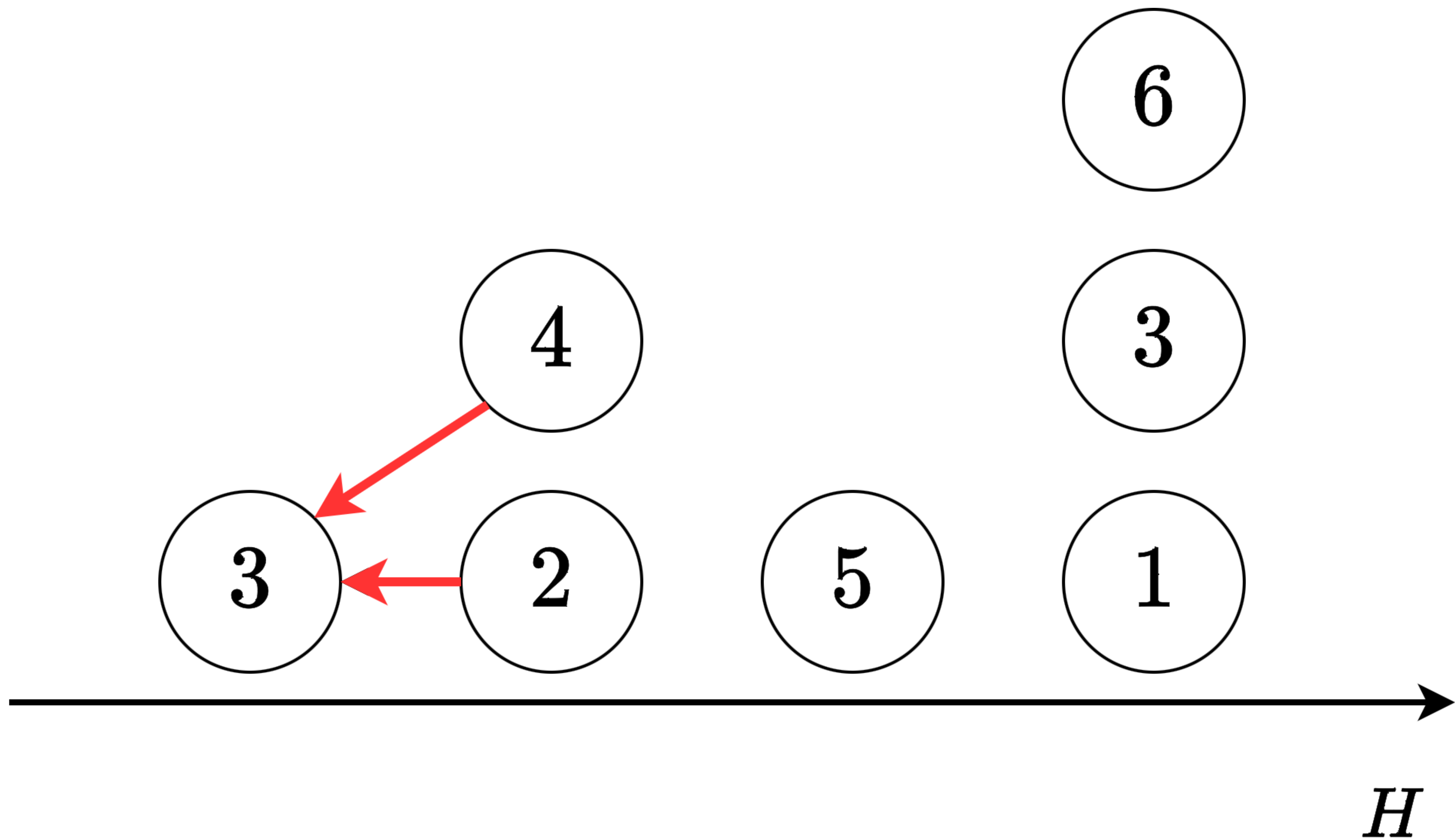
- ホテルを建てる地点以外は、それより真に標高の低い地点が必要
- 盛土工事終了後、標高が一番低い地点はただ一つ存在し、そこにホテルを建てる
- 使わない接続設備を建設してもムダ
- 特に、拡張工事は $N - 2$ 回以下
- 以下の 2 ステップで考えられる
 1. 盛土工事で標高調節（このとき一番低い地点はただ一つに）
 2. 辺を繋ぎつつ、入次数が 1 を超えたら適宜拡張工事

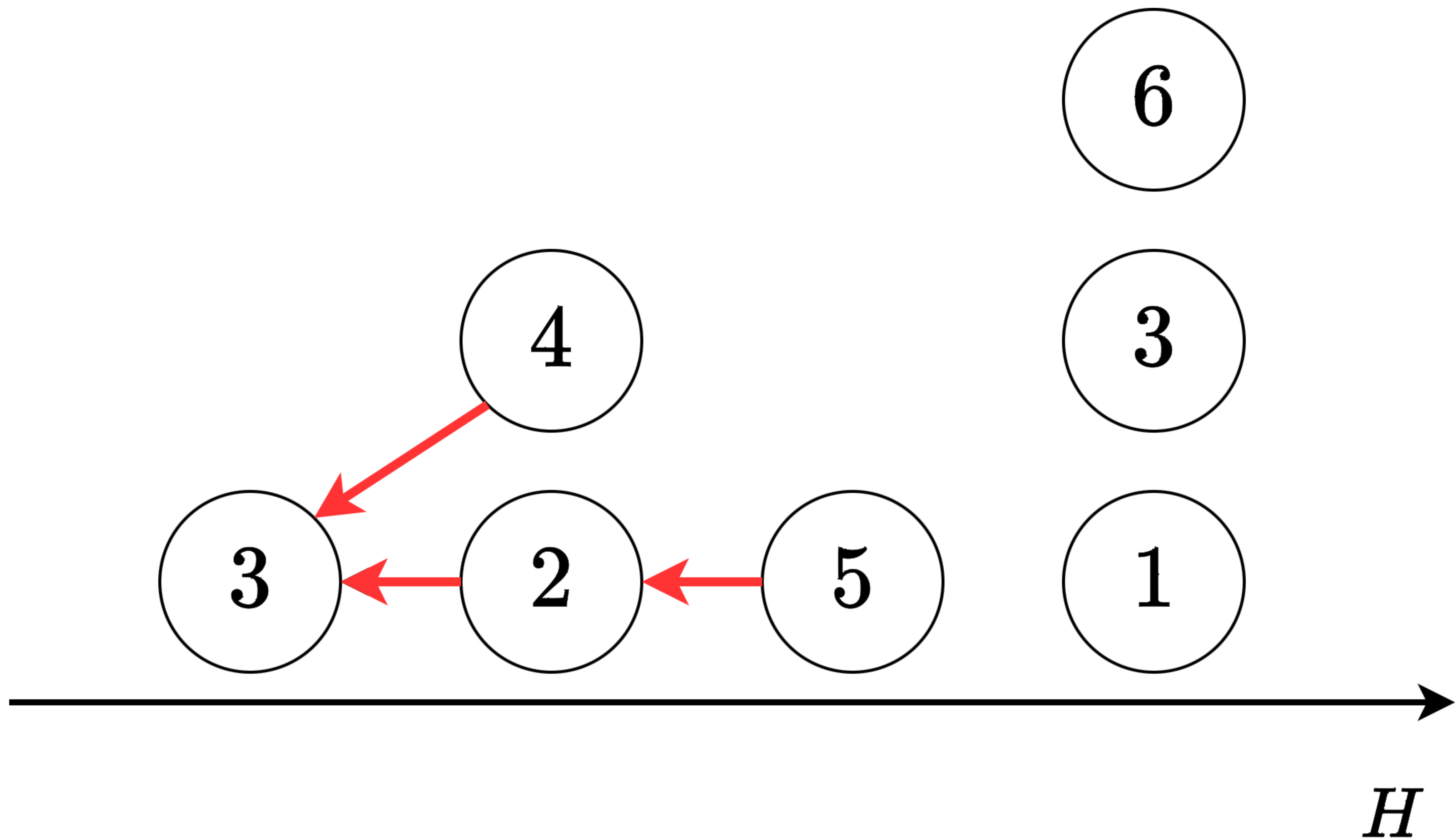
小課題 1 (5点)

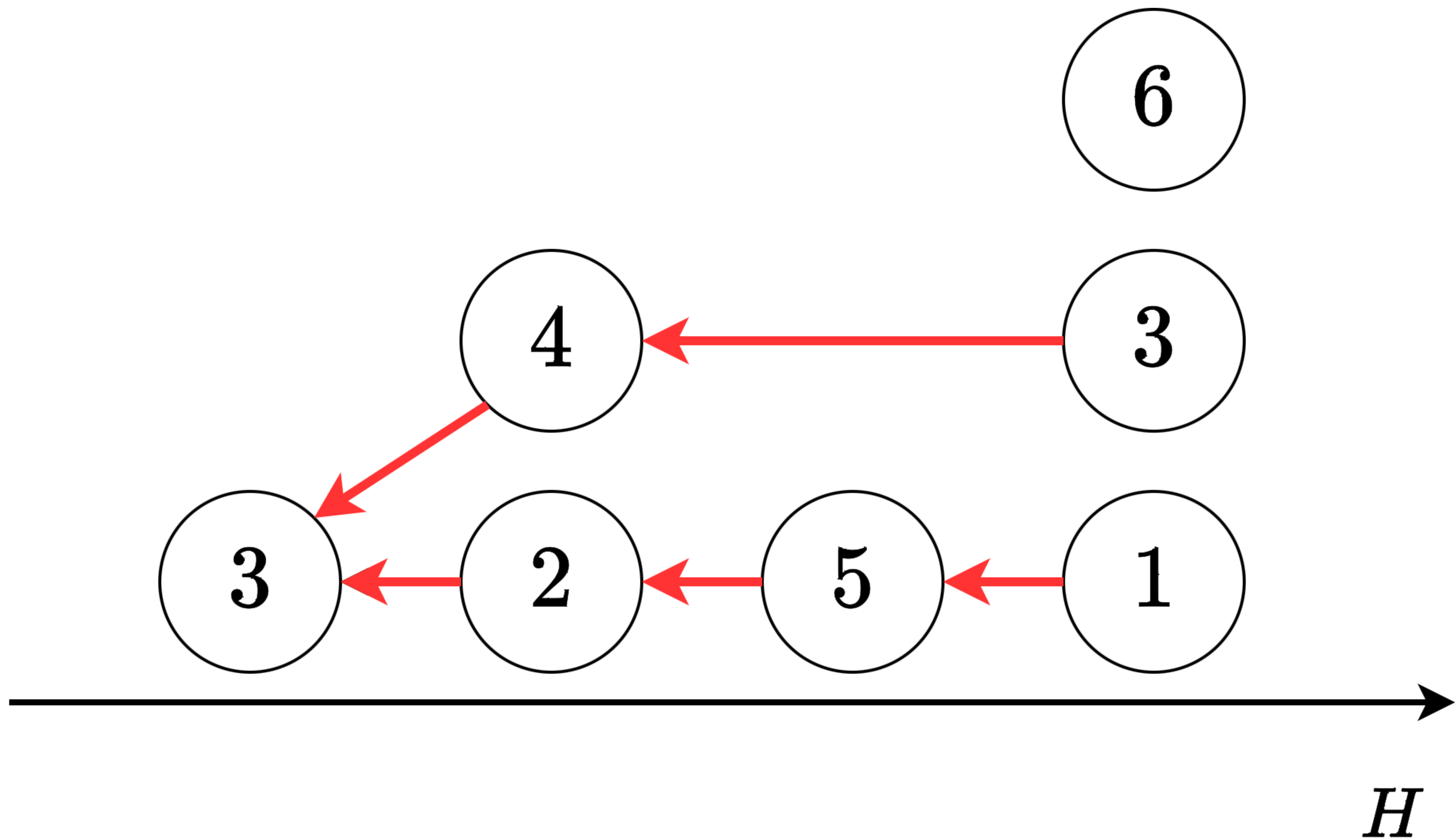
- $K \geq 10^5, H_i \leq 300, C_i \leq 100$
- $N \max C_i < K$ なので、盛土工事はなるべくしたくない
- H_i の最小値をとる地点が複数個ある場合、一つを除いて 1 回ずつ盛土工事が必要
 - どの一つか？ $\rightarrow H_i$ が最小の地点のうち、 C_i が最小のもの
- それ以外の盛土工事は不要
- 標高が定まったとき、どうやって拡張工事のコストを求めるか？

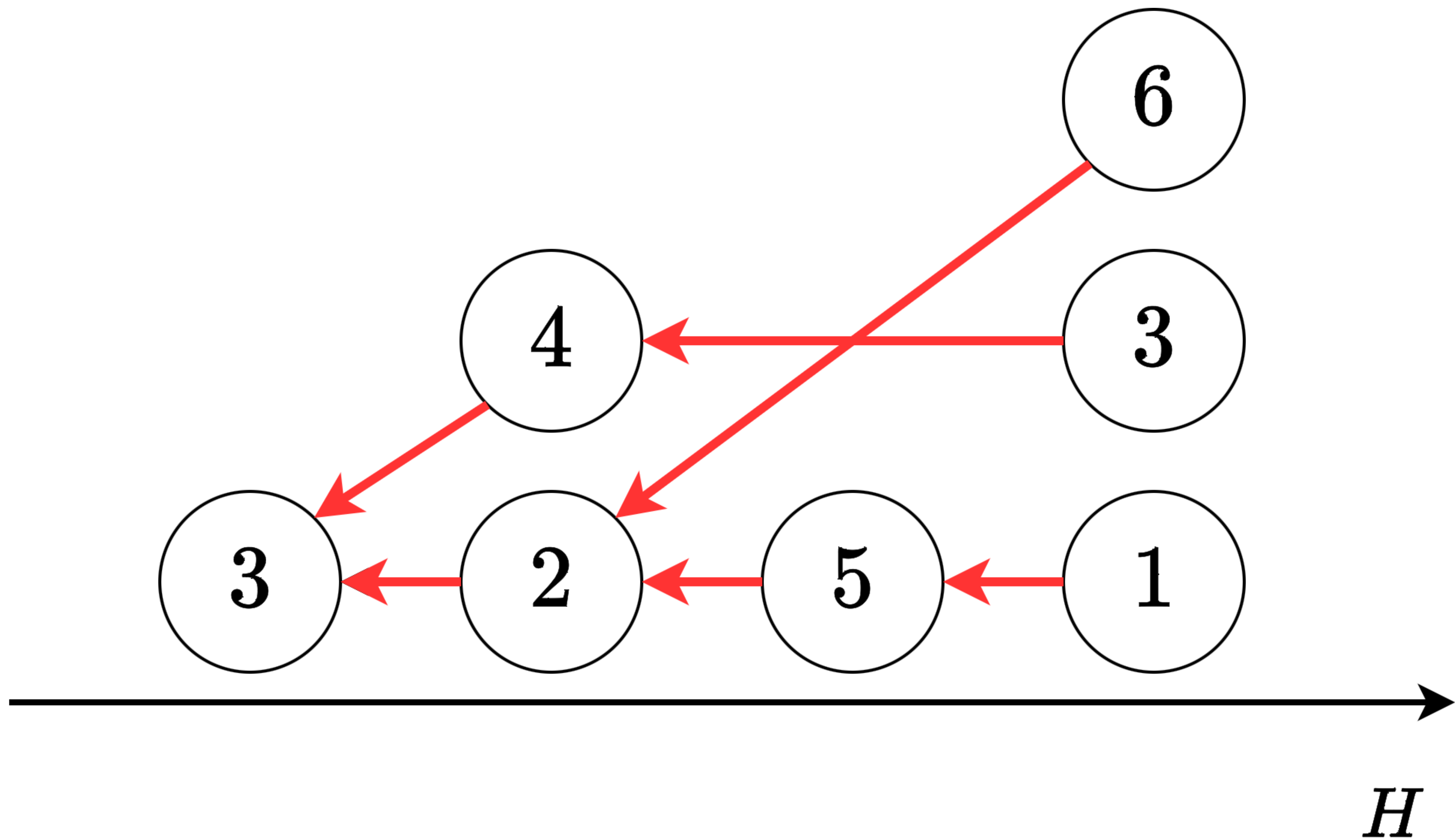












小課題 1 (5点)

- 標高が低い順に貪欲に繋ぎ先を決める
 - 接続設備が余っているならそのうちのどこかに繋ぐ
 - 余っていないなら、自分より標高が低い地点のうち C_i が最小のものに拡張工事を施して、そこに繋ぐ
- 計算量 $O(N), O(N^2)$ など

小課題 3 （9 点）

- $N \leq 10, H_i \leq 10$
- 復習：標高が定まったとき、拡張工事のコストの計算は、
 - 標高の低い順に繋ぎ先を決めることで行う
 - 必要な情報：
 - 余っている接続設備の個数
 - 自分より標高が低い地点の C_i の最小値
- 標高は盛土工事によって変化するが、低い方から高い方へしか移動しない
- $j = 1, 2, \dots$ の順に、最終的な標高が j になる地点の集合を決めて行けばよさそう

小課題 3 (9 点)

- $\text{dp}[i][S][j]$:
 - 標高 i まで見た
 - すでに標高を確定させた地点の集合が S
 - 余っている接続設備の個数が j
- 遷移：元の標高が $i + 1$ 以下で S に含まれない地点たちの中から、標高 $i + 1$ で確定させる地点の集合を選ぶ
- i は $\max H_i + N$ 程度まで見れば十分（それより標高を上げる意味はない）
- 計算量 $O(3^N N(N + \max H_i))$ など

小課題4 (33点)

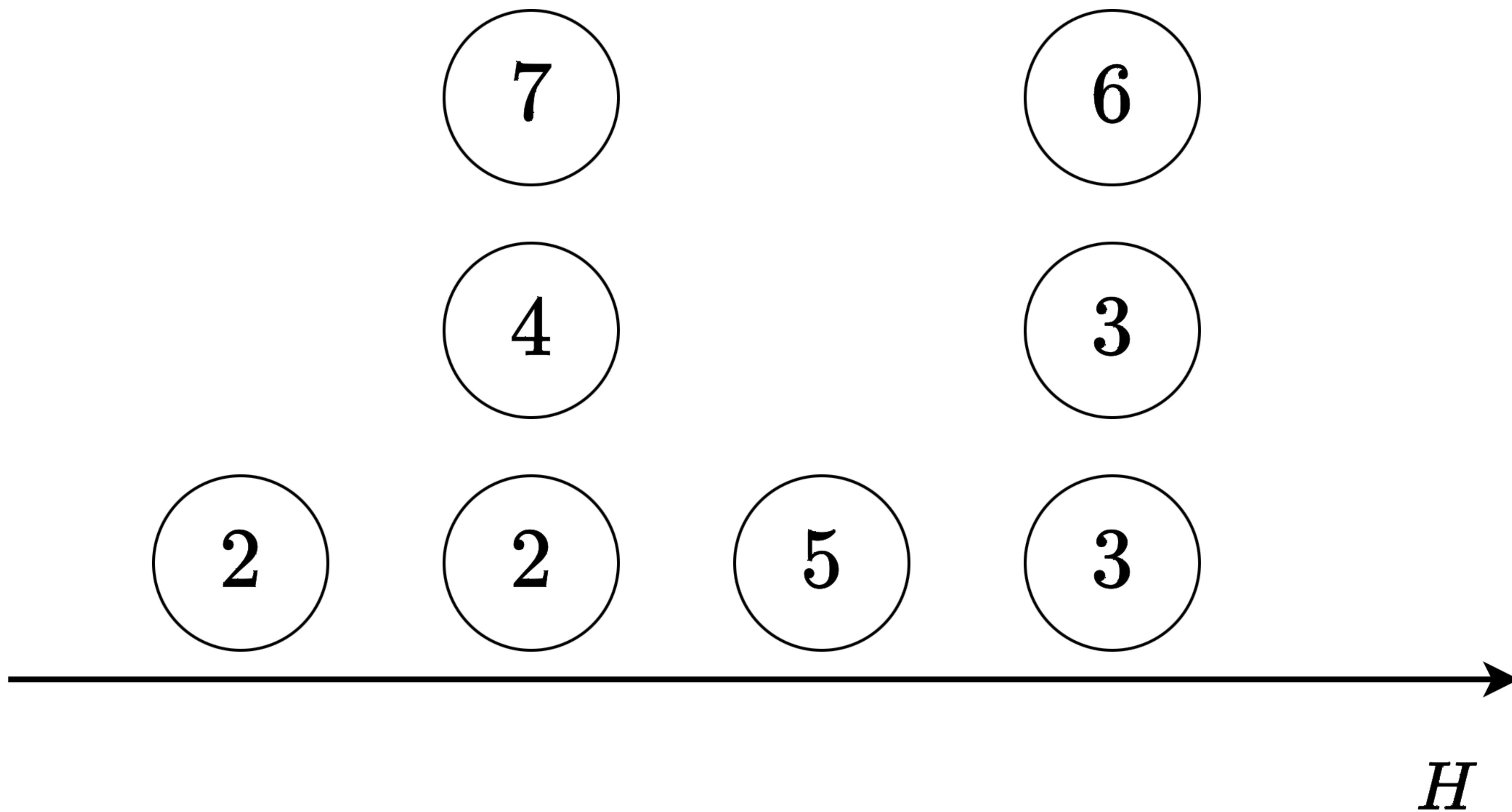
- $N \leq 40, H_i \leq 40$
 - 集合 S は持てない
 - 本当に必要な情報は何か？
 - 元の標高が i 以下なのに S に入っていないもの（**繰上げ**と呼ぶ）の個数
 - S 内での C_i の最小値
 - S の代わりに、繰上げの個数と S 内で C_i が最小になる地点の番号を持てばいい
- $dp[i][S][j]$ (小課題3)
 - 標高 i まで見た
 - すでに標高を確定させた地点の集合が S
 - 余っている接続設備の個数が j

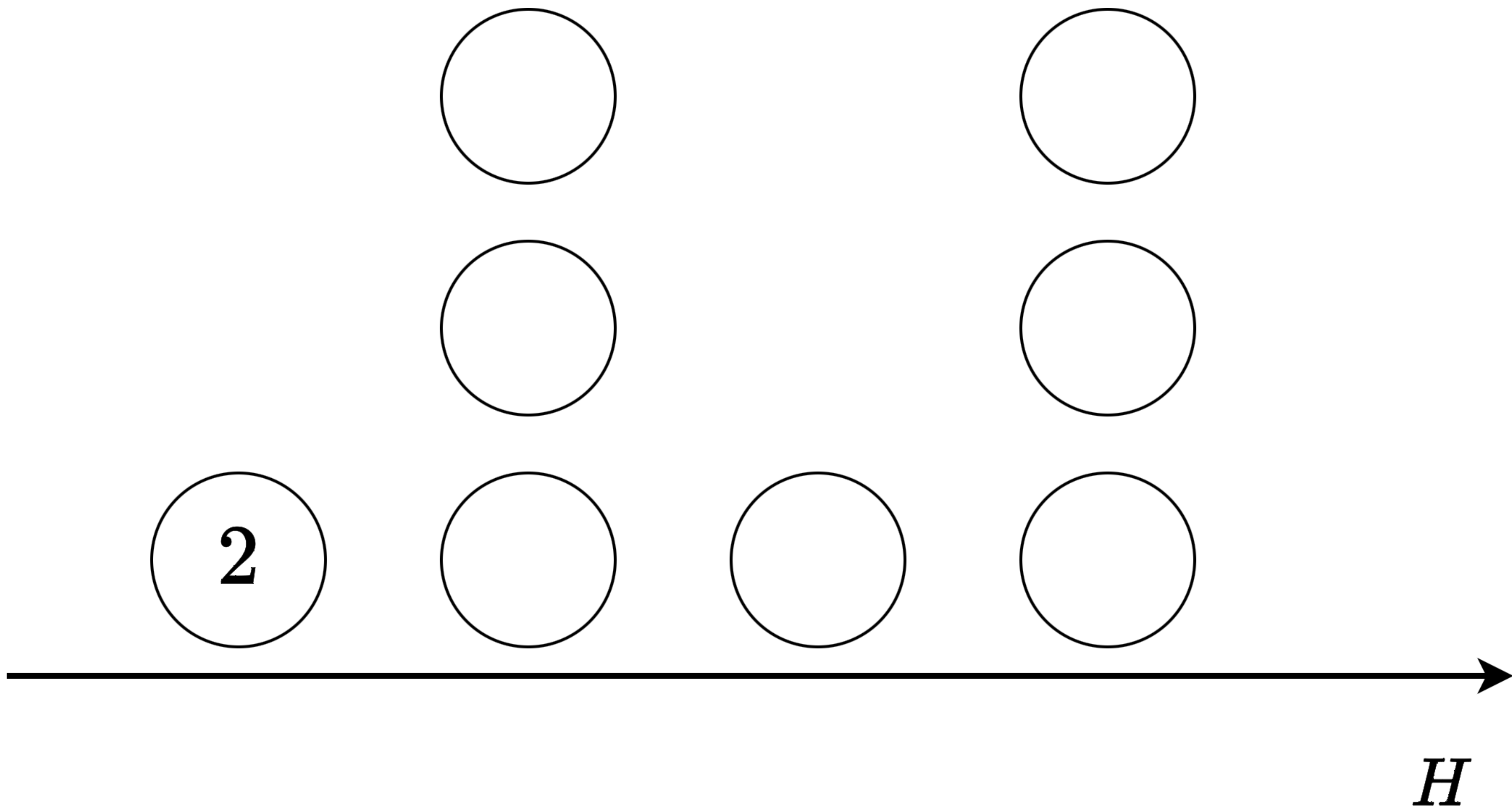
小課題4 (33点)

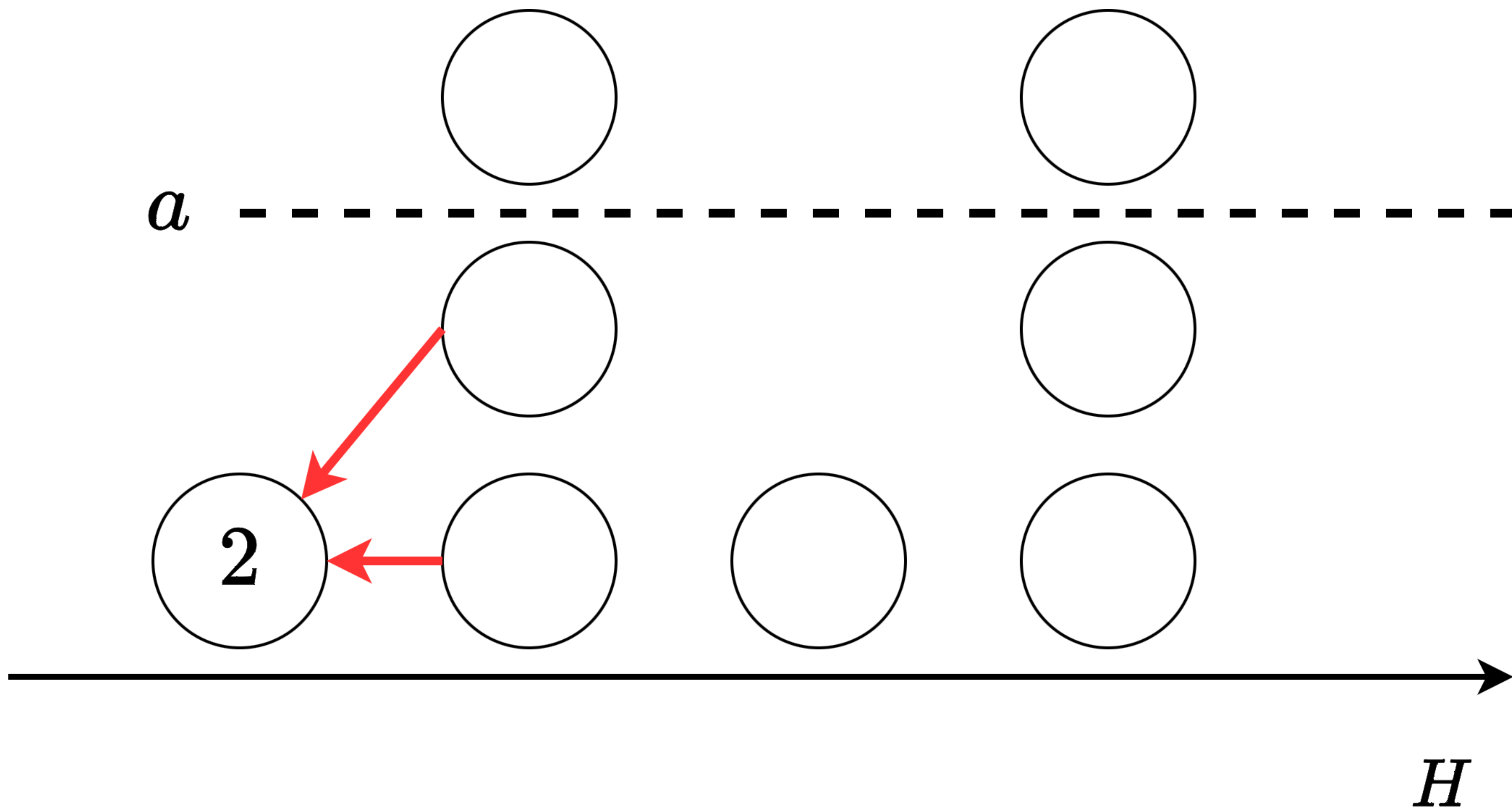
- $dp[i][j][k][l]$:
 - 標高 i まで見た
 - 余っている接続設備の個数が j
 - 繰上げの個数が k
 - 標高を確定させた地点の中で C_i が最小となる地点の番号が l
- 遷移：候補のうち C_i が小さい順に標高 i で確定させていくべきなので、高々 N 通り
- 計算量 $O(N^4(N + \max H_i))$
- 思ったより大変かも？

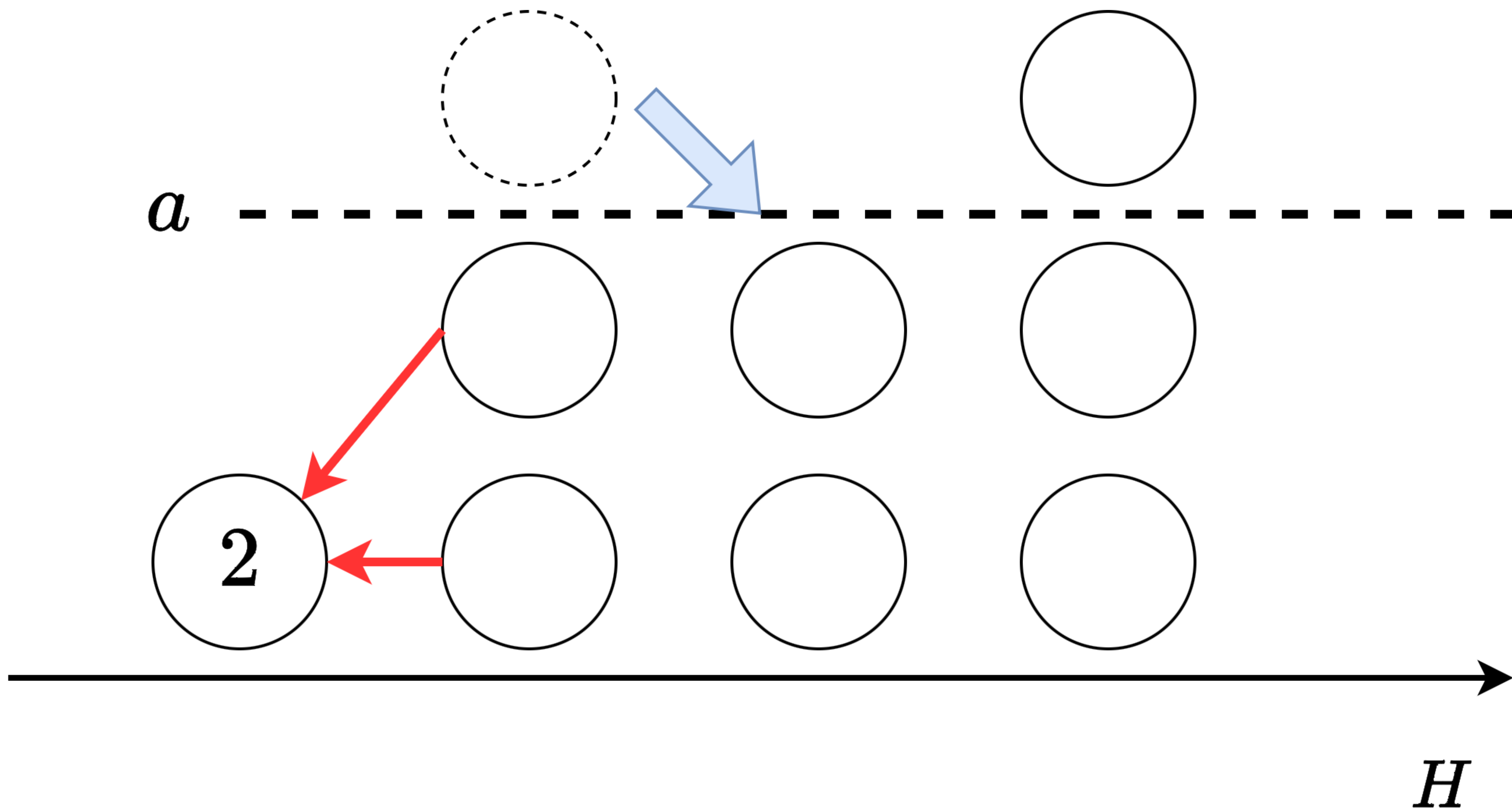
小課題 2 （1 2 点）

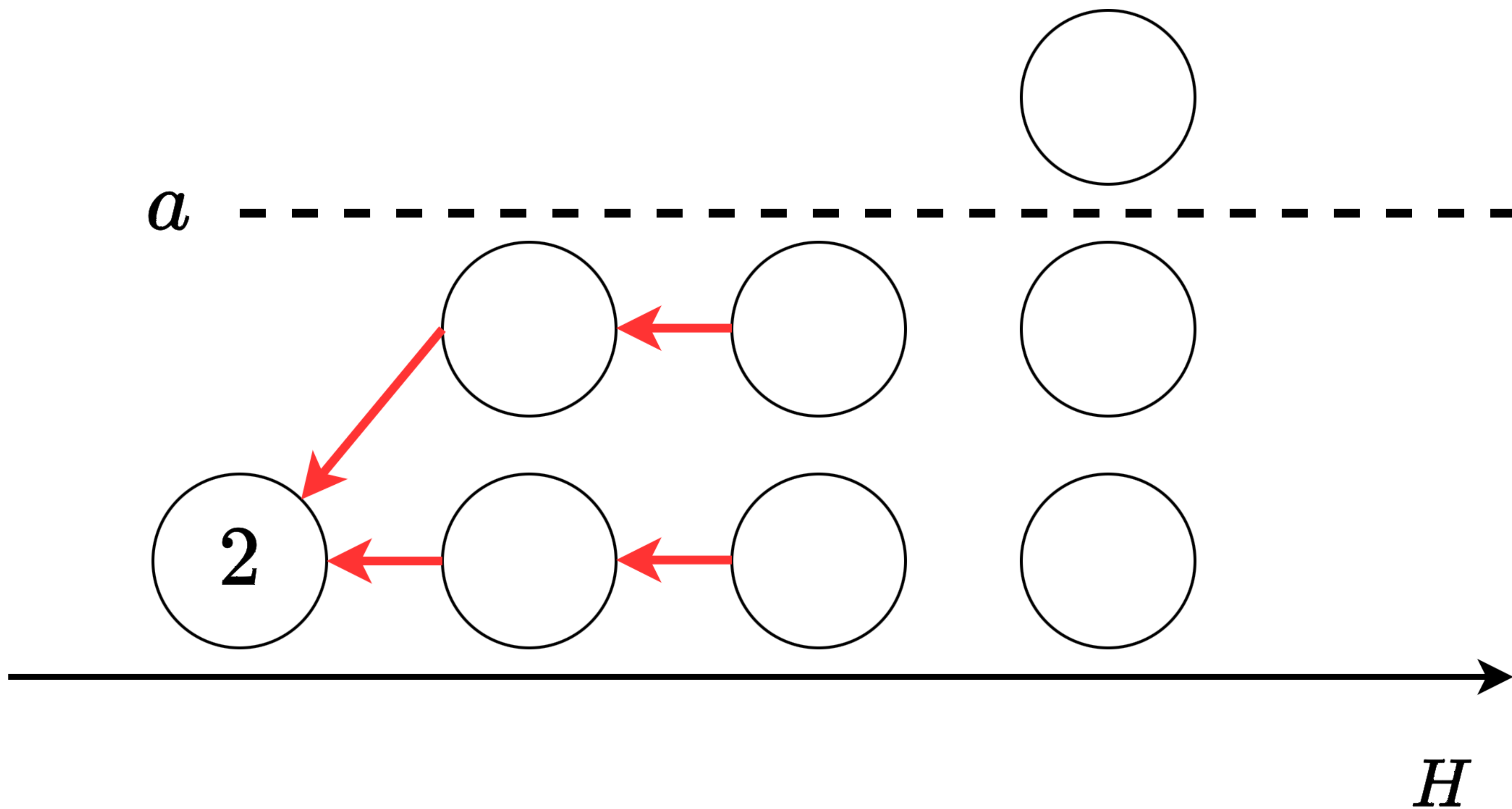
- $H_1 \leq H_i, C_1 \leq C_i, N \leq 300, H_i \leq 300$
- 地点 1 にホテルを建てる（& 地点 1 の標高は増やさない）
- 地点 1 以外に拡張工事を施す必要はない
- 地点 1 に拡張工事を施す回数 a を固定してみる

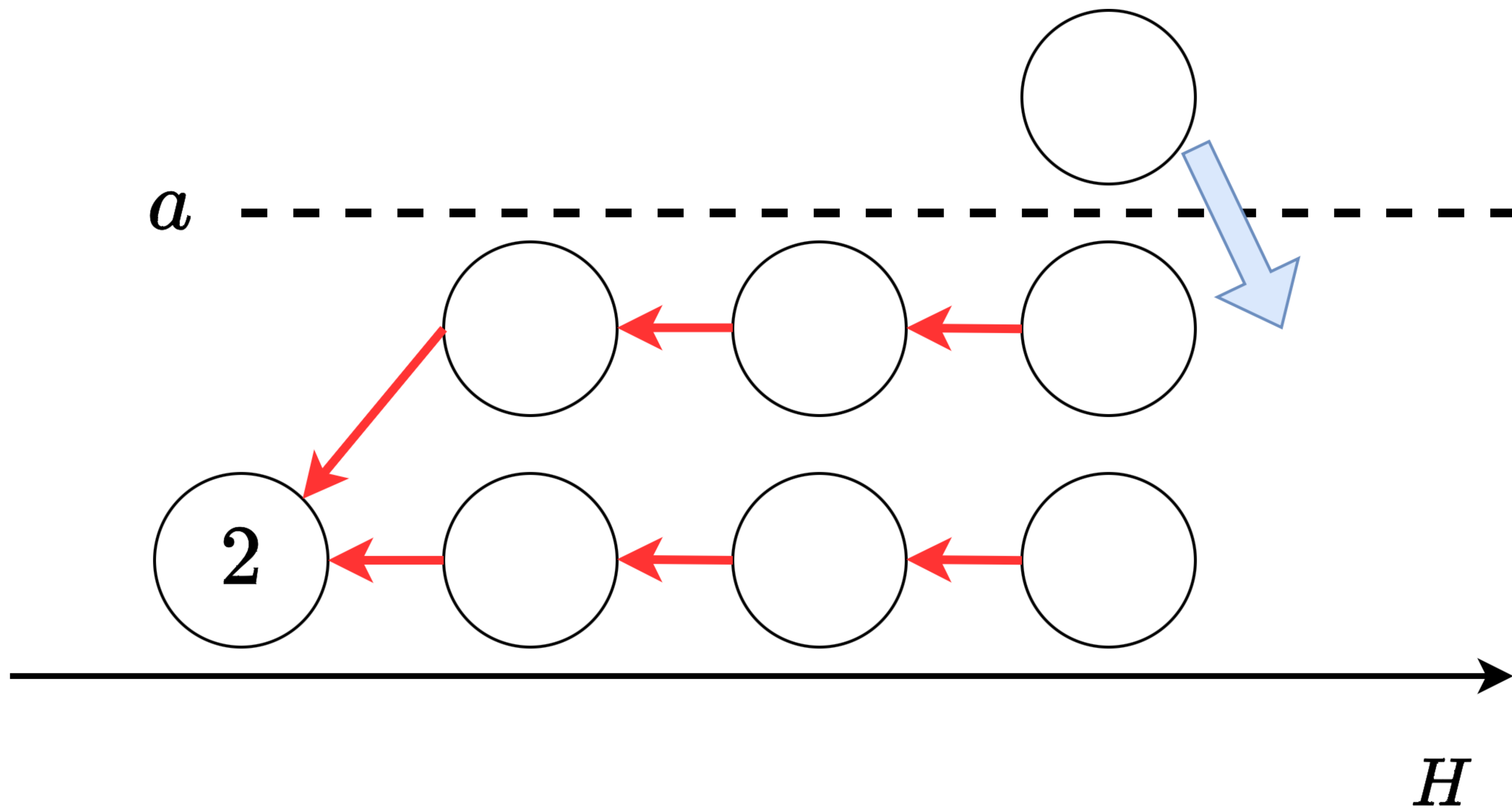


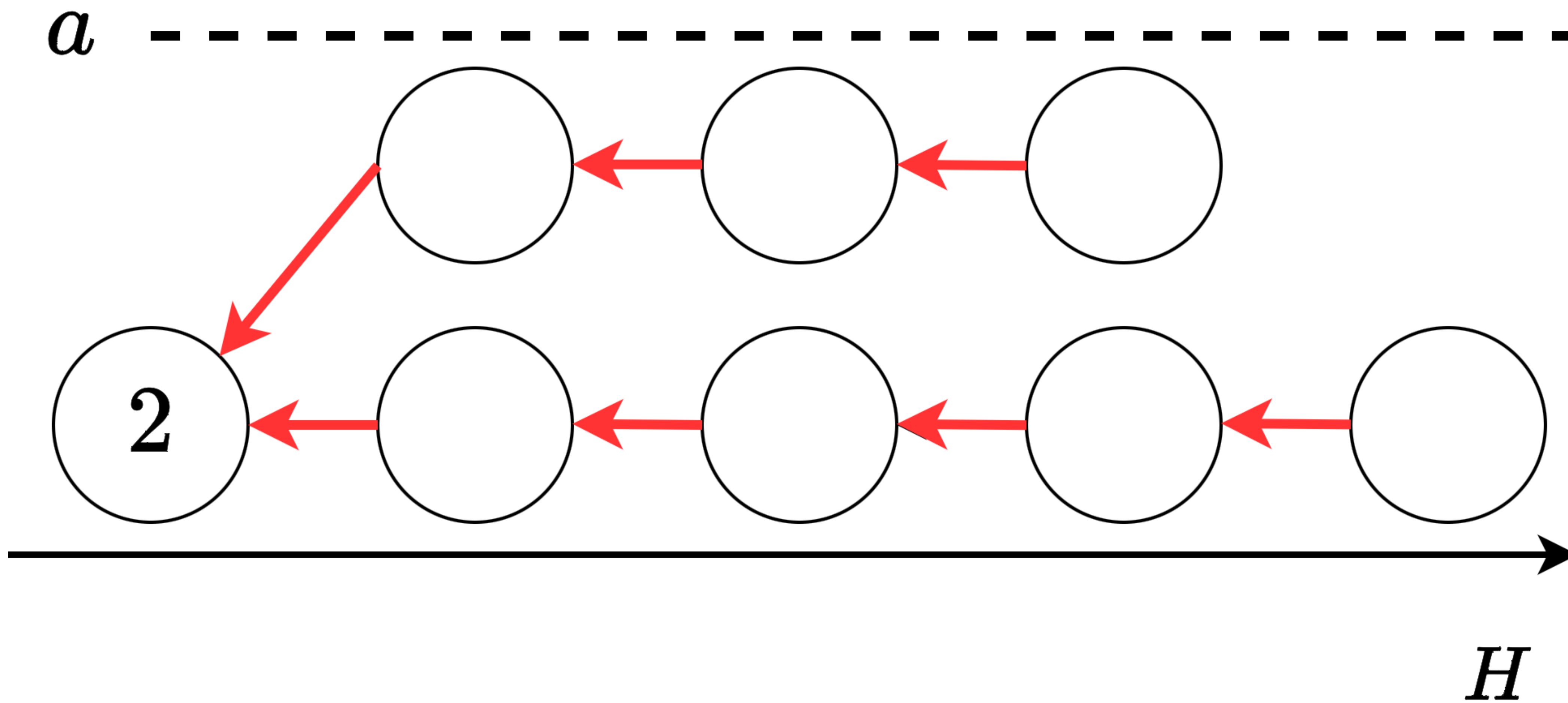












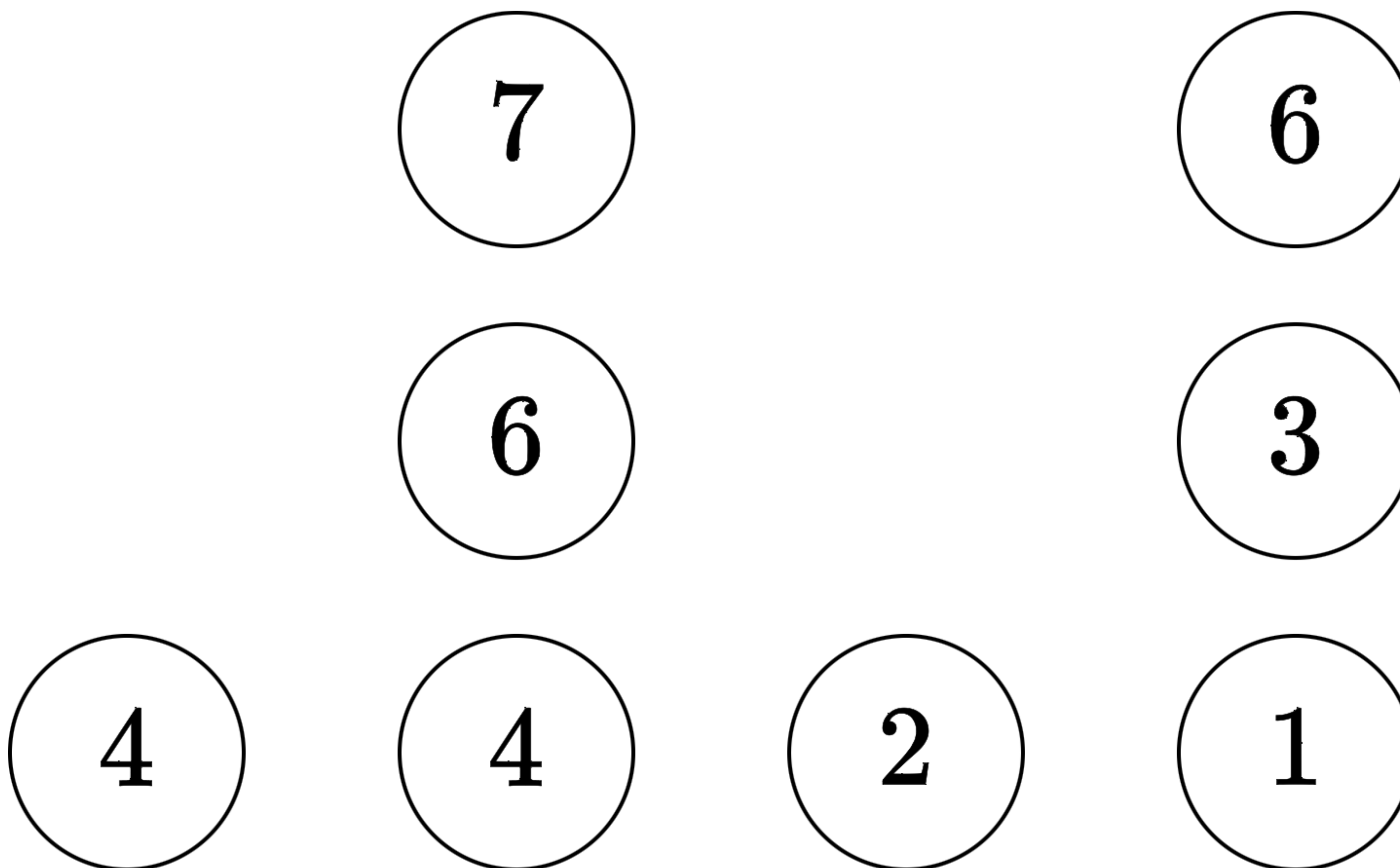
小課題2 (12点)

- $H_1 \leq H_i, C_1 \leq C_i, N \leq 300, H_i \leq 300$
- 地点 1 に拡張工事を施す回数 a を固定してみる
- 「同じ標高の地点は a 個以下」を満たすように盛土工事をする問題
- 明らかに貪欲法
- $O(N^2), O(N \max H_i)$ など

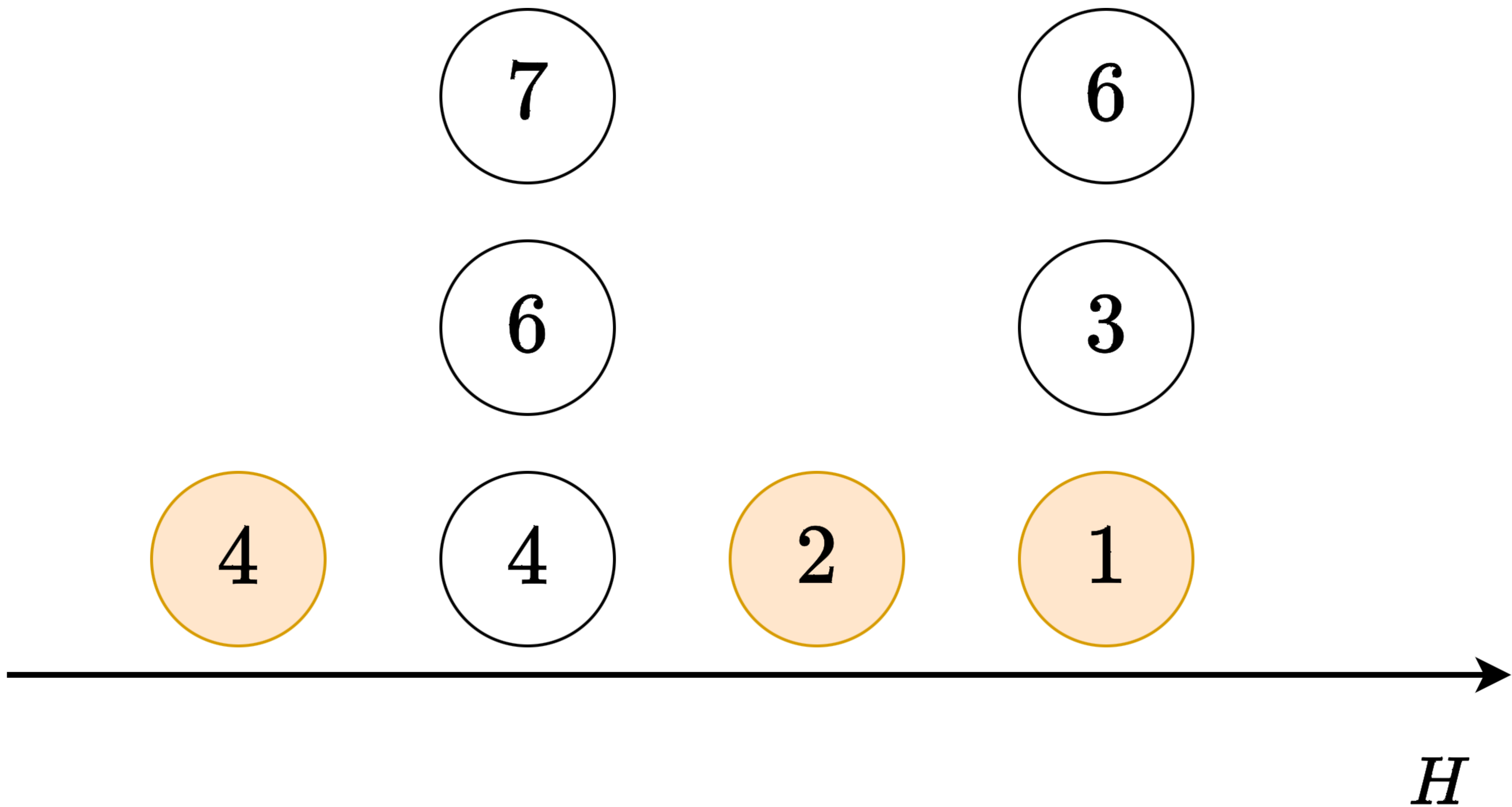
小課題 5 (27点、累計86点)

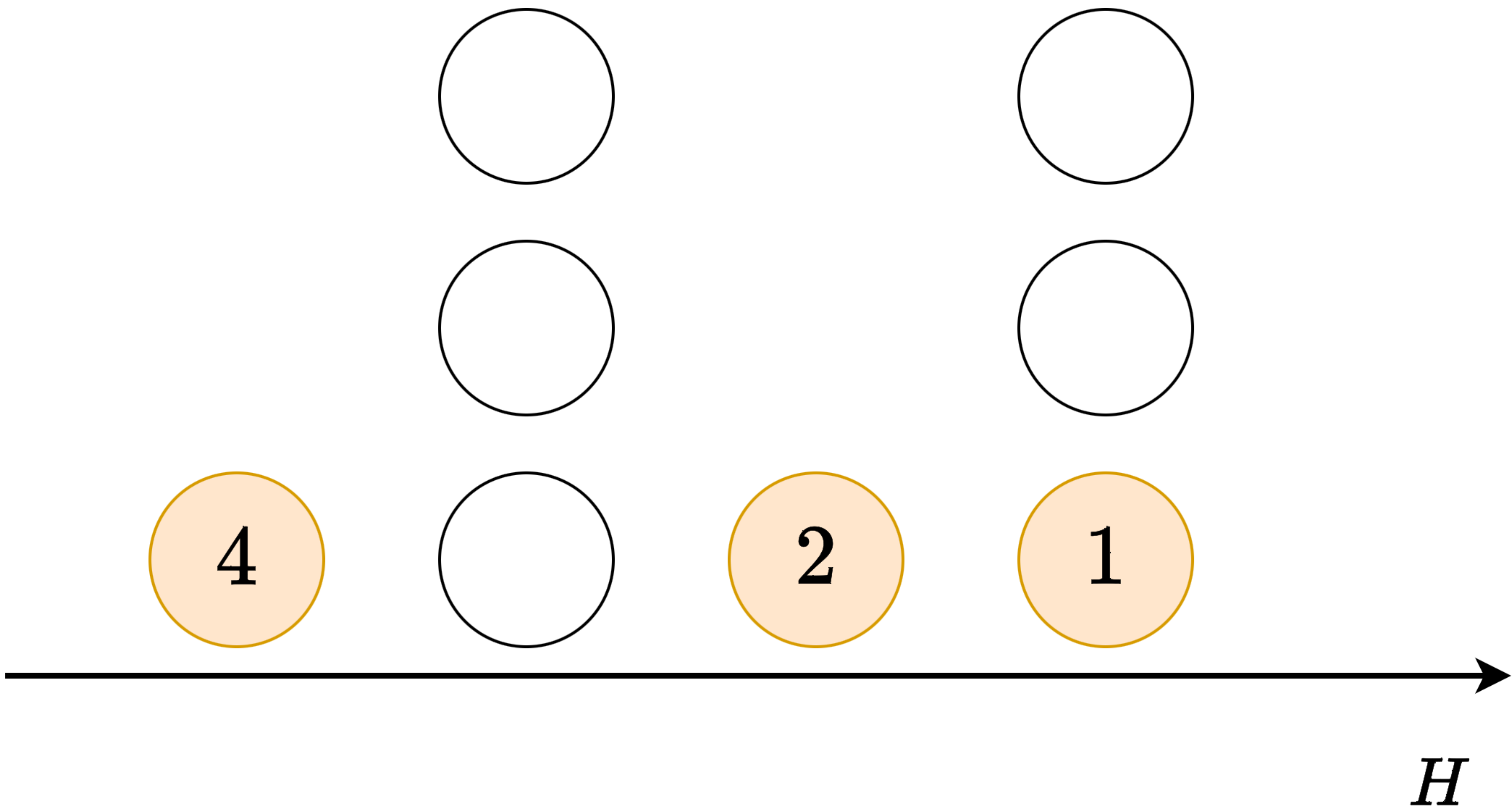
- $N \leq 300, H_i \leq 300$
- どれか削りたい

- $dp[i][j][k][l]$ (小課題 4)
 - 標高 i まで見た
 - 余っている接続設備の個数が j
 - 繰上げの個数が k
 - 標高を確定させた地点の中で C_i が最小となる地点の番号が l



H





小課題5 (27点、累計86点)

- $N \leq 300, H_i \leq 300$
- どれか削りたい

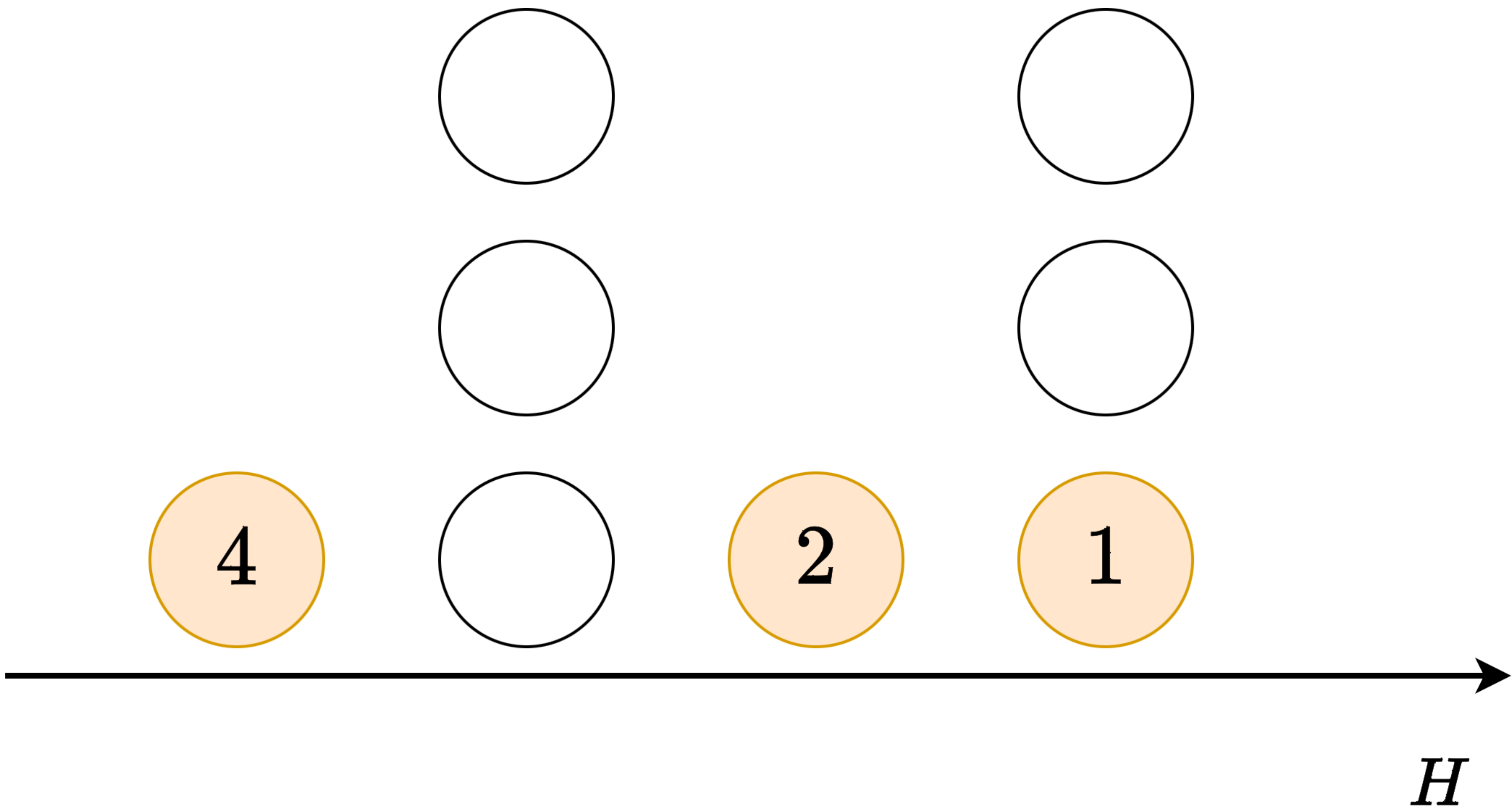
いらないう →

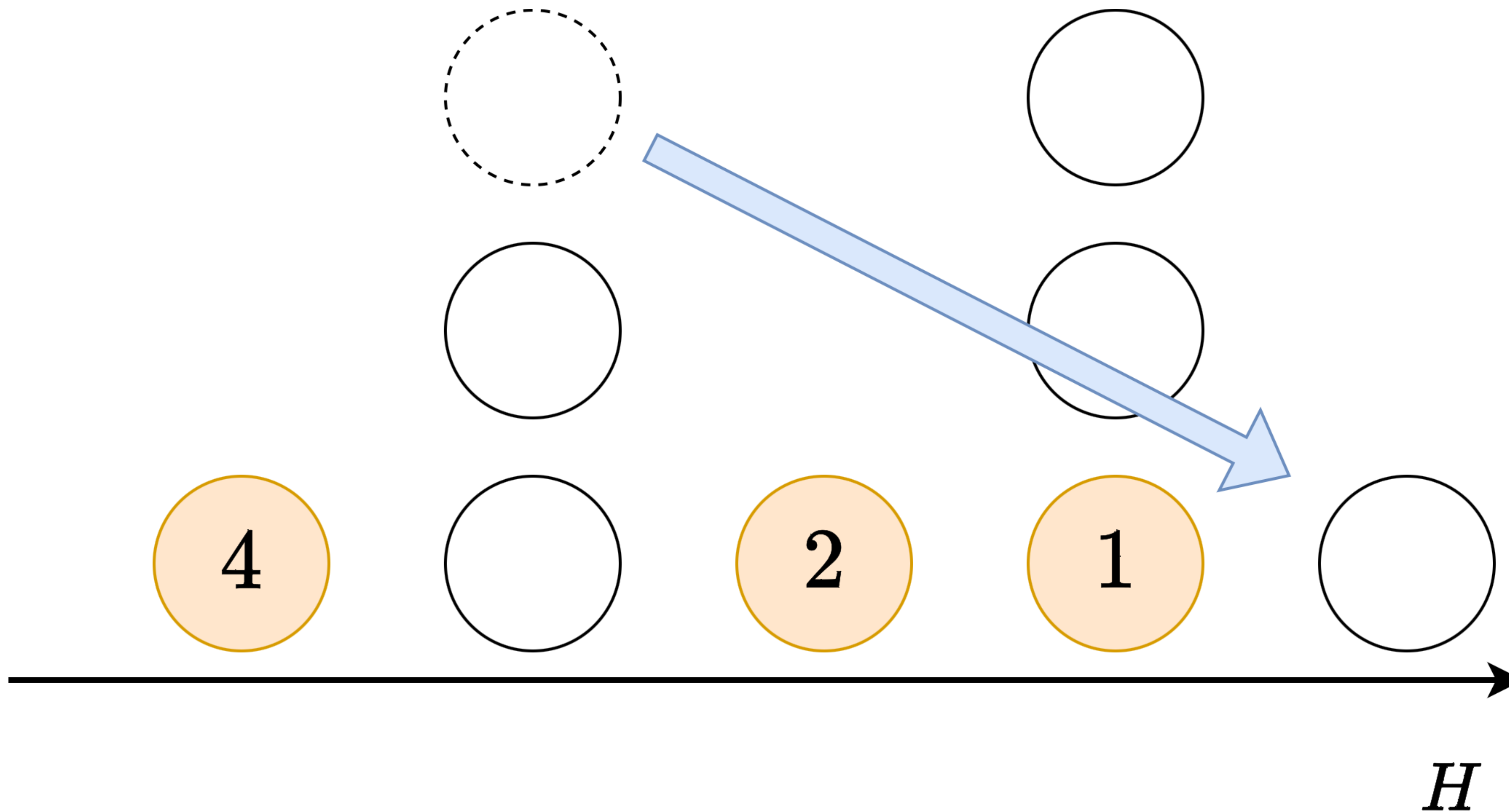
- $dp[i][j][k][l]$ (小課題4)

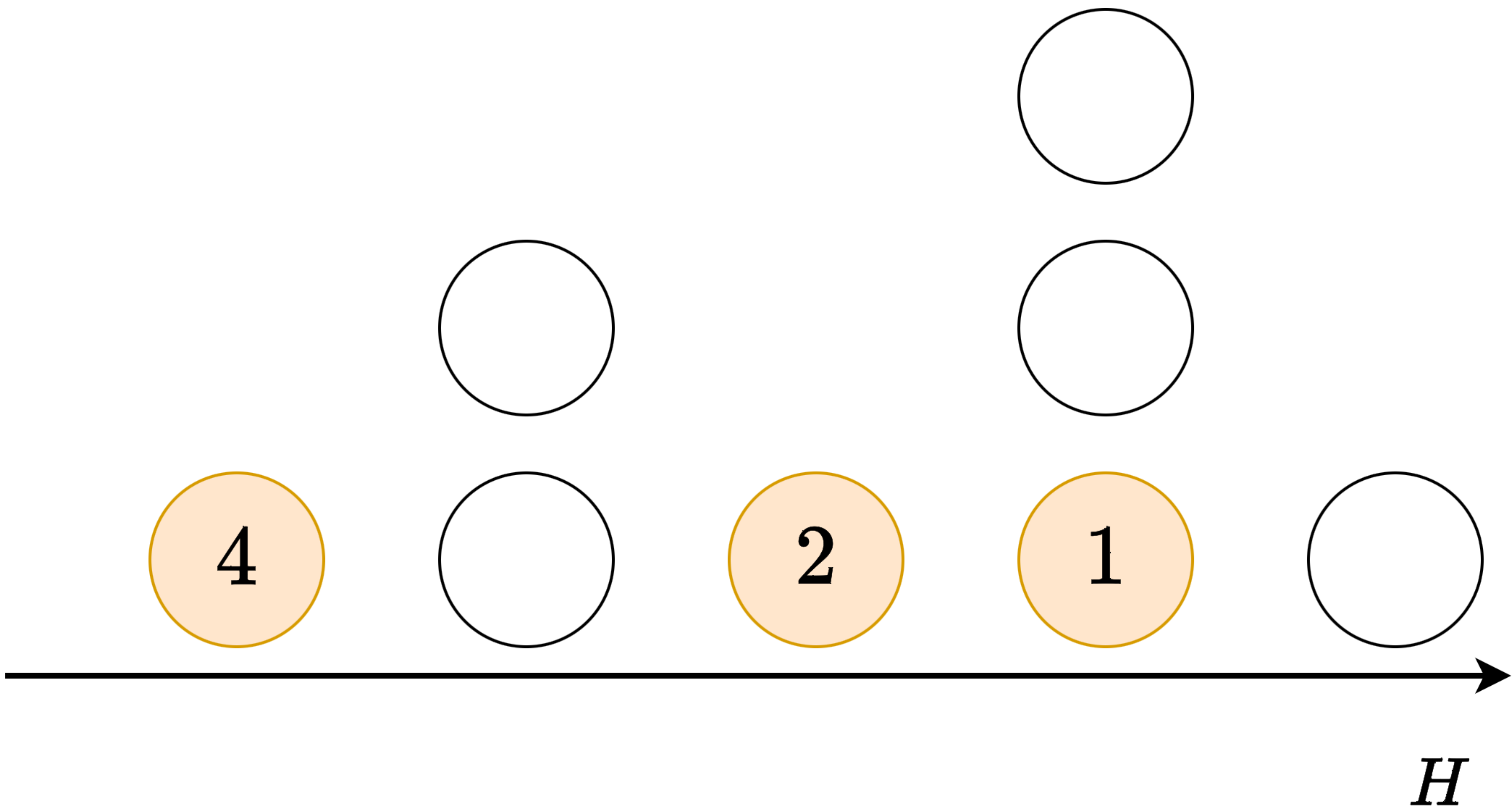
- 標高 i まで見た
- 余っている接続設備の個数が j
- 繰上げの個数が k

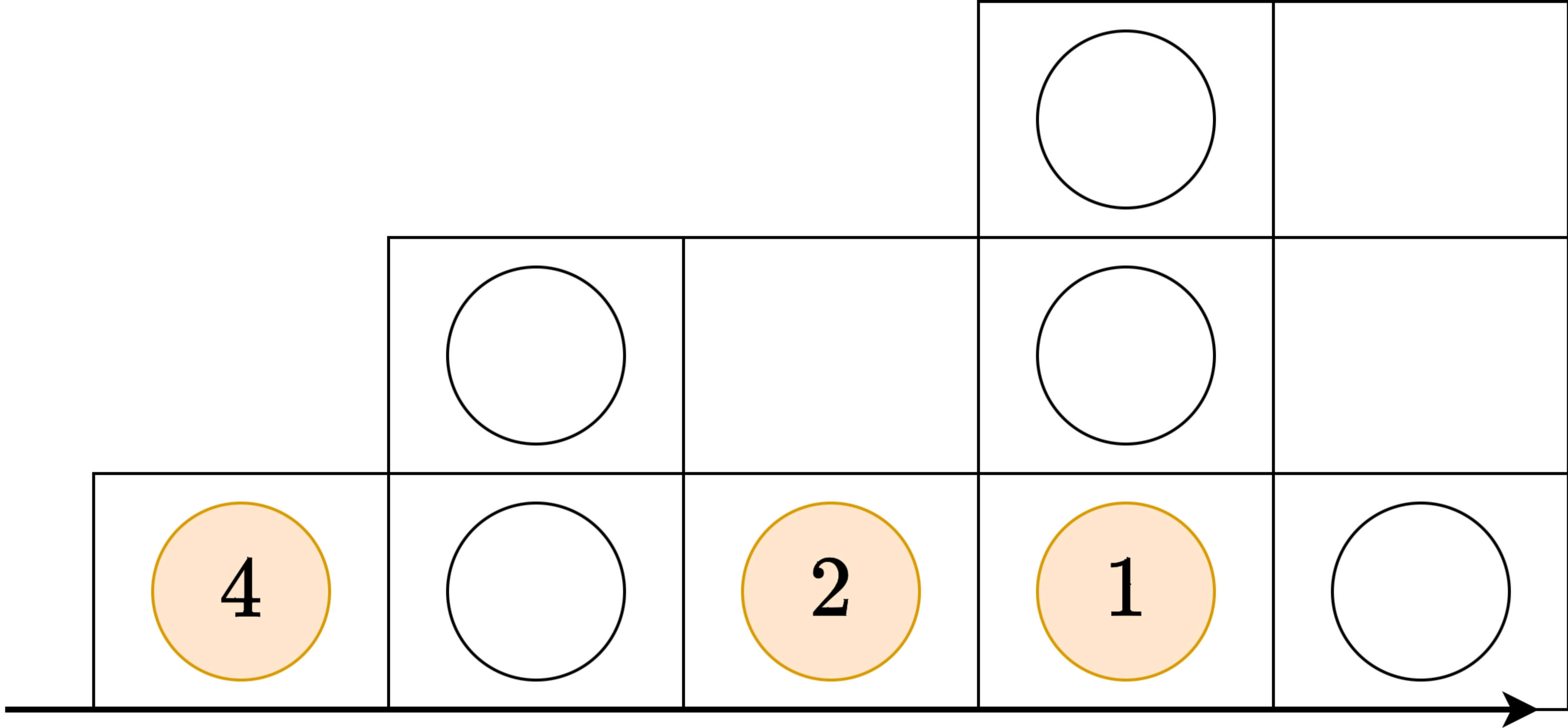
- ~~• 標高を確定させた地点の中で C_i が最小となる地点の番号が l~~

- ただし遷移が $O(N)$ なので $O(N^3(N + \max H_i))$

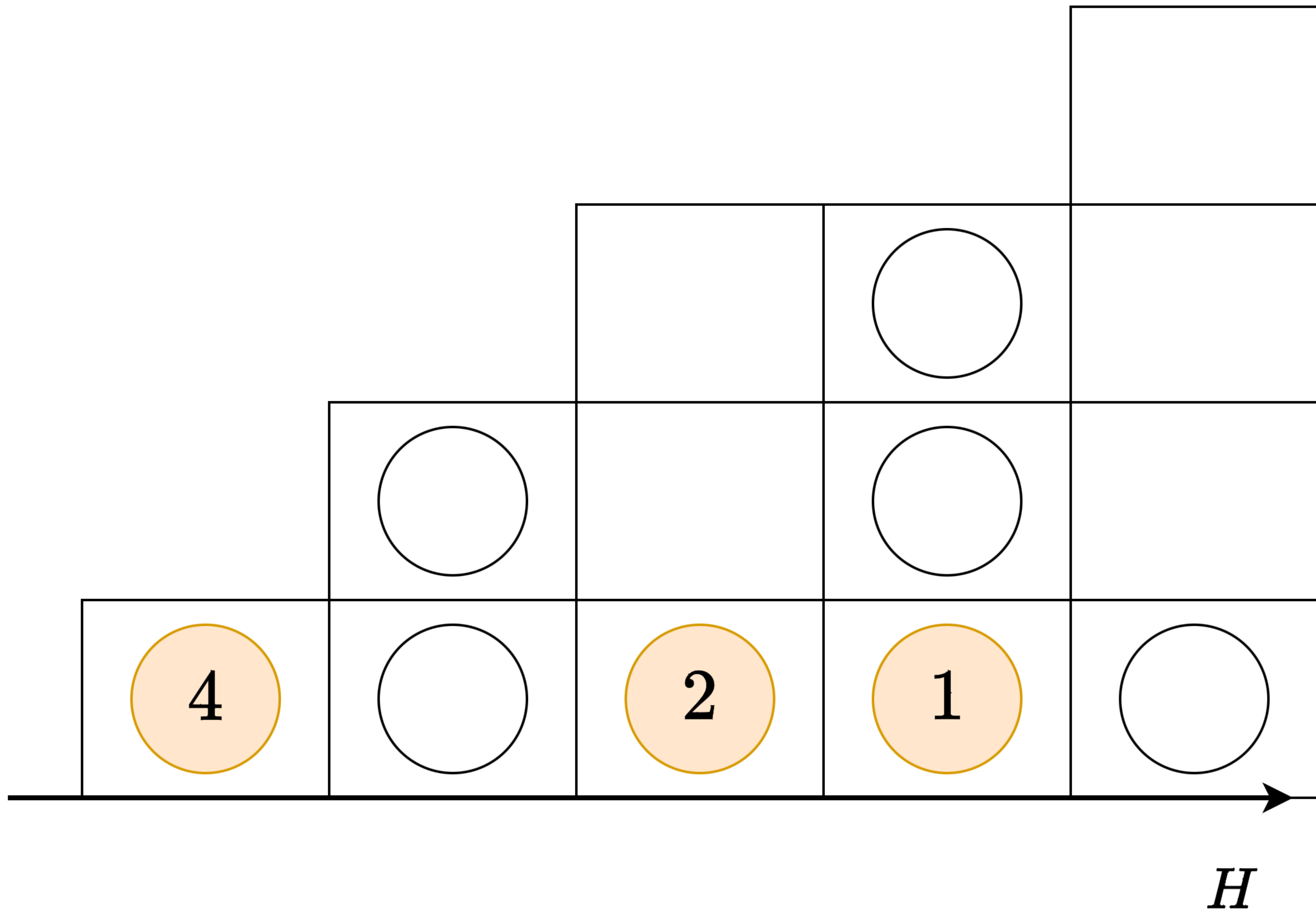








H



小課題5（27点、累計86点）

- 高さが単調増加になるヒストグラムを作って、その中に収まるようにボールを格納していく
- ヒストグラムの高さが変わるタイミング、格納しきれなくてボールを右に動かすタイミングでコストが生じる

小課題5 (27点、累計86点)

- $dp[i][j][k]$:
 - 標高 i まで見た
 - ヒストグラムの高さが j
 - $i - 1$ までで格納しきれなくて繰り上がったボールの数が k
- 遷移 :
 - $dp[i][j+1][k] \leftarrow dp[i][j][k] + f(i)$
 - $f(i)$ は $\{C_x \mid H_x < i\}$ の最小値
 - $dp[i+1][j][r] \leftarrow dp[i][j][k] + K \times r$
 - $r = \max\{k + (\text{元の標高が } i \text{ である地点の数}) - j, 0\}$

小課題5 (27点、累計86点)

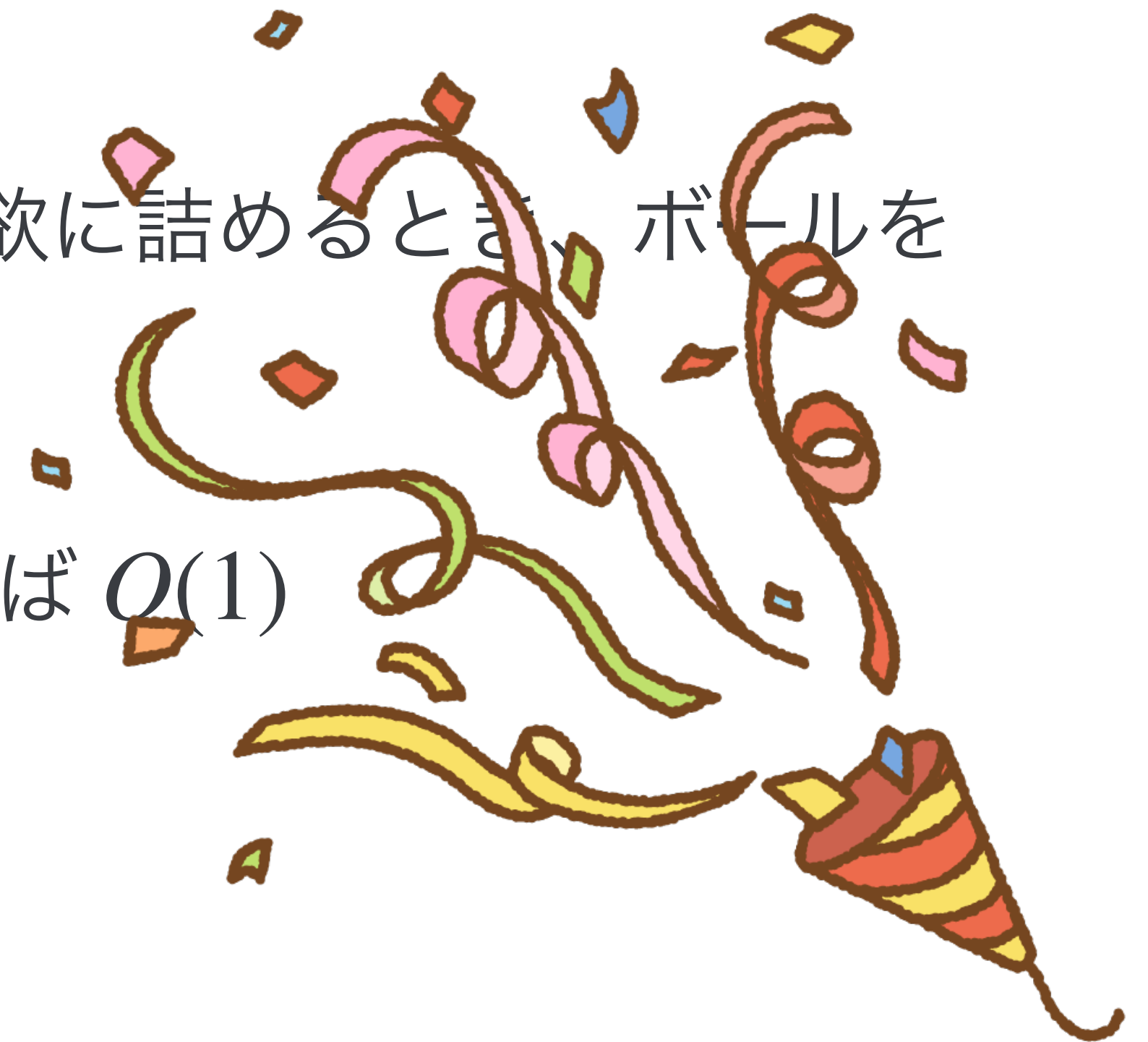
- よって、計算量 $O(N^2(N + \max H_i))$

小課題 6 (満点)

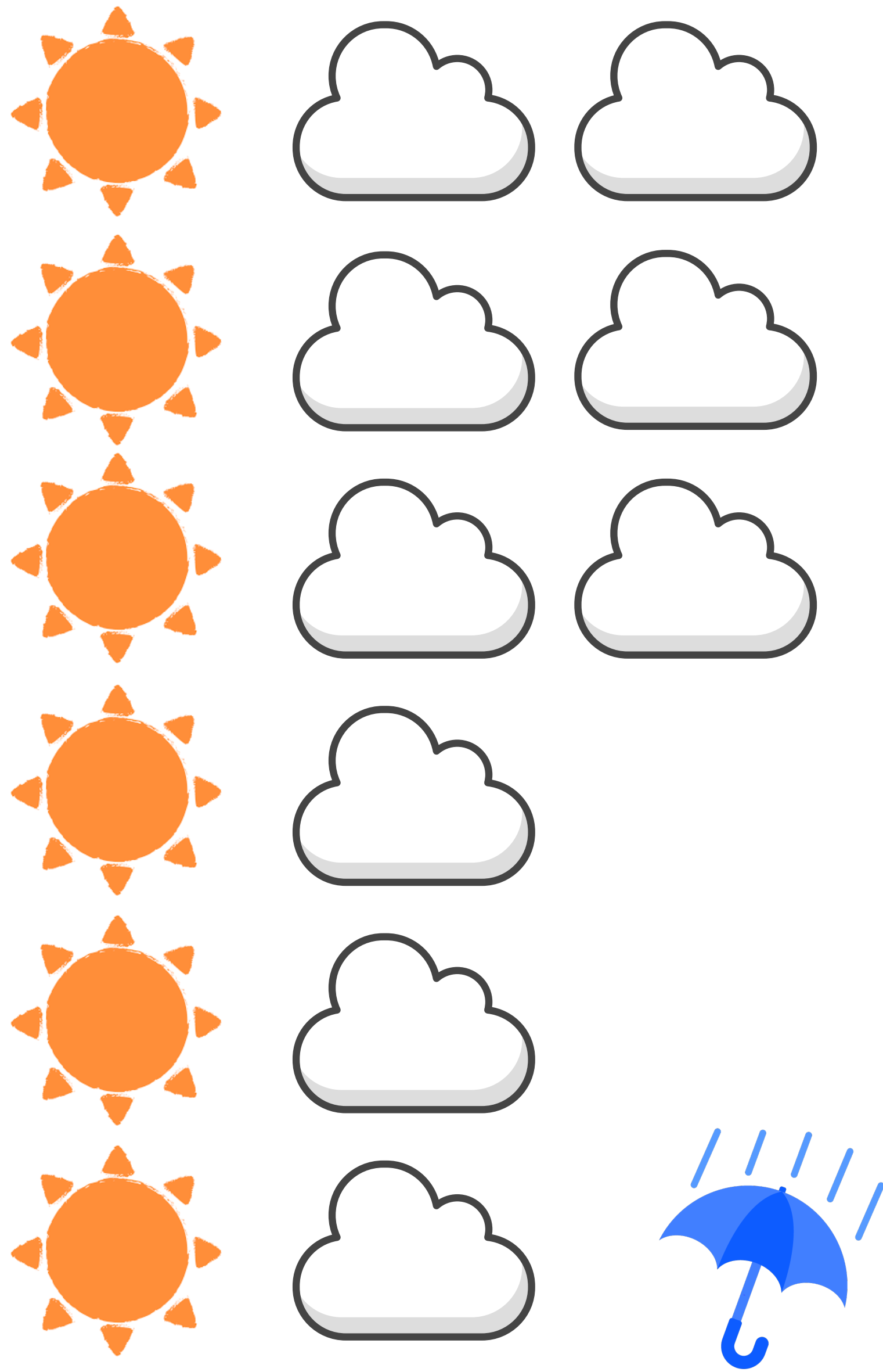
- $N \leq 300, H_i \leq 10^9$
- 座標圧縮したい
- ヒストグラムの高さが変わりうる場所、新たなボールが追加される場所だけ考えればいい
- ~~$\{1, 2, \dots, 10^9\}$ を $\{H_1, H_2, \dots, H_N\}$ に圧縮できる~~
- $\{1, 2, \dots, 10^9\}$ を $\{H_1, H_1 + 1, H_2, H_2 + 1, \dots, H_N, H_N + 1\}$ に圧縮できる

小課題 6 (満点)

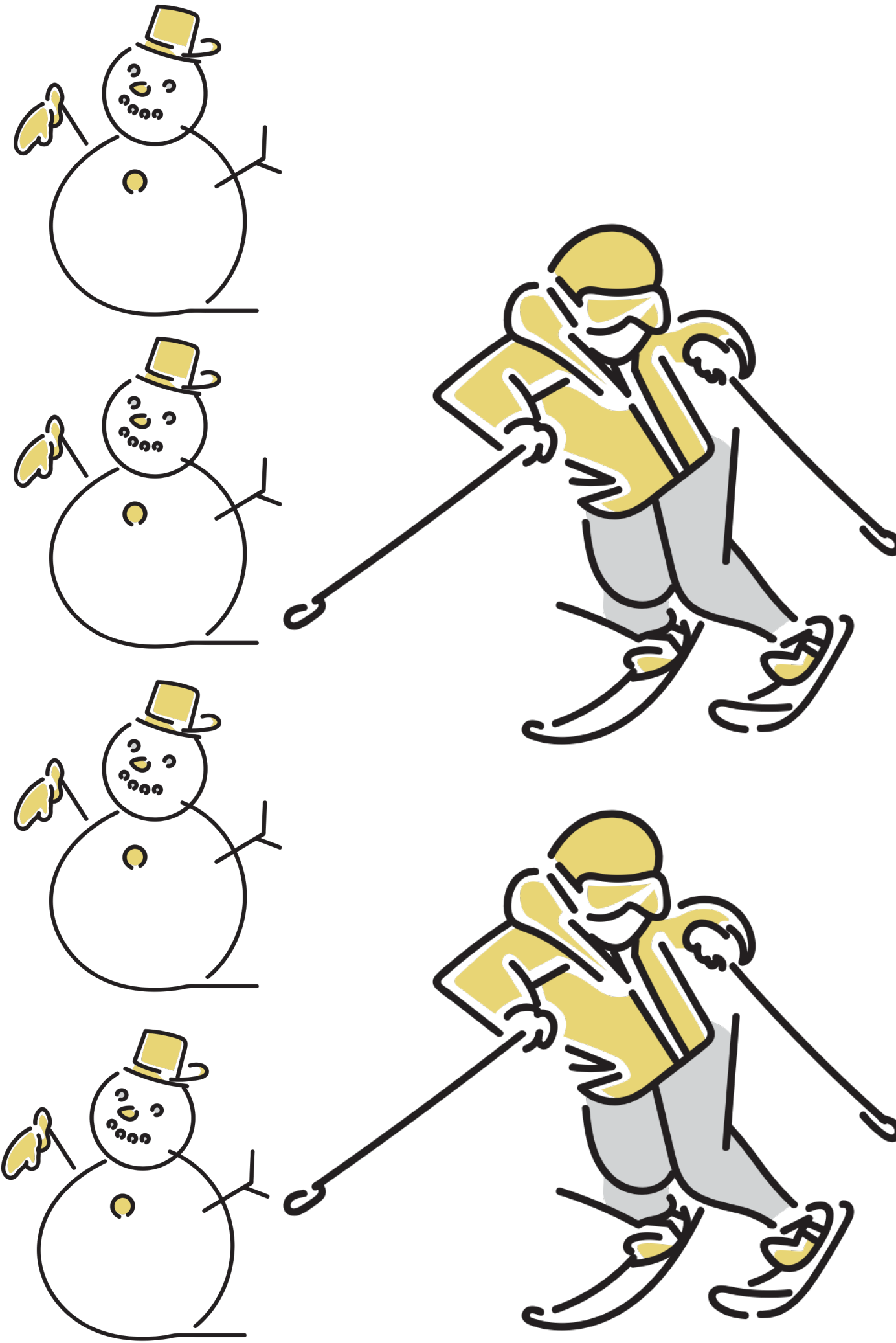
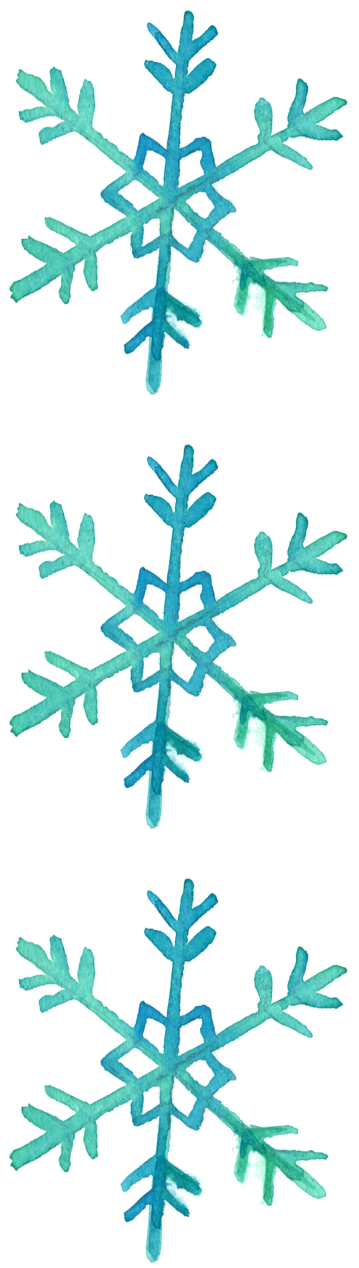
- $O(N)$ 個の地点を除いて、ヒストグラムの高さの変更やボールの追加は起こらない
- それらの地点の間では、そこまでに繰り上がってきたボールを、長方形領域に左から貪欲に詰め込む
- $f(h, w, n) : h \times w$ の長方形領域に n 個のボールを左から貪欲に詰めるとき、ボールを移動させる回数の和は？
- 最初の h 個が 0、次の h 個が 1、... となるので算数をすれば $O(1)$
- 計算量 $O(N^3)$



得点分布



0 5 12 17 26 59 86 100



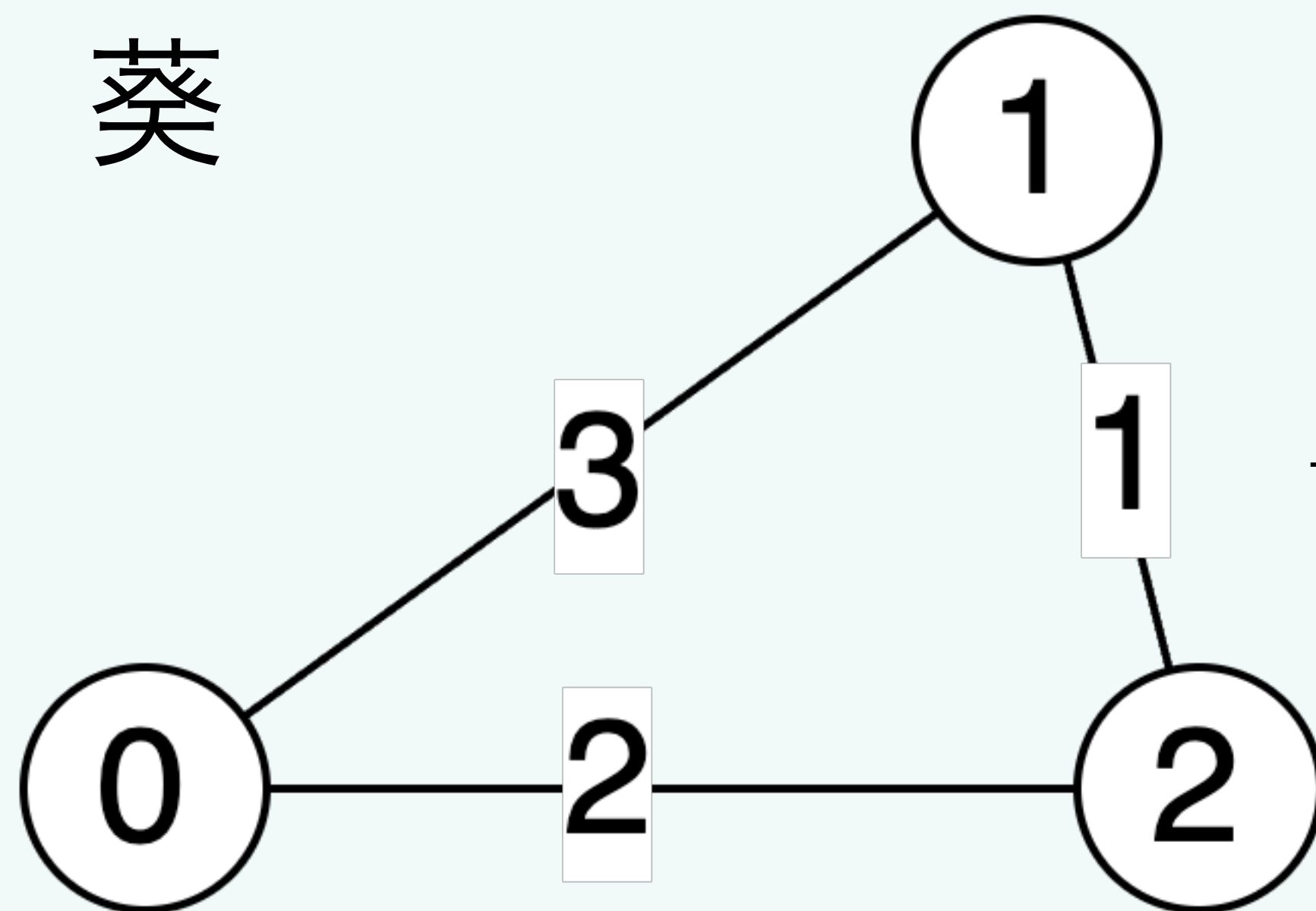
スパイ 3

解説担当：星井智仁

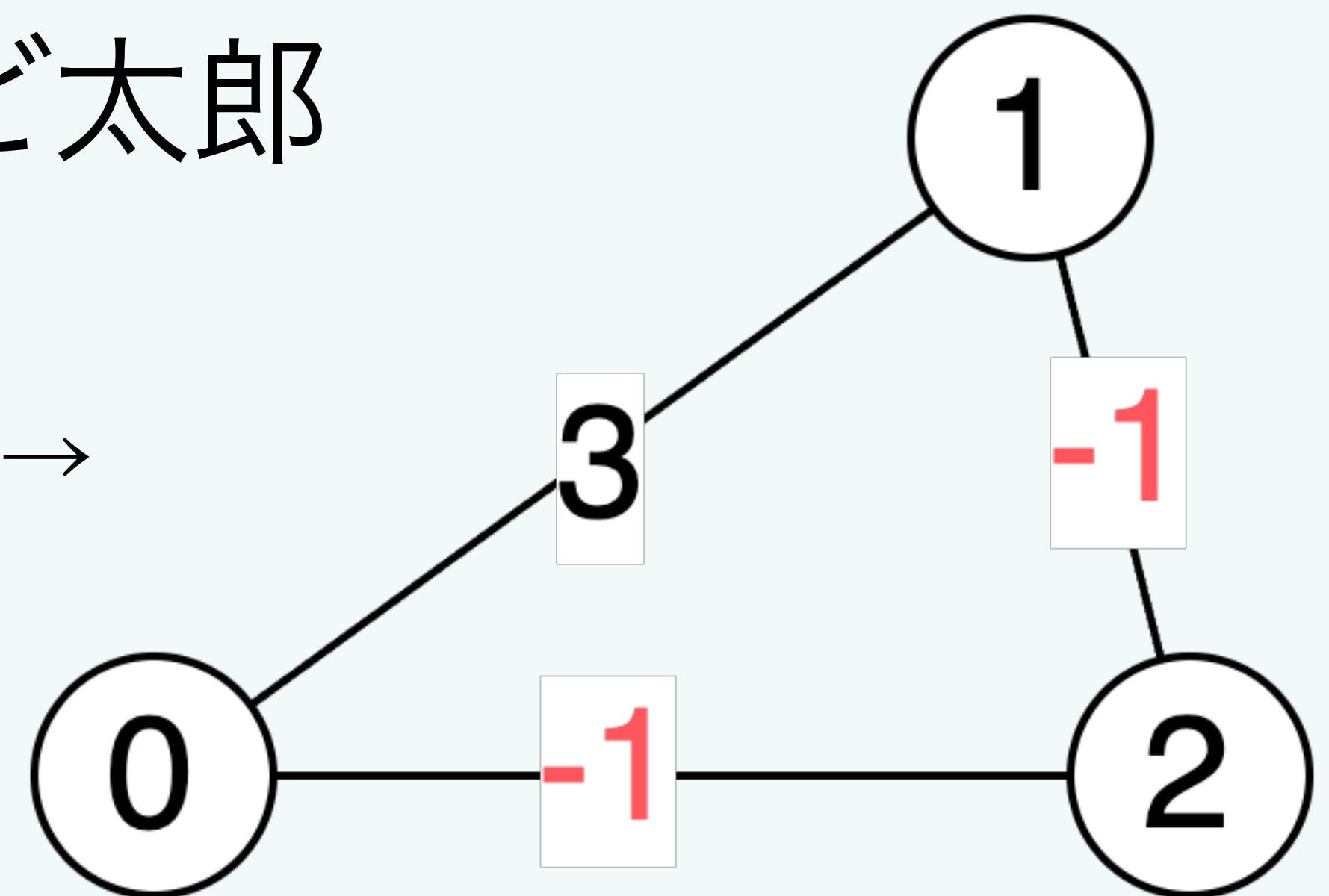
問題概要

N頂点M辺のグラフの一部の辺の**長さ**の情報を失いました
01文字列を送って頂点0からQ個の頂点への最短**経路**を教えた

葵



ビ太郎



→010101101→

文字列を送る

制約

$N \leq 10\,000$ 頂点の数

$M \leq 20\,000$ 辺の数

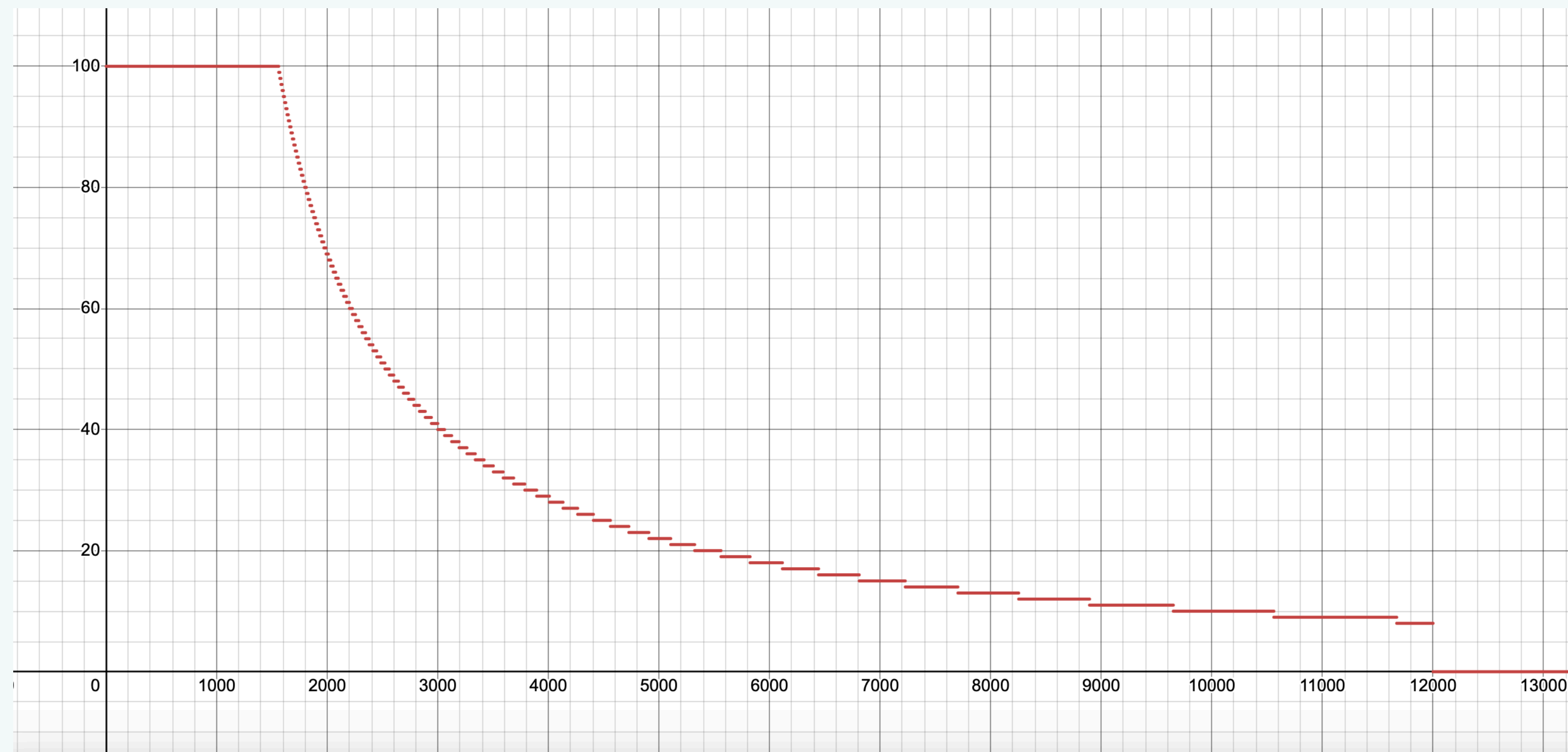
$C_i \leq 10^{12}$ 辺の長さ

$Q \leq 16$ 最短経路を知りたい頂点の数

$K \leq 300$ 距離がわからない辺の数

配点

$$\left\lfloor \frac{100\,000}{L - 560} \right\rfloor \text{点}$$



$L = 12000$ (8点)

わからない辺の長さを全て送る
ビ太郎側でDijkstra法をして最短経路を調べる

$$L = K \log \max C_i = 12000$$

$L = 2240000$ (0点)

葵側でDijkstra法をして、経路を調べる
それぞれの頂点について、経路をそのまま送る

$$L \approx NQ \log N \approx 2\,240\,000$$

$L = 4800$ (23点)

距離がわかるところの経路は送らなくてもビ太郎側で特定できる

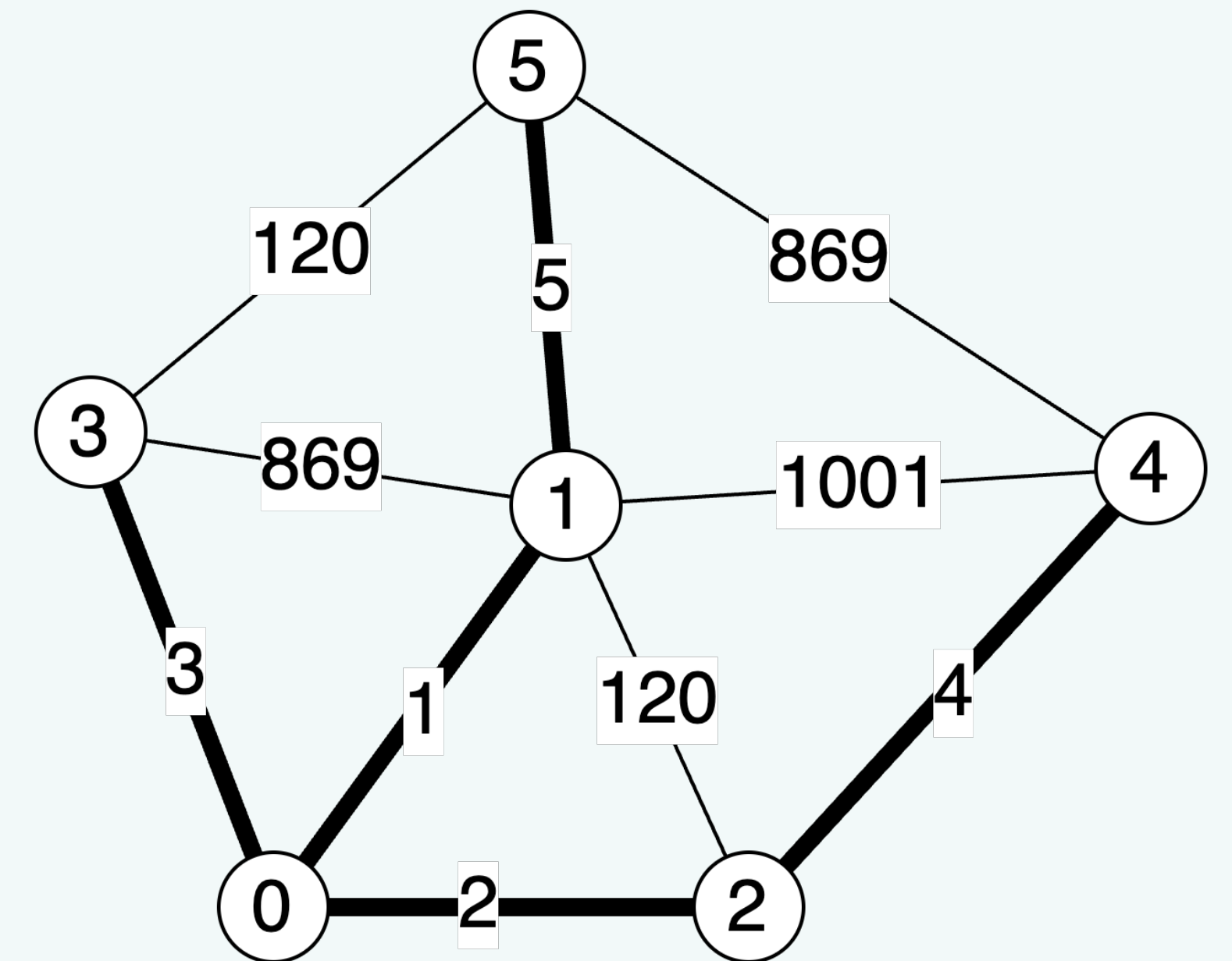
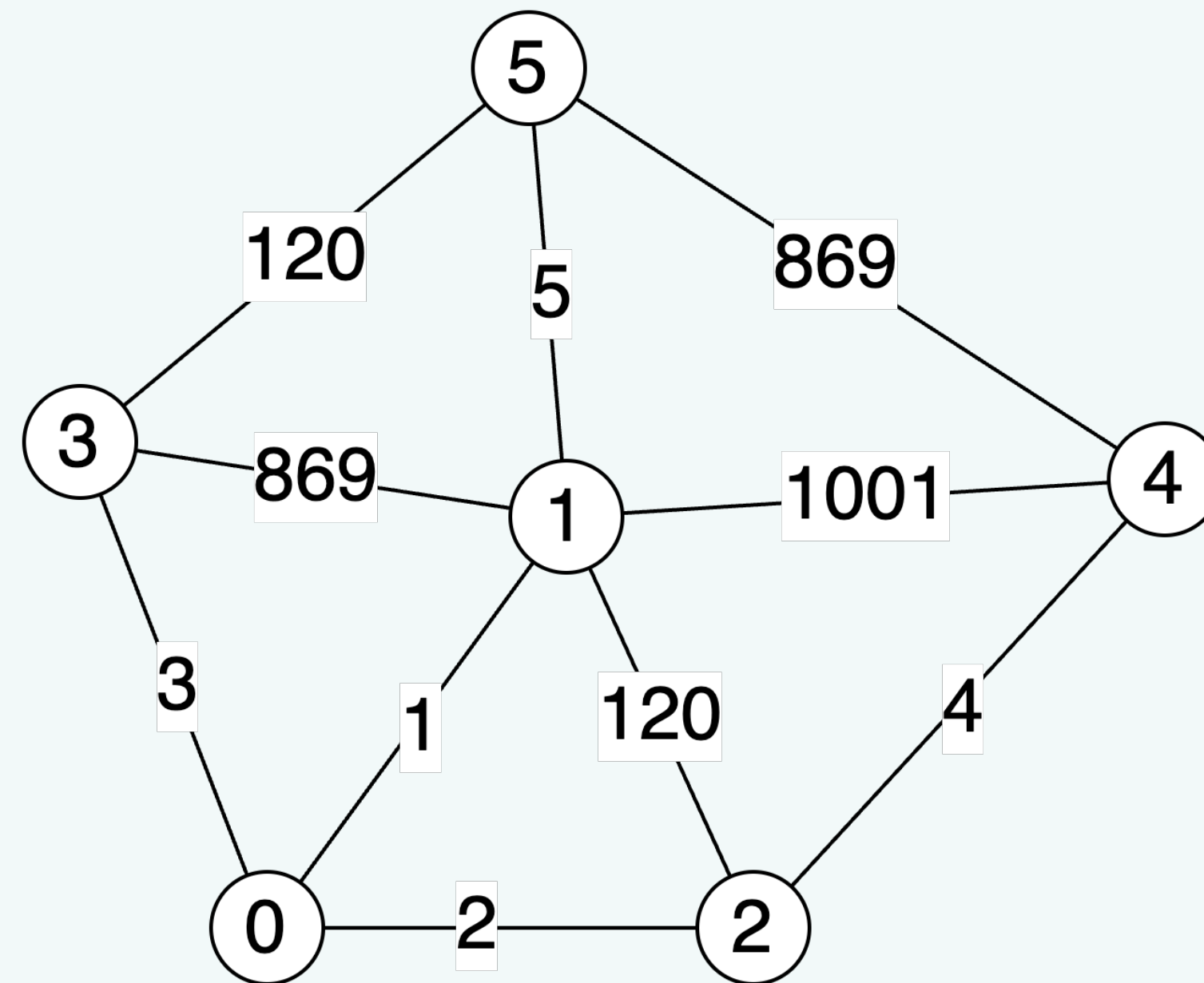
それぞれの頂点について、距離がわからない辺を使うか否かを送る

使う辺の長さを1に、使わない辺の長さをMAXとして最短経路を調べると、復元ができる

$$L = QK = 4800$$

$L = 20000$ (0点)

最短路木を作る



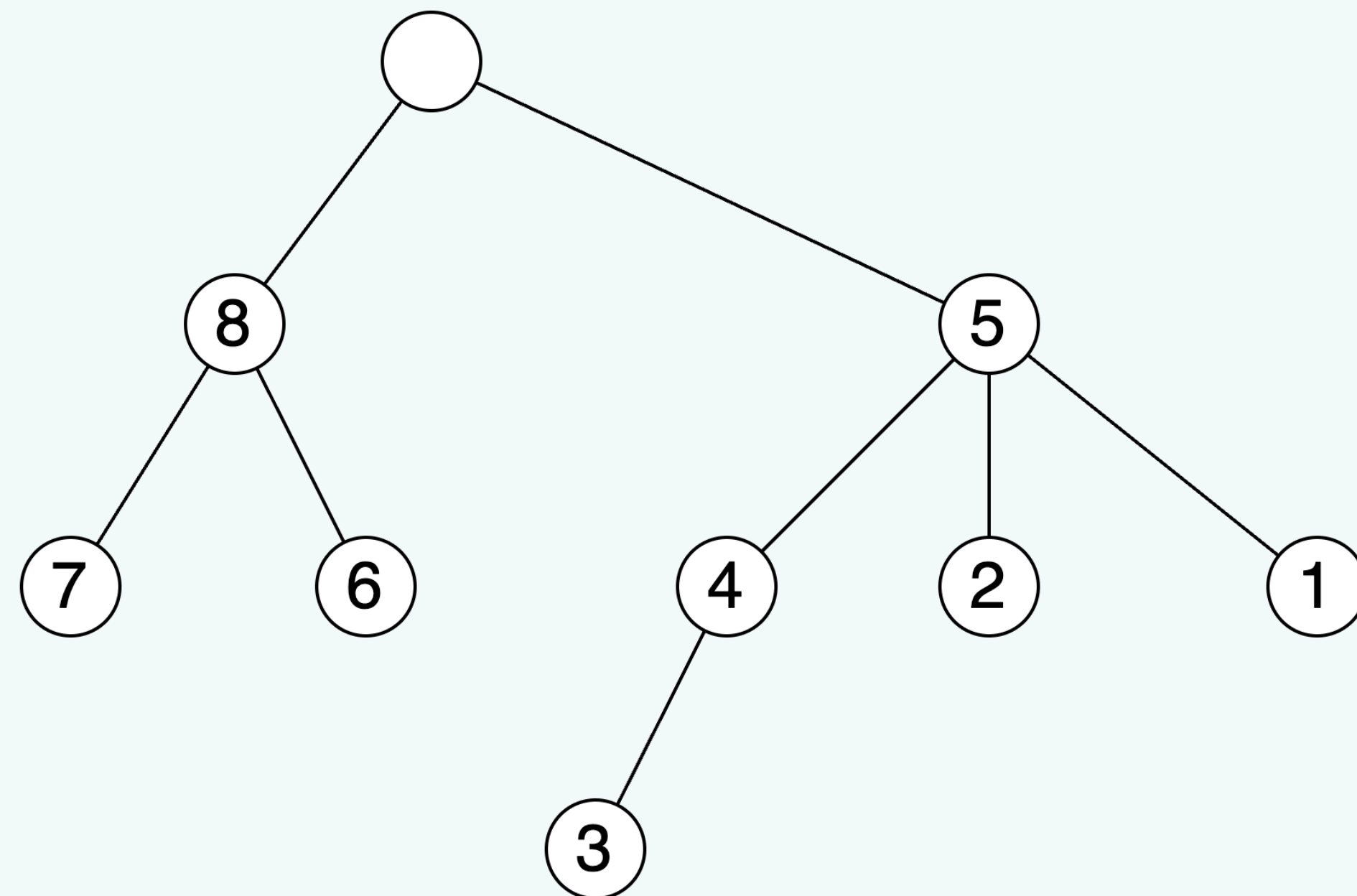
全ての辺について最短路木に含まれているか否か調べる

$L = M = 20\,000$

L = 2445 (53点)

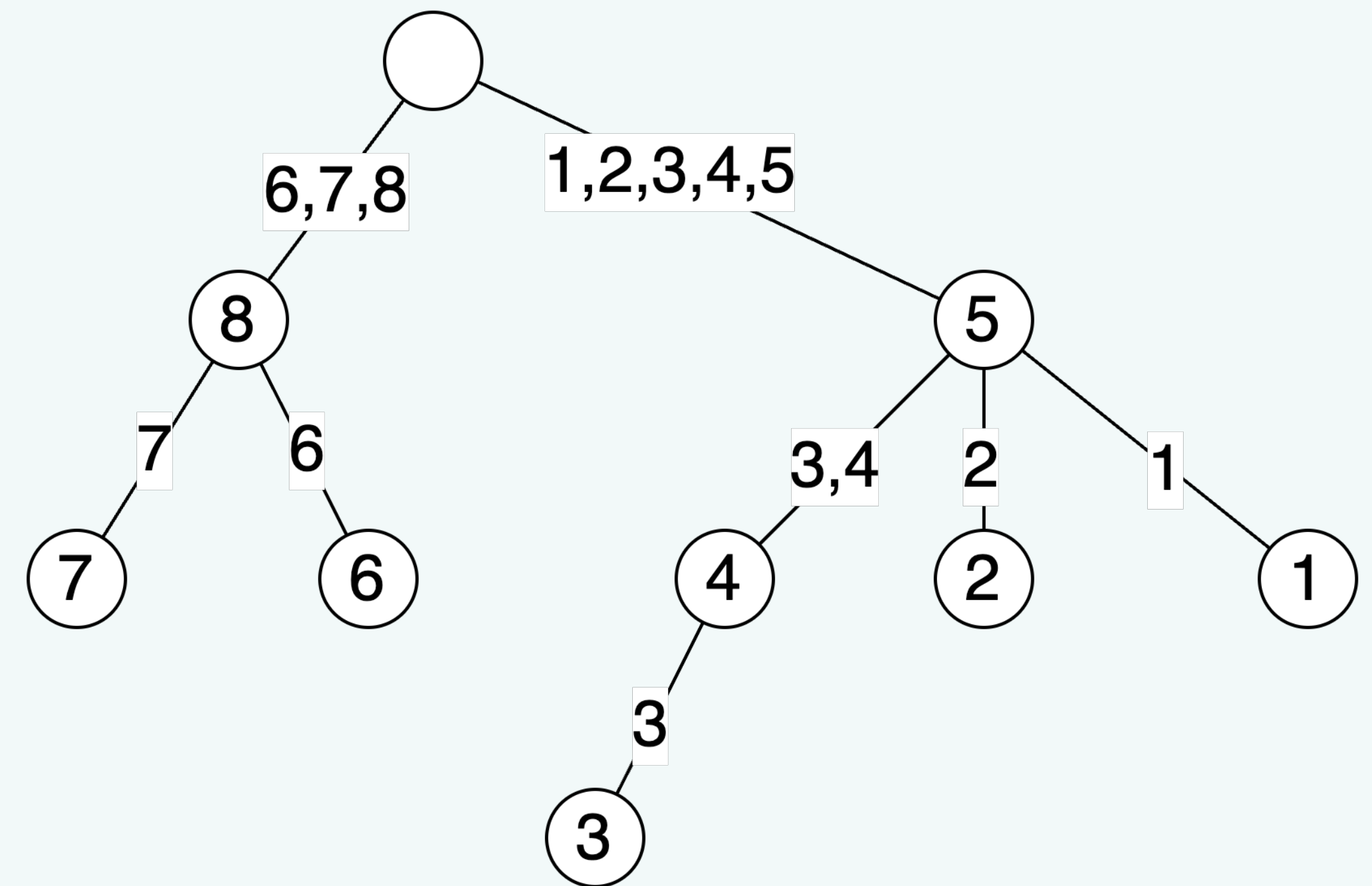
それぞれの辺について、各クエリで使うか使わないかを知りたい

最短路木についてDFSの帰りがけ順に頂点番号を振る



L = 2445 (53点)

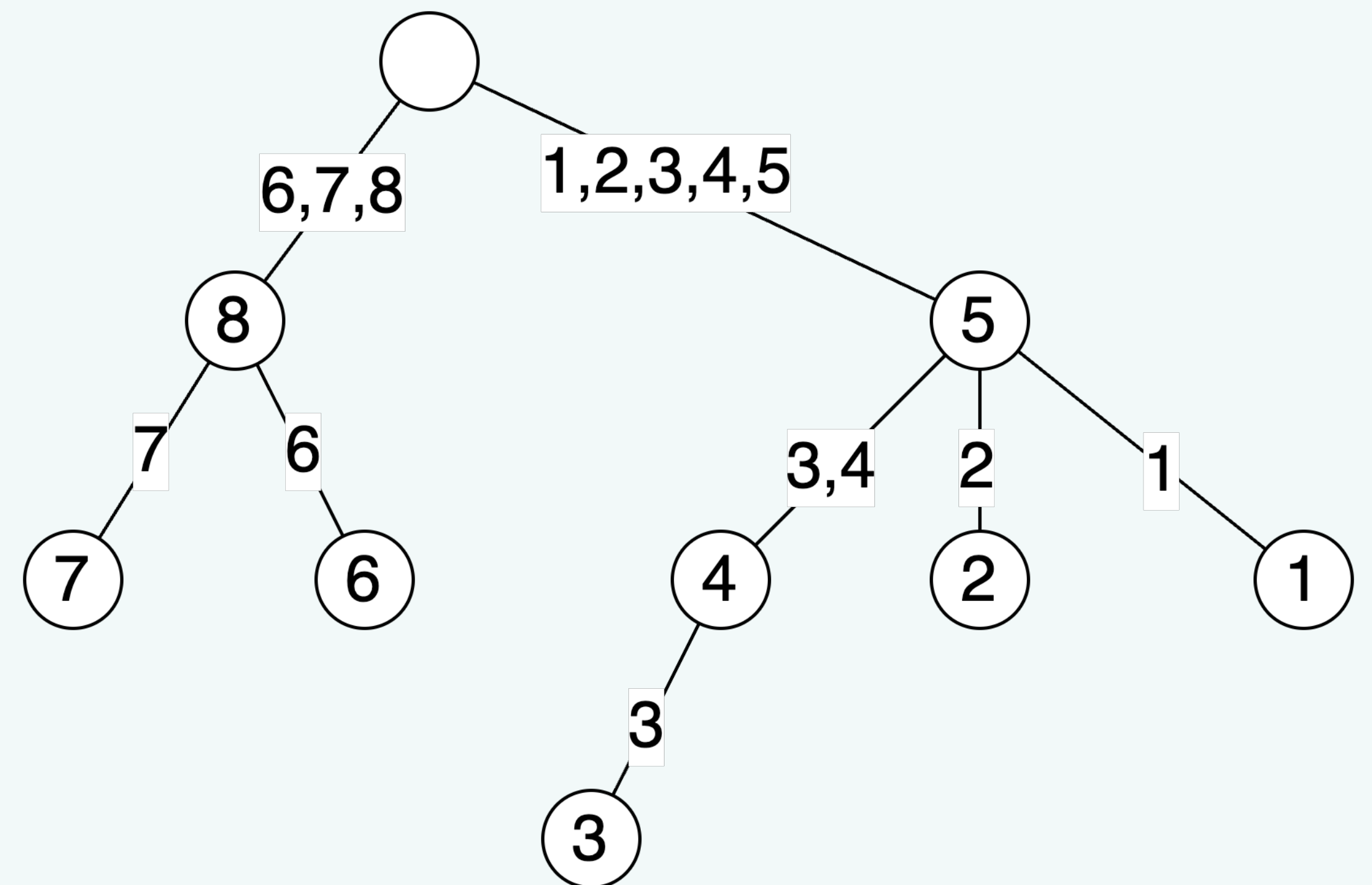
DFSの帰りがけ順に頂点番号を振る
→それぞれの辺の子となる頂点は区間となる！！



右図は全頂点についてやっているが、
クエリのQ頂点のみについて調べる

L = 2445 (53点)

Q頂点について，訪れる順番を調べる
並べ方は $Q!$ 通り $\rightarrow \log Q!$ (順番をそのまま送ると $Q \log Q$)



L = 2445 (53点)

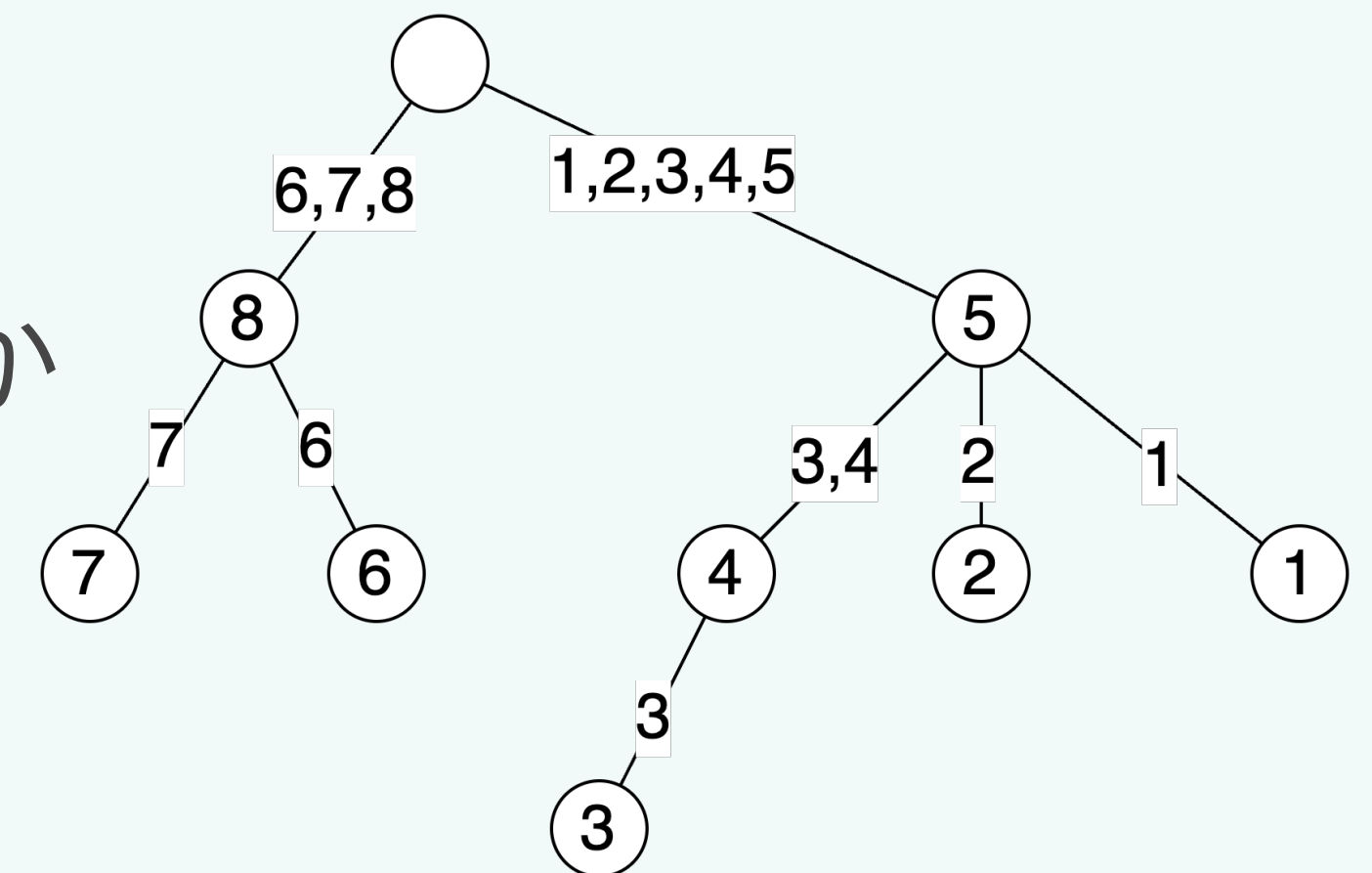
K辺について、区間を送る

それぞれに左端と右端のペアを送ると長さ $2 \log Q = 8$

区間は $\frac{Q(Q+1)}{2} + 1 = 137$ 通りなので多倍長整数などを使うと
区間に含まれない場合

1辺あたり7文字強で送ることも可能

$$L = K \times 2 \log Q + \log Q! = 2445 \text{ とか}$$



L = 1665 (90点)

区間は $\frac{Q(Q+1)}{2} + 1 = 137$ 通り

137通り全てが現れますか？

実は $2Q = 32$ 通りしか現れません！！！！！！

$L = 1665$ (90点)

32通り の内訳

使わない：1通り， それ以外： $2Q-1$ 通り

1つだけの区間 $[i, i]$ が Q 通り， 隣同士の区間がマージされていく
ことを考えると， 他に現れるのは $Q-1$ 通り

N 人のトーナメントでは $N-1$ 戦行われますみたいな話

$L = 1665$ (90点)

$\frac{Q(Q+1)}{2} + 1 = 137$ 通りのうち, $Q+1$ 通りは必ず現れるので

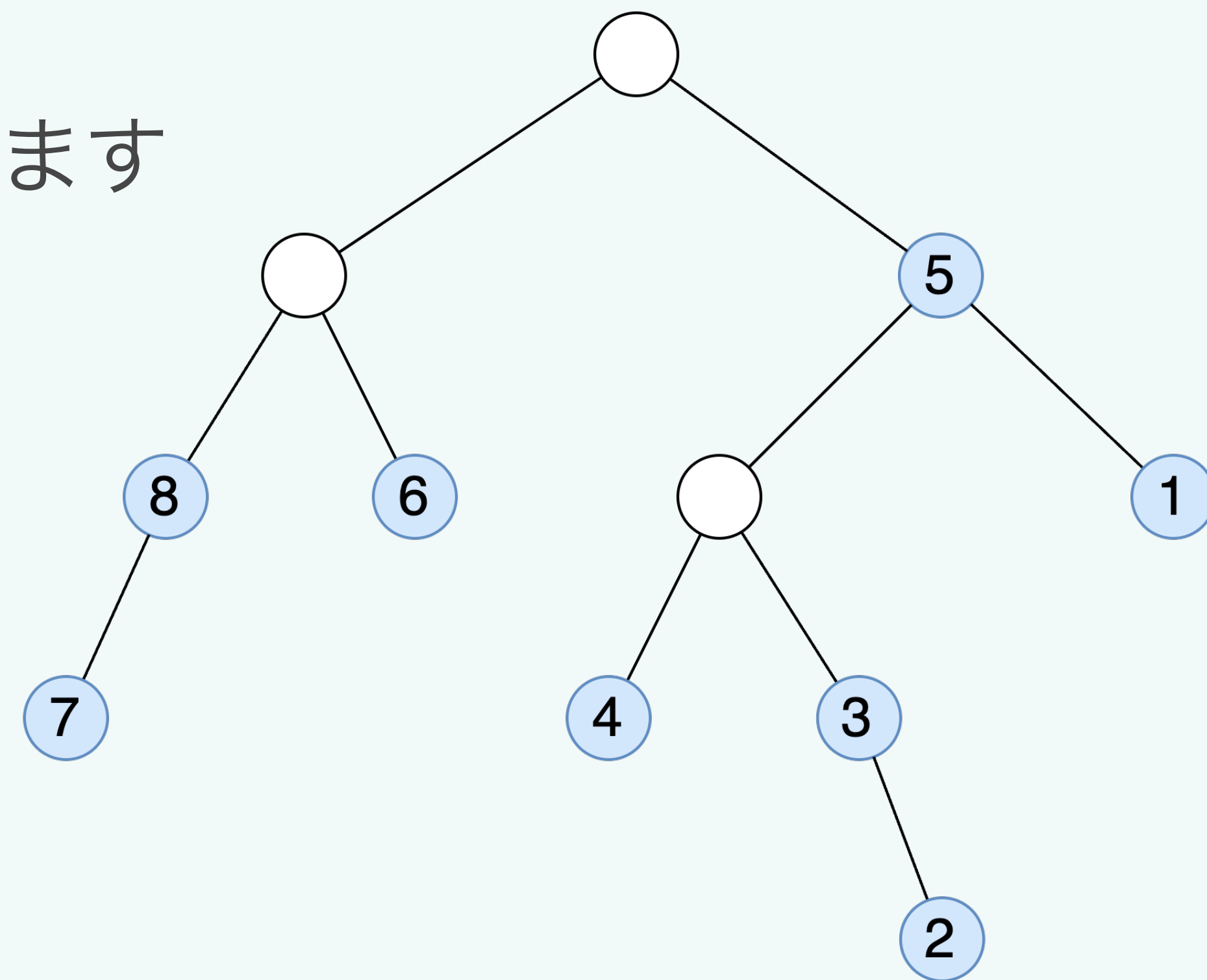
残りの $\frac{Q(Q-1)}{2}$ 通りについて現れるか現れないかを01で書く

$$L = \frac{Q(Q-1)}{2} + K \log 2Q + \log Q! = 1665$$

L = 1553 (100点)

「どの区間が現れるか」をもっと縮めることができます

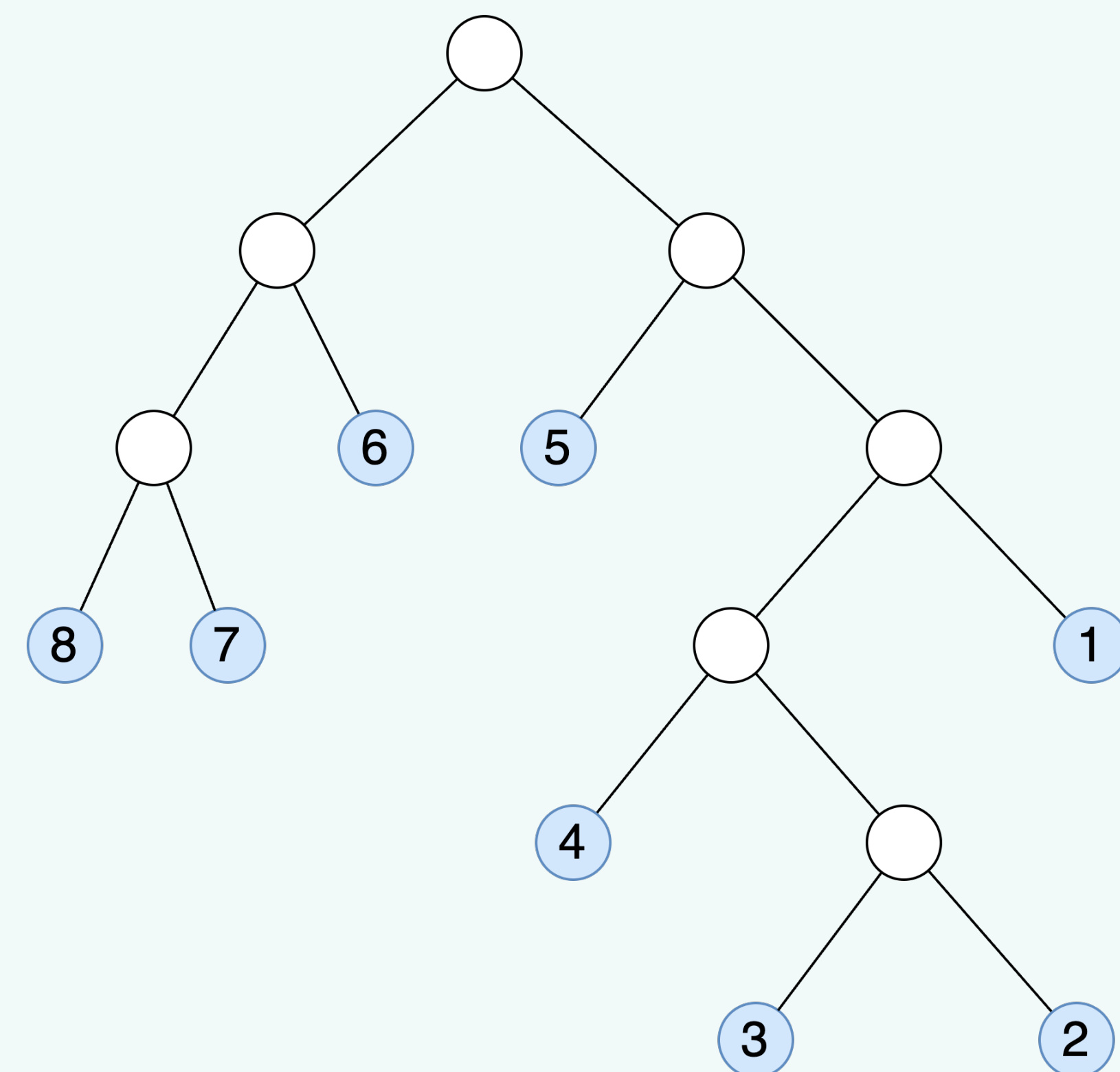
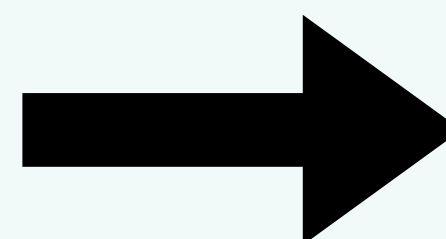
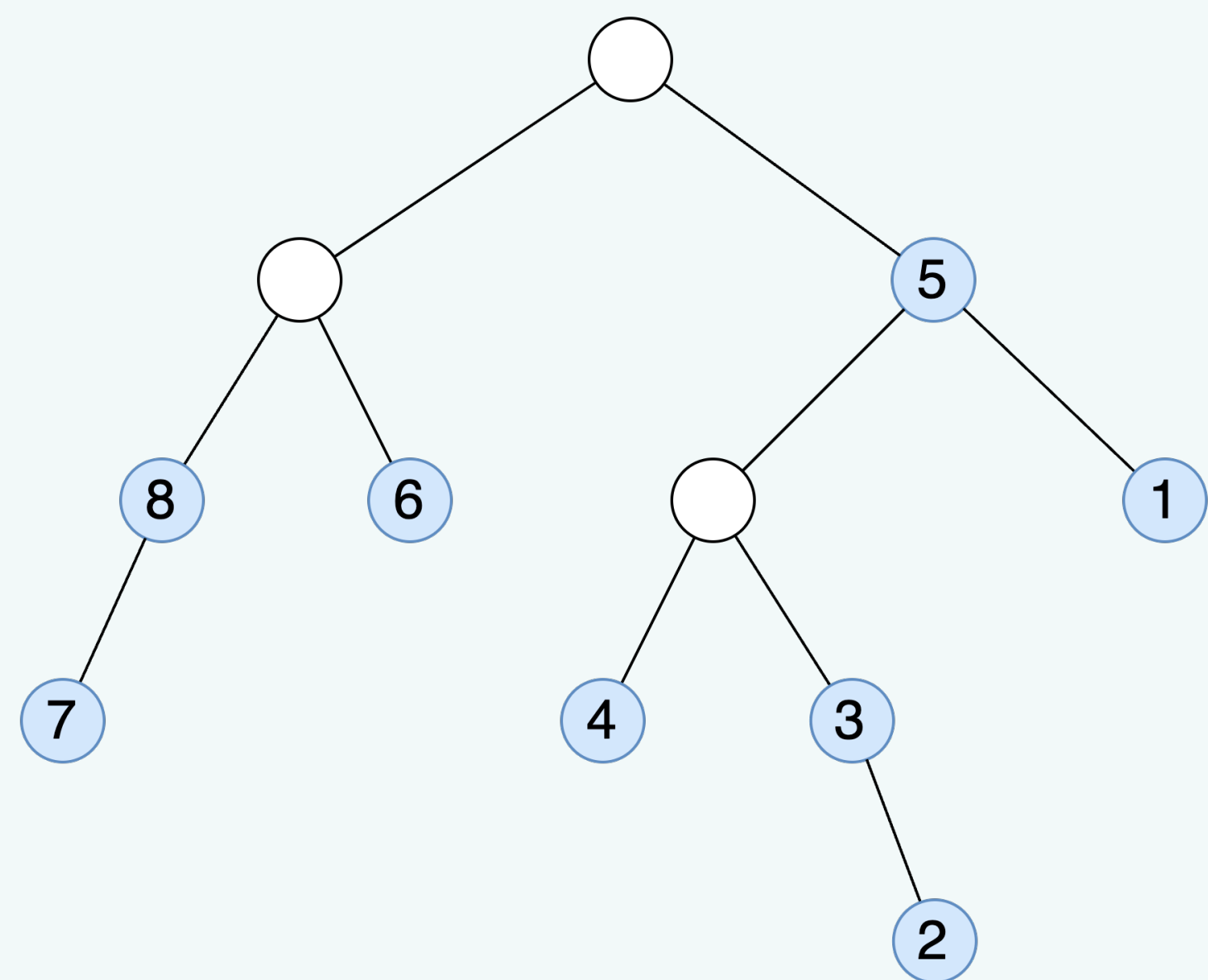
クエリに含まれる頂点に注目して木を圧縮します
(Auxiliary Tree というらしい)



L = 1553 (100点)

適切に辺を足して二分木にします

二分木がわかると現れる区間がわかる



$L = 1553$ (100点)

葉のみにラベルがついた $2Q+1$ 頂点の二分木は何通り？

$(2Q-3)!! = 1 \times 3 \times \dots \times (2Q-3) = 6\,190\,283\,353\,629\,375$ 通りあります

整数と二分木を一対一対応させると

$$L = K \log 2Q + \log [(2Q-3)!!] = 1553$$

L = 1553 (100点)

葉のみにラベルがついた $2Q+1$ 頂点の二分木が $(2Q-3)!!$ となる証明

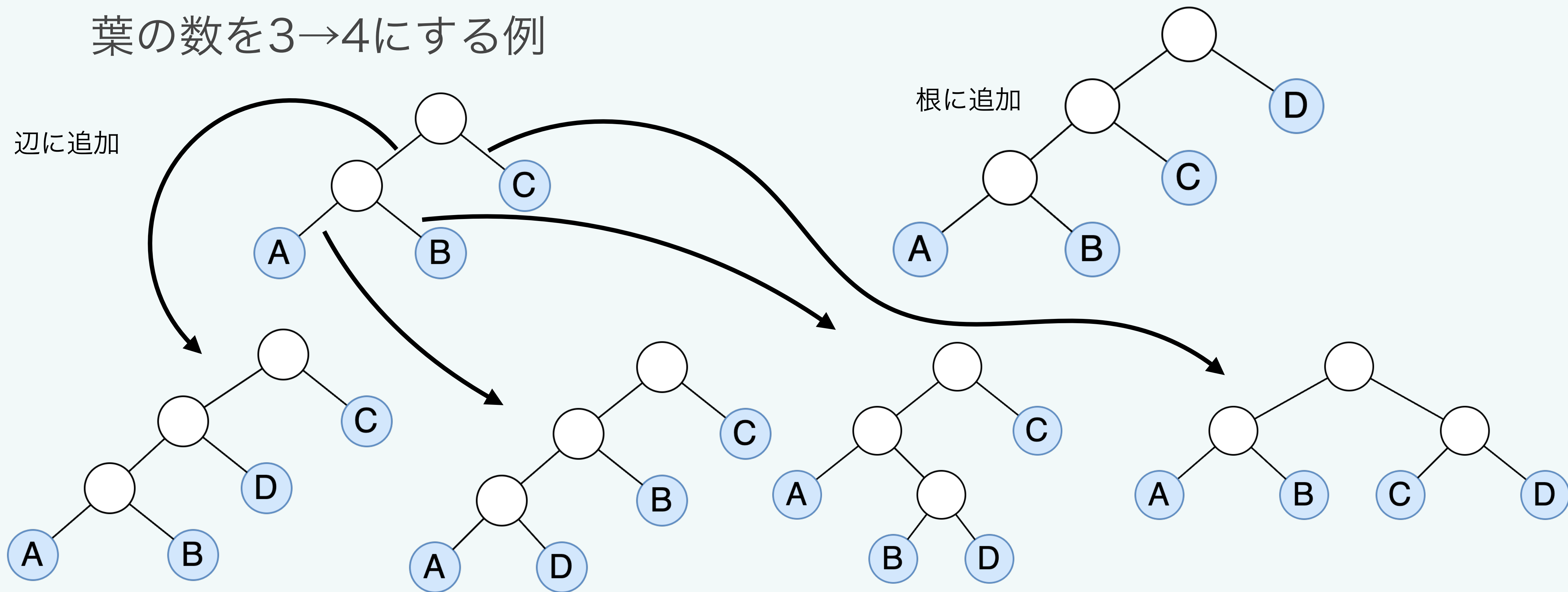
葉を追加していくことを考える

葉の数を K 個 $\rightarrow K+1$ 個にするときに追加する方法が $2K-1$ 通り

辺に対して追加操作を行う or 根に追加操作を行う

L = 1553 (100点)

葉の数を3→4にする例



得点分布

0点：14人

38点：1人

8点：9人

23点：2人

48点：1人

L = 1350 (100点)

チューター陣はより良い解法を見つけました！



square1001 2024/03/21 22:54

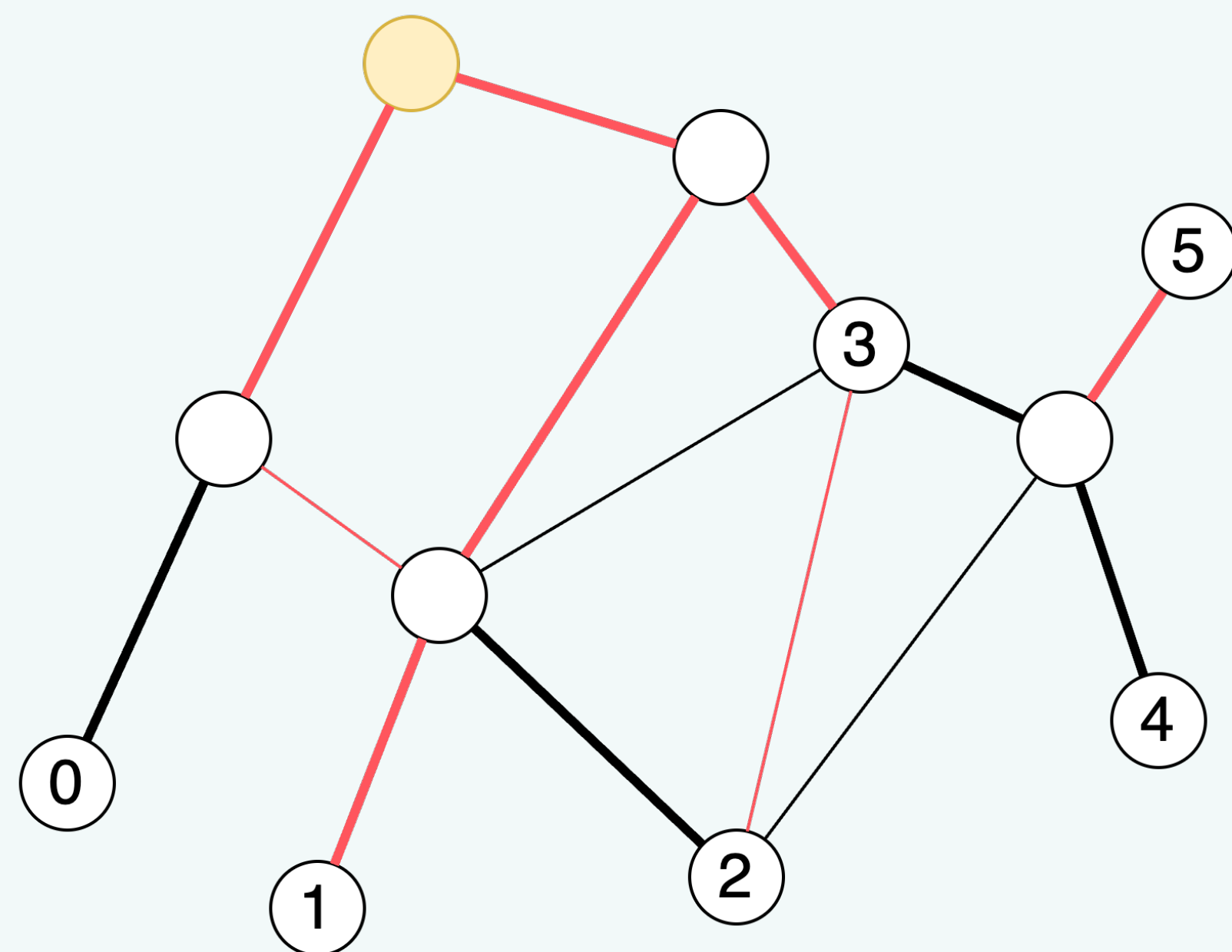
解法の概要

- ・ 4800 bit 解法は各クエリごとに「K 個の忘れた辺それぞれについて最短経路に含まれるかどうか」を送るものであるが、実際にはこれは冗長です
- ・ その代わりに、以下の情報を送ることを考えます
 - ・ K 個の忘れた辺それぞれについて「何番目のクエリで初めてこの辺を通ったか（存在しなければ -1）」を送る（合計 $Q+1$ 通り）
 - ・ 2 番目以降のクエリに対して「頂点 0 からゴールまでたどるときに、最後に『直前のクエリまでに一度以上通った忘れた辺』の番号（存在しなければ -1）」を送る（合計 $K+1$ 通り）
- ・ 以上の情報だけで、4800 bit 解法に必要な情報が復元でき、4800 bit 解法で最短経路が復元できます
- ・ 送るべき情報量は $K \log(Q+1) + (Q-1) \log(K+1)$ bit なので、 $K = 300, Q = 16$ では 1349.743... bit と計算される、つまり 1350 bit で解けます

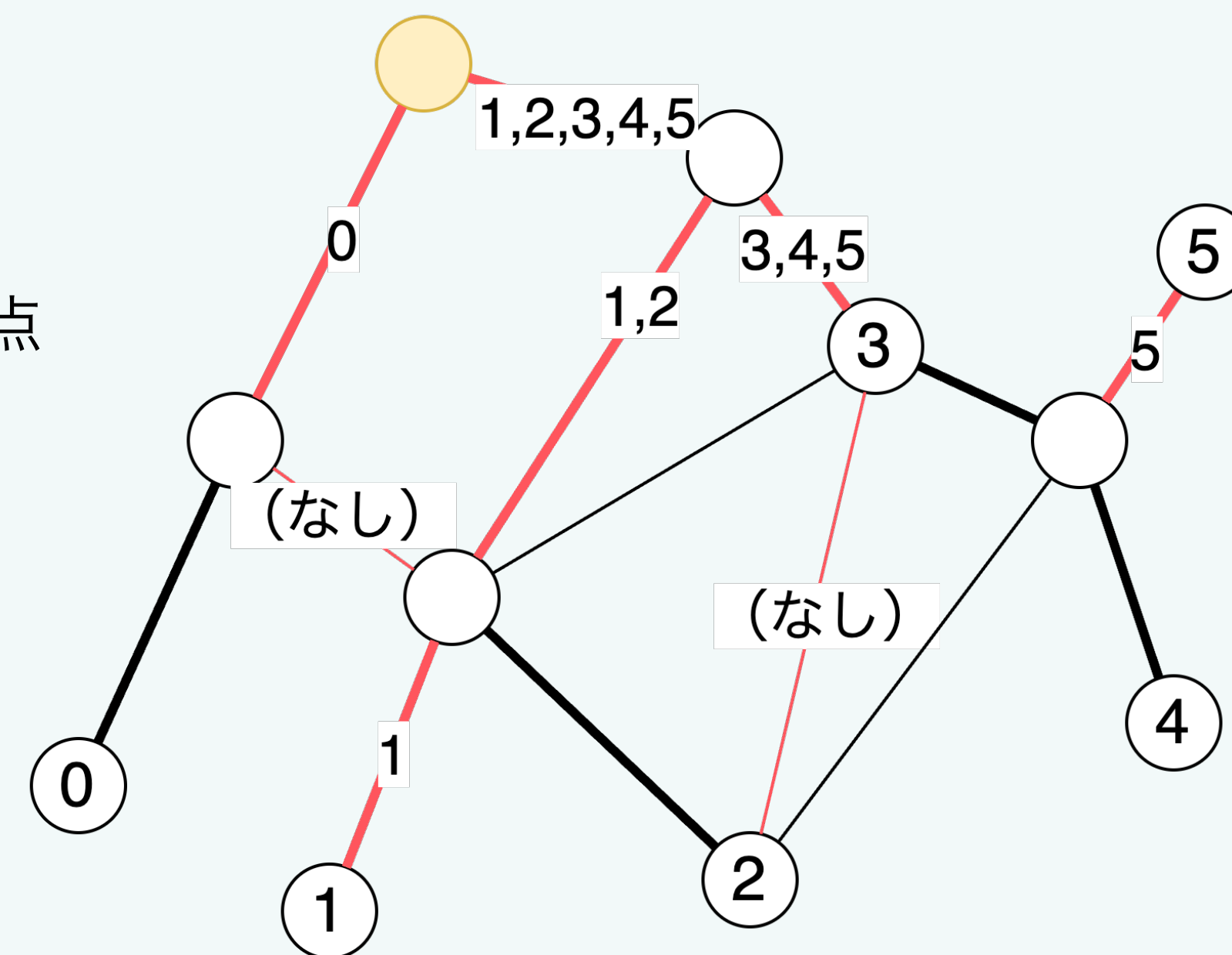
L = 1350 (100点)

L = 4800 解法

左図のグラフに対し，右図で辺に書かれている情報を送信する

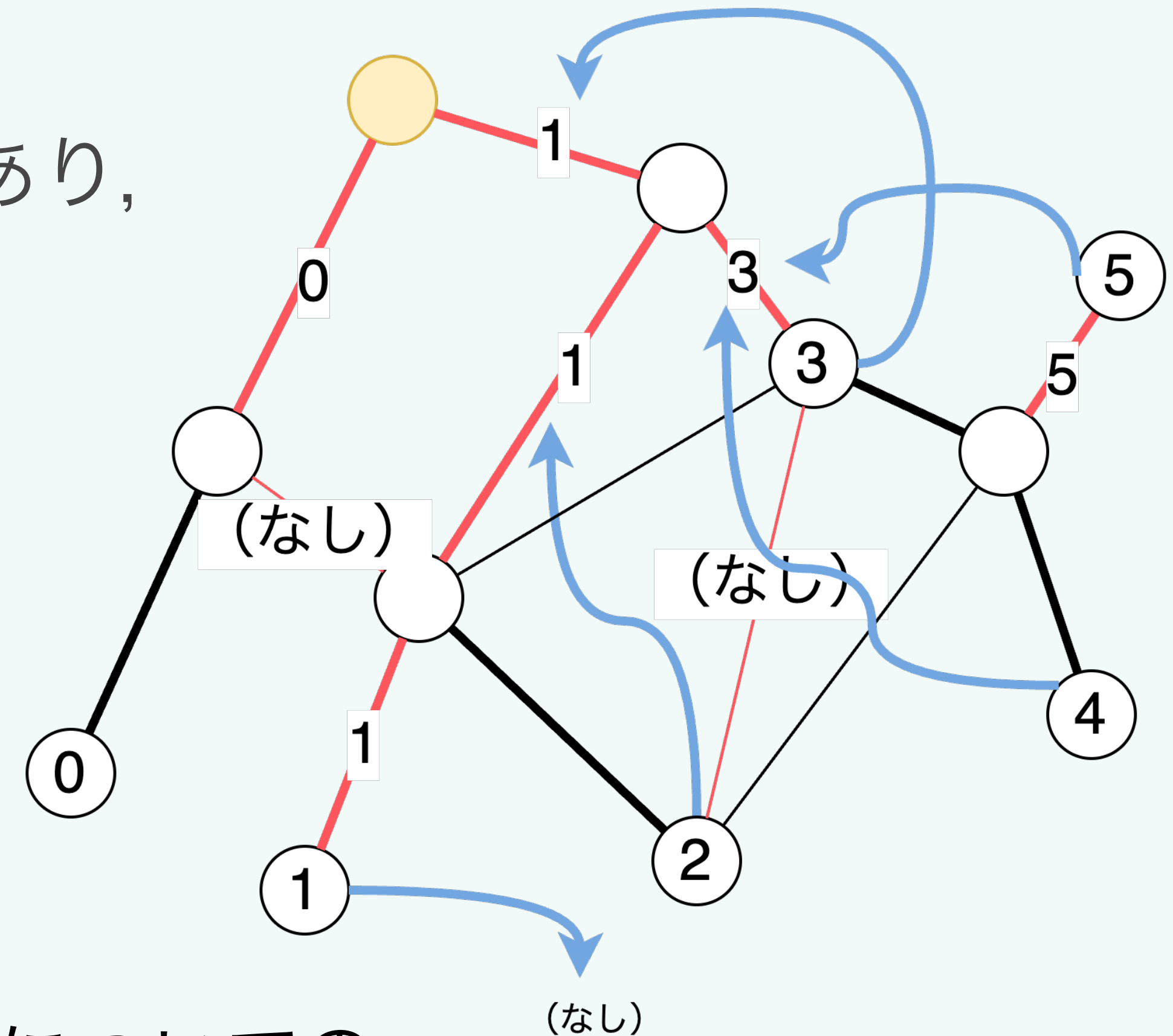
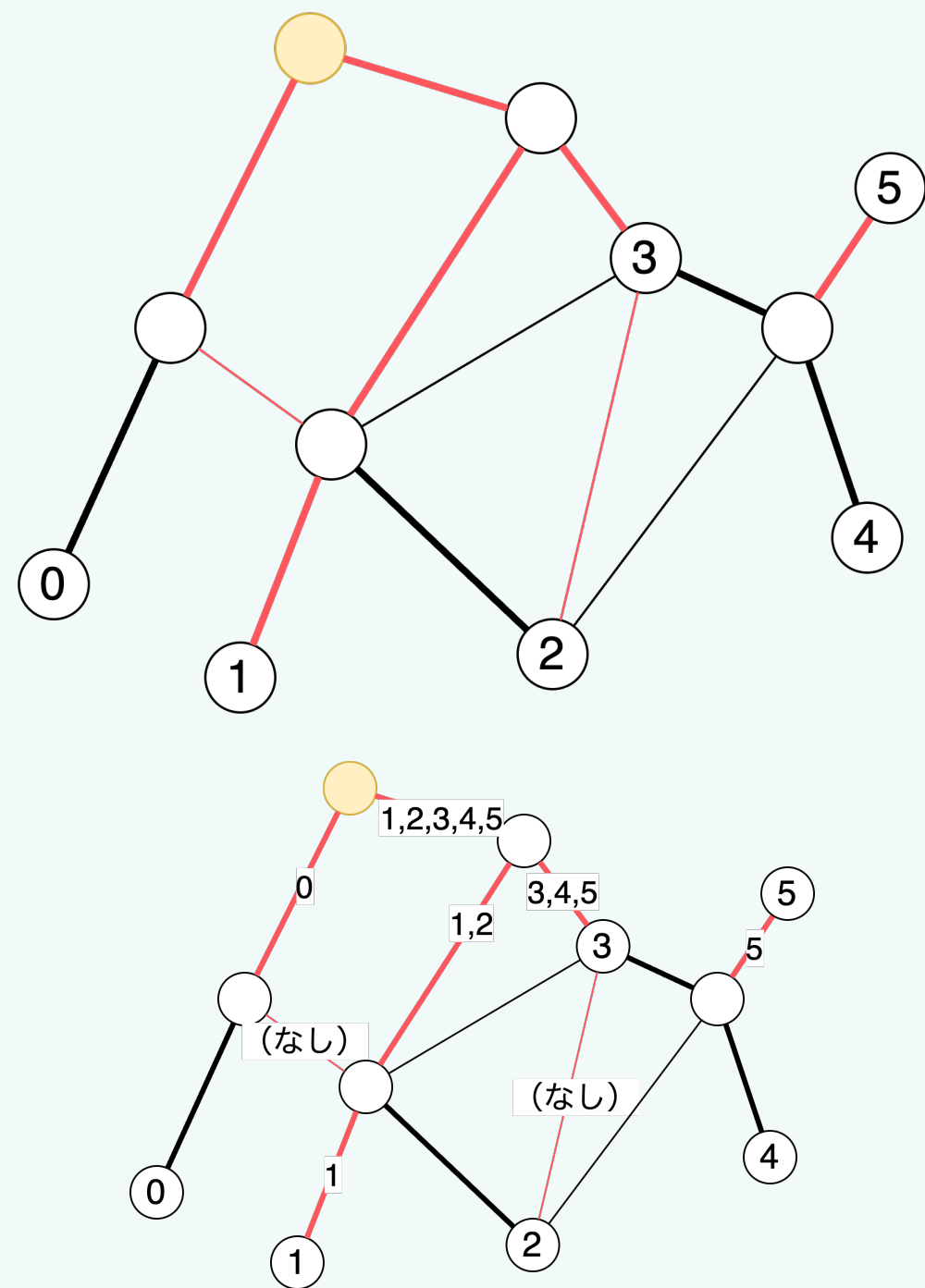


黄色の頂点：頂点0
番号付き頂点：クエリで聞かれる頂点
赤色の辺：長さがわからない頂点
太い辺：最短路木に含まれる辺



L = 1350 (100点)

各辺について書かれる情報は冗長であり、
実際には右図のような情報で十分



自分より若い番号のクエリについての
経路から分岐するところの辺の番号を記録

$L = 1350$ (100点)

各隠されている辺についてクエリ番号を記録

$K \log(Q + 1)$

自分より若い番号のクエリについての経路から分岐するところの辺の番号

$(Q - 1) \log(K + 1)$

多倍長整数などを使ってエンコードすると、長さ1350でエンコードできる

ボードゲーム

— 解説担当: 西本将樹 (maspy) —

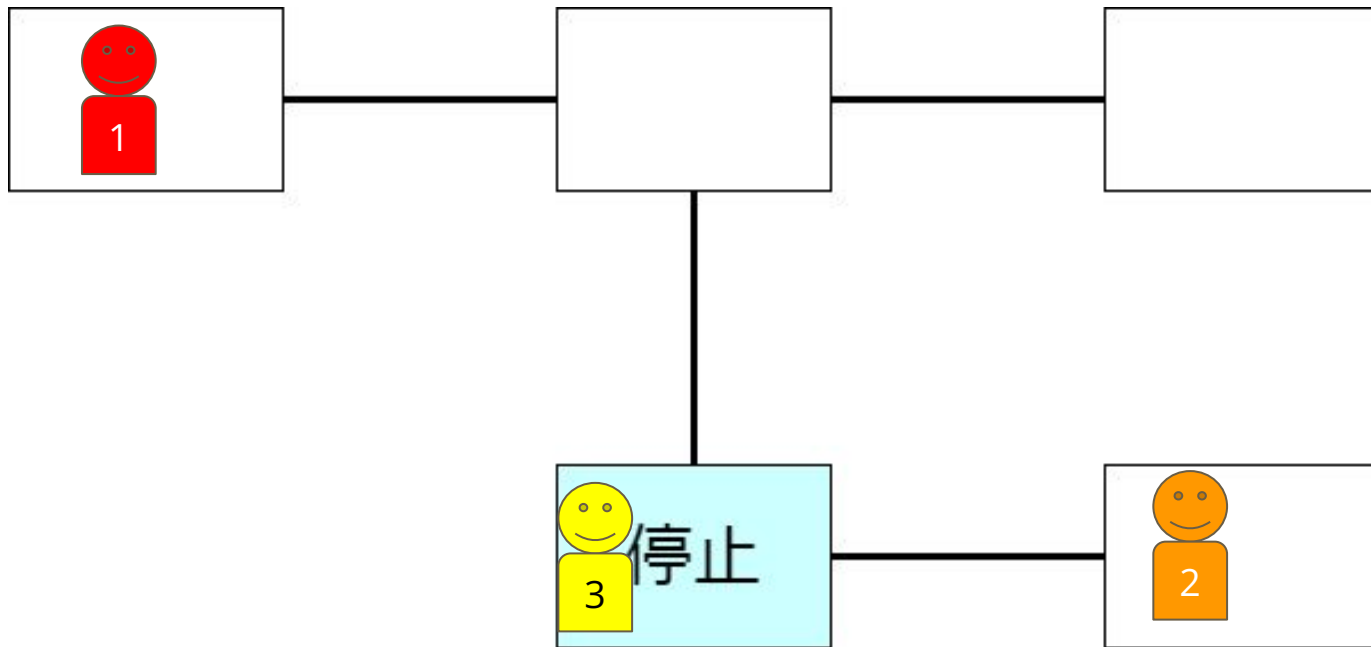
問題概要

- K 個のコマがある
- 動かし方のルールがある(問題文をよく読みましょう)
- 目標を達成するまでの最小操作回数を求めよ

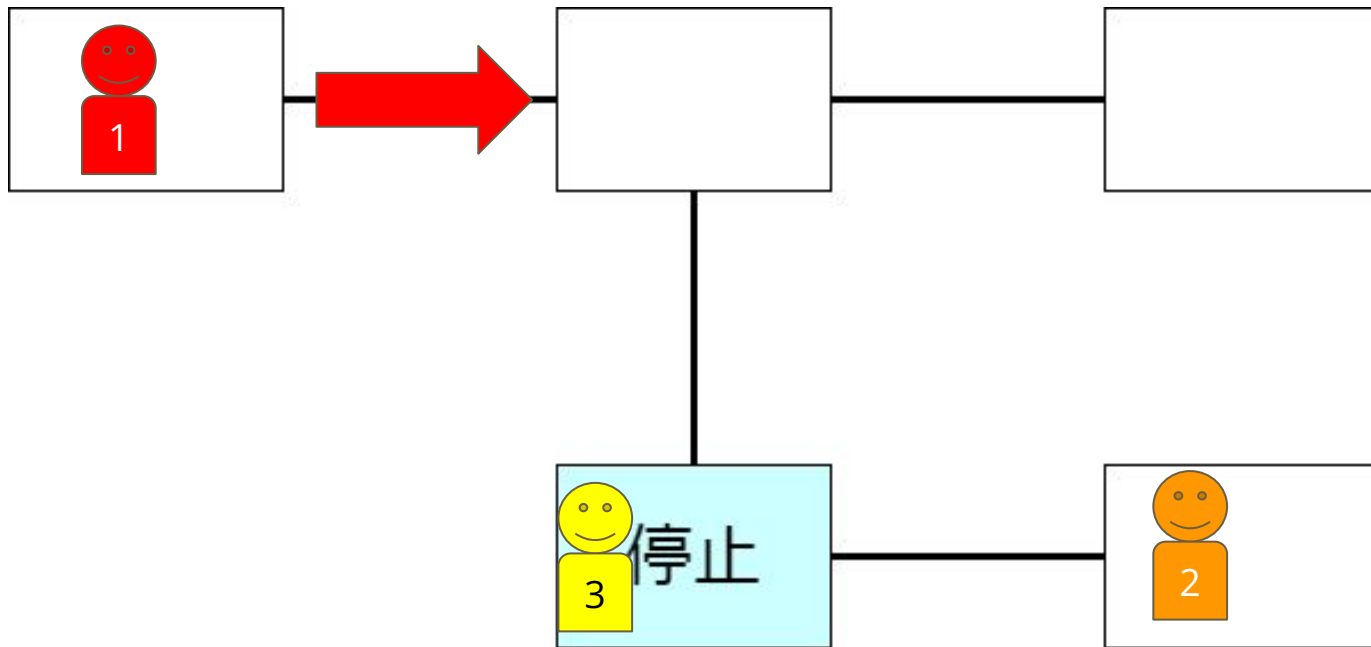
K 人ゲームと言っているが, 勝負要素等はなし.

1 人が動かすと思ってもよい.

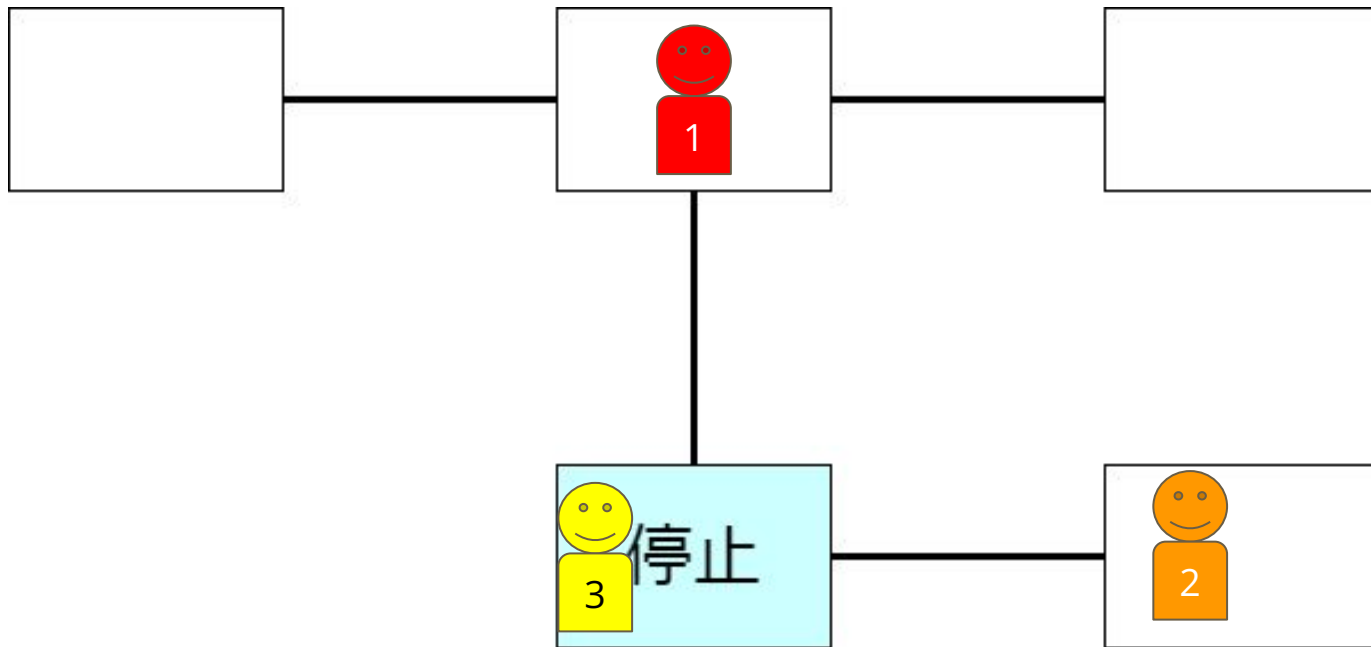
問題概要(動かし方のルール)



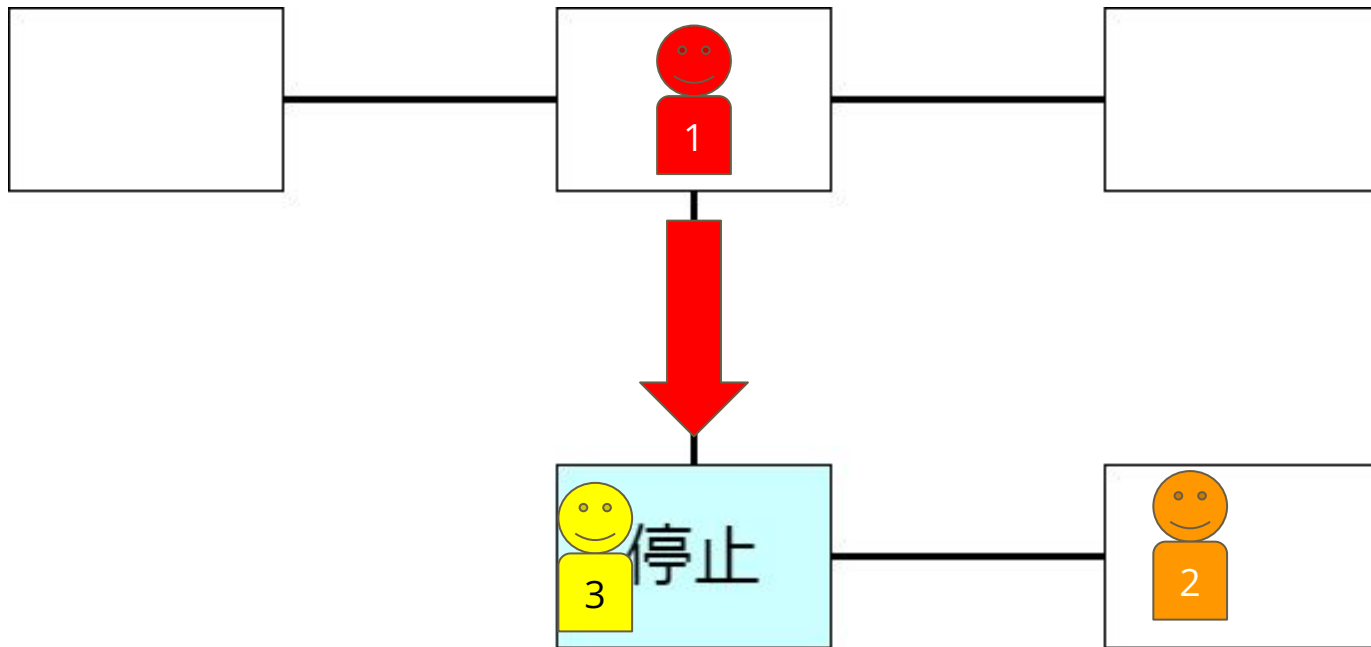
問題概要(動かし方のルール)



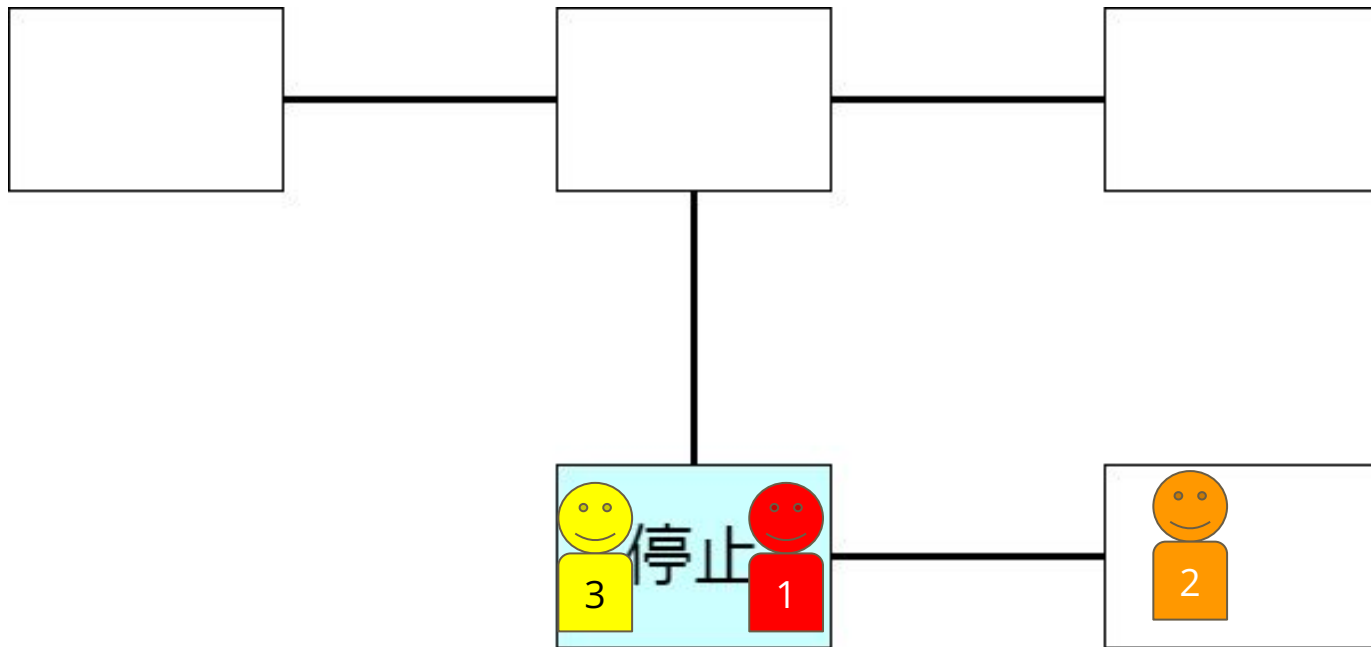
問題概要(動かし方のルール)



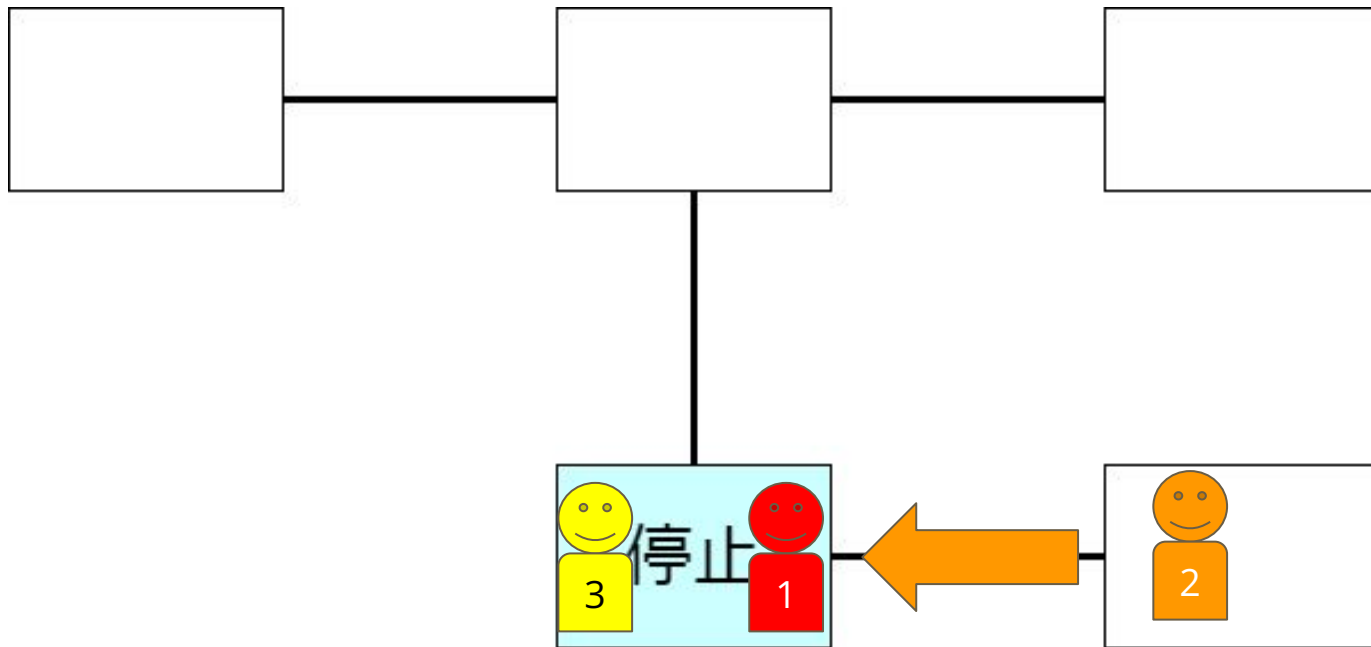
問題概要(動かし方のルール)



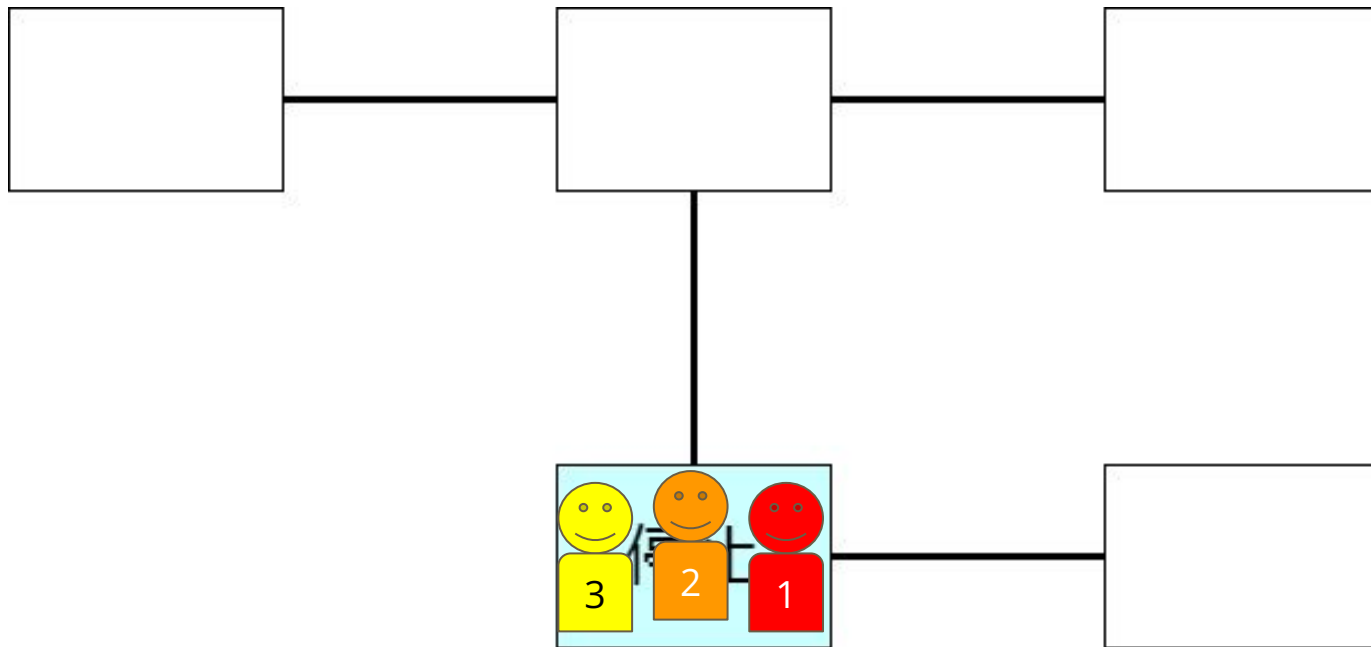
問題概要(動かし方のルール)



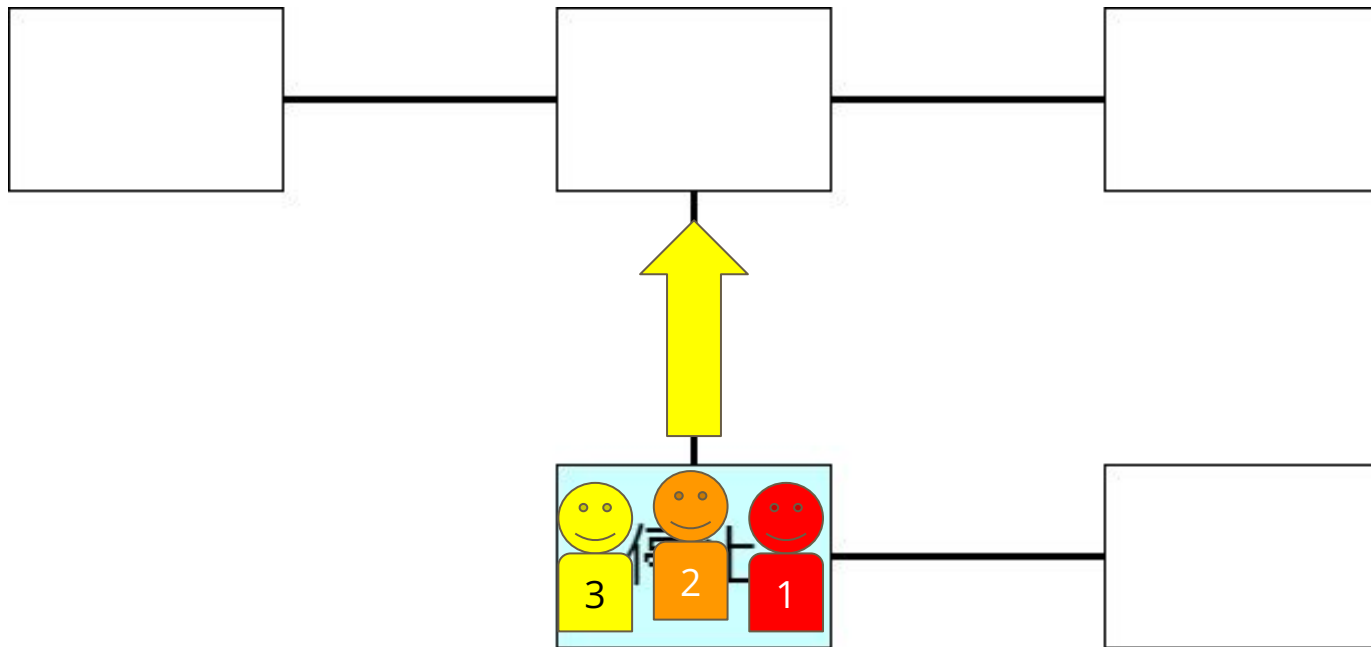
問題概要(動かし方のルール)



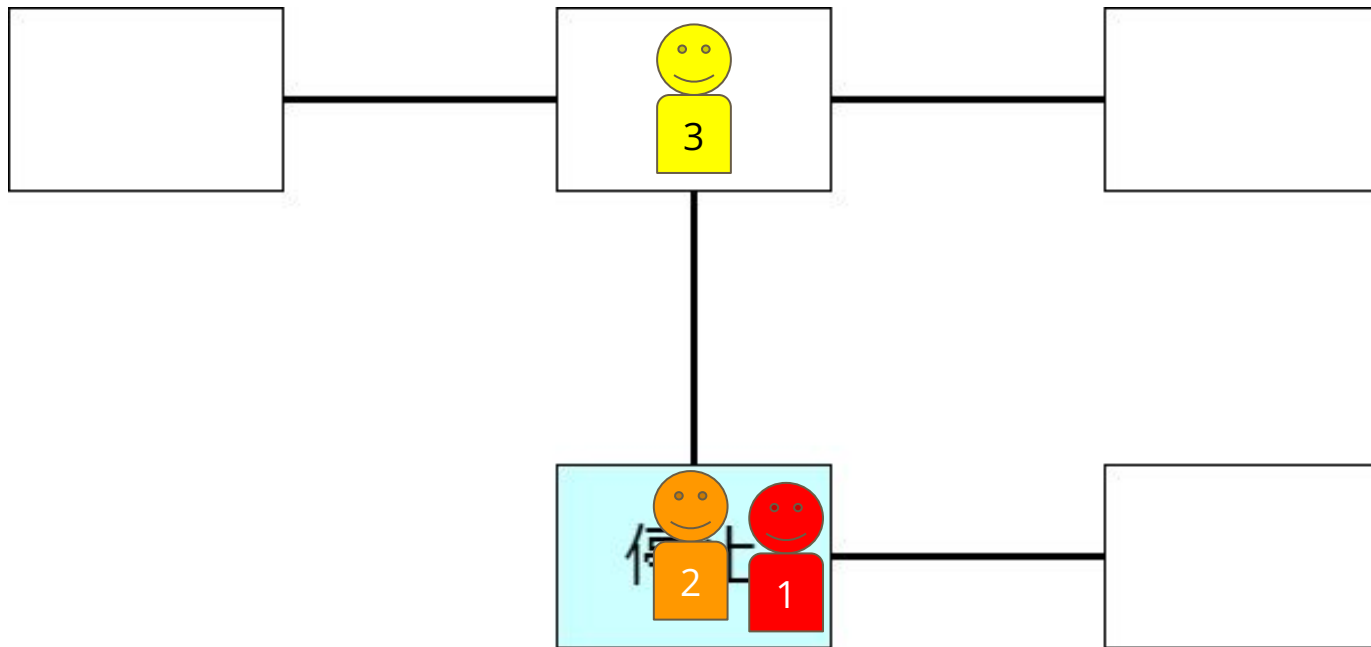
問題概要(動かし方のルール)



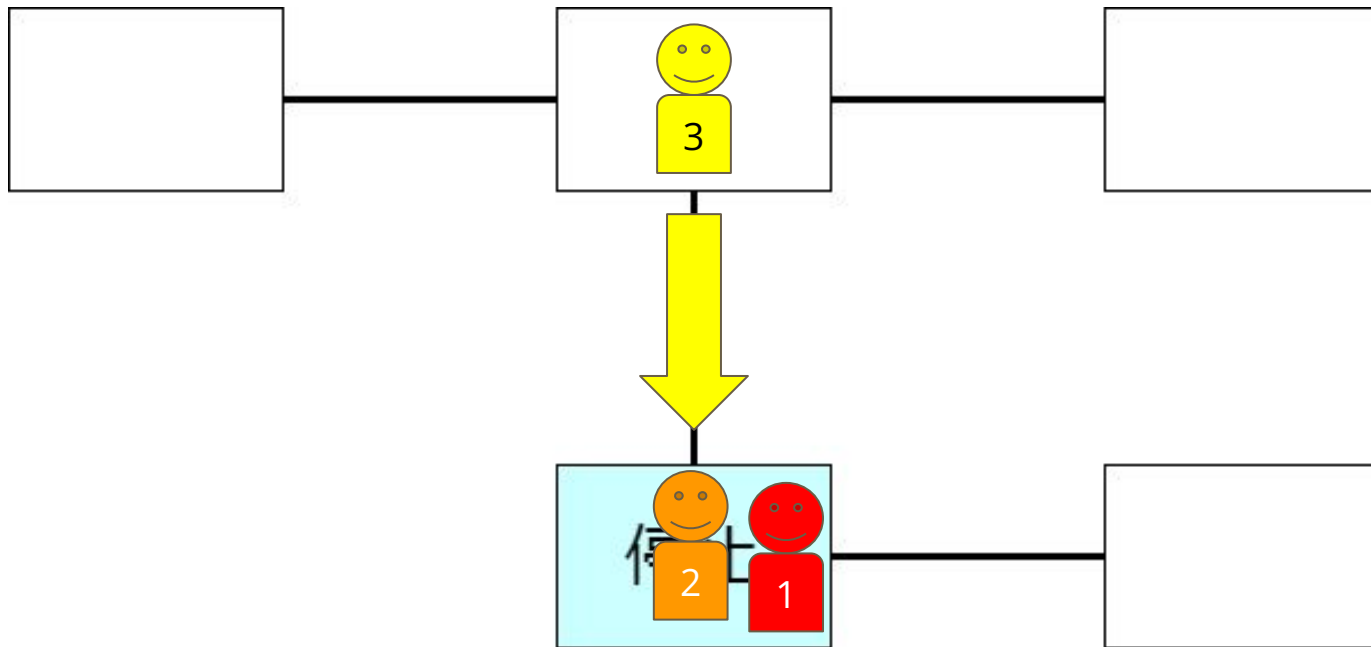
問題概要(動かし方のルール)



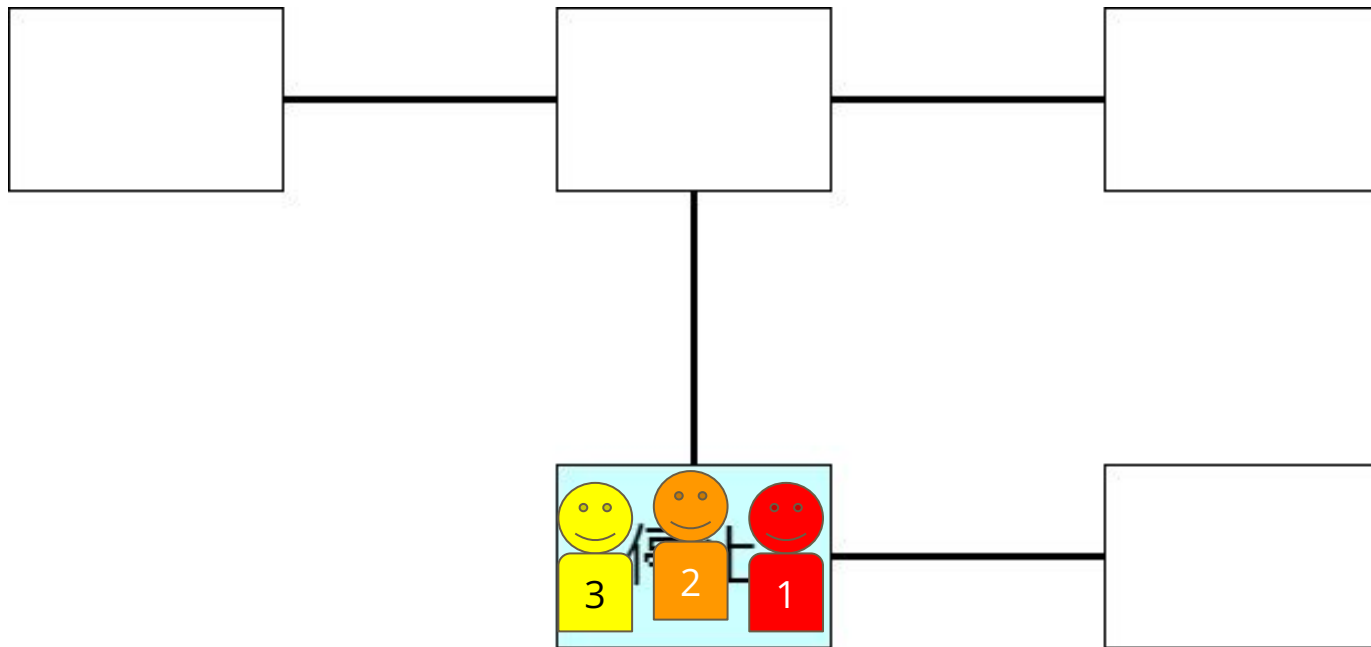
問題概要(動かし方のルール)



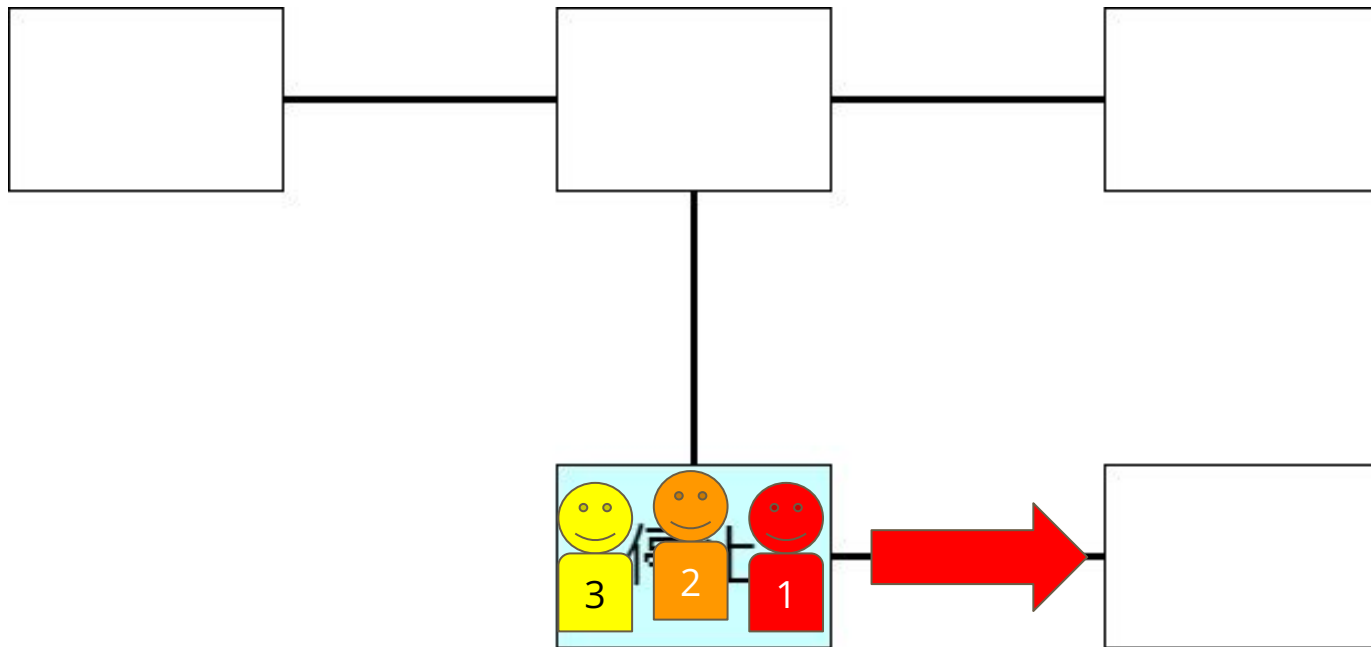
問題概要(動かし方のルール)



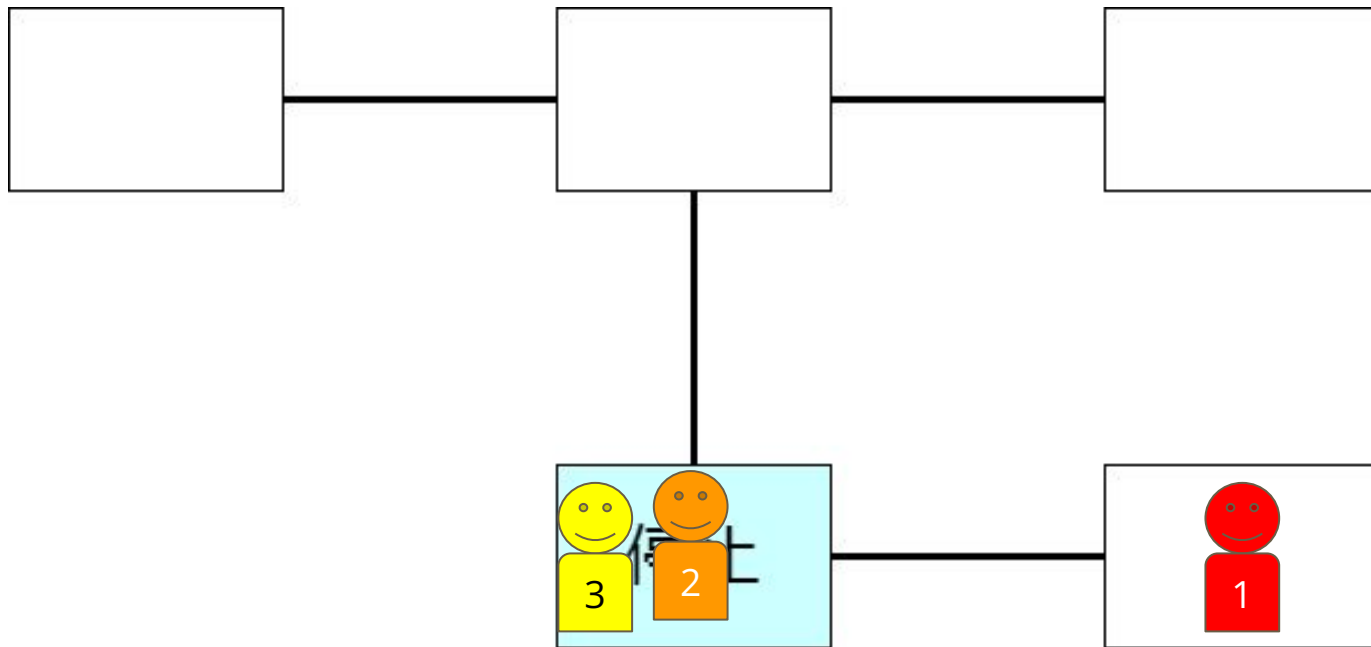
問題概要(動かし方のルール)



問題概要(動かし方のルール)



問題概要(動かし方のルール)



小課題1: 停止マスが0個 (JOI 3pts, JOIG 7pts)

小課題1: 停止マスが0個

- 単にプレイヤー 1 が動き続ける問題です.
- すべての辺重みが 1 であるような, 単一始点最短路問題
- 幅優先探索 (BFS) で解ける
- 計算量は $O(N+M)$

小課題1: 停止マスが0個

最短路を求める手法としては次が有名です

- (01) - BFS
- Dijkstra
- Warshall Floyd
- Bellman Ford, SPFA

計算量は？ 負辺はあってもよい？ 負閉路は検出可能？

小課題2: 停止マスが1個 (JOI 7pts, JOIG 12pts)

小課題2: 停止マスが1個

- プレイヤ 1 は次のどちらか
 - マス T まで停止マスを踏まずに進む
 - マス T まで停止マスを 1 回踏んで進む
- 他のプレイヤ
 - 停止マス(唯一)まで進む

小課題2: 停止マスが1個

- プレイヤ 1 は次の**どちらか** → **最後に考える**
 - マス T まで停止マスを踏まずに進む → **計算可能**
 - マス T まで停止マスを 1 回踏んで進む → **計算可能**
- 他のプレイヤー
 - 停止マス(唯一)まで進む → **計算可能**

小課題2: 停止マスが1個

【解法】

- プレイヤ 1: 停止マスを 0 回・1 回踏んで T に行くときの移動回数を計算する
- 他のプレイヤー: 停止するまでの移動回数を計算する

小課題2: 停止マスが1個

【解法】

- プレイヤ 1: 停止マスを 0 回・1 回踏んで T に行くときの移動回数を計算する

→ 2N 状態の最短路問題

- 他のプレイヤ: 停止するまでの移動回数を計算する

→ 停止マスを始点として最短路

小課題2: 停止マスが1個

【実装の注意】

停止マスの扱いに注意しよう

- プレイヤ 1 の場合: 到達する瞬間は数えない
- 他のプレイヤの場合: 停止マスから距離 0 だとターン終了まで 2 回かかる

小課題3: 停止マスが2個 (JOI 7pts, JOIG 12pts)

小課題3: 停止マスが2個

- プレイヤ 1 は次のどれか
 - マス T まで停止マスを踏まずに進む
 - マス T まで停止マスを 1 回踏んで進む
 - マス T まで停止マスを 2 回踏んで進む
- 他のプレイヤ
 - 停止マスのどちらかまで進む
 - 2 回目のターンは 1 or 2 手で終わらせる

小課題3: 停止マスが2個

- プレイヤ 1 は次の**どれか** → **最後に決める**
 - マス T まで停止マスを踏まずに進む → **計算可能**
 - マス T まで停止マスを 1 回踏んで進む → **計算可能**
 - マス T まで停止マスを 2 回踏んで進む → **計算可能**
- 他のプレイヤー
 - 停止マスのどちらかまで進む → **計算可能**
 - 2 回目のターンは 1 or 2 手で終わらせる → **計算可能**

小課題3: 停止マスが2個

答えを求めるには

- プレイヤ 1: 停止マスを 0 回・1 回・2 回踏んで T に行くときの移動回数を計算する

→ **3N 状態の最短路問題**

- 他のプレイヤ: 停止するまでの移動回数を計算する, 2 回目のターン終了にかかる時間を計算する

→ **最短路問題 2 回 / 停止マスが隣接しているかチェック**

小課題4 : N, M, K は 3000 以下 (JOI 19pts, JOIG 23pts)

小課題4: N, M, K は 3000 以下

これまでの解法の通り, プレイヤ 1 が停止マスを踏む回数 n を固定したときのことを計算することを考えましょう.

- **プレイヤ 1 について計算したいもの**
 - 停止マスを n 回踏んでマス T に到達するときの移動回数
- **他のプレイヤについて計算したいもの**
 - 自分のターンを n 回消化するためにかかる移動回数
- $n = 0, 1, \dots, N$ すべてに対してこれが求まれば解ける

小課題4: N, M, K は 3000 以下

- プレイヤ 1 について計算したいもの

- 停止マス n 回踏んでマス T に到達するときの移動回数

→ これは **N^2 状態の最短路問題**

$\text{dist}[k][T] :=$ マス T に到達し, 到達時点で停止を k 回行っているときの最小移動回数 (T での停止は含めない)

小課題4 : N, M, K は 3000 以下

- **他のプレイヤーについて計算したいもの**
 - 自分のターンを n 回消化するためにかかる移動回数

この部分について, 2 つの解法を紹介します. 「2 つめ」は小課題5のときに扱います.

小課題4: N, M, K は 3000 以下

次の N^2 状態の最短路問題と考えることができます.

- マス v で自分のターンを始めて, k ターン消化を目標としている状態 (v, k)
- v, w が隣接しているとき, (v, k) から (w, k) や $(w, k-1)$ に進める
- (v, k) から $(\text{any}, 0)$ への距離は？

小課題4: N, M, K は 3000 以下

次の N^2 状態の最短路問題と考えることができます.

- マス v で自分のターンを始めて, k ターン消化を目標としている状態 (v, k)
- v, w が隣接しているとき, (v, k) から (w, k) や $(w, k-1)$ に進める
- (v, k) から $(any, 0)$ への距離は？

→ **$(any, 0)$ から逆向きに最短路を求めればよい**

小課題1～4まとめ (JOI 36pts, JOIG 54pts)

ここまで解けていれば、問題に登場するすべての要素を正しく理解して計算に組み込んでいるといってい良いでしょう。

考えるべき状態は基本的なものばかりで、計算も BFS だけと基本的ですが、それでもすべての実装を正確にこなすのは結構大変だと思います。

少しだけ解説者なりに、**実装のためのアドバイス**をします。

実装のためのアドバイス

まず細かい計算式を正確に作るためには、変数の意味を正確に定義して、それに沿って考えることが大切です.

$\text{dist}[k][T] :=$ 停止マス k 回でマス T に居るときの移動回数



$\text{dist}[k][T] :=$ マス T に到達し, 到達時点で停止を k 回行っているときの最小移動回数 (T での停止は含めない)

実装のためのアドバイス

自分が定義した $\text{dist}[k][T]$ が、マス T で停止マスを踏むタイミングを考慮しているのか等を明確に答えられることを確認してから実装に臨みましょう. ソースコードにコメントを入れておくことも有効だと思います.

```
// dist[k][v]  
// -- v につくまでにターンエンドを k 回やった  
// -- v でのターンエンドは含まない  
vc<vc<int>> dist(N, vc<int>(N, infty<int>));
```

実装のためのアドバイス

実装ミスを減らすことも大事ですが、実装ミスをしたときのリカバリも超重要です。以下の 2 つを手早く実装できるようにしておくとても便利です。

愚直な解法

- なるべく問題文で言われている通りに実装する
- 計算量は気にしない

ランダムケース生成

- 制約を満たす小さいケースを適当に作る

実装のためのアドバイス

【愚直な解法】例えばコマの場所・次に動かせるコマの番号を状態とした最短路問題を解けばよいです．とにかく全部の状態を考えて最短路問題，というのはかなりよく使います．

```
using T = pair<vc<int>, int>; // コマの場所・次に動かせるコマ
map<T, int> dist;
deque<T> que;
auto upd = [&](vc<int> X, int k, int x) -> void {
    T t = {X, k};
    if (!dist.count(t)) { chmin(ANS[X[0]], x), dist[t] = x, que.push_back(t); }
};
upd(X, 0, 0);
while (!que.empty()) {
```

実装のためのアドバイス

【ランダムケース生成】私は Python をよく使います. 何らかの言語の乱数の扱いに慣れておきましょう. 自分の実装が扱えている通りのケースであれば厳密に問題の制約に合わせる必要也没有ありません. 画像のコードでは多重辺も作っています.

```
N = randint(2, 8)
M = randint(N - 1, N + 3)
K = randint(2, 4)
print(N, M, K)

for v in range(2, N + 1):
    print(randint(1, v - 1), v)
for _ in range(M - (N - 1)):
    a = randint(1, N - 1)
    b = randint(a + 1, N)
    print(a, b)

print("".join(random.choice("01") for _ in range(N)))
print(*[randint(1, N) for _ in range(K)])
```


小課題5: $K = 2$ (JOI 24pts, JOIG 26pts)

小課題5以降では小課題4までの解法の計算量を改善することが問われています.

小課題5: $K = 2$

- 小課題4での解法
- プレイヤ 1 について計算したいもの
 - 停止マスから n 回踏んでマス T に到達するときの移動回数

→ **計算したいものが N^2 通りもあるので無理**

n の範囲を絞り込んだりするのも難しい (n が巨大な解と n が小さな解の比較も明らかではない)

→ プレイヤ 1 のコスト・他プレイヤのコストを個別に求めようとすると厳しい

小課題5: $K = 2$

【解法の大筋】

- 他のプレイヤーの移動回数は, まあまあ簡単な n の関数になる.
- →「**総コスト** = プレイヤ 1 の移動回数 + 他のプレイヤーの移動回数」を直接扱える

小課題5: $K = 2$

【他のプレイヤーの移動回数について】

2 回目以降のターンの過ごし方にはどのようなものがあるか？

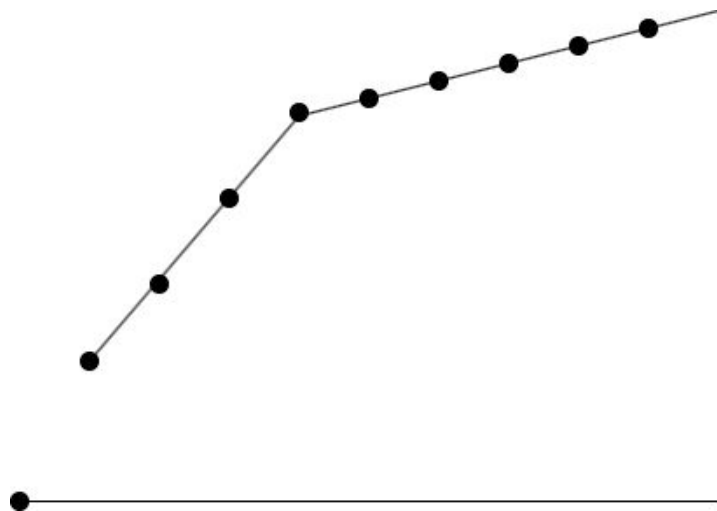
- 1 手かけてターンを終了する
- 2 手かけてターンを終了する
- 将来 1 手かけてターンを終了できるようになることを目指して移動

小課題5: $K = 2$

【他のプレイヤーの移動回数について】

だいたいこんな形の関数になる

- 0 回のときは 0
- 途中まで傾き 2
- 途中から傾き 1



小課題5: $K = 2$

【他のプレイヤーの移動回数について】

- 途中まで傾き 2 → 最短路問題(簡単)
- 途中から傾き 1

→ $dp[v] :=$ 頂点 v からスタートしてどこかでコストが $f(n)=n+c$ にできるような c の最小値

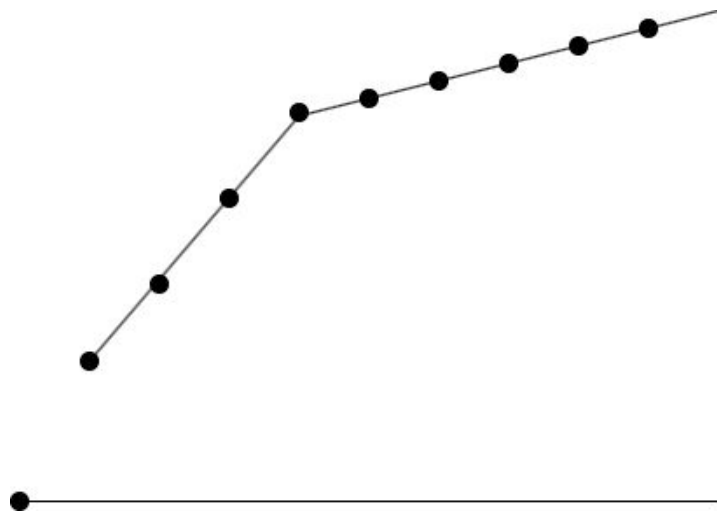
→ これも最短路問題みたいに計算できる

小課題5: $K = 2$

【他のプレイヤーの移動回数について】

左の方と右の方で挙動が違う

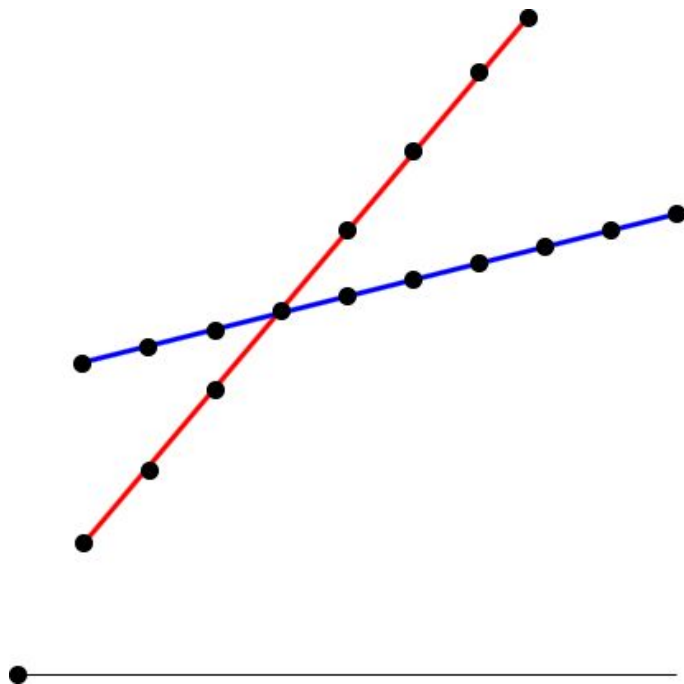
→ 結局プレイヤー 1 の停止回数を
正確に把握する必要がないか？



小課題5: $K = 2$

【他のプレイヤーの移動回数について】

- 最小移動回数は 2 つの min
- min じゃない方も好きなように選べることにしてしまっって解けばよい



小課題5: $K = 2$

【解法】

- プレイヤ 2 の最小移動回数の関数を表す 2 つの 1 次関数を求める.
- それぞれの 1 次関数を使うと決めてしまつて 2 回解く.
- 両プレイヤーの移動回数の総和が, $2N$ 状態の最短路問題で解けるようになる.

小課題5: $K = 2$

【解法】

1 次関数 $an + b$ を使うとき, v から w への移動について,

- 間にプレイヤー 2 のターンがはさまるとき: コスト $1 + a$
- プレイヤ 1 のターンが継続するとき: コスト 1

として最短路を計算すればよい.

($S[v]$ が 1 かつ v が $X[0]$ ではないときコスト a 発生)

小課題5: $K = 2$

【計算量】

- Dijkstra $\rightarrow O((N+M)\log N)$
- コストが 2 種類しかないことを使って $O(N+M)$ 時間で解くこともできる
 - 最後の移動コストが c_1 のものを入れておく que_1
 - 最後の移動コストが c_2 のものを入れておく que_2
 - 2 つの que の先頭のうち小さいものを取り出す

小課題5: $K = 2$

【実装注意点】

- 停止 0 回とその他は区別する必要があります.
- 初期位置が停止マスの隣であるとき, 次のような関数になります.

停止回数	0	1	2	3	4
移動回数	0	1	3	5	7

小課題6 : $K \leq 100$ (JOI 9pts, JOIG 7pts)

小課題6: $K \leq 100$

小課題 5 の解法がそのまま $K \leq 100$ に一般化可能です.

プレイヤー 2, 3, 4, ... の移動回数の総和は,

- 各プレイヤーは傾き 1, 2 の 1 次関数を持っている.
- 各プレイヤーが使う 1 次関数をひとつずつ選んで足す.

として形で考えることができます.

小課題6: $K \leq 100$

プレイヤー 1 以外の戦略は、傾き $K-1, K, \dots, 2(K-1)$ の 1 次関数によって表すことができます。

これらの 1 次関数に対して小課題5と同様にして答を計算できます。

$O(K(N+M))$ 時間で解けます。

小課題7 : $N, M, K \leq 30000$ (JOI 23pts, JOIG 12pts)

小課題8 : $N, M, K \leq 50000$ (JOI 9pts, JOIG 4pts)

最後の 2 つの小課題は制約が類似しており、必要な考察に大きな違いはありません。

小課題7 : $N, M, K \leq 30000$

小課題8 : $N, M, K \leq 50000$

- 小課題4での解法
- プレイヤ 1 について計算したいもの
 - 停止マスから n 回踏んでマス T に到達するときの移動回数

→ 計算したいものが N^2 通りもあるので無理

n の範囲を絞り込んだりするのも難しい (n が巨大な解と n が小さな解の比較も明らかではない)

小課題7 : $N, M, K \leq 30000$

小課題8 : $N, M, K \leq 50000$

- 小課題4での解法
- プレイヤ 1 について計算したいもの
 - 停止マスから n 回踏んでマス T に到達するときの移動回数

→ 計算したいものが N^2 通りもあるので無理

n の範囲を絞り込んだりするの難しい (n が巨大な解と n が小さな解の比較も明らかではない)

→ **しかし K が大きければ n の範囲を絞り込める**

小課題7 : $N, M, K \leq 30000$

小課題8 : $N, M, K \leq 50000$

- 停止マス 1 回あたり, 最低でもコスト $K-1$ がかかる.
- 停止マスを n 回余分に使った場合, プレイヤ 1 の移動回数で $n(K-1)$ 回得しないと損失回復できない.
- 停止マスの回数は最小回数から見て $+N/(K-1)$ 以下であるときに限定できる.

$O(N^2/K)$ 状態の最短路問題 $\rightarrow K$ が大きければ高速.

小課題7 : $N, M, K \leq 30000$

小課題8 : $N, M, K \leq 50000$

【解法まとめ】

- それぞれの他プレイヤーのコスト関数を求める
- それらを足し合わせたコスト関数を求める(累積和などを使う)
- K が小さい(or コスト関数の傾きの種類数が少ない) 場合
→ 小課題6の解法を使う
- K が大きい(or コスト関数の傾きの種類数が多い)
→ 停止マスの回数も状態とした最短路問題
→ 適切に枝狩りをすると $O(N^2/K)$ 状態
- 適切に使い分けると, $O((N+M)^{1.5})$ 時間で解ける.

小課題7 : $N, M, K \leq 30000$

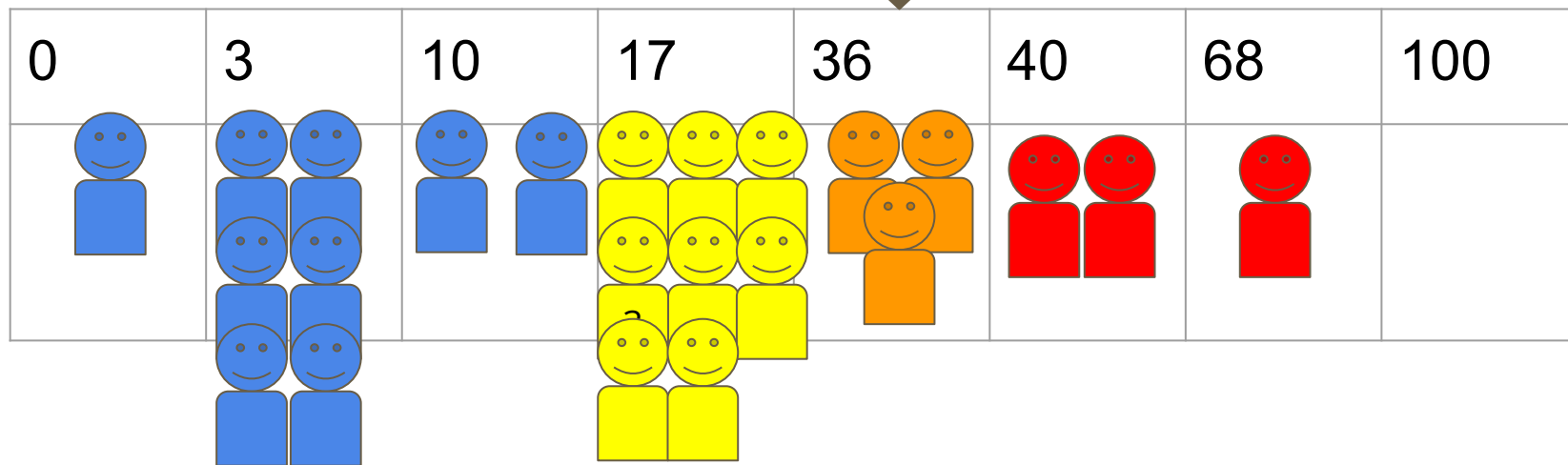
小課題8 : $N, M, K \leq 50000$

ある程度高速な実装をすれば小課題8まで通ります. TLE してしまった場合には以下のような高速化手法をいくつか試すとよいでしょう.

- 2 種の解法のしきい値を調整
- Dijkstra 法を避けて線形時間の最短路アルゴリズムを使う
- 枝狩りの計算式をより丁寧にやる
- 最後の解法を $O(N)$ メモリで実装する
- ...

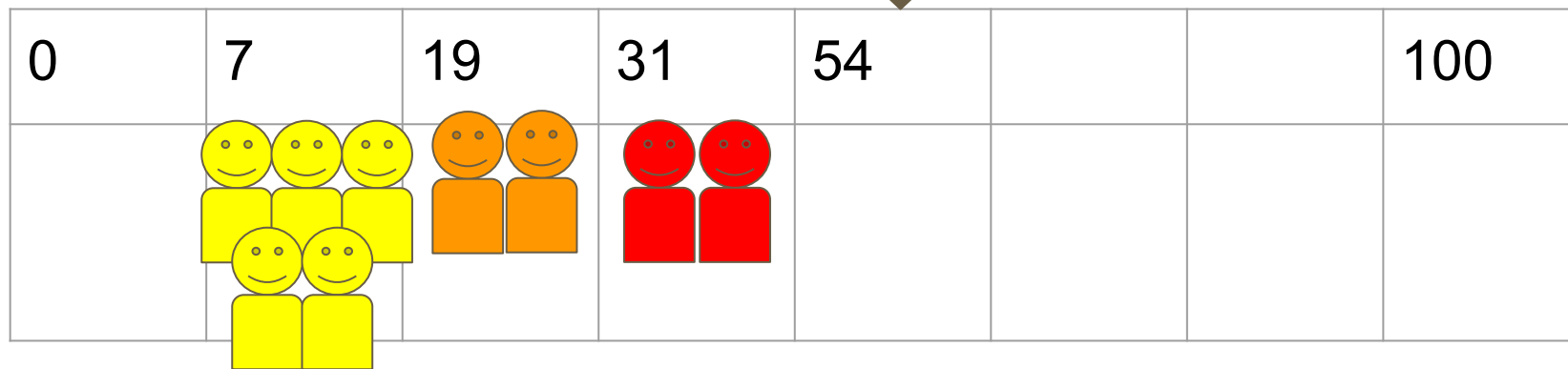
得点分布(JOI, 23人)

小課題1～4



得点分布 (JOIG, 9人)

小課題1 ~ 4



Tricolor Lights 解説

ynymxiaolongbao

問題概要

- AnnaとBrunoの二人が協力して、ディ太郎に勝ちたい
- 長さ N のボードがあり、Annaは各場所をR, G, B のいずれかの色で塗る
- 各場所には一色だけ禁止色 s_i があり、その色では塗れない

R G G B R B B G

問題概要

- Annaが整数 L を指定する
- ディ太郎がボードの長さ L の区間を切り取り、その色の並びをBrunoに伝える
- Brunoはディ太郎が切り取った場所がどこかを答える

RGGBRBBG

→答え3

←→
長さ5

問題概要

- Lは大きくした方が簡単、Nは小さい方が簡単
- 小課題 4 まで：Lの上限は130固定で、Nが段々大きくなる
- 小課題 5 ：Nが大きい値で固定で、Lの上限が段々小さくなる

R G G B R B B G

→ 答え 3

← 長さ 5 →

小課題 1 ($N \leq 131 = L + 1$)

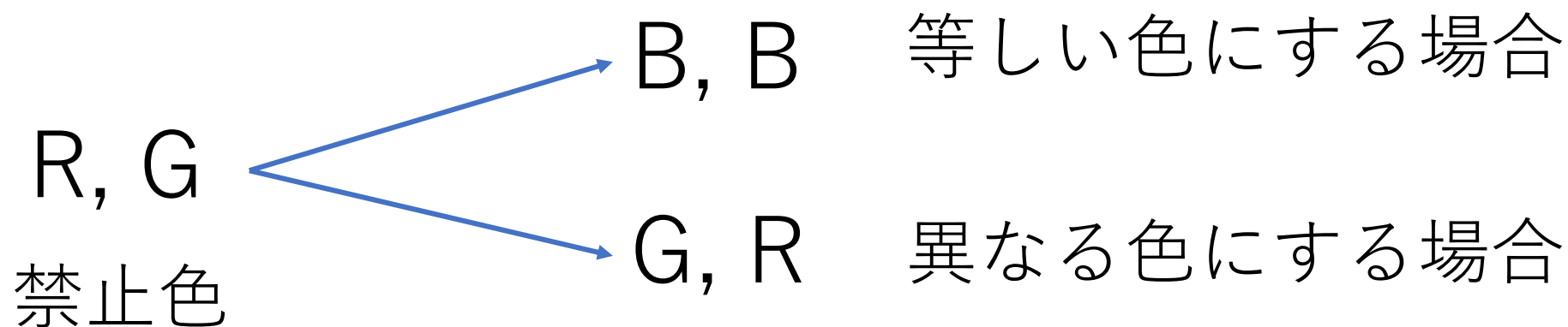
- $N \leq 130$ のとき、 $L = N$ に設定すれば答えはいつも1
- $N = 131$ のとき、 $L = 130$ にすると、1~130なのか2~131なのか判定できれば良い
- 思ったより難しい

小課題 1 ($N \leq 131 = L + 1$)

- 気持ち：一つの桁ではちゃんと情報を伝えられない
- たとえば、「この桁がRだったらこういう意味」と決めておいたとしても、その桁をRにすることが禁止されてると、そうできない
- →二つの桁を使って情報を伝える

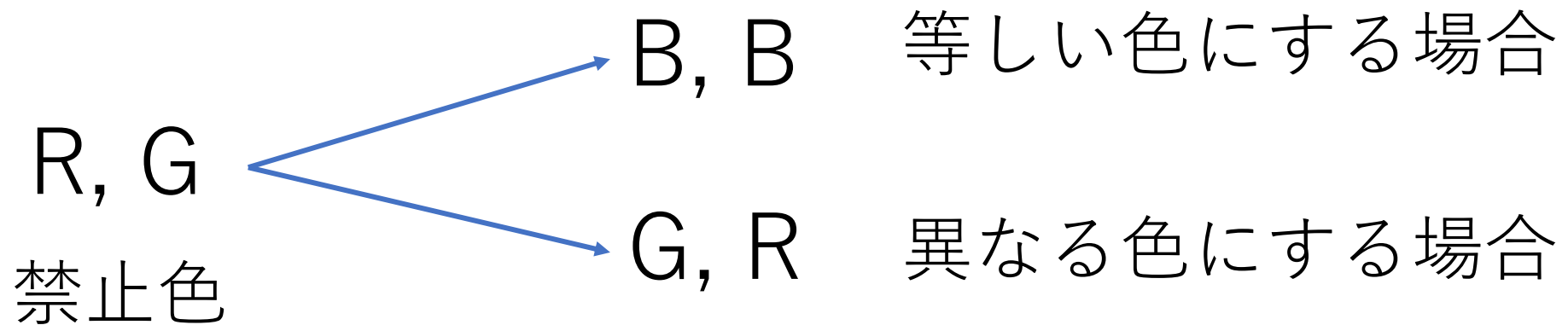
小課題 1 ($N \leq 131 = L + 1$)

- 二つの桁を使って情報を伝える
- 二つの桁が等しいか／異なるか を自由に決められる



小課題 1 ($N \leq 131 = L + 1$)

- 等しくする場合：禁止されている色はR,G,Bのうち高々二種類
→残りの色を選んで両方それで塗る
- 異なるようにする場合：塗り方は $2 \times 2 = 4$ 通りあるが、等しい色にする方法は3通りしかないなので、少なくとも1通りでは異なる色になる（鳩ノ巣原理）



小課題 1 ($N \leq 131 = L + 1$)

- Anna (色を塗る人) は、1桁目と130桁目を同じ色に塗って、2桁目と131桁目を異なる色に塗る
- Bruno (場所を当てる人) は、受け取った色の並びの左端と右端が同じ色かを見て、同じなら1、異なるなら2と答えれば良い

小課題 2 ($N \leq 250 \sim 2L$)

- 隣り合う二つが等しいか異なるかに注目
- 値が同じことを0、異なることをXと書く
- Anna（色を塗る人）：隣あう二つの違いが図のように並ぶようにする

$X \cdots X \ 0 \ X \cdots X$



128 128

小課題 2 ($N \leq 250 \sim 2L$)

- まず0になる（色が等しい）二か所の色を決める
- Xになる（色が異なる）場所は、0に近い方から順に、三色のうち隣の色・禁止色のどちらでもない色を選んで行く



小課題 2 ($N \leq 250 \sim 2L$)

- Bruno (場所を当てる人) : 場所が $1, 2, \dots, 129$ のとき、0がある位置は $129, 128, \dots, 1$ になっているので、 $130 -$ 「0がある位置」で場所が分かる

$X \cdots X \ 0 \ X \cdots X$



128 128

小課題 3 ($N \leq 380 \sim 3L$)

- Anna : 隣あう二つの違いが図のように並ぶようにする
- ?はAnna制御できずよく分からなくなった値とする
- ?は0かXのどちらかとしてBrunoに伝えられる

$X \cdots X \ ? \ 0 \ X \cdots X \ ? \ 0 \ X \ ? \ 0 \ X \cdots X$

126 122 125

小課題 3 ($N \leq 380 \sim 3L$)

- Bruno : 0が二回以上出てくるので、その距離を見てどこの0か判定
- 値が0のとき、Annaが0と?のどちらで送ったか分からない
- 本当の0の次は必ずXで、?の次は必ず0なので、分かる

X.....X ? 0 X.....X ? 0 X ? 0 X.....X



126 122 125

小課題 4 ($N \leq 7000 \sim L^2/2$)

- 0の間の距離が $L-7, 3, L-8, 4, L-9, \dots, L/2$ になるようにする
- どの長さ L の区間にも0が2つ以上出てくるので、その距離を見るとどここの0か分かる

...X.....X ? 0 X.....X ? 0 X...X ? 0 X.....X...



The diagram shows a sequence of characters: "...X.....X ? 0 X.....X ? 0 X...X ? 0 X.....X...". Below this sequence, there are four orange curved brackets, each spanning the distance between two consecutive '0's. Under each bracket is a number: 20, 101, 21, and 100. These numbers represent the distances between the 0's in the sequence.

20 101 21 100

小課題 5 ($N \leq 5 \times 10^5, L = 28 \sim 130$)

- $L = \log N \times \text{定数}$ くらいにしないといけない
- 2進数とか3進数を使う
- 大ブロックに分ける解法
 $L = 130 \rightarrow 60 \rightarrow 40 \rightarrow 33 \rightarrow 31 / 32$ (天才)
- 小ブロックに分ける解法 (de Bruijn 列)
 $L = 51(\text{乱択}) \rightarrow 34 \rightarrow 31 \rightarrow 29 \rightarrow 28$

大ブロックに分ける解法

$L = 130 \rightarrow 60 \rightarrow 40 \rightarrow 33 \rightarrow 31 / 32$ (天才)

$L = 130$ (40点)

- * を、Annaが自由に決めた0かXの値とする
- 図の長さ59のブロックを繰り返す
- **...* の部分でブロックの番号を2進数14桁で書く

? *..... ? * ? 0 X.....X

14×2

29

$L = 130$ (40点)

- 0の後にXが29個並ぶのはブロックの右端の部分だけ
→どこからブロックが始まるか分かる
→そこに書いてある2進数を見て、そのブロックの番号を求める
→どこを切り出したのか分かる

? * ? * ? 0 X X

14×2

29

$L = 60$ (50点)

- ブロックが完全に含まれている必要はない
- 困るのは、ブロックが途中で切れて、図のようになるとき
- 二進数が完全には読めない

? *... ? * ? 0 X.....X ? *... ? *

$L = 60$ (50点)

- ブロックの番号 x の下 k 桁と $x + 1$ の上 $14 - k$ 桁が分かる
- x の下 k 桁が $1 \cdots 1$ なら、 x の上 $14 - k$ 桁は、 $x + 1$ の上 $14 - k$ 桁 -1
- そうでない場合、 x の上 $14 - k$ 桁は $x + 1$ の上 $14 - k$ 桁と同じ
→ブロックの番号が分かる

? *... ? * ? 0 X.....X ? *... ? *

$L = 40$ (58点)

- 2進数ではなく 3進数が使える
- R, G, Bを0, 1, 2とみなして、その差 (mod 3) を自由に決められる

? * ? * ? 0 X X

9×2

19

$L = 40$ (58点)

- * を0,1,2のAnnaが自由に決められる値とする
- 図の長さ39のブロックを繰り返す
- **...* の部分でブロックの番号を3進数9桁で書く

? *..... ? * ? 0 X.....X

9×2

19

$L = 33$ (80点)

- 前の解法は、右端以外の所で1が沢山連続しなければ良い
- $** \dots *$ の部分でブロックの番号を3進数9桁で書く

? * ? * ? 0 X X

9×2

19

$$L = 33 \quad (80\text{点})$$

- 図のように、間に3つごとに0を挟めばよい（9桁なので、ちょうど3つずつに分かれる）
- 右端で必要な $X \cdots X$ の長さが減る

$? * \cdots ? 0 ? * \cdots ? 0 ? * \cdots ? 0 X \cdots \cdots X$

$3 \times 2 \quad 3 \times 2 \quad 3 \times 2 \quad 8$

$$L = 31 \quad (88\text{点})$$

- 前の解法では、3進数9桁の間に0を2つ挟んで、*の部分でXが4連続しないようにした
- 3進数10桁であってXが4連続しないものを選ぶと、減る

$L = 32$ (天才) (84点)

- 3進数が書けるが、あえて1と2だけを使って2進数14桁を書く
- * を、Annaが自由に決められる1か2の値とする
- 右端が工夫されている

? * ? * ? X 0



14 × 2

$L = 32$ (天才) (84点)

- 0になりうるのは 0 と ? の場所
- 0が一つしかない場合：そこが右端の0
- 0が二つ以上ある場合：右端の0だけ、次の0との距離が奇数
→ 右端の0の位置が分かる

? * ? * ? X 0



14 × 2

小ブロックに分ける解法 (de Bruijn 列)

$L = 34 \rightarrow 31 \rightarrow 29 \rightarrow 28$

de Bruijn 列

- k 進数 N 桁のde Bruijn 列… k 進数 n 桁の文字列は k^N 通りあるが、それらがちょうど一回ずつ長さ N の文字列として現れるような列
- これは巡回させることができる
- 以下のグラフの一筆書きに対応する
- 頂点： k 進数 $N-1$ 桁の各文字列
- 辺： k 進数 N 桁の各文字列に対応し、その前 $N-1$ 桁から後ろ $N-1$ 桁の頂点へ有向辺

de Bruijn 列

- 有向グラフが一筆書きできる条件…連結&すべての頂点で入次数と出次数が等しい
- 前頁のグラフはその条件を満たす
→de Bruijn 列は存在

de Bruijn 列の拡張

- 逆に、グラフがその条件を保っていれば、de Bruijn 列のようなものが作れる
- k 進数 N 桁の文字列のうち、いくつかを取り除いても、それらがちょうど一回ずつ出てくるような列が作れることがある
- 例えば、「3進数11桁の文字列のうち、0が一回以上出てくるもの」についても、それらがちょうど一回ずつ出てくるような列が作れる（グラフが、連結で各頂点の入次数と出次数が等しいため）
- これを拡張 de Bruijn 列と呼ぶことにする

$L = 34$ (76点)

- ? * X を1ブロックとしてこれを繰り返す
- * の部分には、「3進数11桁のうち一回以上0が出てくるもの」
からなる拡張de Bruijn 列を書く

? * X

? * X

⋮

? * X

$L = 34$ (76点)

- 11ブロック分見て、mod 3ごとに、一回以上0が出てくるか見る
- 結果がxoxのとき、* になっている余りはoの所
- 結果がooxのとき、* になっている余りは右側のoの所
- Brunoは拡張de Bruijn列から整数を復元すれば良い

? * X

? * X

:

? * X

$L = 31$ (88点)

- $? * ? * X$ を1ブロックとしてこれを繰り返す
 - $*$ の部分二つには、
 - 「3進数6桁のうち一回以上0が出てくるもの (665個)」
 - 「そこから0...0を除いたもの (664個)」
 - かなる拡張de Bruijn 列をそれぞれ繰り返し書く
 - 6ブロック分見て、mod 5ごとに、一回以上0が出てくるか見る
 - 結果がNoになる余りは?とXの所だけ
- 前の $L=32$ 解法と同様に、次の場所との距離が奇数な所がX

$$L = 31 \quad (88\text{点})$$

- 周期が665, 664 のde Bruijn 列を合わせると、周期は $\text{LCM}(665, 664)$ になる
- Brunoは得た二つの整数から中国剰余定理を使って場所を復元
- $5 \times \text{LCM}(665, 664) \geq 500000$ で、通る

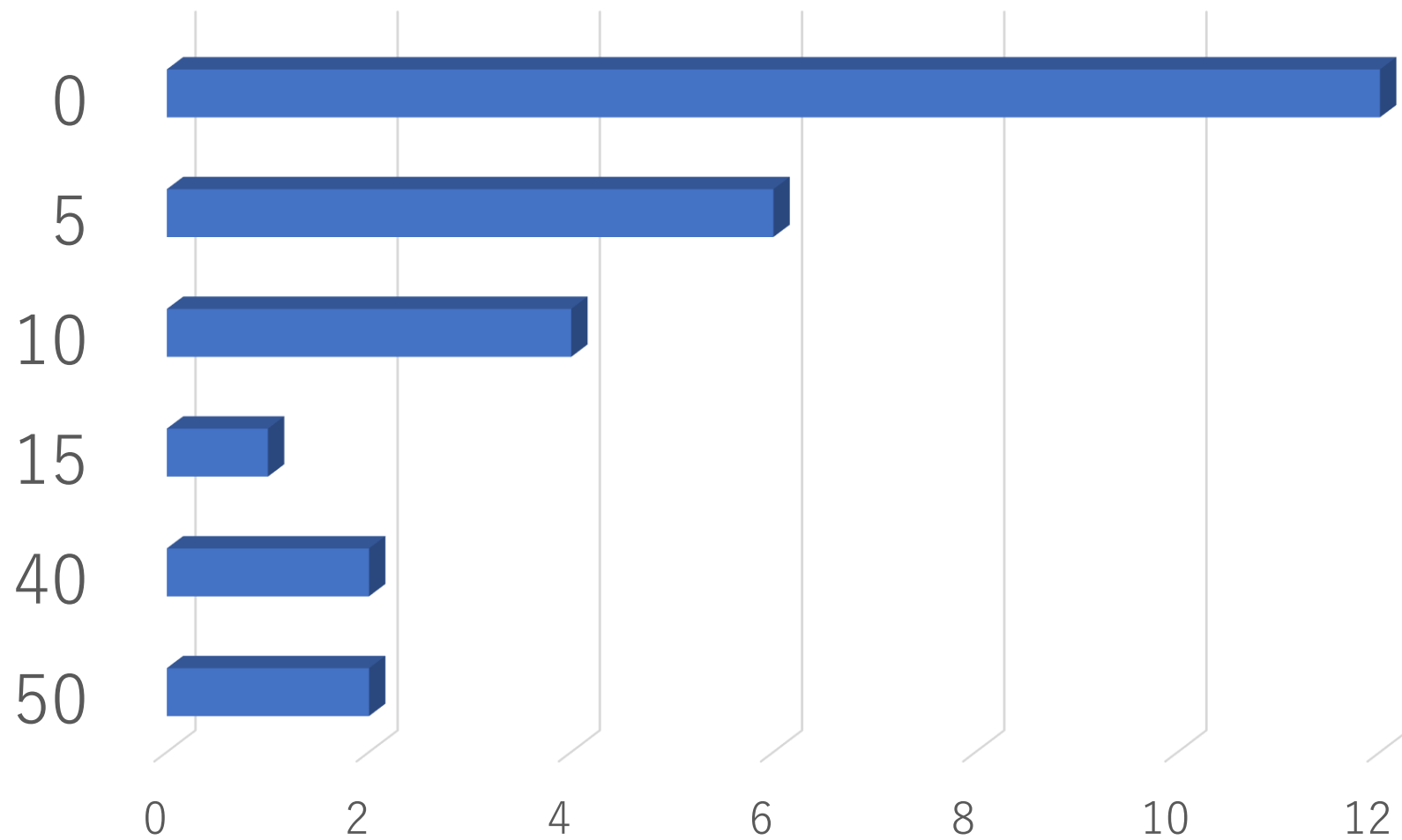
$L = 29$ (96点)

- $? * ? * ? * X$ を1ブロックとしてこれを繰り返す
- $*$ の部分三つには、
- 「3進数4桁のうち一回以上0が出てくるもの (65個)」
- 「そこから0000を除いたもの (64個)」
- 「そこから0101と1010を除いたもの (63個)」
- かなる拡張de Bruijn 列をそれぞれ繰り返し書く
- 余りの特定、復元 (中国剰余定理) も先ほどと同様
- $7 \times \text{LCM}(65, 64, 63) \geq 500000$ で、通る

$L = 28$ (100点)

- $? * ? * ? * ? * X$ を1ブロックとしてこれを繰り返す
- $*$ の部分四つには、
- 「3進数3桁のうち一回以上0が出てくるもの (19個) 」
- 「010,101を除いたもの (17個) 」
- 「010,101,000を除いたもの (16個) 」
- 「010,101,020,202を除いたもの (15個) 」
- かなる拡張de Bruijn 列をそれぞれ繰り返し書く
- $9 \times \text{LCM}(19, 17, 16, 15) \geq 500000$ で、通る

得点分布 (JOI)



得点分布 (JOIG)

10点 2人

0点 7人

JOI 2023/2024 春季トレーニング Day2

とてもたのしいたのしい家庭菜園

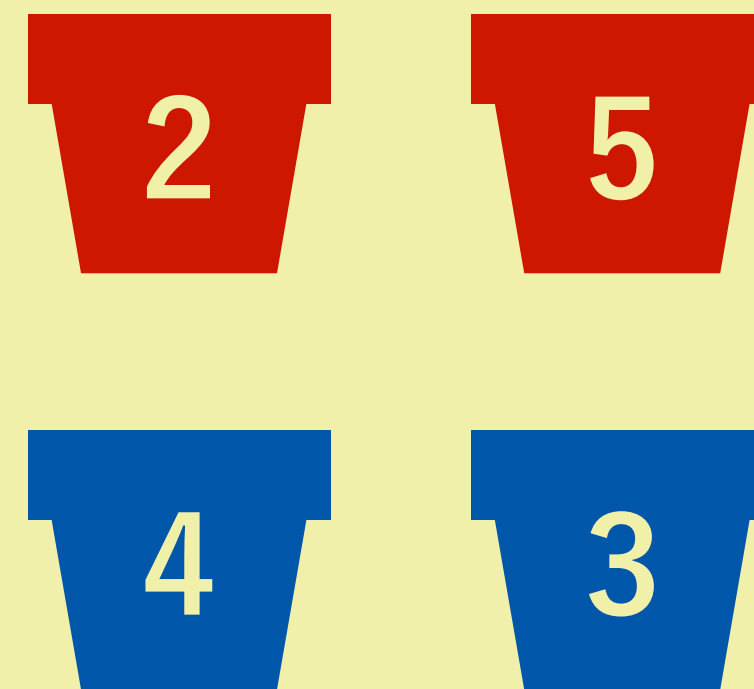
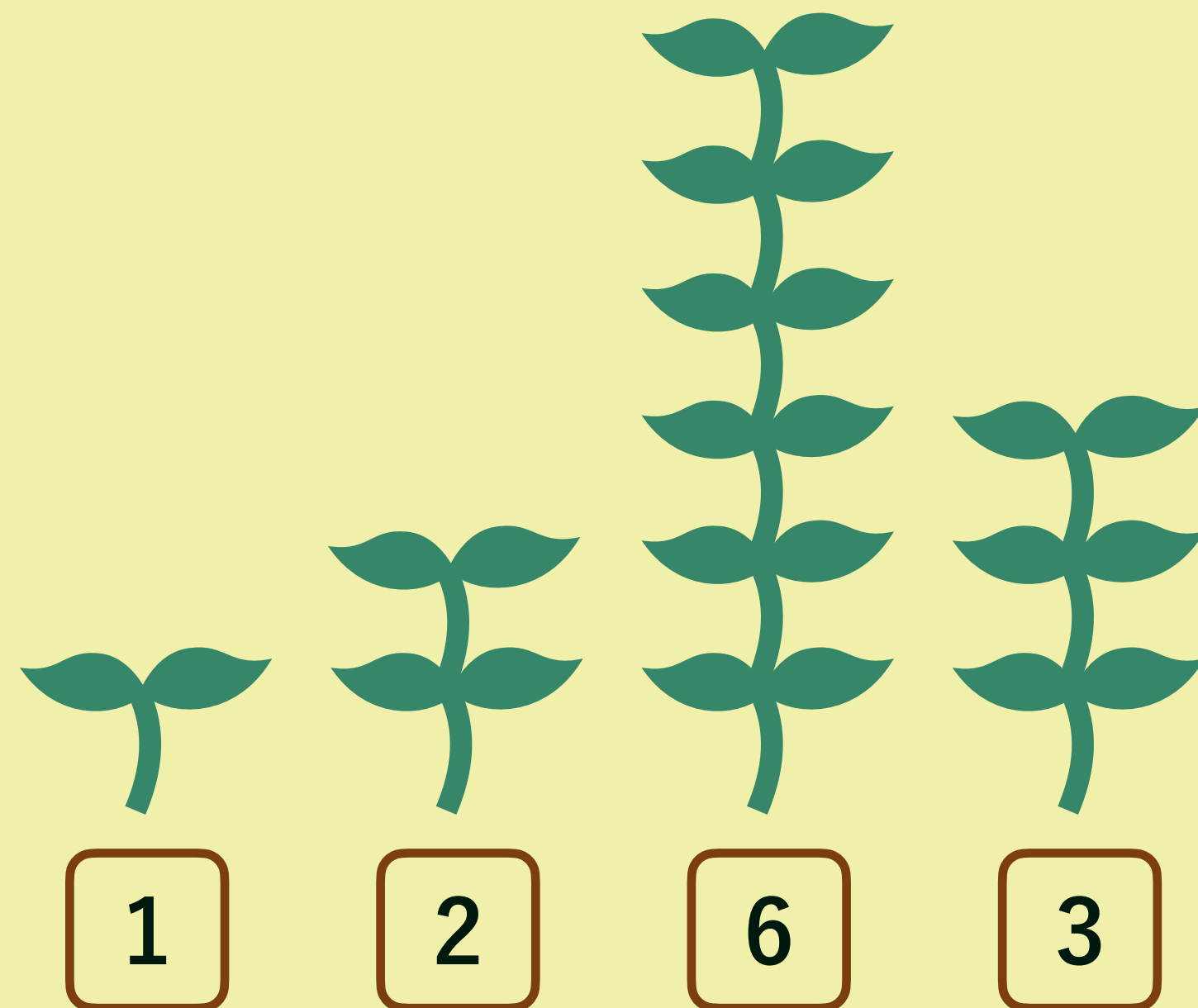
Growing Vegetables is Fun 5

解説 (解説担当 : KoD)



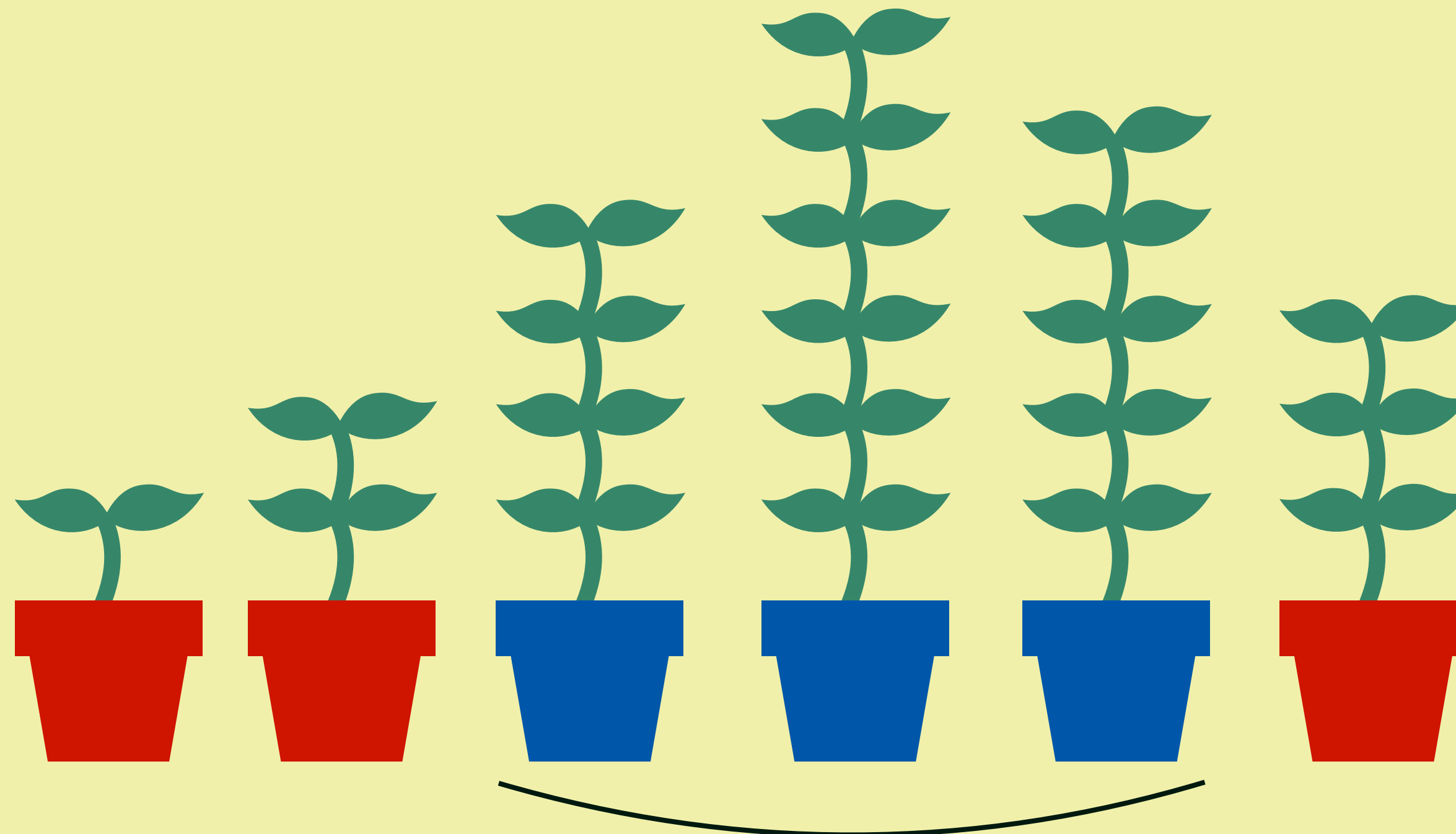
問題概要

- $2N$ 個の苗, N 個の赤い植木鉢, N 個の青い植木鉢がある
 - N は 300 000 くらい
- それぞれに大きさが定まっている
 - 苗の大きさは山なりに並ぶ



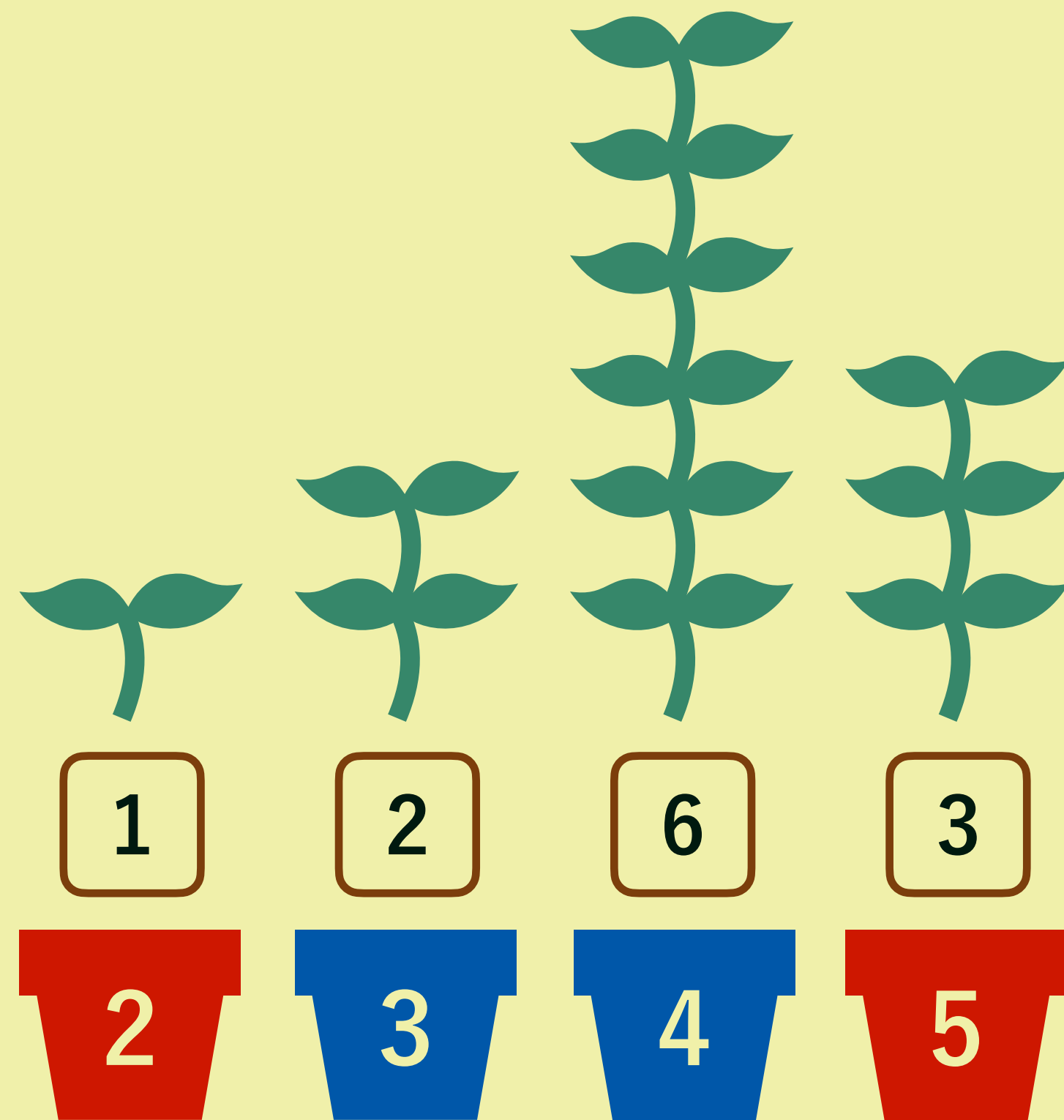
問題概要

- それぞれの苗に植木鉢を割り当てる
 - どれか N 個の連続する植木鉢が同じ色でなければならない (美しい並び)



問題概要

- 「苗と植木鉢の大きさの差の最大値」を最小化したい



差（育てにくさ）：左から順に 1, 1, 2, 2

最大値（疲労度）： $\max(1, 1, 2, 2) = 2$

小課題 1 ($N \leq 5$)

- ・ 植木鉢の並び $(2N)!$ 通りを全て試す
- ・ それぞれ美しい並びになっているか確認し, になっているなら疲労度を計算
- ・ 計算量は $O((2N)! \times N)$



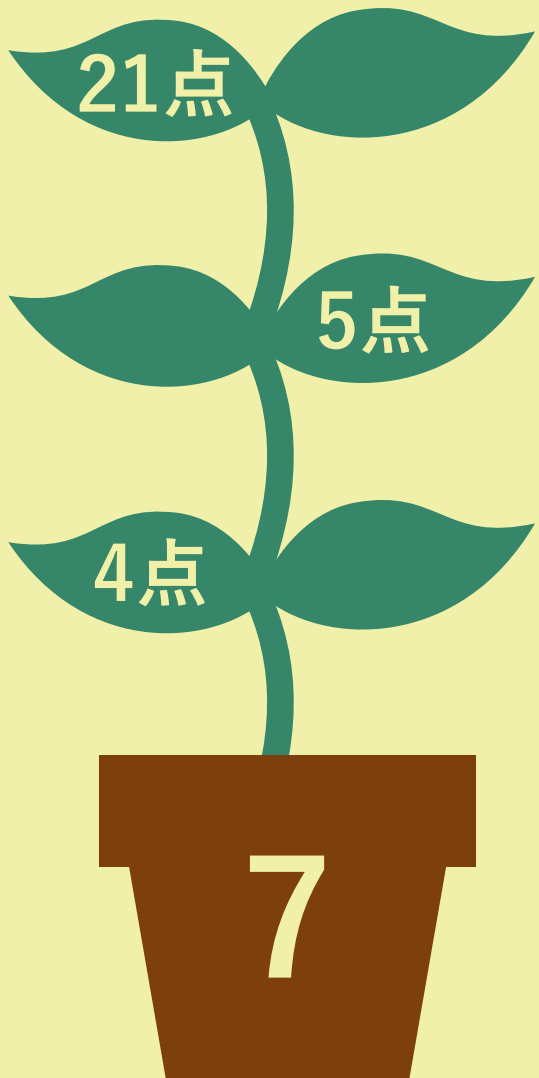
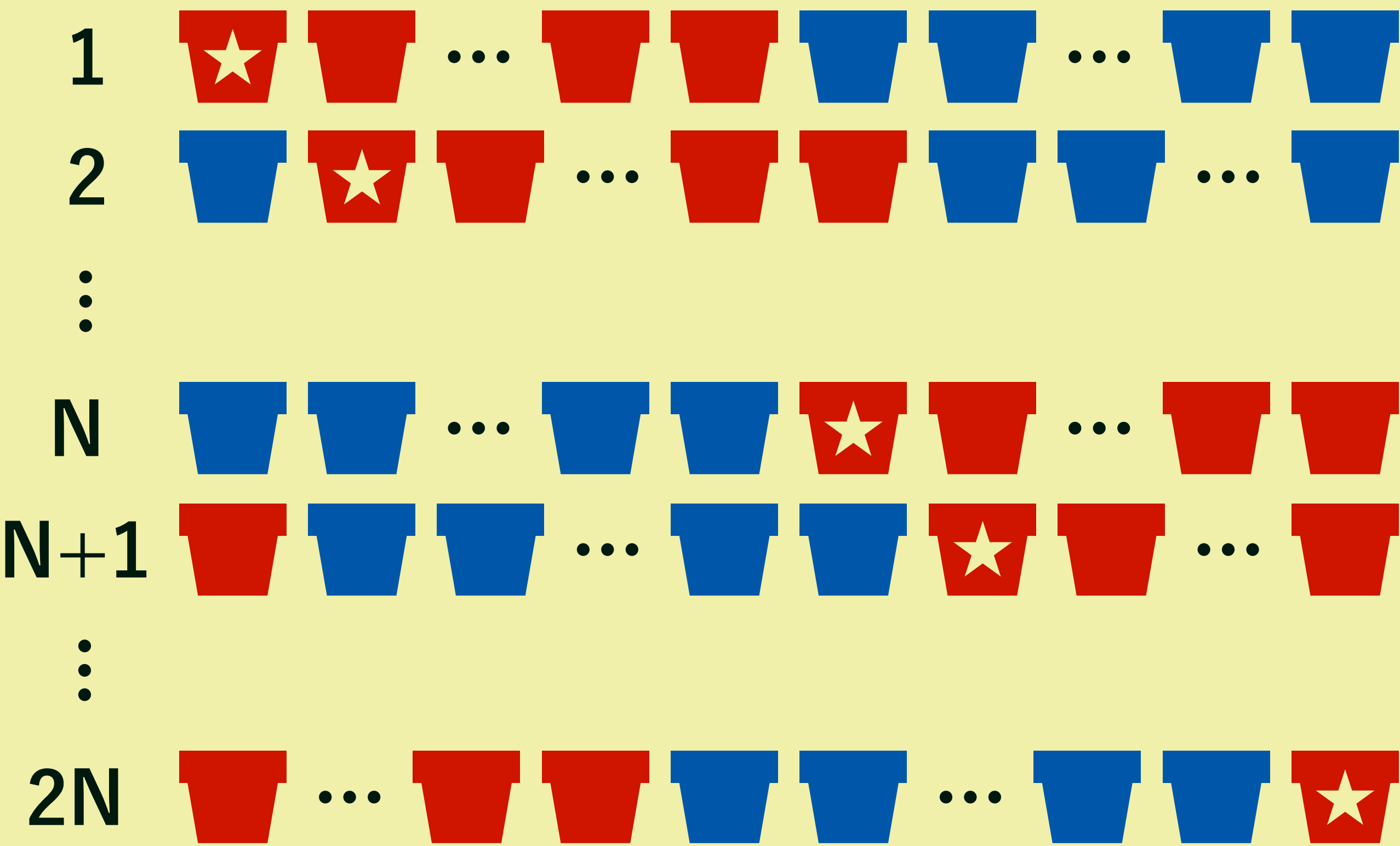
小課題 2 ($N \leq 10$)

- ・ 小課題 2 の順列全探索の代わりに bit DP を行う
- ・ 詳しい説明は割愛



小課題 3 ($N \leq 2\,000$)

- 色だけに注目すると, ありうる配置は $2N$ 通り
(★は青から赤に切り替わる位置)

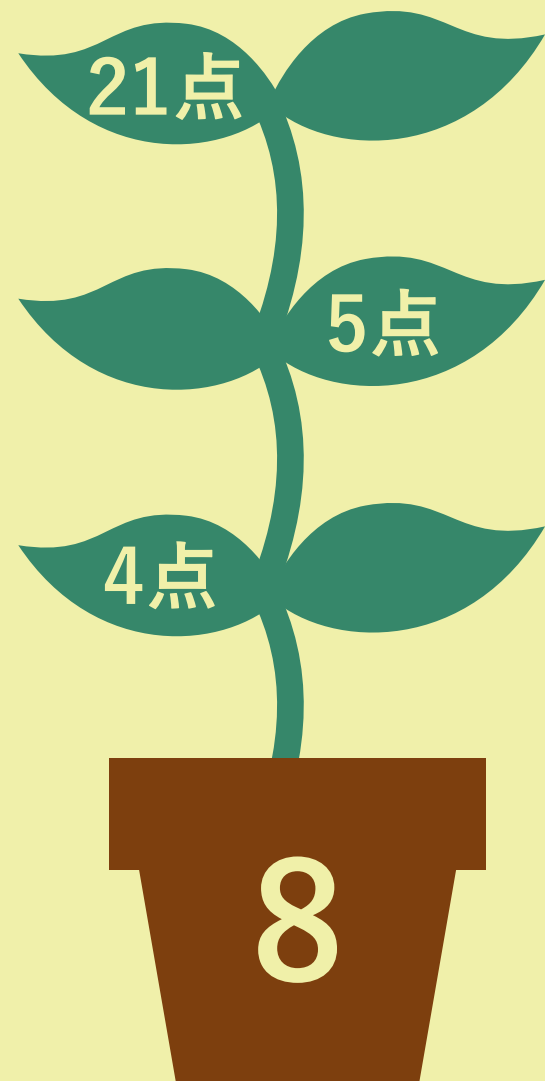


小課題 3 ($N \leq 2\,000$)

- これら $2N$ 通りを全て調べる
- 各色の植木鉢がどの苗の区間に対応するか決まるので、以下の問題が解ければよい

<問題>

長さ N の数列 X, Y が与えられる。 Y の要素を好きに並べ替えるとき、 $\max(|X_i - Y_i|)$ を最小化せよ。

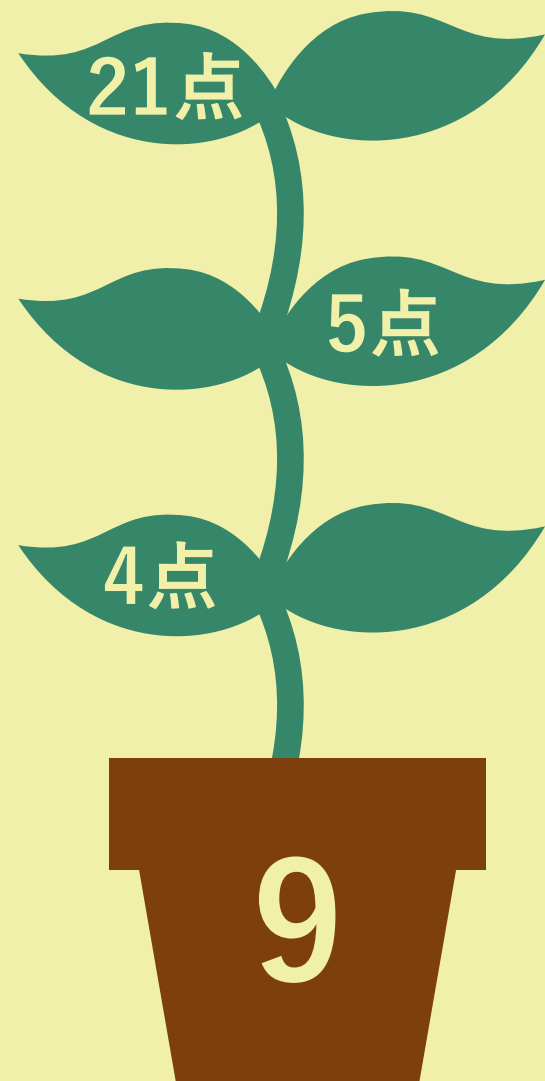


小課題 3 ($N \leq 2\,000$)

<問題>

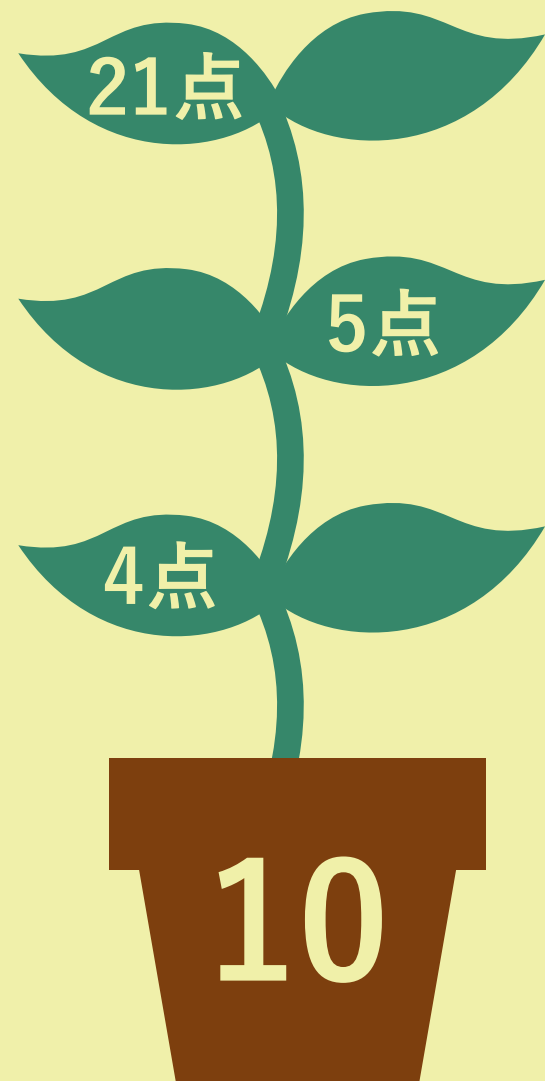
長さ N の数列 X, Y が与えられる。 Y の要素を好きに並べ替えるとき、 $\max(|X_i - Y_i|)$ を最小化せよ。

- X_i の大小と Y_i の大小が一致するように並べ替えればよい
- X, Y を両方ソートして $\max(|X_i - Y_i|)$ を計算すれば OK
- (証明) $X_i \leq X_j \wedge Y_i \geq Y_j$ のとき Y_i, Y_j を swap しても損しない



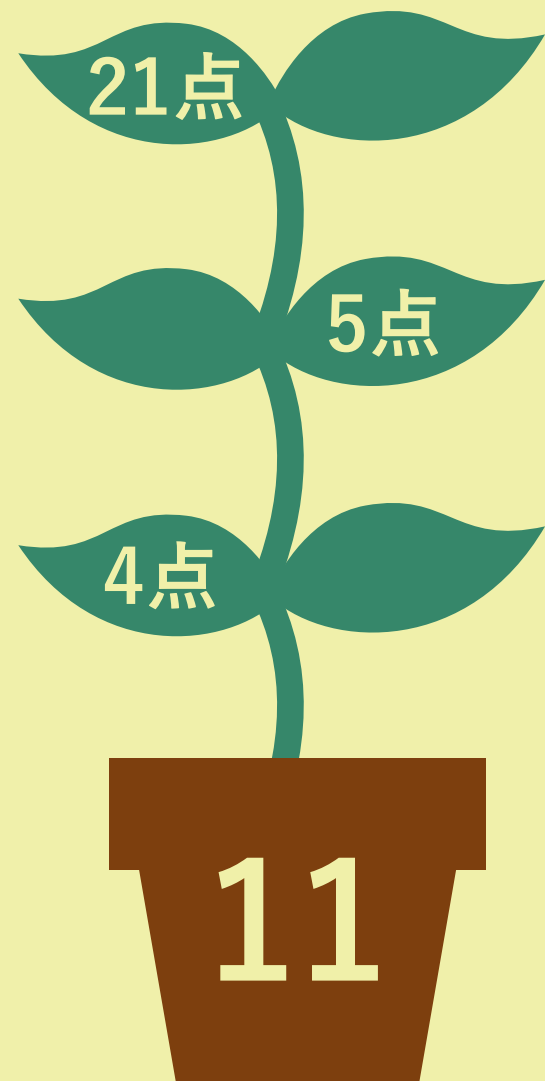
小課題 3 ($N \leq 2\,000$)

- (証明) $X_i \leq X_j \wedge Y_i \geq Y_j$ のとき Y_i, Y_j を swap しても損しない
 - swap 前後の $\max(|X_i - Y_i|, |X_j - Y_j|)$ はどう変化するか？
- $\max(X_i - Y_i, Y_i - X_i, X_j - Y_j, Y_j - X_j) \geq \max(X_i - Y_j, Y_j - X_i, X_j - Y_i, Y_i - X_j)$
 - 右辺の各項について, それより大きな左辺の項があることを確認



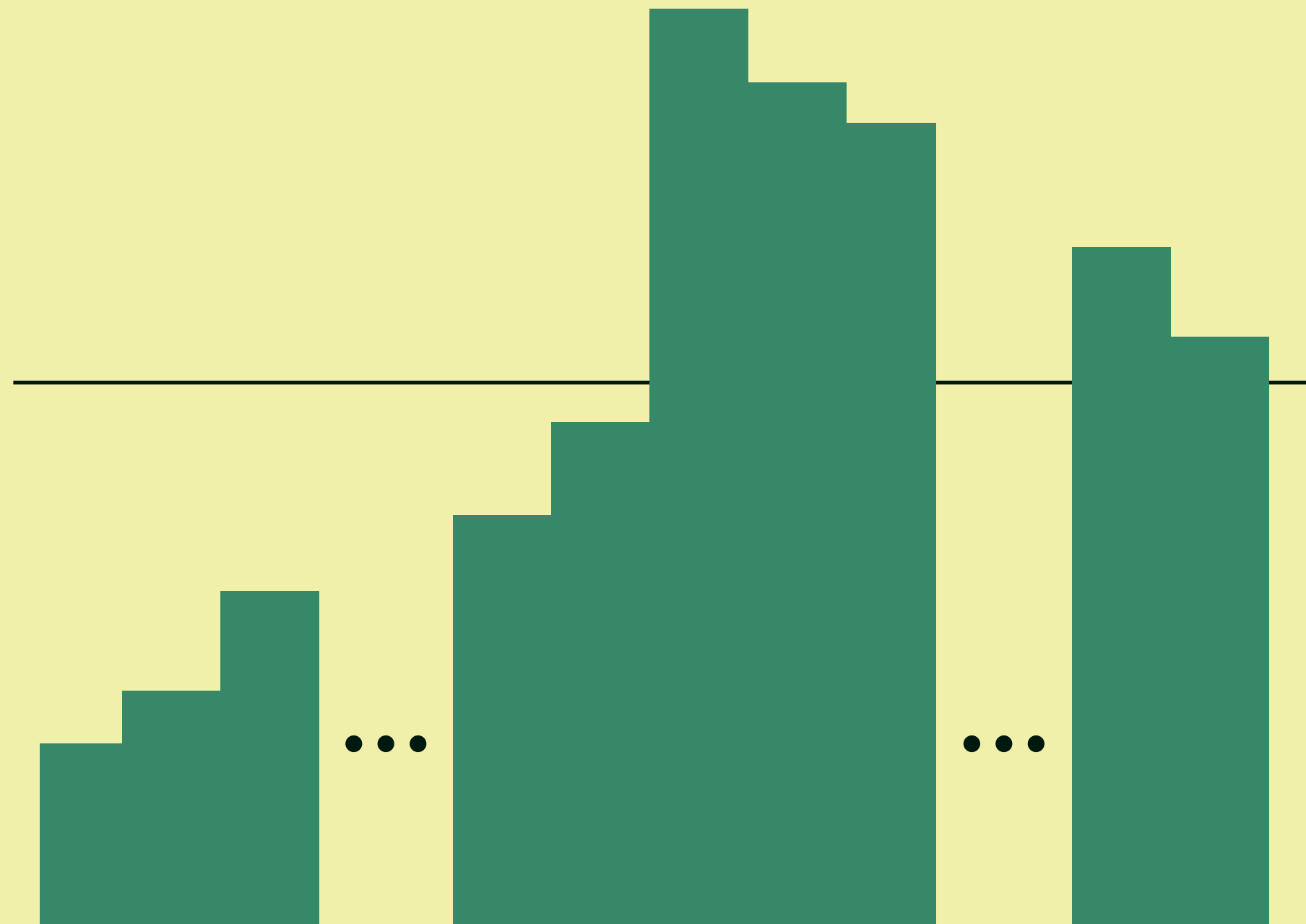
小課題 3 ($N \leq 2\,000$)

- ・ 植木鉢が青から赤に変わる位置を固定したとき、最悪 $\Theta(N \log N)$ で疲労度の最小値が求められる
- ・ 丁寧に実装すれば $O(N)$ でもできる
- ・ $O(N^2 \log N)$ で解けた



小課題 4 (A_i の値は全て異なり, かつ $A_N < A_{2N}$)

- ・ 色の配置を決めた時に毎回 $\Omega(N)$ かかっているのが大変
- ・ 毎回のシミュレーションを独立なものとして捉えるのではなく共通する性質を考える
- ・ この小課題では苗の大小が簡潔なので, これを利用したい



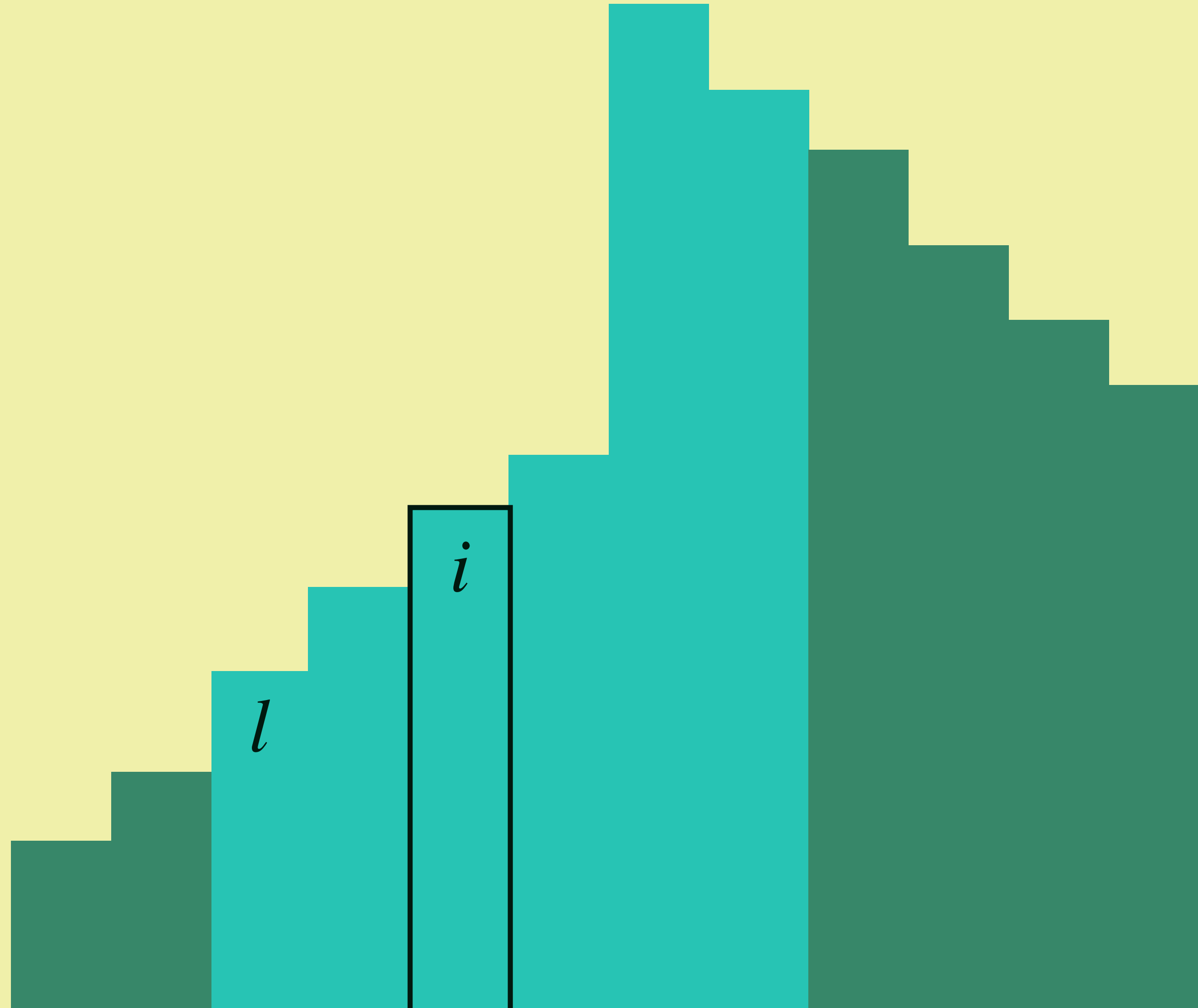
小課題 4 (A_i の値は全て異なり, かつ $A_N < A_{2N}$)

- ・ 植木鉢の要素はあらかじめソートしておく
- ・ 同色の植木鉢を区間 $[l, l + N - 1] (1 \leq l \leq N)$ に割り当てるとき
苗 i に割り当てられる植木鉢の添え字は？
- ・ つまり, 苗 i は区間 $[l, l + N - 1]$ で何番目に小さい？



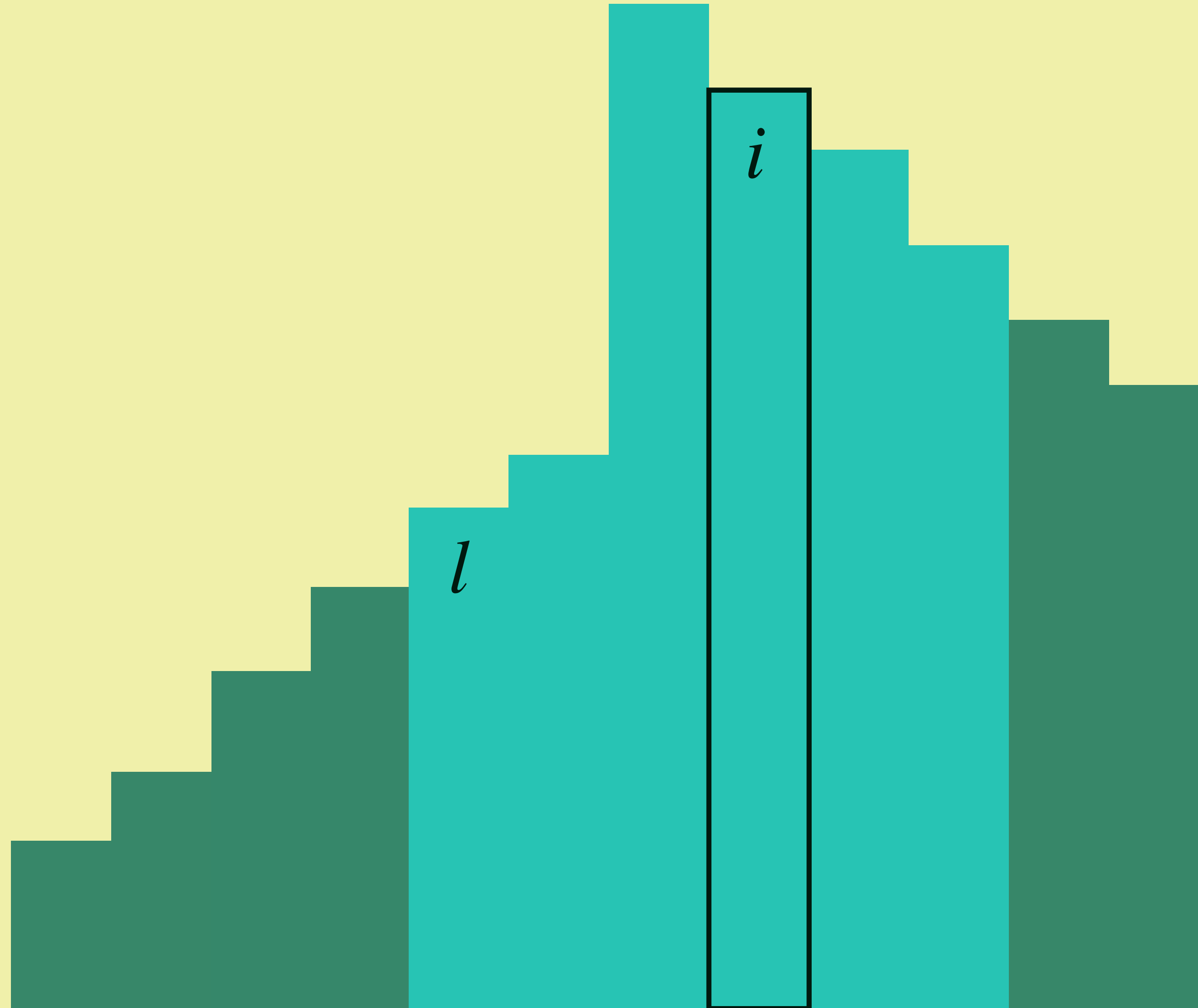
小課題 4 (A_i の値は全て異なり, かつ $A_N < A_{2N}$)

- $1 \leq i \leq N$ のとき $i - l + 1$



小課題 4 (A_i の値は全て異なり, かつ $A_N < A_{2N}$)

- $N + 1 \leq i \leq 2N$ のとき $2N - i + 1$ (l によらない)



小課題 4 (A_i の値は全て異なり, かつ $A_N < A_{2N}$)

- $N + 1 \leq l \leq 2N$ の場合も -1 倍して考えれば同じ
- 苗が対応する植木鉢の添字が区間の左端に関する式で表せた
 - しかも傾き 0 または -1 の高々一次関数
- しかし, 添字が簡単にわかってても, 差の絶対値の $\max$ を高速に求めるのは難しい



小課題 4 (A_i の値は全て異なり, かつ $A_N < A_{2N}$)

- まだ使っていない典型考察がある
- この問題は, ざっくり言うと「最大値の最小化」



小課題 4 (A_i の値は全て異なり, かつ $A_N < A_{2N}$)

→ 二分探索

- ・ 「差を全て K 以下にできるか？」と考える
- ・ 赤&青それぞれについて, その色の植木鉢を
区間 $[l, l + N - 1]$ の苗に割り当てたときに, 区間内のすべての苗と
植木鉢の大きさの差が K 以下にできるかどうかを
 $l = 1, 2, \dots, 2N$ について求めたい



小課題 4 (A_i の値は全て異なり, かつ $A_N < A_{2N}$)

- 差の上限 K が指定されたとき, それぞれの苗について割り当てることのできる植木鉢の添字の範囲は区間になる
- 植木鉢の添字は左端の関数で表せる
- $\uparrow$ の考察二つを合わせると, ある苗が条件を満たすような左端の範囲も区間になることがわかる



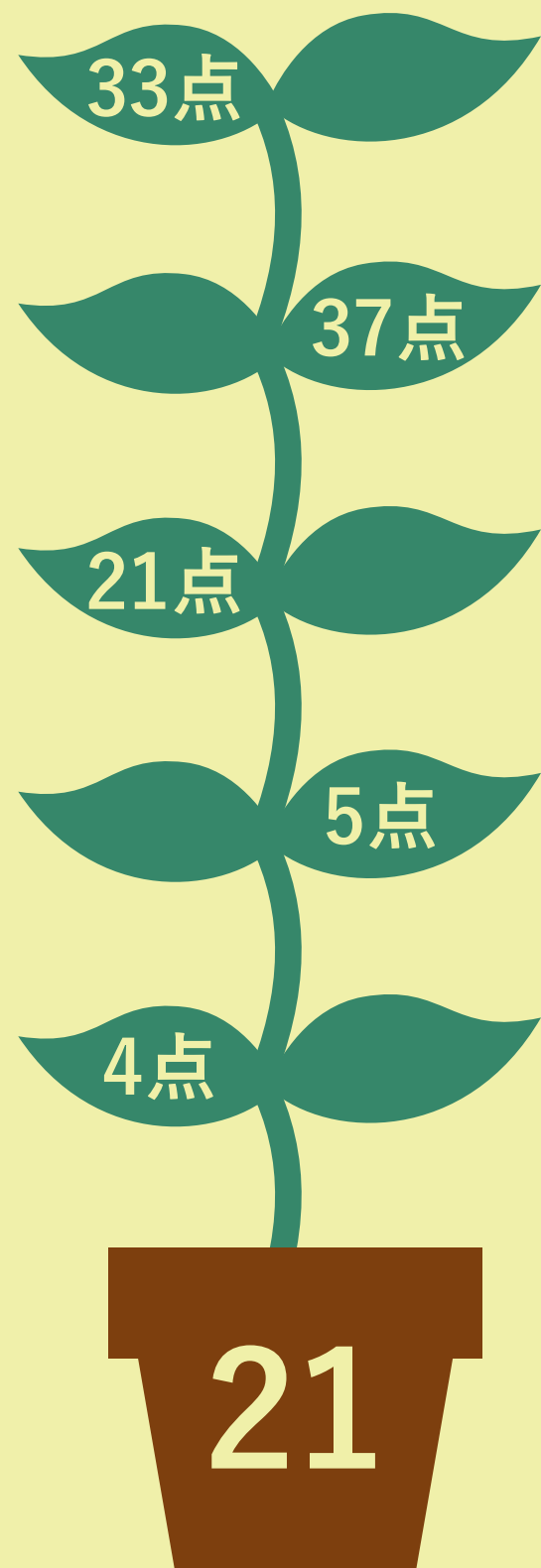
小課題 4 (A_i の値は全て異なり, かつ $A_N < A_{2N}$)

- N 個の苗が同時に条件を満たすような左端の位置を求める
- これにより, 赤&青それぞれについて, どの左端ならすべての苗が条件を満たすかを $O(N)$ で求められる
- この情報から, 疲労度 K 以下の配置が存在するかを $O(N)$ で判定できる
- 全体で $O(N \log \max A)$



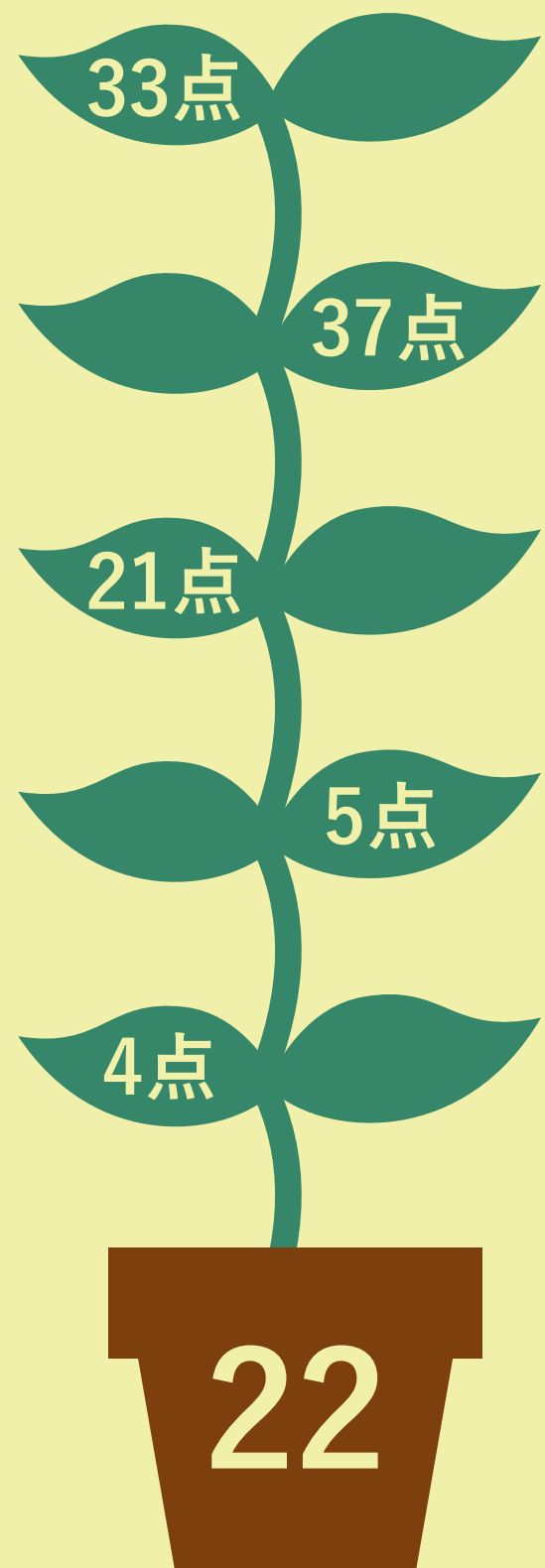
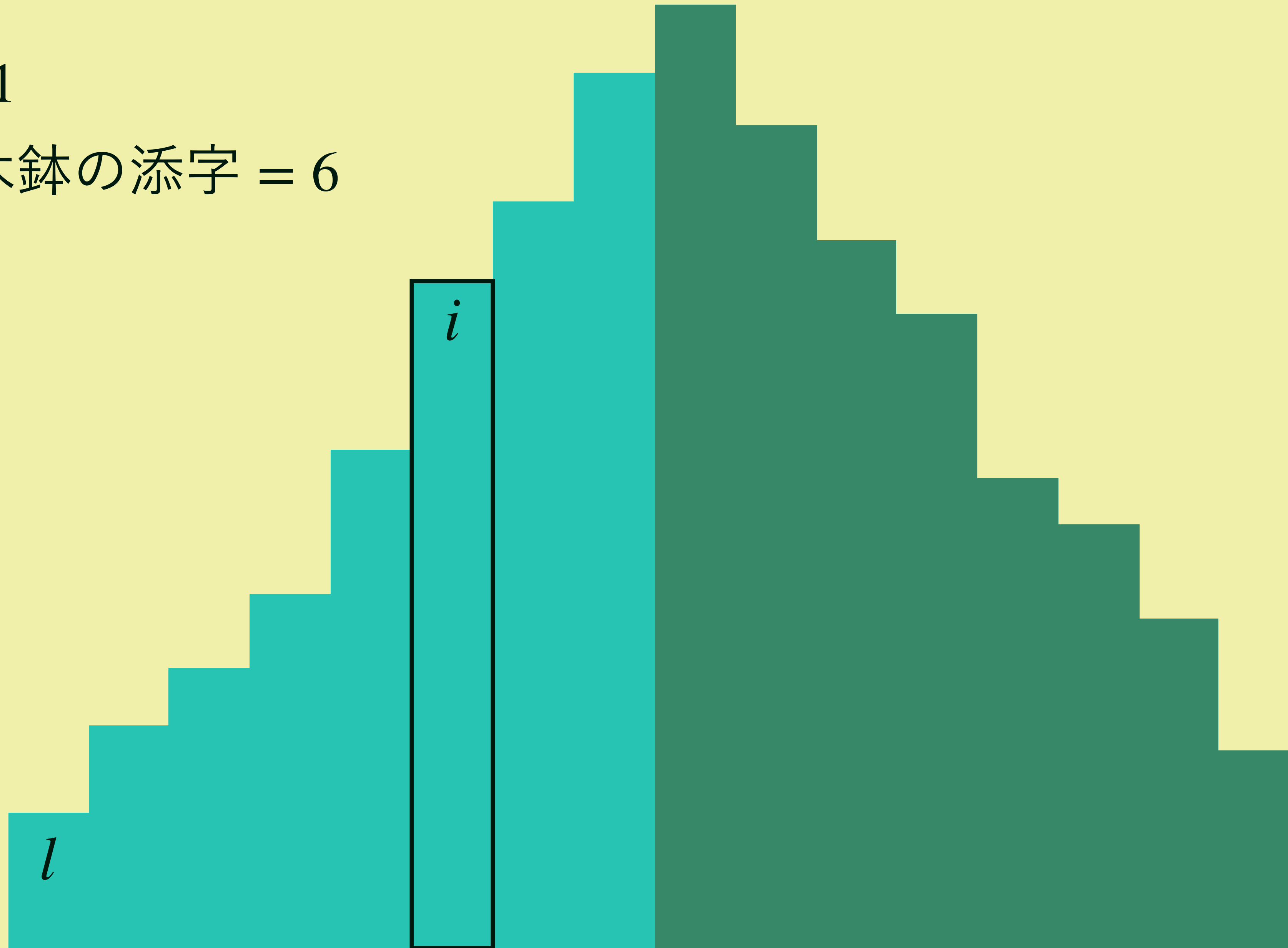
小課題 5 (追加の制約はない)

- ・ 苗の大きさは特殊ではないが, 苗が対応する植木鉢の添字が簡単に表せれば小課題 4 と同様に解ける
- ・ 区間をスライドさせていって変化を観察



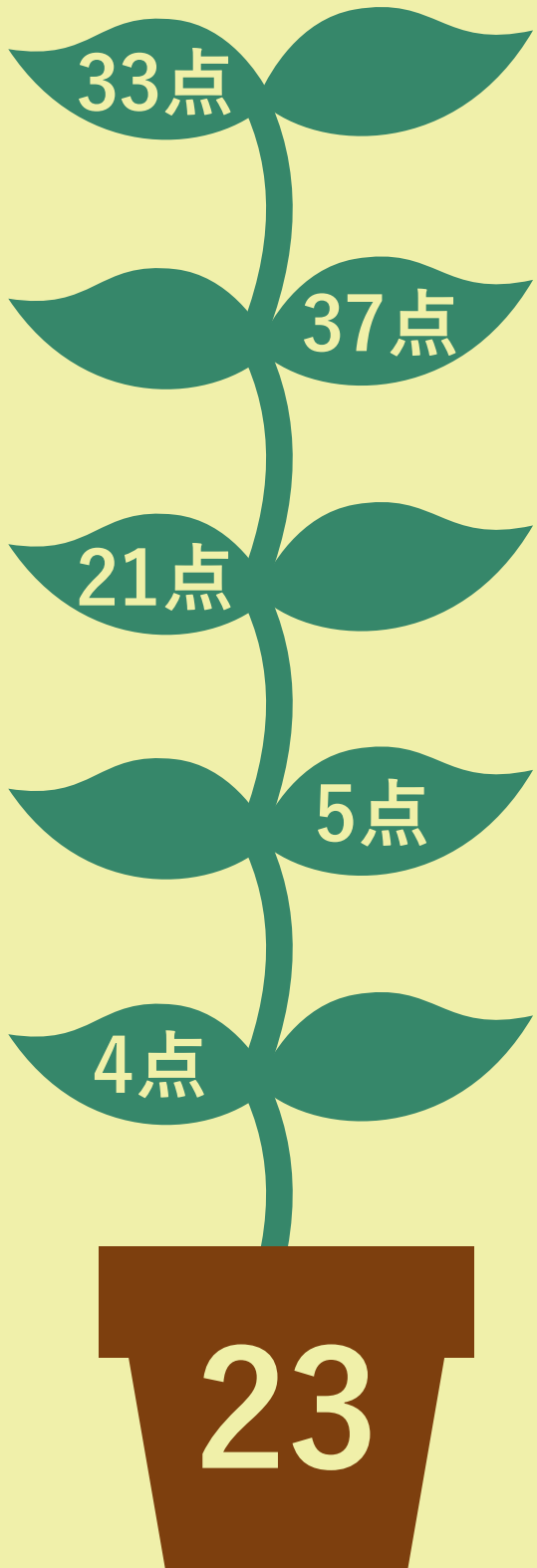
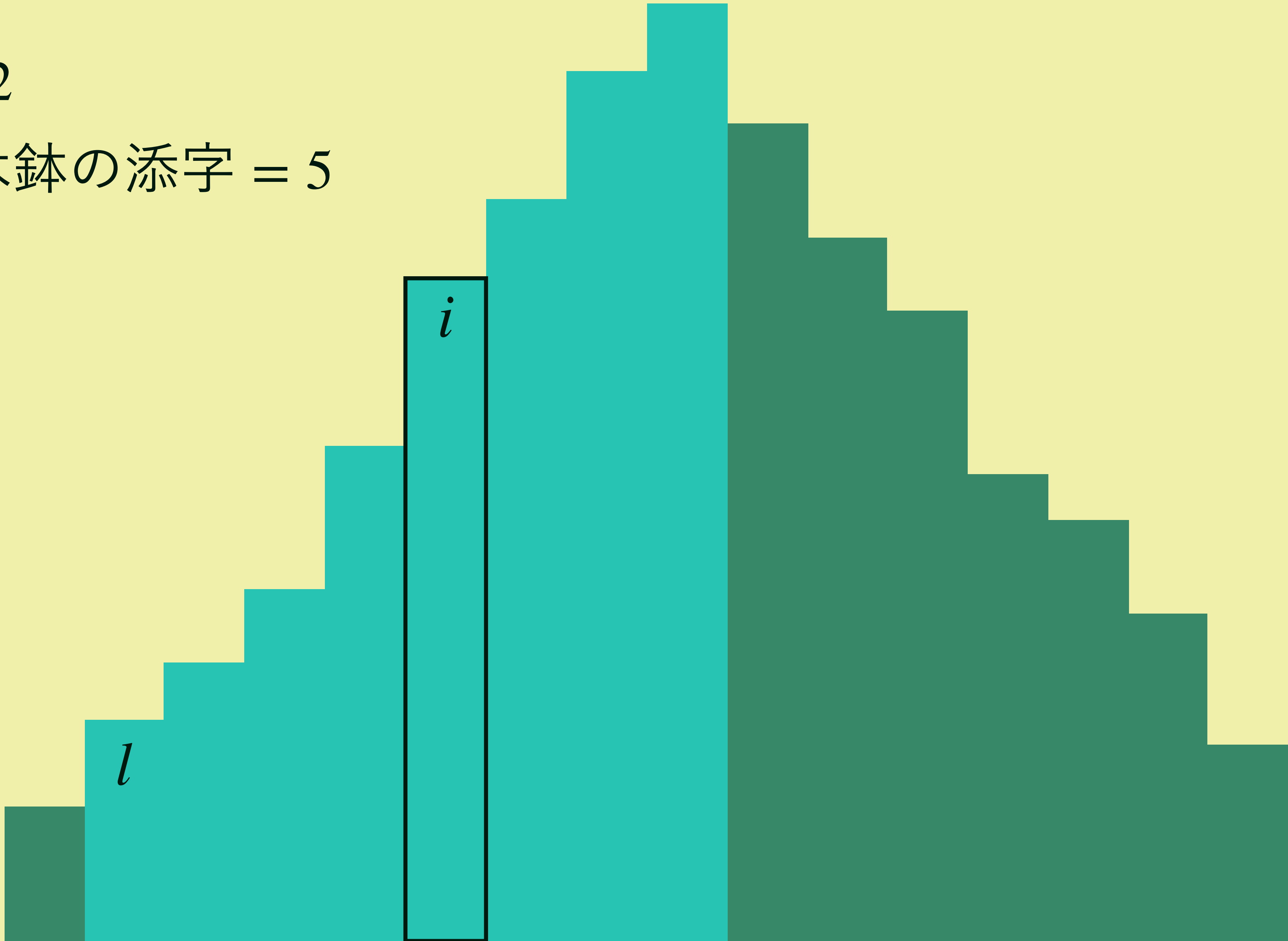
小課題 5 (追加の制約はない)

- $l = 1$
- 植木鉢の添字 = 6



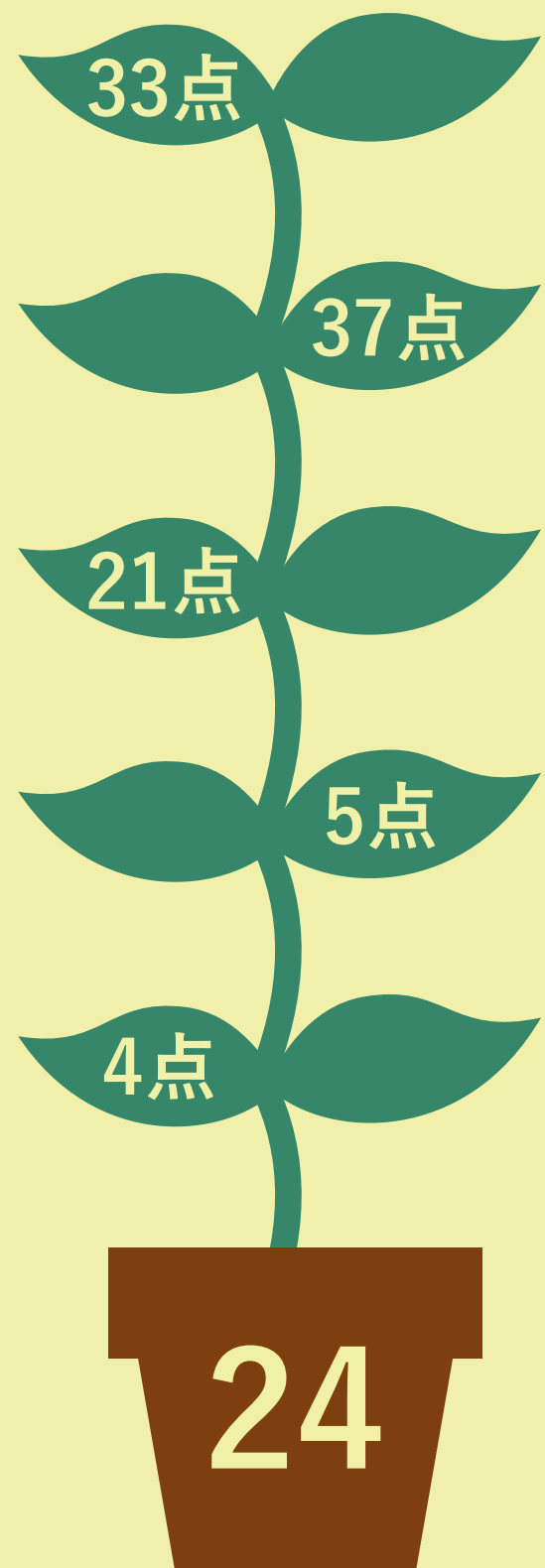
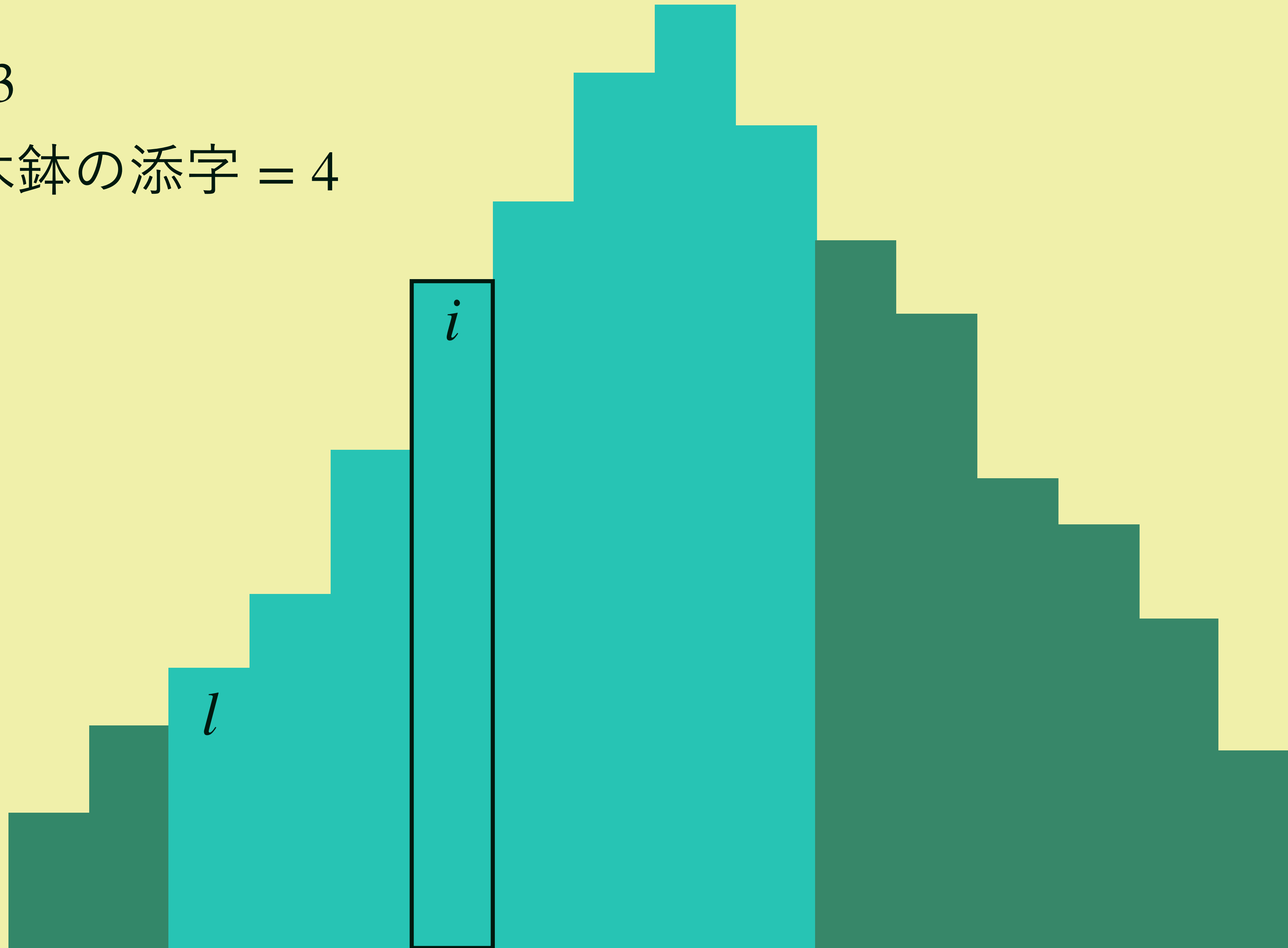
小課題 5 (追加の制約はない)

- $l = 2$
- 植木鉢の添字 = 5



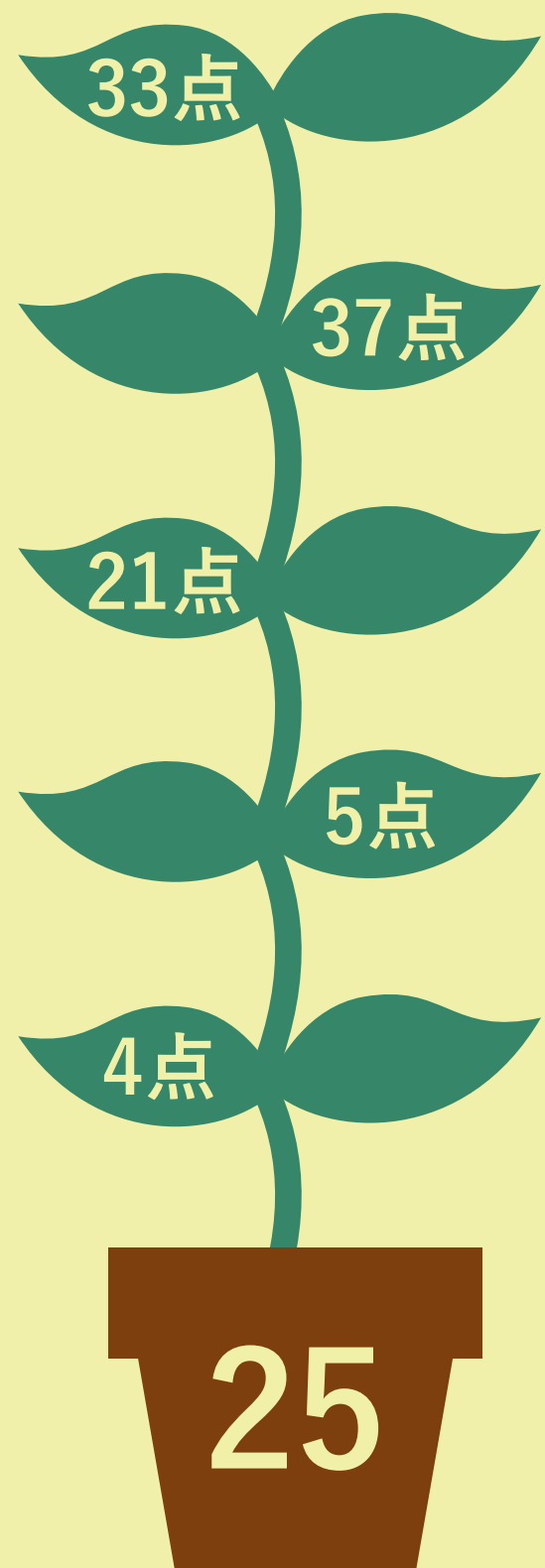
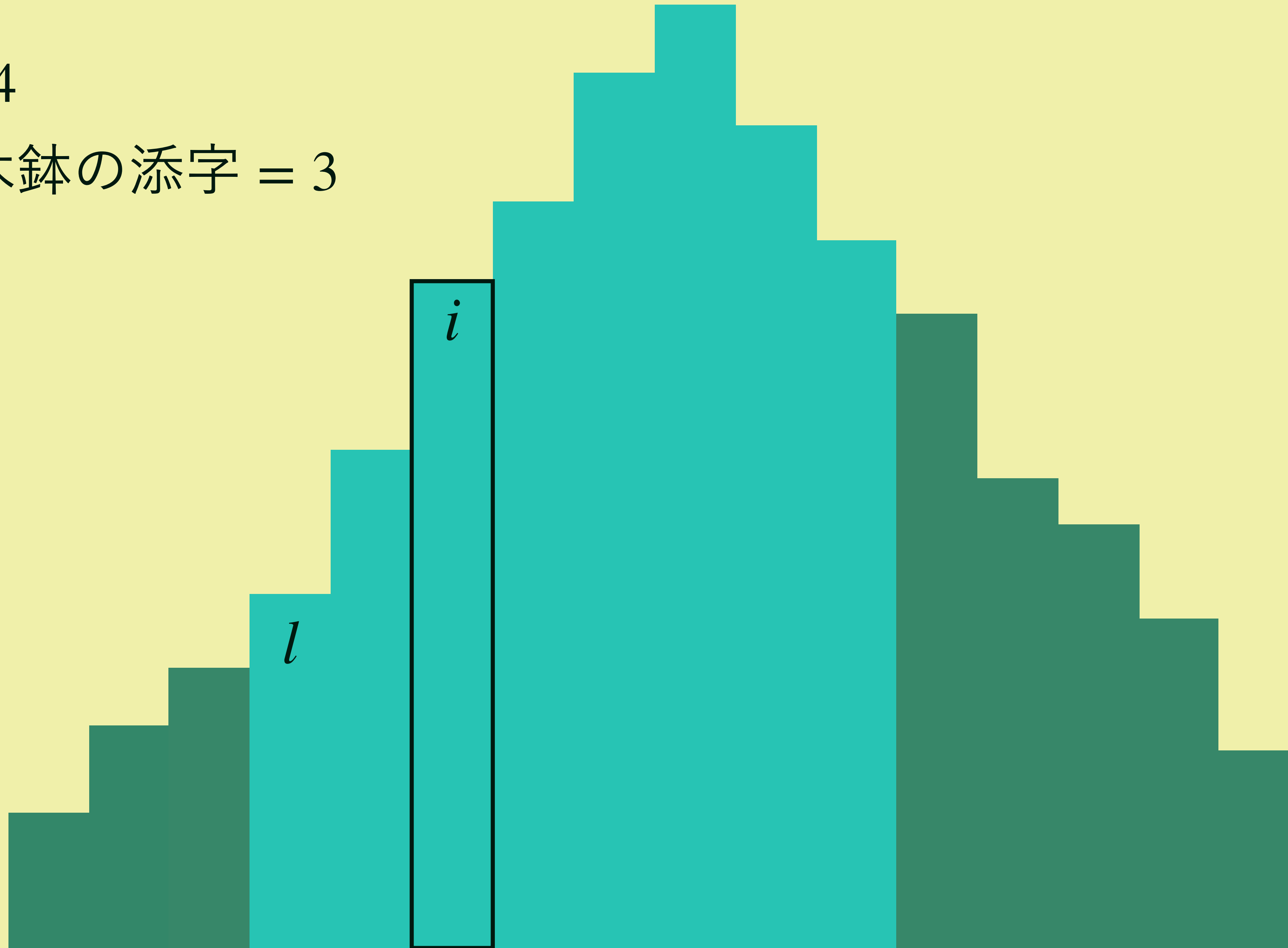
小課題 5 (追加の制約はない)

- $l = 3$
- 植木鉢の添字 = 4



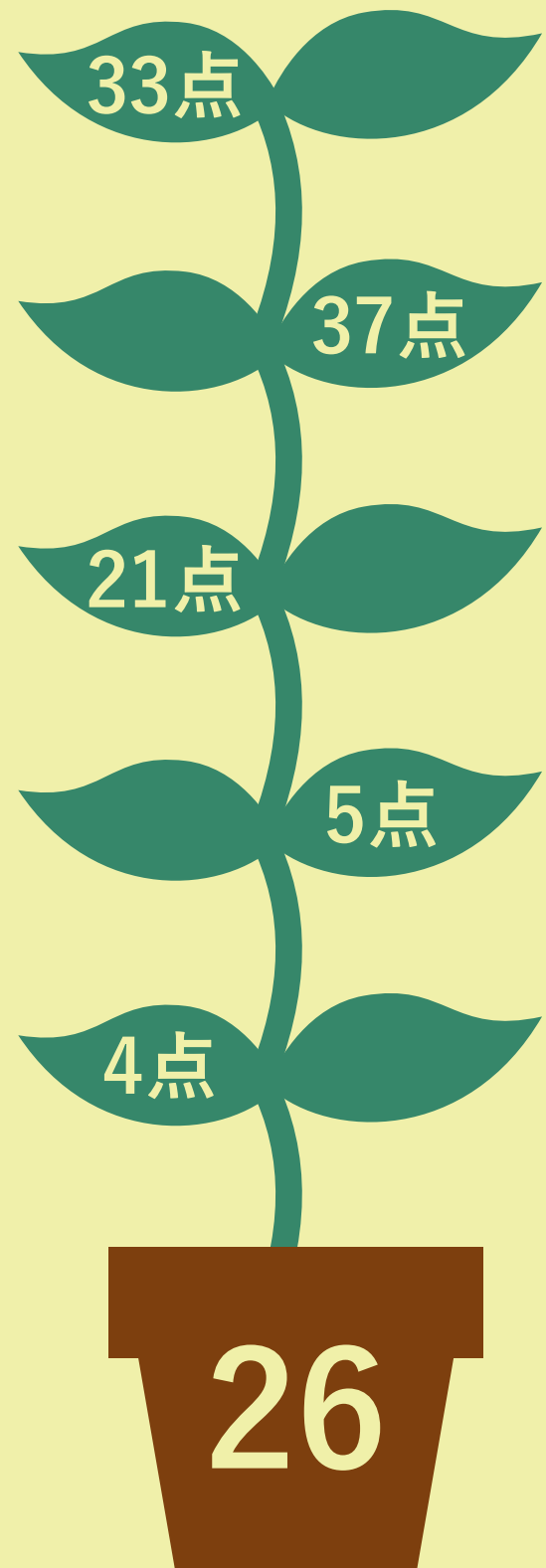
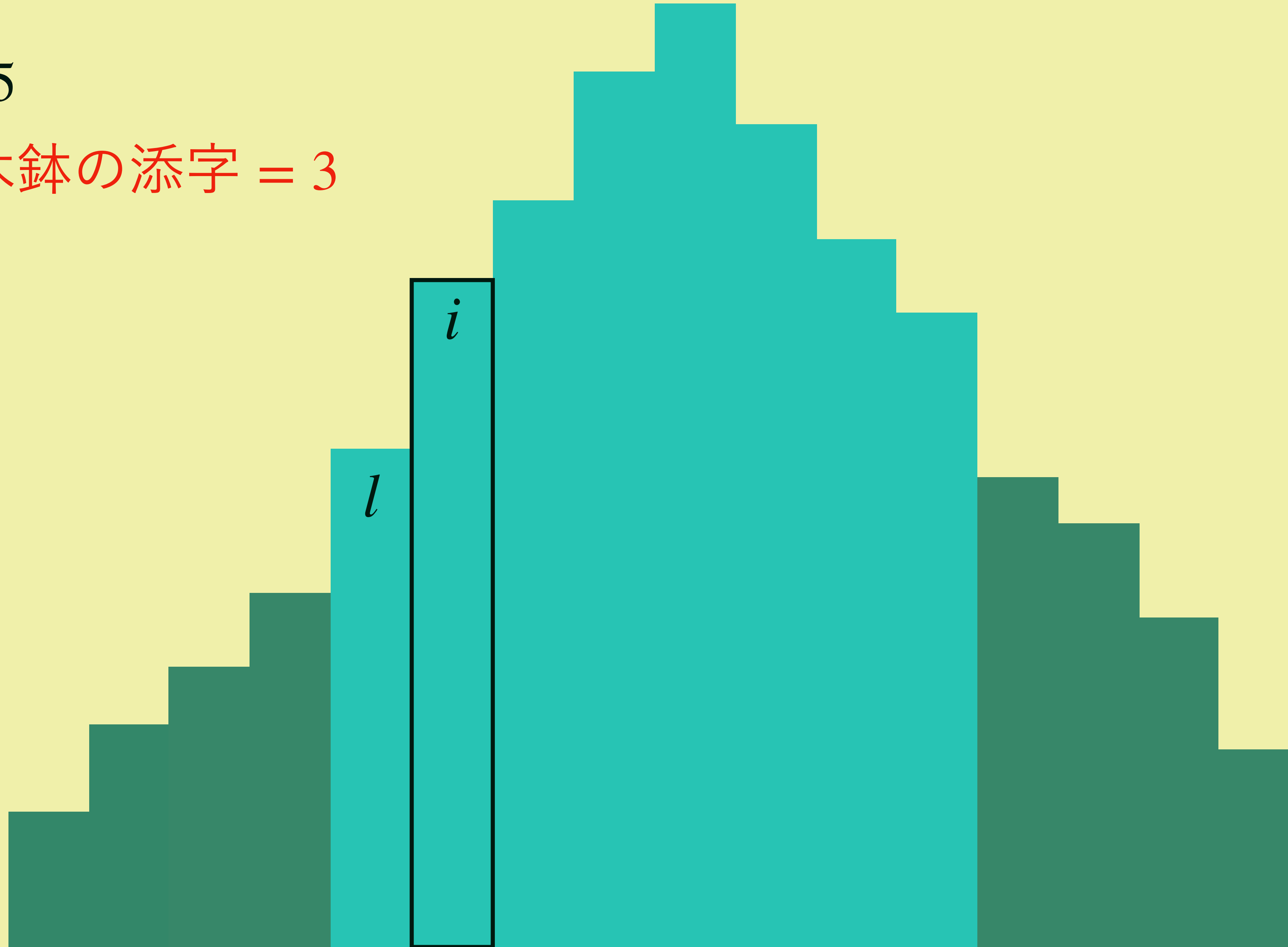
小課題 5 (追加の制約はない)

- $l = 4$
- 植木鉢の添字 = 3



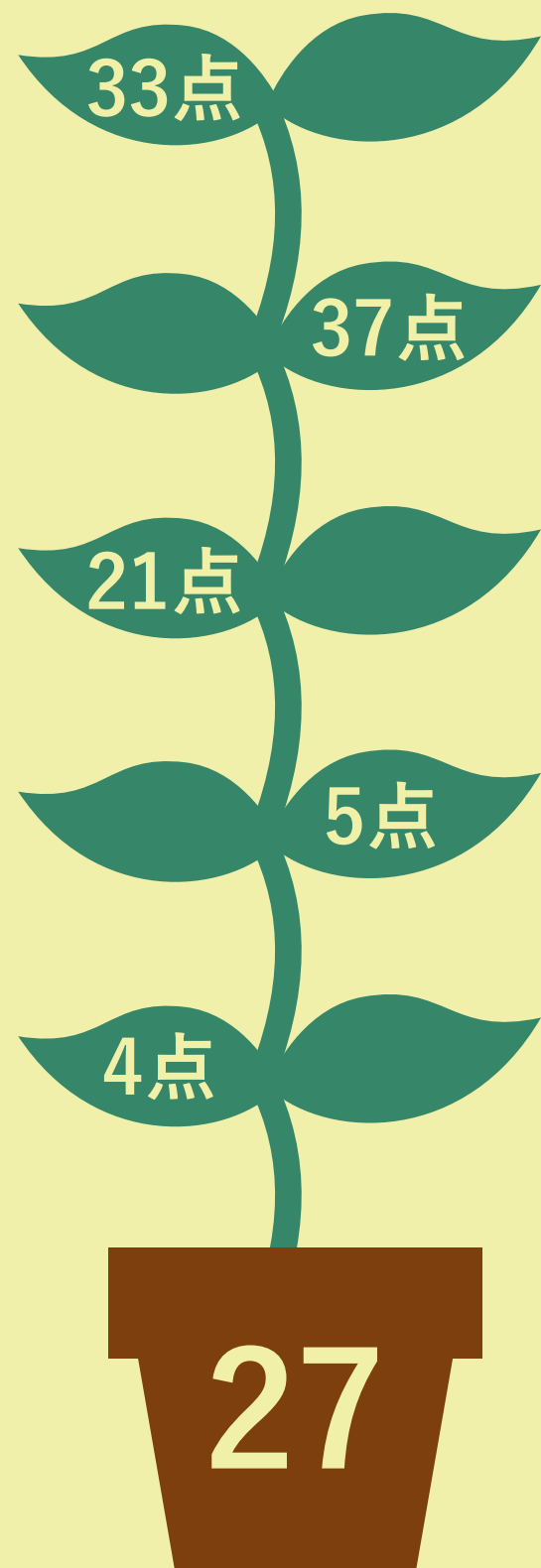
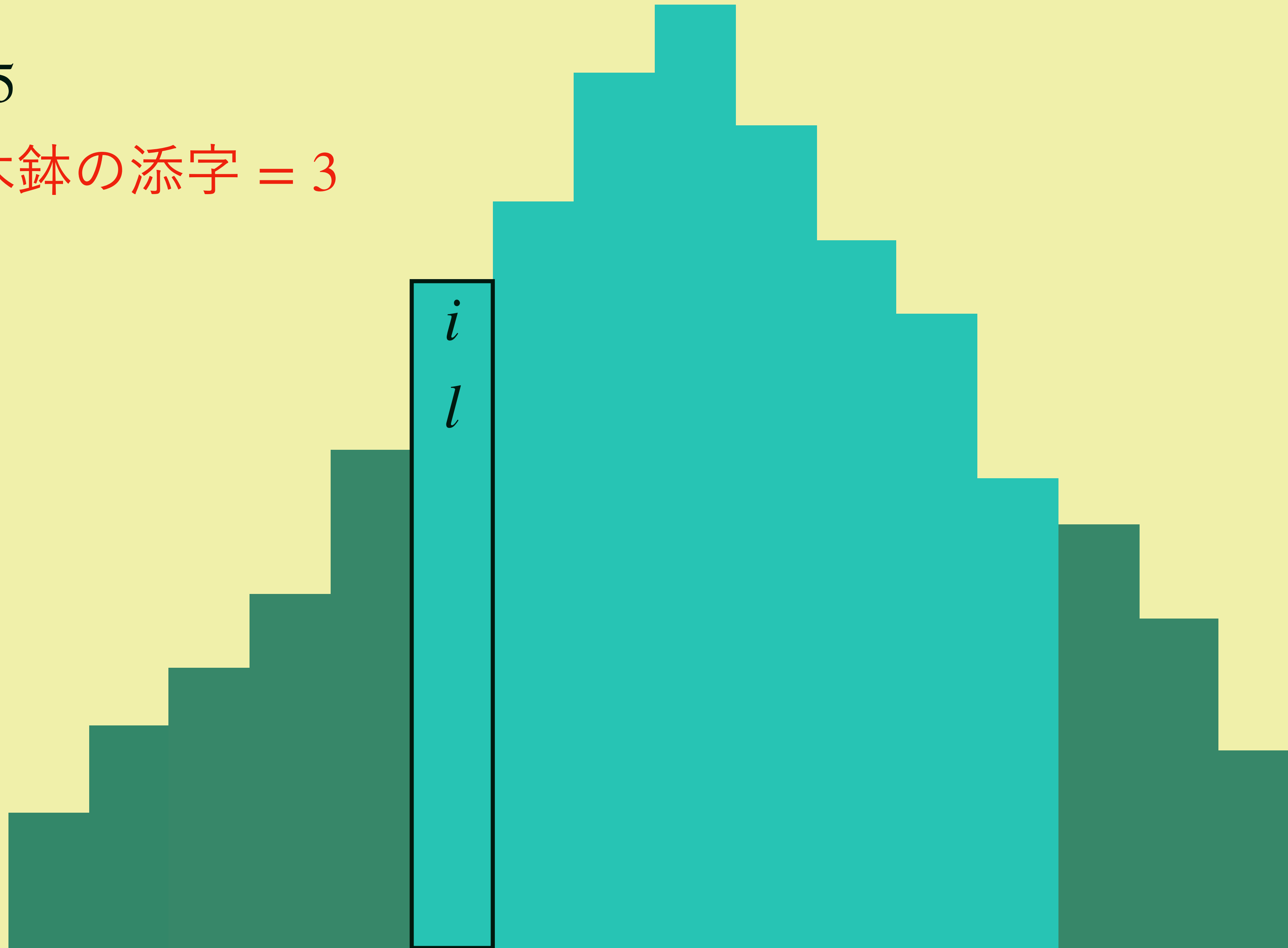
小課題 5 (追加の制約はない)

- $l = 5$
- 植木鉢の添字 = 3



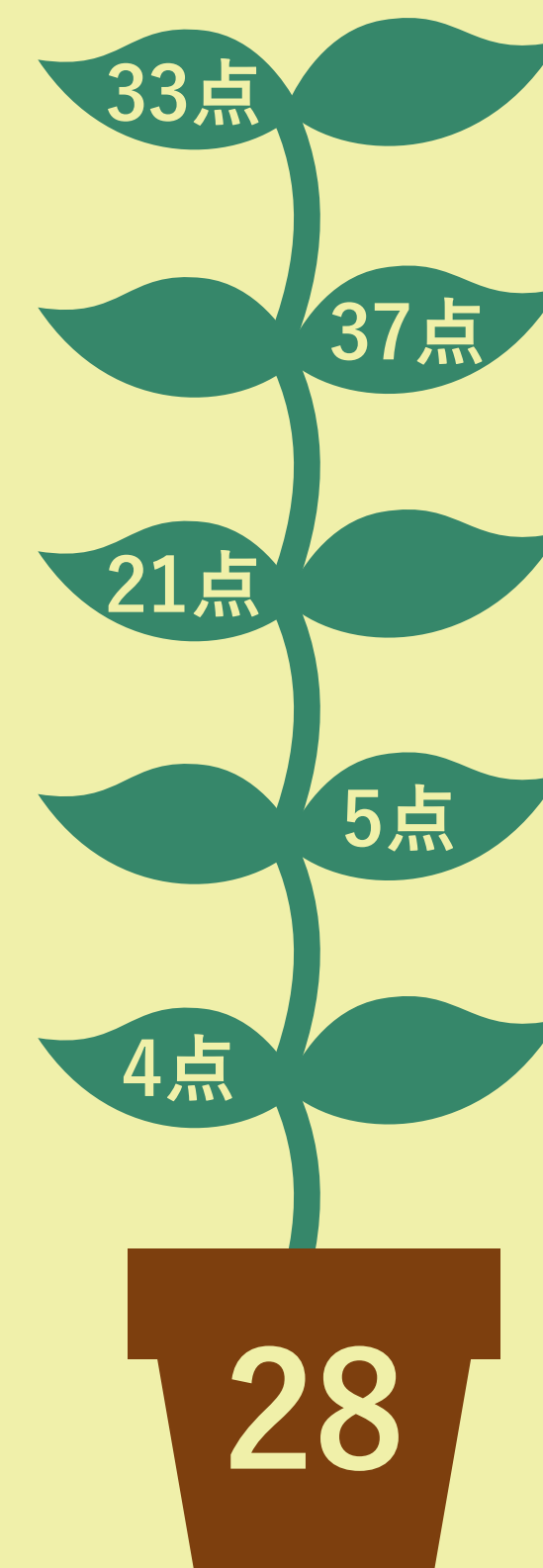
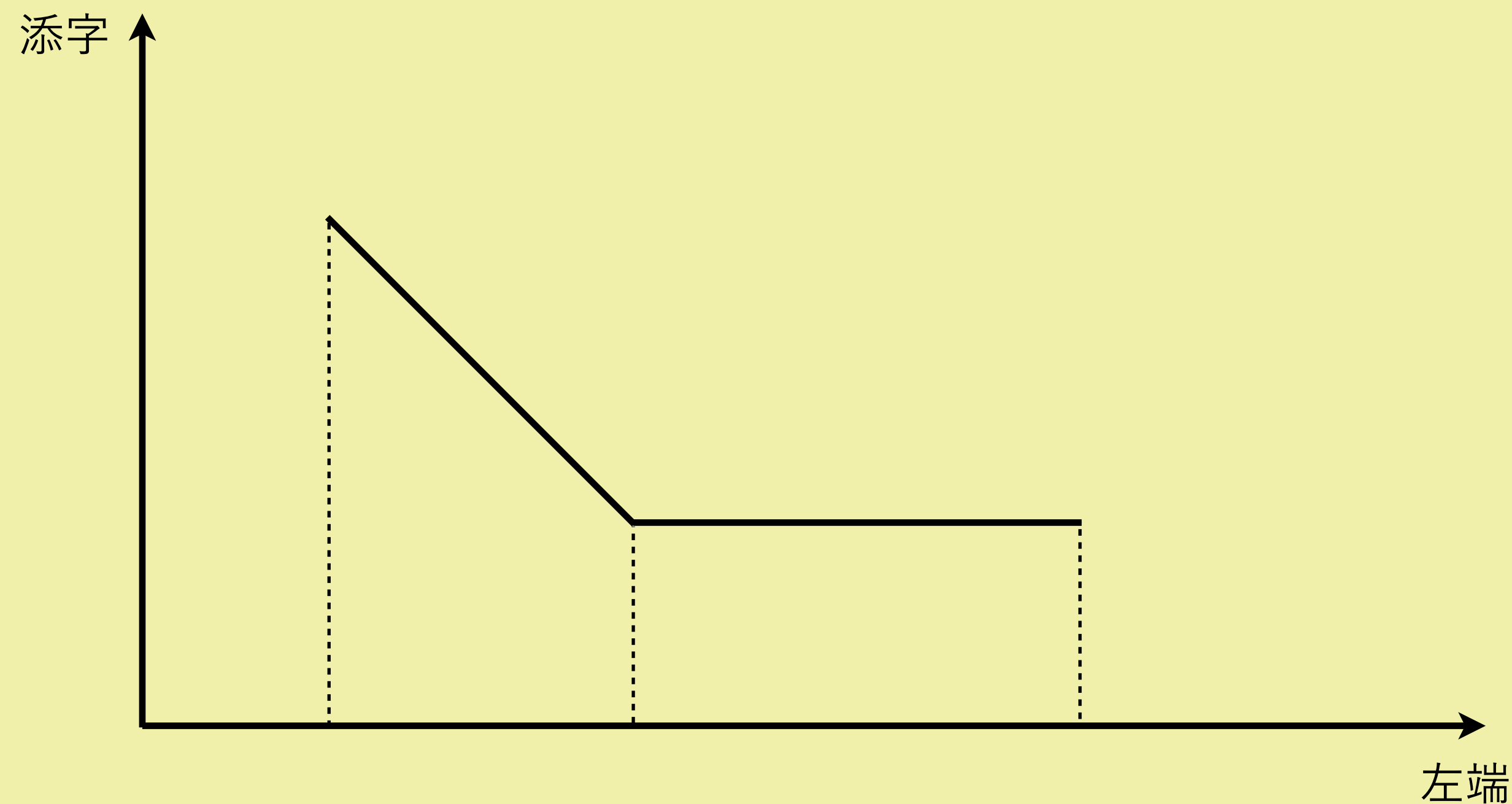
小課題 5 (追加の制約はない)

- $l = 5$
- 植木鉢の添字 = 3



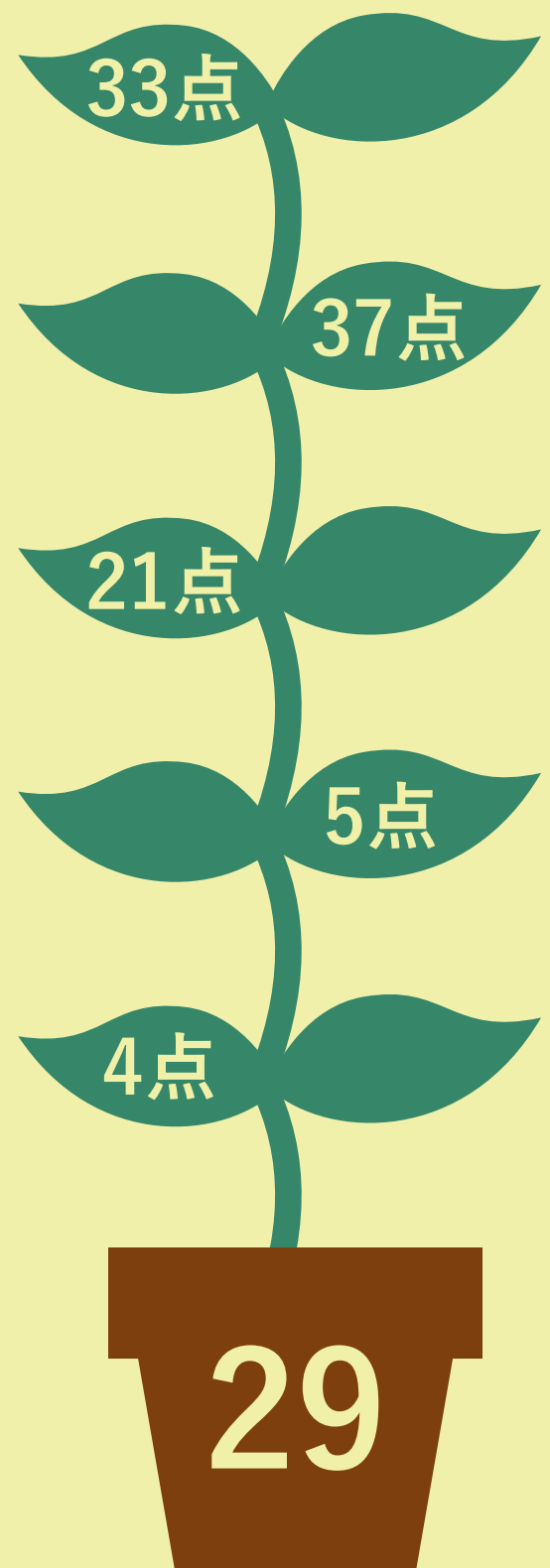
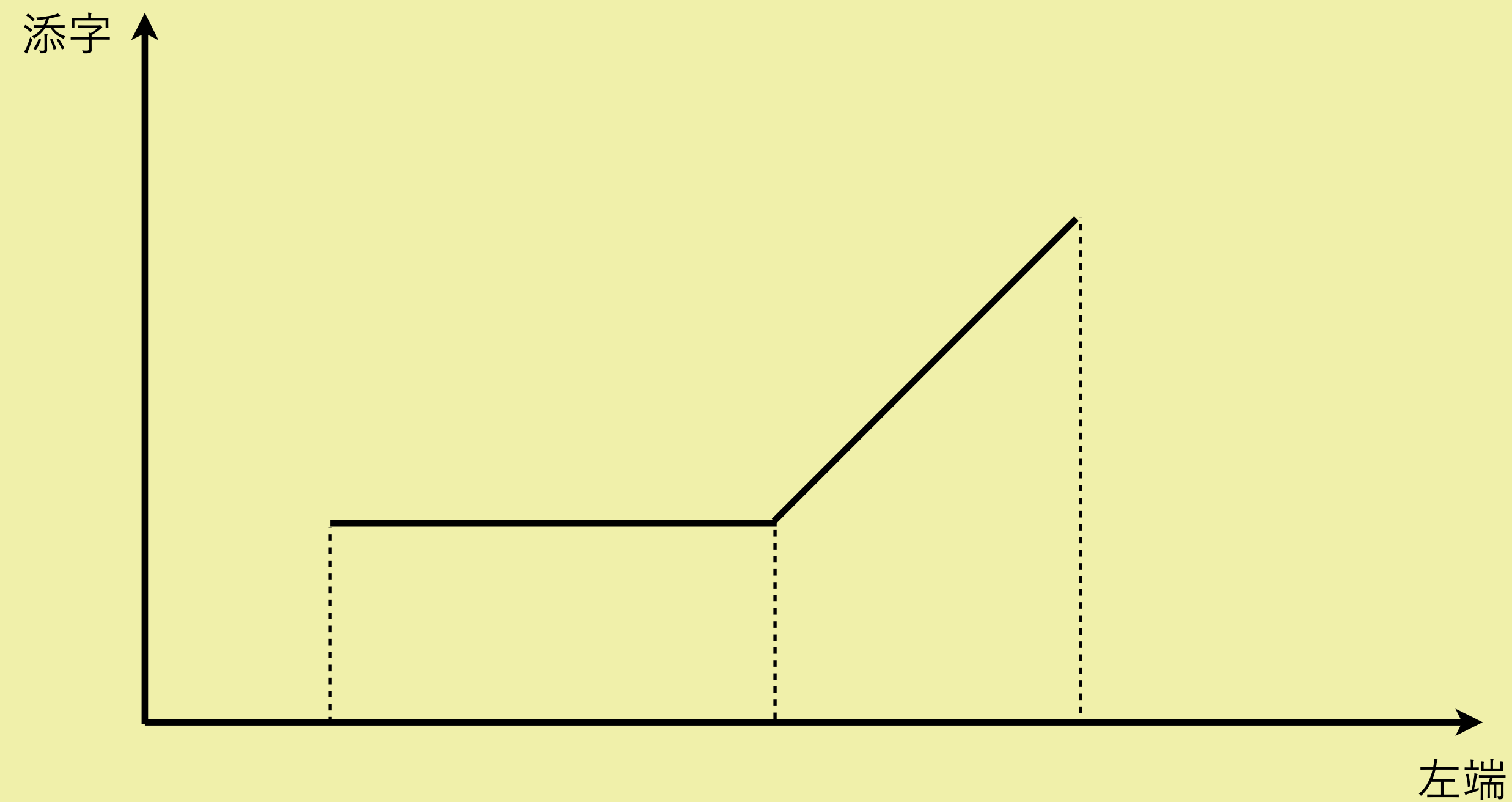
小課題 5 (追加の制約はない)

- $1 \leq i \leq N$ のとき, 区間を右にずらしたときに入ってくる苗が苗 i より大きければ添字が減り, 小さければそのまま
- ↓ グラフにするとこんな感じ



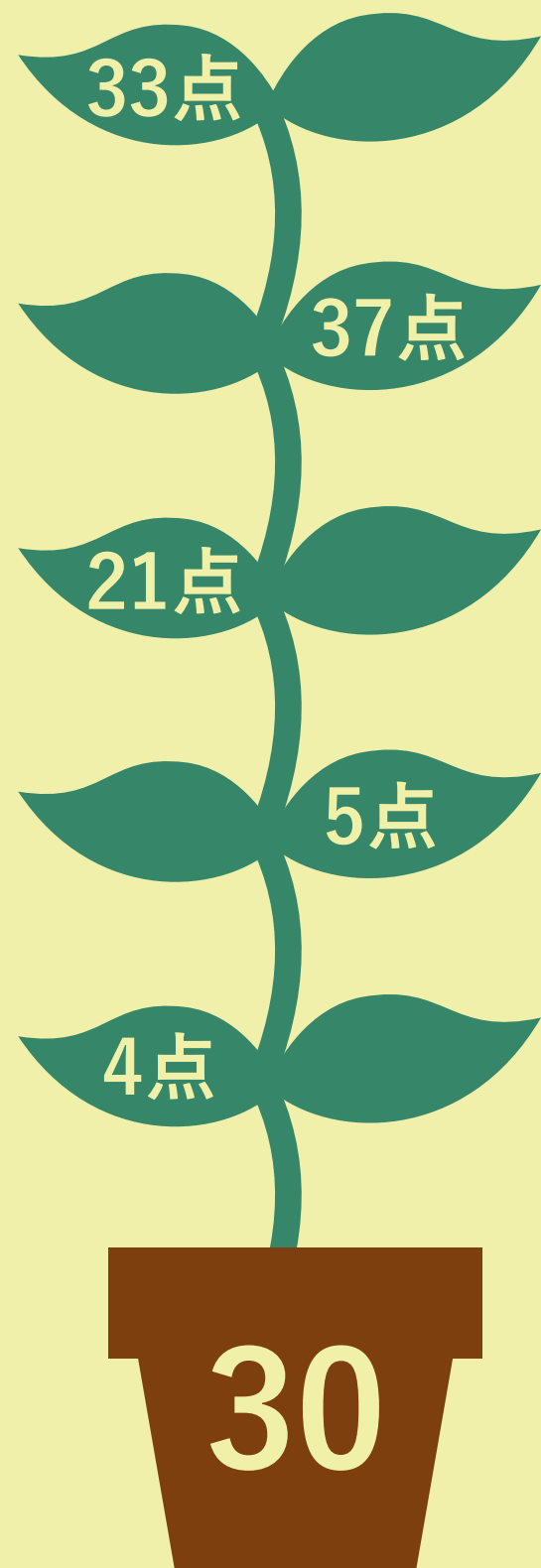
小課題 5 (追加の制約はない)

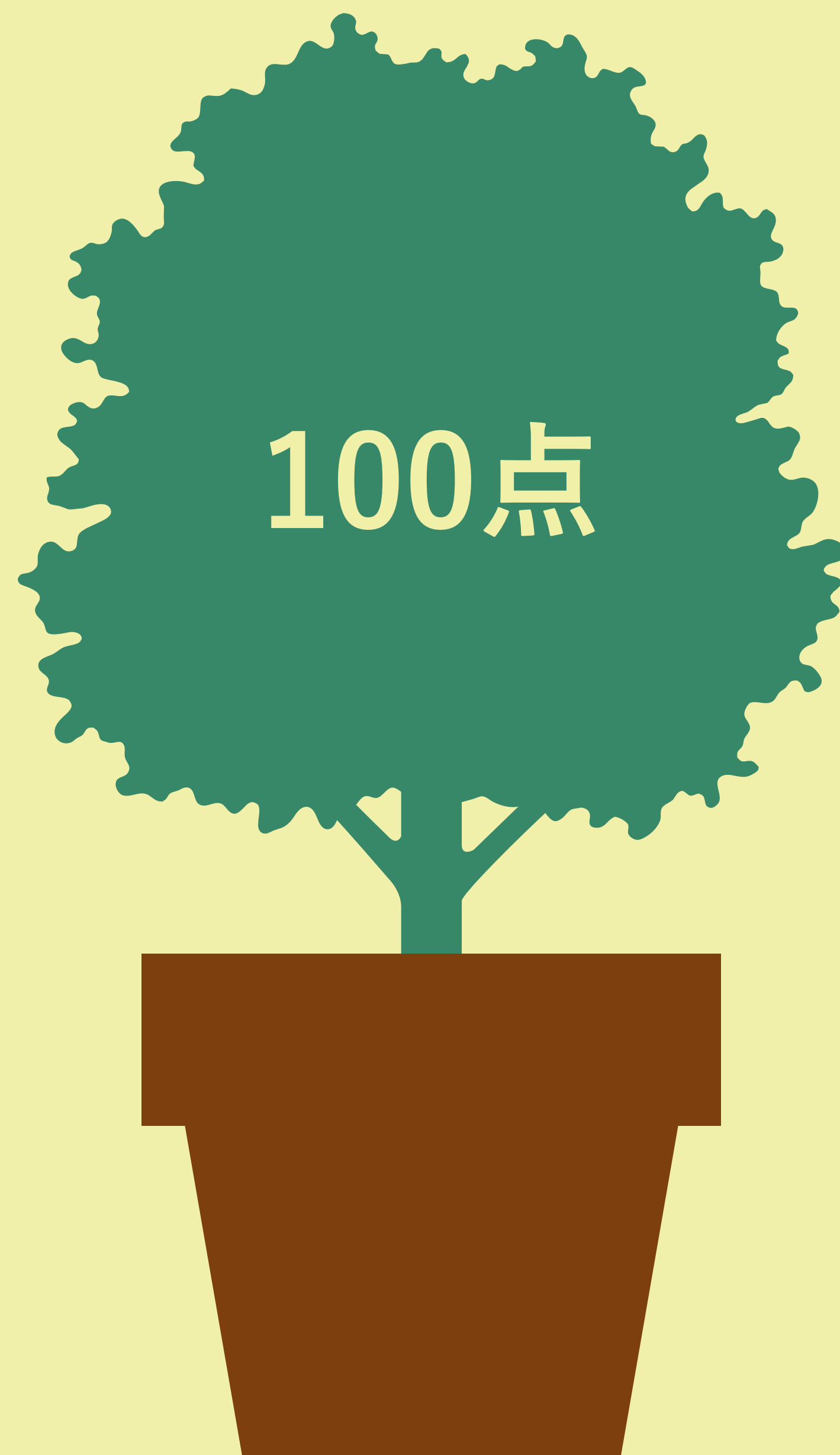
- $N + 1 \leq i \leq 2N$ のときも考えるとこんな感じ



小課題 5 (追加の制約はない)

- この値が苗 i との大きさが K 以下である植木鉢の区間に含まれるような左端の範囲を求めればよい
- グラフの形を決める値は, 尺取り法などで求められる
- あとは小課題 4 と同じで $O(N \log \max A)$
- 丁寧に実装

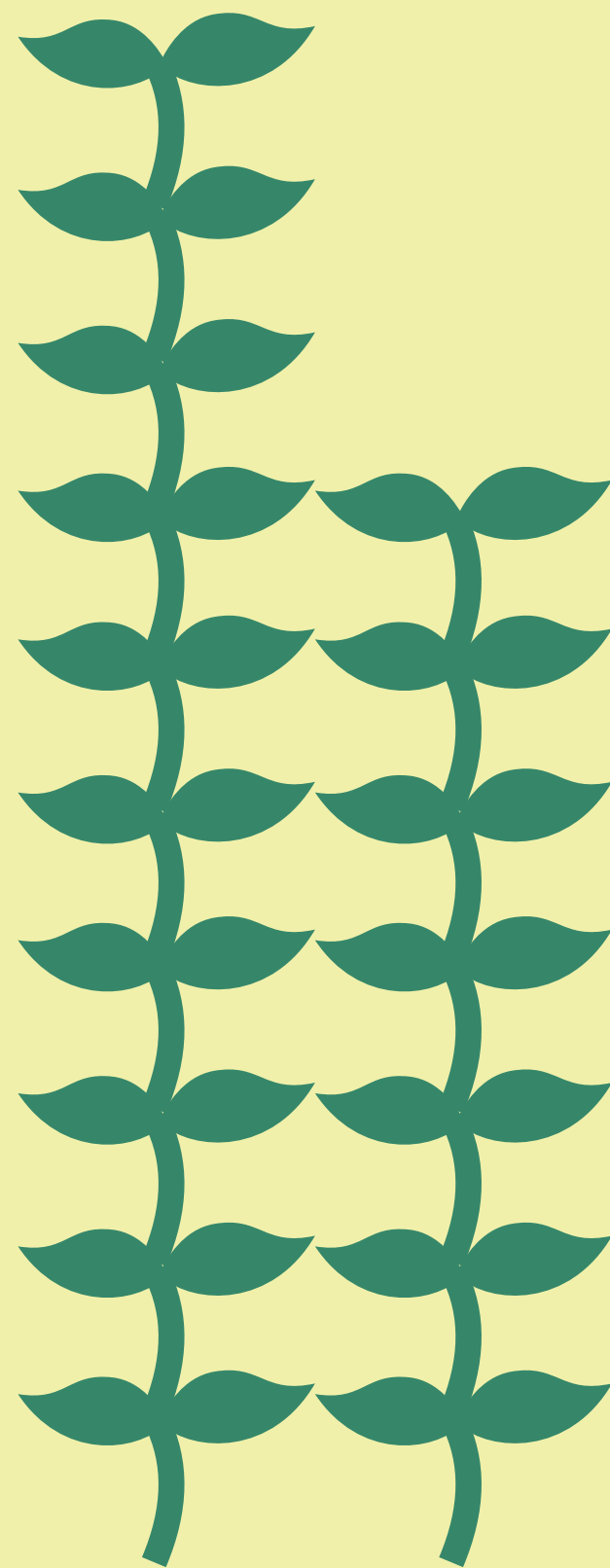




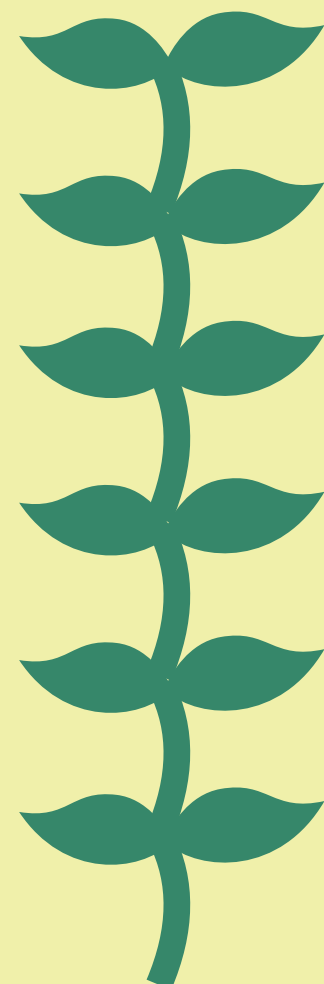
得点分布



0点



30点



67点

100点

カード収集 (Card Collection)

高谷悠太

2024 年 3 月 23 日

問題概要

N 枚のカードが一行に並んでいる。各カードは整数の組 (S_i, V_i) に対応している。JOI 君は、以下の操作を $N - 1$ 回繰り返して 1 枚のカードを作る。

隣り合う 2 枚のカード $(a, b), (c, d)$ を選び、

$(\max(a, c), \max(b, d))$ または $(\min(a, c), \min(b, d))$

に置き換える。

M 枚のカードが与えられるので、それぞれのカードについて、JOI 君が最終的に作ることができるか判定せよ。

制約

1. $N \leq 20, M \leq 10$.
2. $N \leq 2000, M \leq 10$.
3. $N \leq 200000, M \leq 10$.
4. $N \leq 200000, M \leq 200000$.

小課題 1: $N \leq 20$, $M \leq 10$

操作途中に現れる各カードは，元のカード列でのある連続する区間に対応する．

小課題 1: $N \leq 20, M \leq 10$

操作途中に現れる各カードは、元のカード列でのある連続する区間に対応する．部分問題として、次の問題を考える．

区間 $[l, r]$ でカード (a, b) を作ることができるか？

小課題 1: $N \leq 20, M \leq 10$

操作途中に現れる各カードは、元のカード列でのある連続する区間に対応する。部分問題として、次の問題を考える。

区間 $[l, r]$ でカード (a, b) を作ることができるか？

この表示により区間 dp が適用できる。

$dp[l][r][a][b] :=$ 部分問題の答え

を動的計画法で求める。

小課題 1: $N \leq 20, M \leq 10$

更新は以下のようになる.

$$\begin{aligned} dp[l][r][\max(a, c)][\max(b, d)] \\ &= dp[l][m][a][b] \& dp[m+1][r][c][d] \\ dp[l][r][\min(a, c)][\min(b, d)] \\ &= dp[l][m][a][b] \& dp[m+1][r][c][d] \end{aligned}$$

小課題 1: $N \leq 20, M \leq 10$

更新は以下のようになる.

$$\begin{aligned} dp[l][r][\max(a, c)][\max(b, d)] \\ &= dp[l][m][a][b] \& dp[m+1][r][c][d] \\ dp[l][r][\min(a, c)][\min(b, d)] \\ &= dp[l][m][a][b] \& dp[m+1][r][c][d] \end{aligned}$$

計算量は $O(N^7)$ だが定数倍が軽く十分高速に動く.

小課題 1: 区間 dp の高速化

- 計算量 $O(N^5)$

$$L_a[l][r][b] := \min\{a \text{ s.t. } dp[l][r][a][b] = \text{true}\}$$

$$R_a[l][r][b] := \max\{a \text{ s.t. } dp[l][r][a][b] = \text{true}\}$$

$$L_b[l][r][a] := \min\{b \text{ s.t. } dp[l][r][a][b] = \text{true}\}$$

$$R_b[l][r][a] := \max\{b \text{ s.t. } dp[l][r][a][b] = \text{true}\}$$

小課題 1: 区間 dp の高速化

- 計算量 $O(N^5)$

$$L_a[l][r][b] := \min\{a \text{ s.t. } dp[l][r][a][b] = \text{true}\}$$

$$R_a[l][r][b] := \max\{a \text{ s.t. } dp[l][r][a][b] = \text{true}\}$$

$$L_b[l][r][a] := \min\{b \text{ s.t. } dp[l][r][a][b] = \text{true}\}$$

$$R_b[l][r][a] := \max\{b \text{ s.t. } dp[l][r][a][b] = \text{true}\}$$

- 計算量 $O(N^3M)$

目標のカード (T_j, W_j) を固定すると、各座標は T_j との大小、 W_j との大小のみが関わる。

$$dp[l][r][<, =, > T_j][<, =, > W_j]$$

小課題 2: $N \leq 2000$, $M \leq 10$

最終的にカード (T_j, W_j) を作るとして、そのカードを作る手順を考える。すると、操作は以下のように表せる。

1. 区間 $[l, r]$ でカード (T_j, W_j) を作る。
2. 以降の操作では、カード (T_j, W_j) を常に保つように操作する。
2. が可能になるカード (T_j, W_j) の両側にあるカードの条件を考える。

小課題 2: $N \leq 2000, M \leq 10$

2. が可能になる条件は以下.

カード (T_j, W_j) の両側について, そちら側が空か, もしくは以下のいずれかでもない.

- $(> T_j, < W_j)$ なるカードのみからなる.
- $(< T_j, > W_j)$ なるカードのみからなる.

つまり各側について, その区間内の整数組の $\min$ が $(\leq T_j, \leq W_j)$ となる, もしくは $\max$ が $(\geq T_j, \geq W_j)$ となることが条件.

小課題 2: $N \leq 2000, M \leq 10$

1. で用いる区間 $[l, r]$ の長さで場合分けをする.

- $l = r$ の場合, 先程の条件で判定可能.
- $l < r$ の場合, ある $l \leq m < r$ について区間 $[l, m]$ で作ったカードと区間 $[m+1, r]$ で作ったカードを用いてカード (T_j, W_j) を作る.

小課題 2: $N \leq 2000, M \leq 10$

1. で用いる区間 $[l, r]$ の長さで場合分けをする.

- $l = r$ の場合, 先程の条件で判定可能.
- $l < r$ の場合, ある $l \leq m < r$ について区間 $[l, m]$ で作ったカードと区間 $[m+1, r]$ で作ったカードを用いてカード (T_j, W_j) を作る.

区間 $[1, l-1]$ と区間 $[r+1, n]$ で作ることができるカードを用いると, 実は $[l, r] = [1, n]$ の場合に帰着できる.

小課題 3: $N \leq 200000$, $M \leq 10$

$[l, r] = [1, n]$ の場合を考える. m の値ごとに場合分けをすると,

- $[1, m]$ で $(T_j, \leq W_j)$, $[m+1, n]$ で $(\leq T_j, W_j)$ を作る場合
- $[1, m]$ で $(T_j, \geq W_j)$, $[m+1, n]$ で $(\geq T_j, W_j)$ を作る場合
- $[1, m]$ で $(\leq T_j, W_j)$, $[m+1, n]$ で $(T_j, \leq W_j)$ を作る場合
- $[1, m]$ で $(\geq T_j, W_j)$, $[m+1, n]$ で $(T_j, \geq W_j)$ を作る場合

の 4 通りに場合分けされる.

小課題 2: $N \leq 2000, M \leq 10$

対称性から、以下の問題に帰着される。

区間 $[a, b]$ にあるカードを用いて、 $(T_j, \leq W_j)$ なるカードを作ることができるか判定せよ。

実際には $[a, b] = [1, m], [m + 1, n]$ の場合について上記の問題を解く。

作るべきカードの条件も $(\leq T_j, W_j)$ や $(T_j, \geq W_j)$, $(\geq T_j, W_j)$ を扱う必要がある。

小課題 2: $N \leq 2000, M \leq 10$

緩和問題として以下を考える.

区間 $[a, b]$ にあるカードを用いて, $(\geq T_j, \leq W_j)$ なるカードを作ることができるか判定せよ.

このとき, 以下の重要な性質がある.

$(< T_j, *)$, $(*, > W_j)$ というカードの集合は, $\max$ と $\min$ の操作に関して閉じている.

つまり, $(\geq T_j, \leq W_j)$ なるカードが 1 枚以上必要.

小課題 2: $N \leq 2000, M \leq 10$

区間 $[a, b]$ にあるカードを用いて, $(\geq T_j, \leq W_j)$ なるカードを作ることができるか判定せよ.

先程の観察を使うと, $l = r$ の場合と同様に緩和問題が判定できる.

- $(\geq T_j, \leq W_j)$ なるカードがあり, そのカードを常に保つように操作でき+るか判定する.
- そのカードの両側について, その区間内の整数組の $\min$ が $(*, \leq W_j)$ となる, もしくは $\max$ が $(\geq T_j, *)$ となることが条件.

小課題 2: $N \leq 2000, M \leq 10$

元の問題を考える.

区間 $[a, b]$ にあるカードを用いて, $(T_j, \leq W_j)$ なるカードを作ることができるか判定せよ.

実は先程の緩和問題に加えて $(T_j, *)$ があれば十分.

- $(\geq T_j, \leq W_j)$ なるカードがあり, そのカードの両側について, その区間内の整数組の $\min$ が $(*, \leq W_j)$ となる, もしくは $\max$ が $(\geq T_j, *)$ となるとする.
- この状況で $(T_j, *)$ があれば, うまく操作をすれば $(T_j, \leq W_j)$ が作れる.

小課題2 (総括) : $N \leq 2000$, $M \leq 10$

1. $(T_j, W_j) = (S_i, V_i)$ なる各カード i について, 両側の条件を調べる.
2. $1 \leq m < n$ なる各 m について $[1, m]$ および $[m+1, n]$ で $(T_j, \leq W_j)$ etc. を作れるか判定する.
3. $(\geq T_j, \leq W_j)$ のカードを作れるかを 1. 同様に調べる.
そして, $(T_j, *)$ のカードがあるか判定する.

計算量はクエリ毎 $O(N^2)$

小課題 3: $N \leq 200000$, $M \leq 10$

小課題 2 の解法でボトルネックになるのは以下の部分.

- $1 \leq m < n$ なる各 m について $[1, m]$ および $[m+1, n]$ で $(T_j, \leq W_j)$ etc. を作れるか判定する.

m を全て試すのは時間が足りない.

→ 最適な m の条件を考えよう!

小課題 3: $N \leq 200000$, $M \leq 10$

- $x := \min\{i \text{ s.t. } S_i = T_j\}$
- $z := \min\{i \text{ s.t. } S_i \geq T_j, V_i \leq W_j, i \text{ の左側で条件成立 } \}$

このとき, $\max(x, z) \leq m$ が必要.

小課題 3: $N \leq 200000$, $M \leq 10$

- $x := \min\{i \text{ s.t. } S_i = T_j\}$
- $z := \min\{i \text{ s.t. } S_i \geq T_j, V_i \leq W_j, i \text{ の左側で条件成立}\}$

このとき, $\max(x, z) \leq m$ が必要.

同様に右側についても,

- $y := \max\{i \text{ s.t. } V_i = W_j\}$
- $w := \max\{i \text{ s.t. } S_i \leq T_j, V_i \geq W_j, i \text{ の右側で条件成立}\}$

とすると, $m < \min(y, w)$ が必要.

小課題 3: $N \leq 200000$, $M \leq 10$

実は $\max(x, z) < \min(y, w)$ ならば十分.

- $\max(x, z) \leq m < \min(y, w)$ なる範囲に条件を満たすものがあればいい.
- z の右側 $[z, m]$ での条件と w の左側 $[m+1, w]$ での条件は確認できていない.
- ちゃんと条件を書くと, 極端な場合 $m = \max(x, z)$ および $m+1 = \min(y, w)$ のいずれかでは必ず成立する!
 - 各自確認してみてください.

小課題 3: $N \leq 200000$, $M \leq 10$

x, y, z, w の計算は $O(N)$ の時間計算量で可能.

実際には対称性を考えて 4 パターン扱う必要があることに注意.

- $[1, m]$ で $(T_j, \leq W_j)$, $[m+1, n]$ で $(\leq T_j, W_j)$ を作る場合.
- $[1, m]$ で $(T_j, \geq W_j)$, $[m+1, n]$ で $(\geq T_j, W_j)$ を作る場合.
- $[1, m]$ で $(\leq T_j, W_j)$, $[m+1, n]$ で $(T_j, \leq W_j)$ を作る場合.
- $[1, m]$ で $(\geq T_j, W_j)$, $[m+1, n]$ で $(T_j, \geq W_j)$ を作る場合.

満点解法: $N \leq 200000, M \leq 200000$

x, y, z, w の計算を高速化する.

- $x := \min\{i \text{ s.t. } S_i = T_j\}$

この計算は, 座標圧縮をしておけば $O(1)$ で可能.

- $z := \min\{i \text{ s.t. } S_i \geq T_j, V_i \leq W_j, i \text{ の左側で条件成立 } \}$

この計算を高速化したい.

満点解法: $N \leq 200000, M \leq 200000$

x, y, z, w の計算を高速化する.

- $x := \min\{i \text{ s.t. } S_i = T_j\}$

この計算は, 座標圧縮をしておけば $O(1)$ で可能.

- $z := \min\{i \text{ s.t. } S_i \geq T_j, V_i \leq W_j, i \text{ の左側で条件成立 } \}$

この計算を高速化したい.

→ 二次元クエリに帰着された!

満点解法: $N \leq 200000$, $M \leq 200000$

二次元クエリに帰着するために, i の左側の条件を取り除きたい.

- その条件以外の条件を満たす最小の i を z' とする.
- z' の左側で条件が成立していれば $z = z'$.
- そうでなければ, z は $i \neq z'$ なる i で残りの条件を満たすものの最小値となる.

満点解法: $N \leq 200000$, $M \leq 200000$

二次元クエリに帰着するために, i の左側の条件を取り除きたい.

- その条件以外の条件を満たす最小の i を z' とする.
- z' の左側で条件が成立していれば $z = z'$.
- そうでなければ, z は $i \neq z'$ なる i で残りの条件を満たすものの最小値となる.

これより, 以下のクエリに帰着される.

- i 番目のペア (S_i, V_i) を取り除く・追加する
- $\min\{i \text{ s.t. } S_i \geq T_j, V_i \leq W_j\}$ を求める.

満点解法: $N \leq 200000$, $M \leq 200000$

- i 番目のペア (S_i, V_i) を取り除く・追加する
- $\min\{i \text{ s.t. } S_i \geq T_j, V_i \leq W_j\}$ を求める.

このクエリには

- 二次元 segment tree: $O(\log^2 N)$
- クエリ先読み + 平面操作: $O(\log N)$

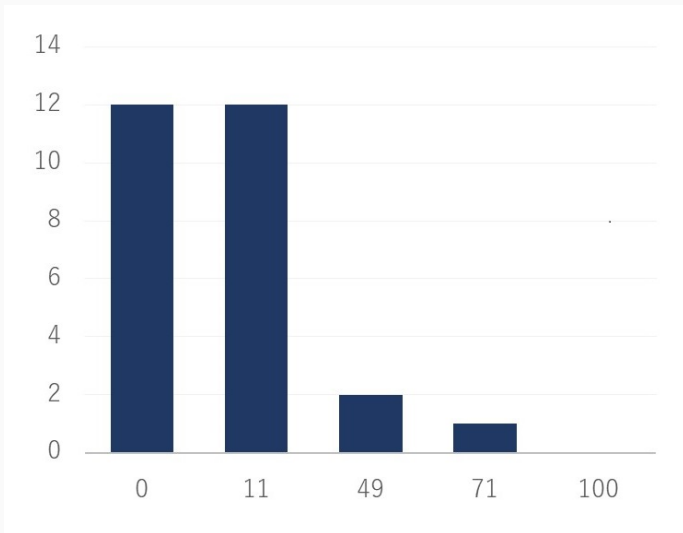
で答えることができる.

V_i, W_j の昇順に追加・応答を行うと, クエリは

$$\min\{i \text{ s.t. } S_i \geq T_j\}$$

となる. これは *segment tree* 上の二分探索で求まる.

得点分布



JOI 2023/2024 春季トレーニング Day 3

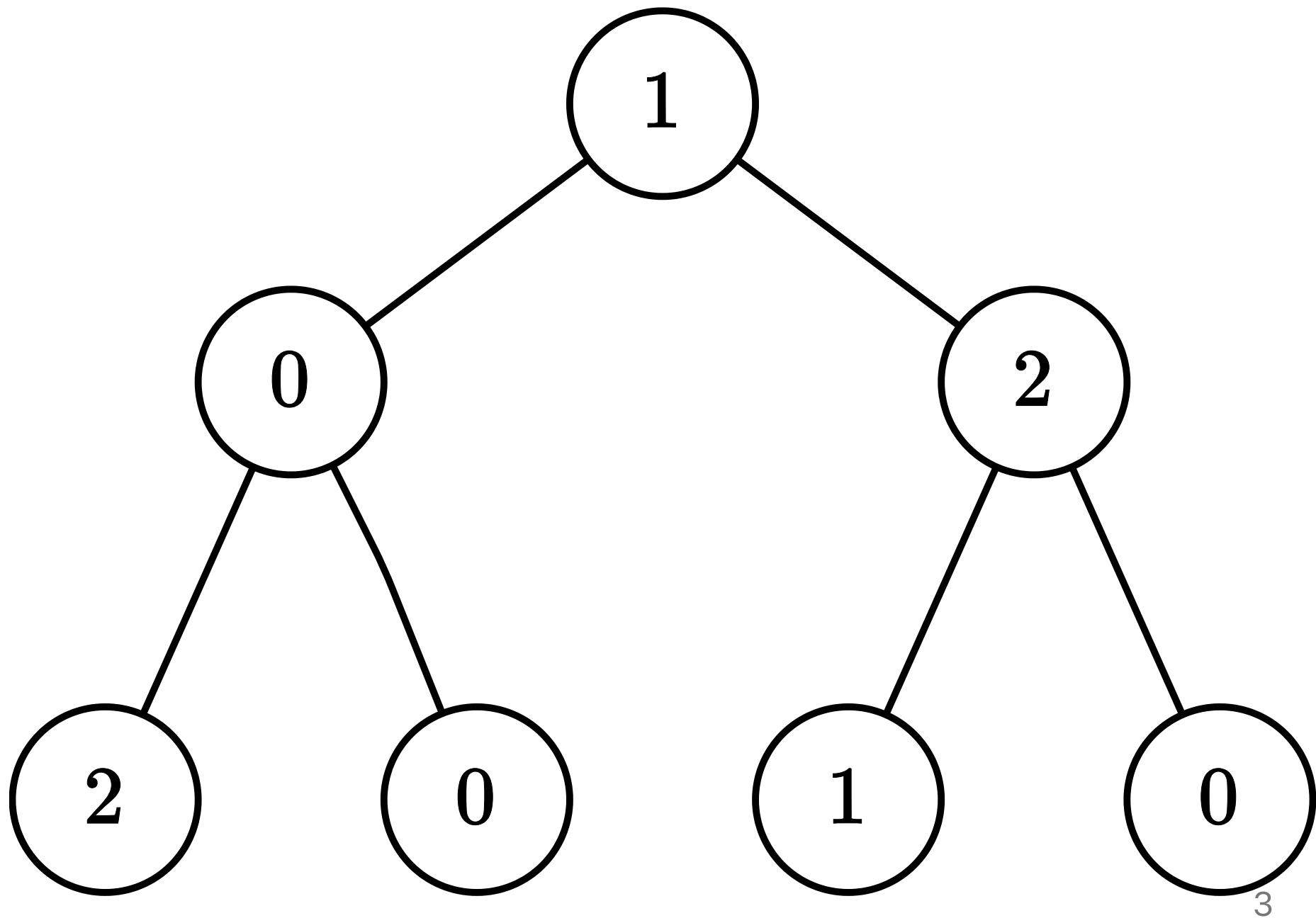
JOI ツアー (JOI Tour) 解説

解説担当 : tatyam

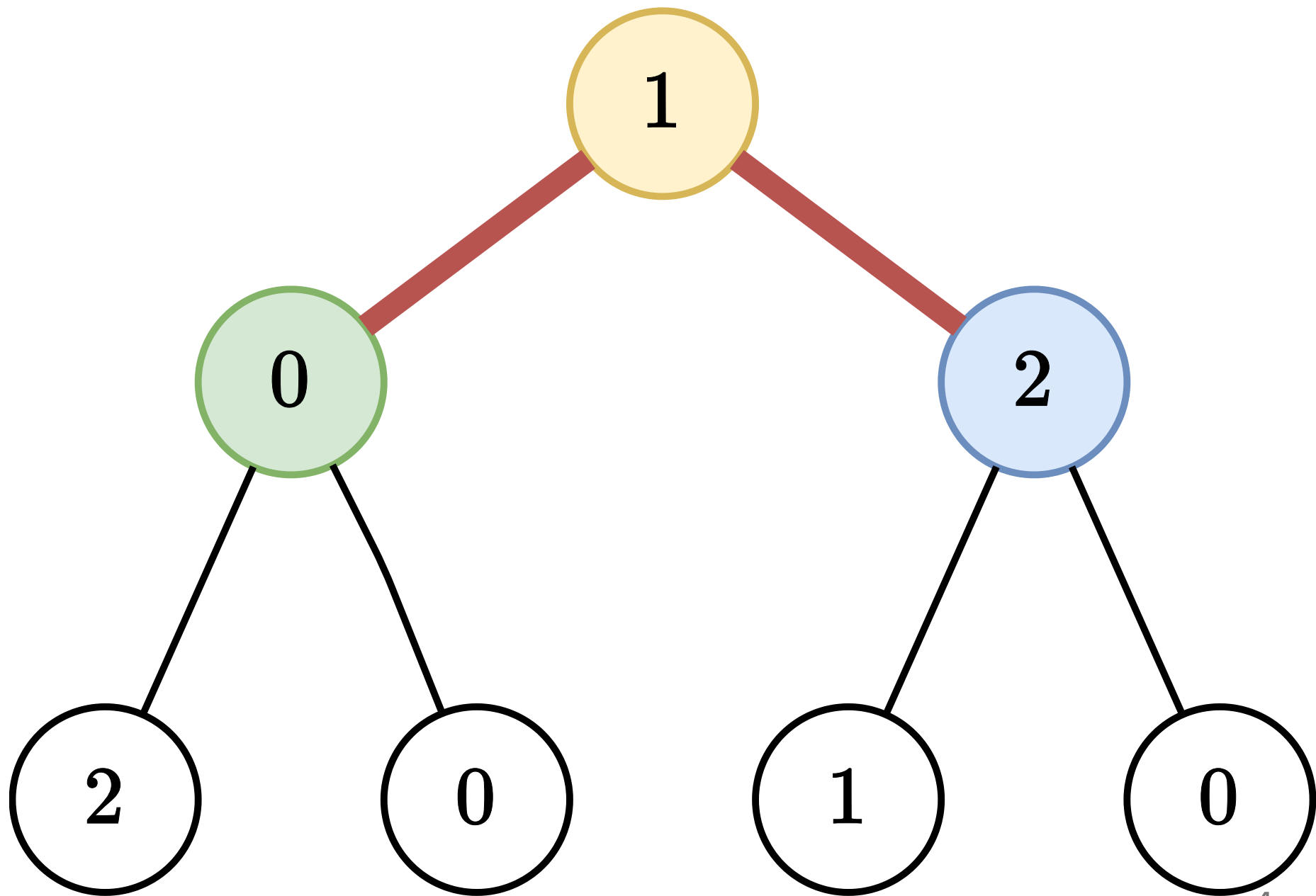
問題概要

- N 頂点の木があって、各頂点には $0, 1, 2$ いずれかの整数が書かれている.
- Q クエリを **オンラインで** 処理:
 - 頂点 X に書かれている整数を Y に変更
 - その後、 $0 - 1 - 2$ パスの個数を出力

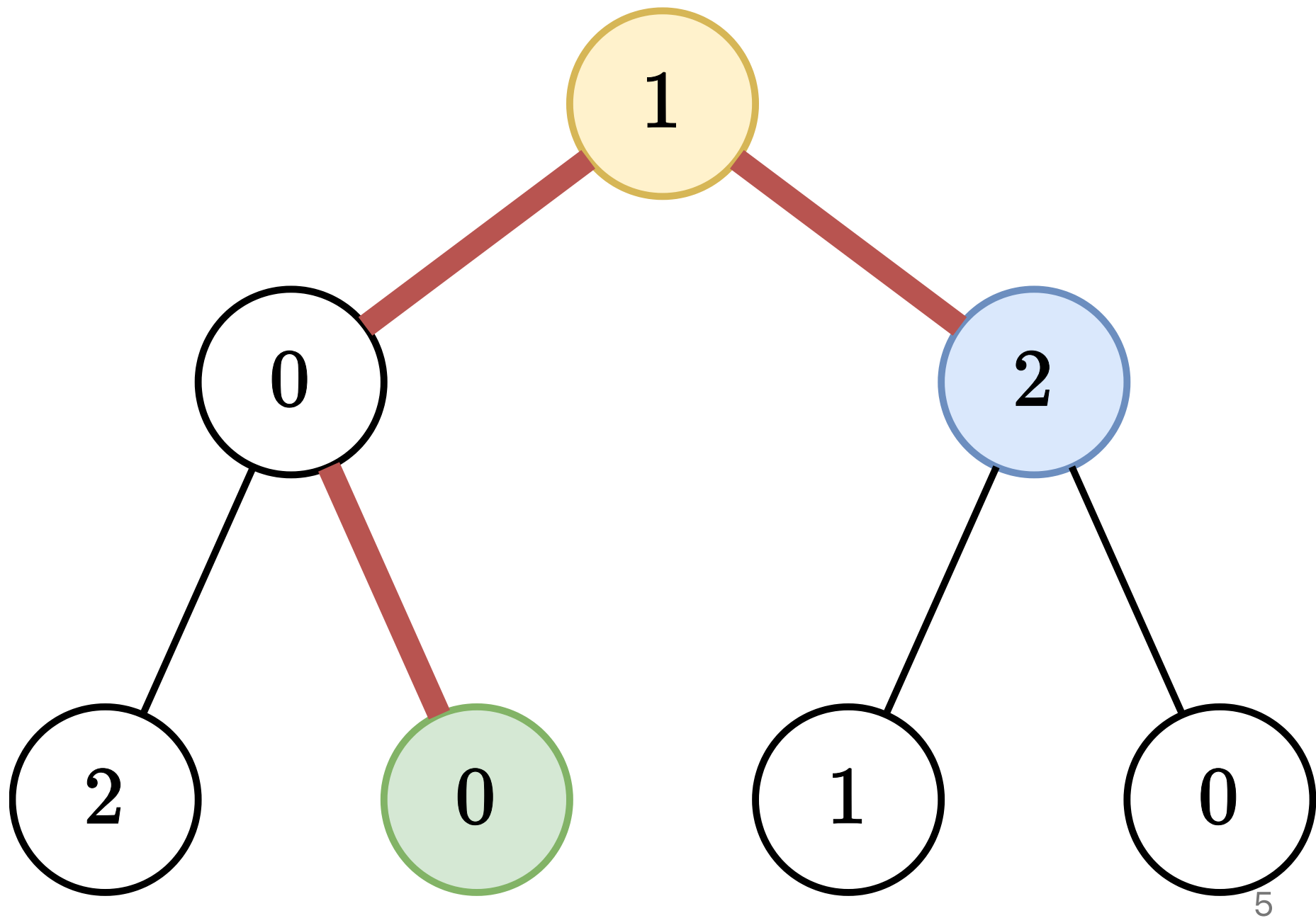
例



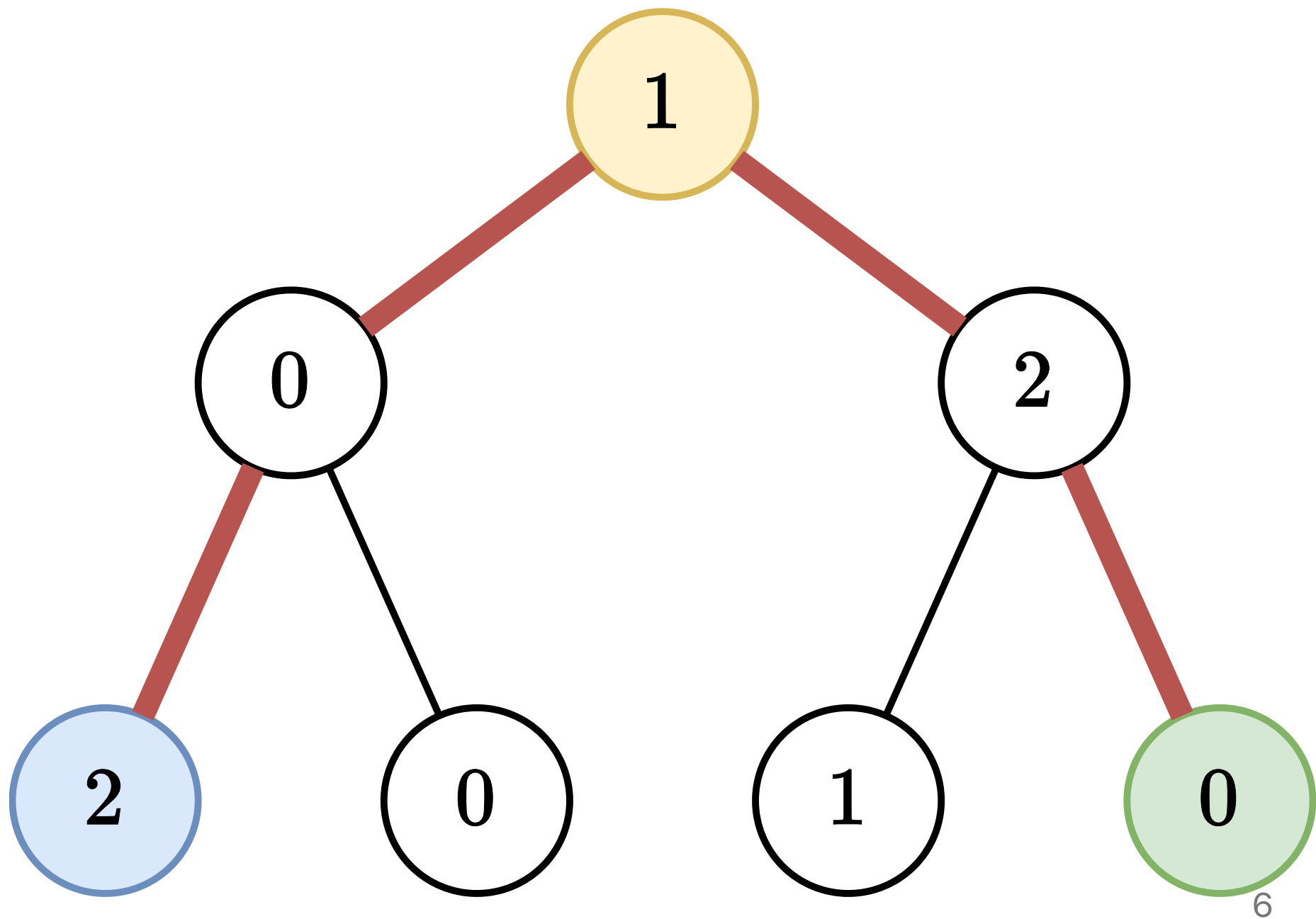
例



例



例



小課題

番号	配点	制約	計算量の例
1.	6 点	$N \leq 400, Q \leq 100$	$O(QN^2)$
2.	8 点	$N \leq 4000, Q \leq 1000$	$O(N^2 + QN)$
3.	6 点	$Q = 0$	$O(N)$
4.	16 点	パス	$O((N + Q) \log N)$
5.	16 点	完全 2 分木	$O((N + Q) \log N)$
6.	34 点	$N \leq 100000, Q \leq 25000$	
7.	14 点	$N \leq 200000, Q \leq 50000$	$O(N \log N + Q(\log N)^2)$

小課題 1 : $N \leq 400, Q \leq 100$ (6 点)

- 一旦変更クエリを忘れてみる
- 木が与えられたとき, $0 - 1 - 2$ パスの個数を $O(N^2)$ 時間で数える

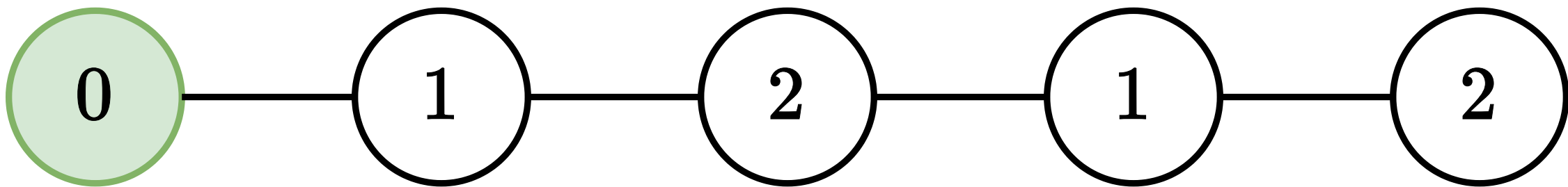
0 — 1 — 2 パスの個数を $O(N^2)$ 時間で数えたい

- 使う 0 を固定してみる

→ 指定された 0 から始まる 0 — 1 — 2 パスの個数を $O(N)$ 時間で数えたい

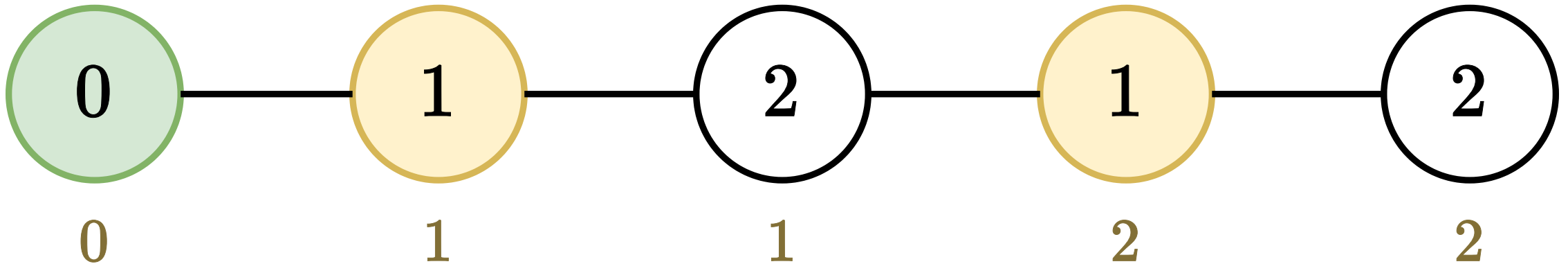
パスの場合は... ?

- 使う 0 を固定してみる



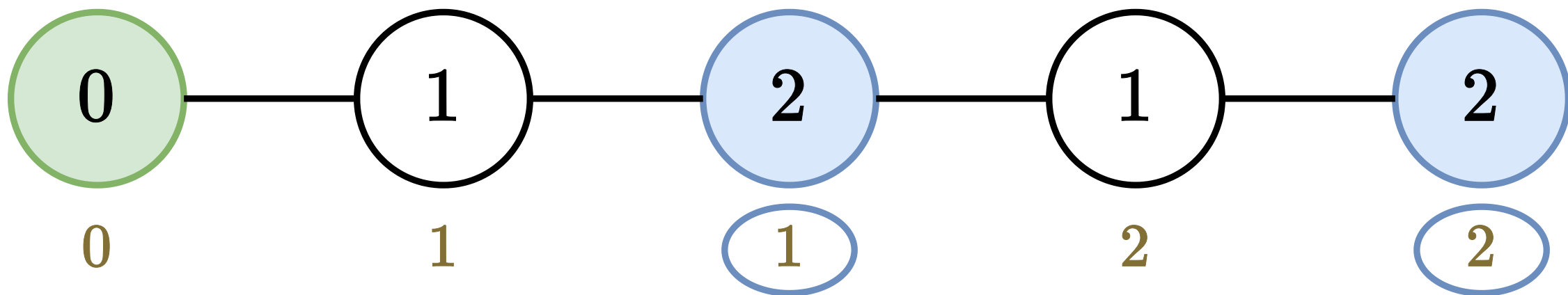
パスの場合は... ?

- 使う 0 を固定してみる
- 0 から各位置までの 1 の個数を求めて...



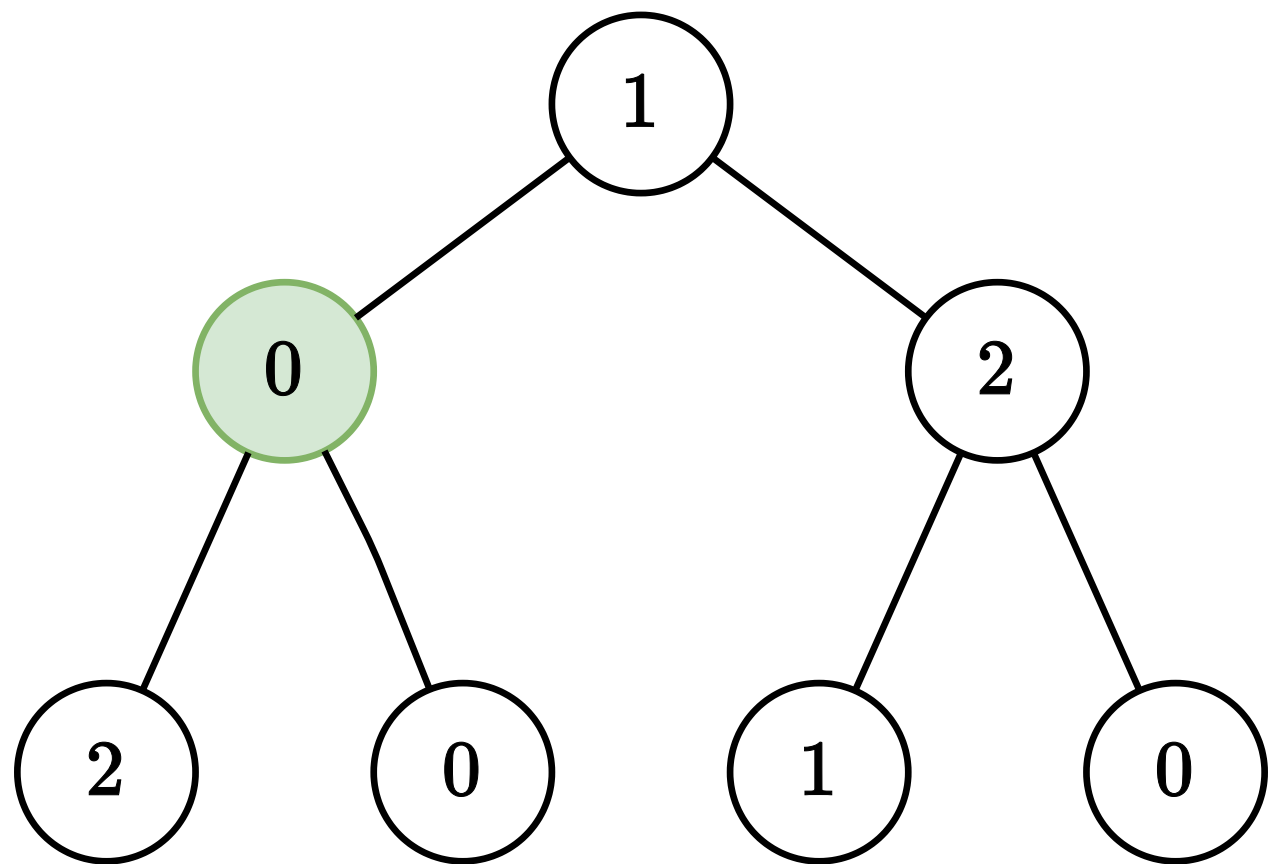
パスの場合は... ?

- 使う 0 を固定してみる
- 0 から各位置までの 1 の個数を求めると,
- 使う 0 と 2 を固定した場合の $0 - 1 - 2$ パスの個数は, $0 - 2$ 間の 1 の個数に等しい



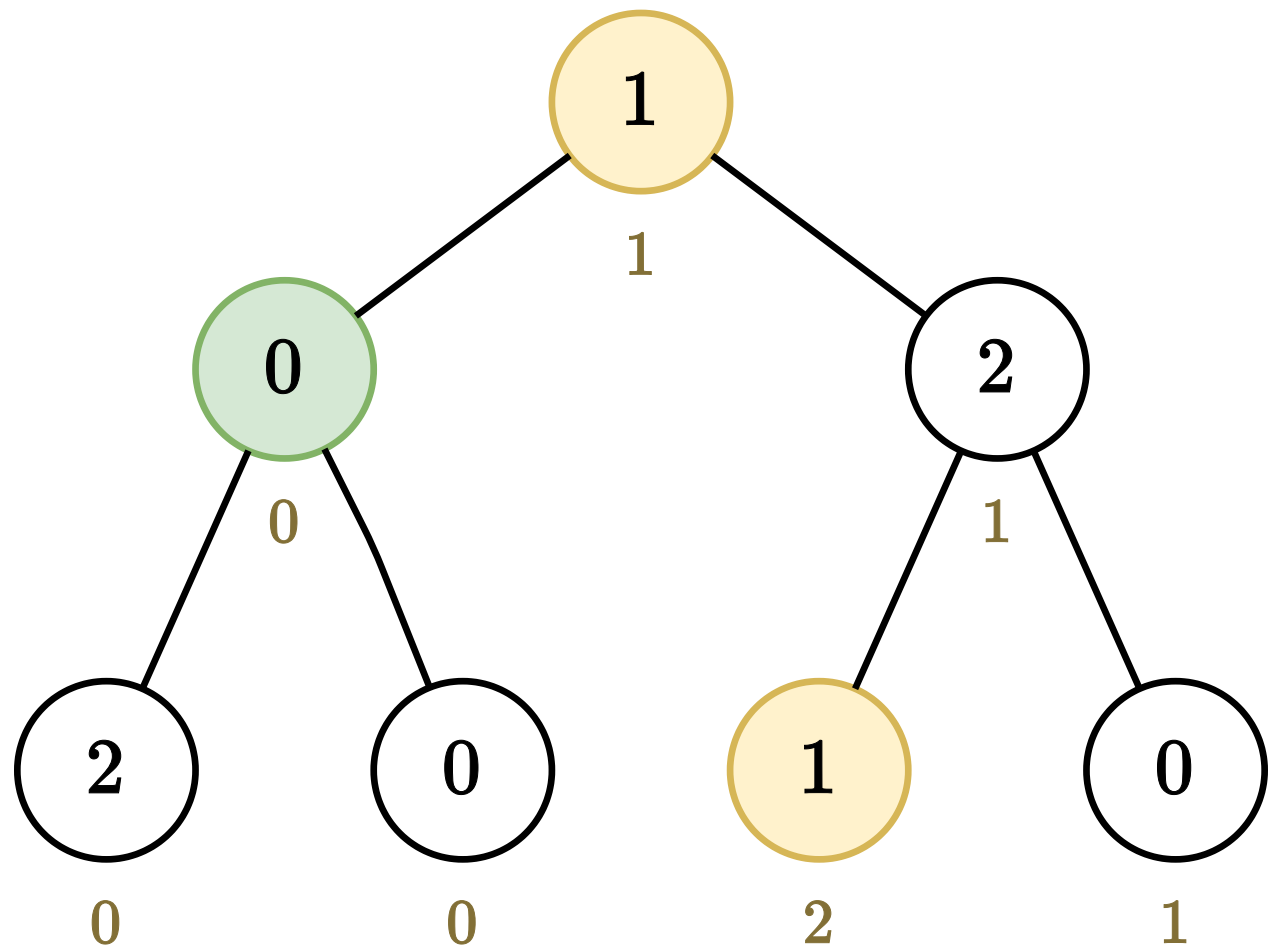
木の場合は... ?

- 同じことを DFS
しながら行う



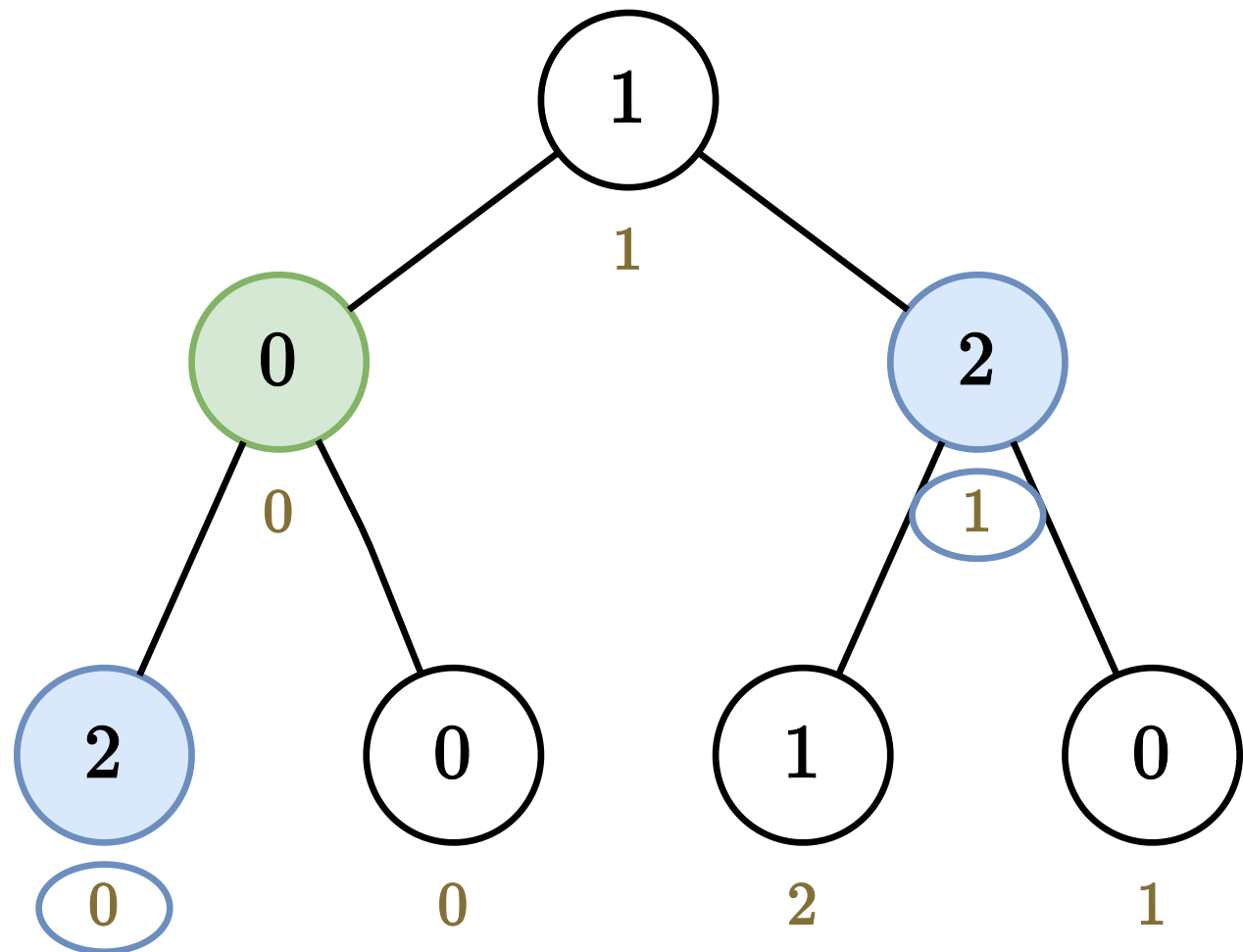
木の場合は... ?

- 同じことを DFS
しながら行う
- 0 から各位置までの 1 の
個数を求めると,



木の場合は... ?

- 同じことを DFS しながら行う
- 0 から各位置までの 1 の個数を求めると,
- 使う 0 と 2 を固定した場合の 0 - 1 - 2 パスの個数は, 0 - 2 間の 1 の個数に等しい



小課題 2 : $N \leq 4000, Q \leq 1000$ (8 点)

- 毎回 $\Theta(N^2)$ 時間掛けると TLE $\rightarrow$ 差分計算をしよう !

小課題 2 : $N \leq 4000, Q \leq 1000$ (8 点)

- 毎回 $\Theta(N^2)$ 時間掛けると TLE $\rightarrow$ 差分計算をしよう !

- 毎回 $\Theta(N^2)$ 時間掛けると TLE $\rightarrow$ 差分計算をしよう！

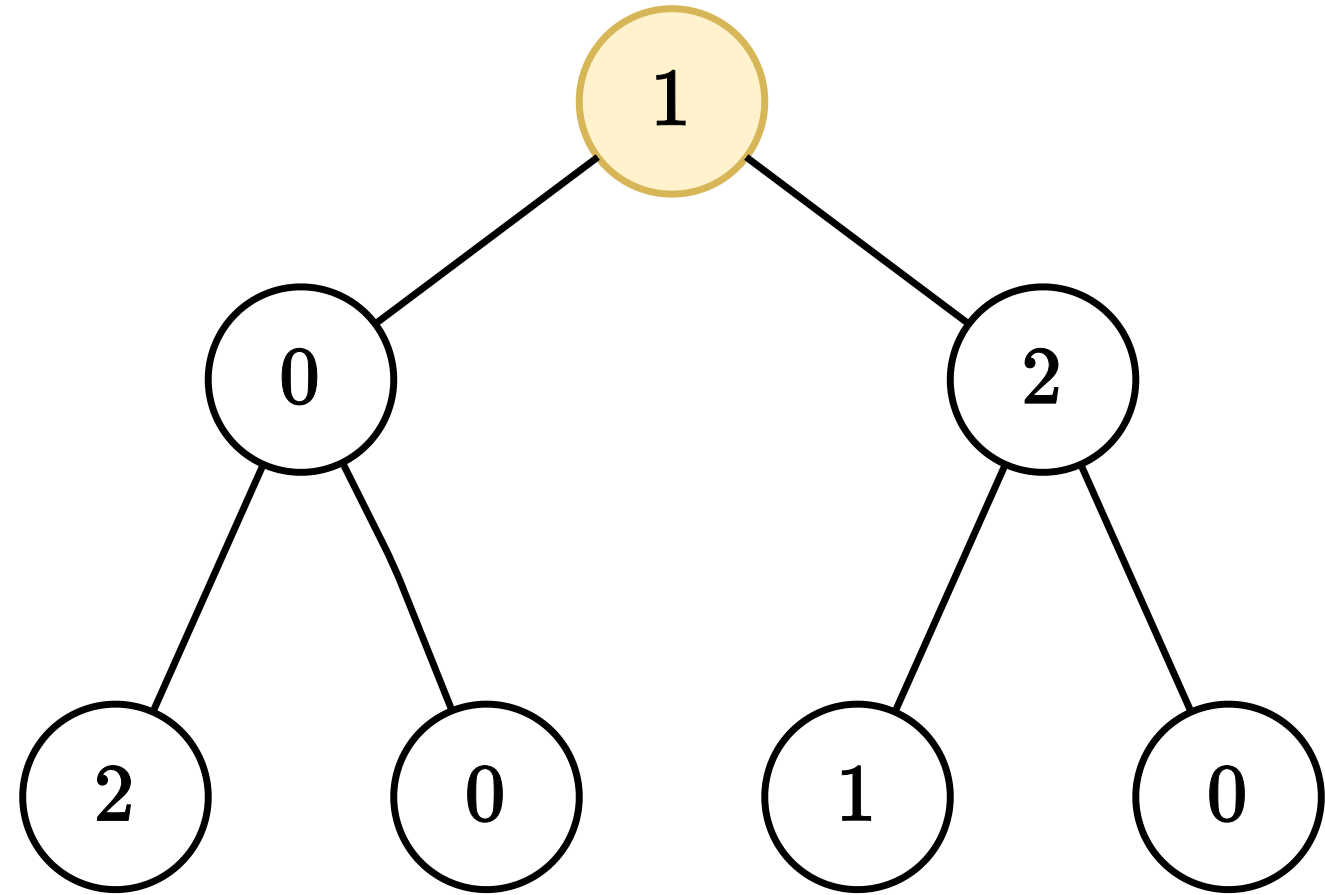
差分計算

1. 頂点 X を使う $0 - 1 - 2$ パスの個数を数えて, 答えから引く
2. F_X を Y に変更する
3. 頂点 X を使う $0 - 1 - 2$ パスの個数を数えて, 答えに足す

頂点 X を使う $0 - 1 - 2$ パスの個数

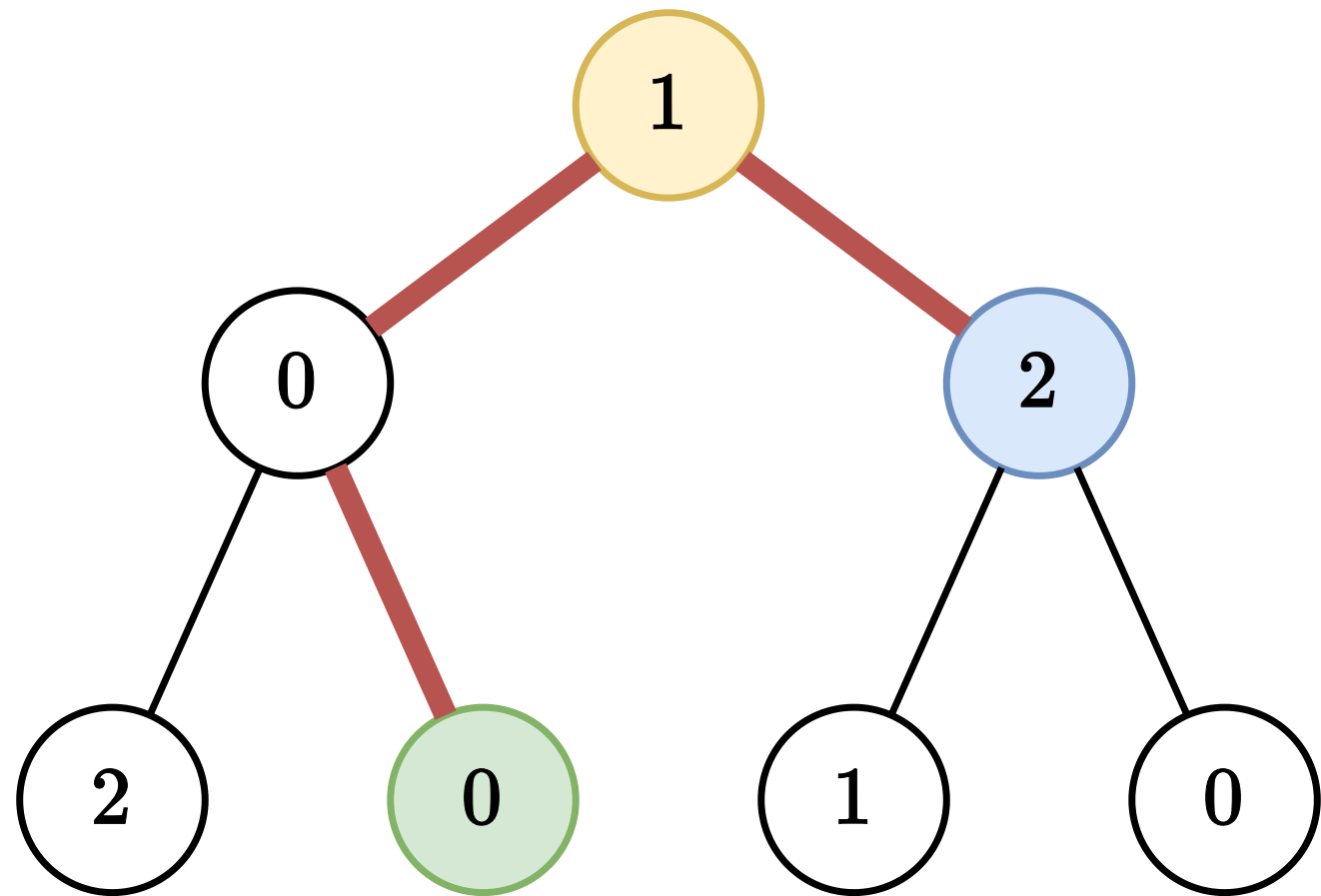
- $F_X = 0$ のとき : さっきやった
- $F_X = 2$ のとき : $0, 1, 2$ を反転させれば, さっきやった
- $F_X = 1$ のとき : ?

1 を指定されたときの
0 - 1 - 2 パスの個数



1 を指定されたときの
0 – 1 – 2 パスの個数

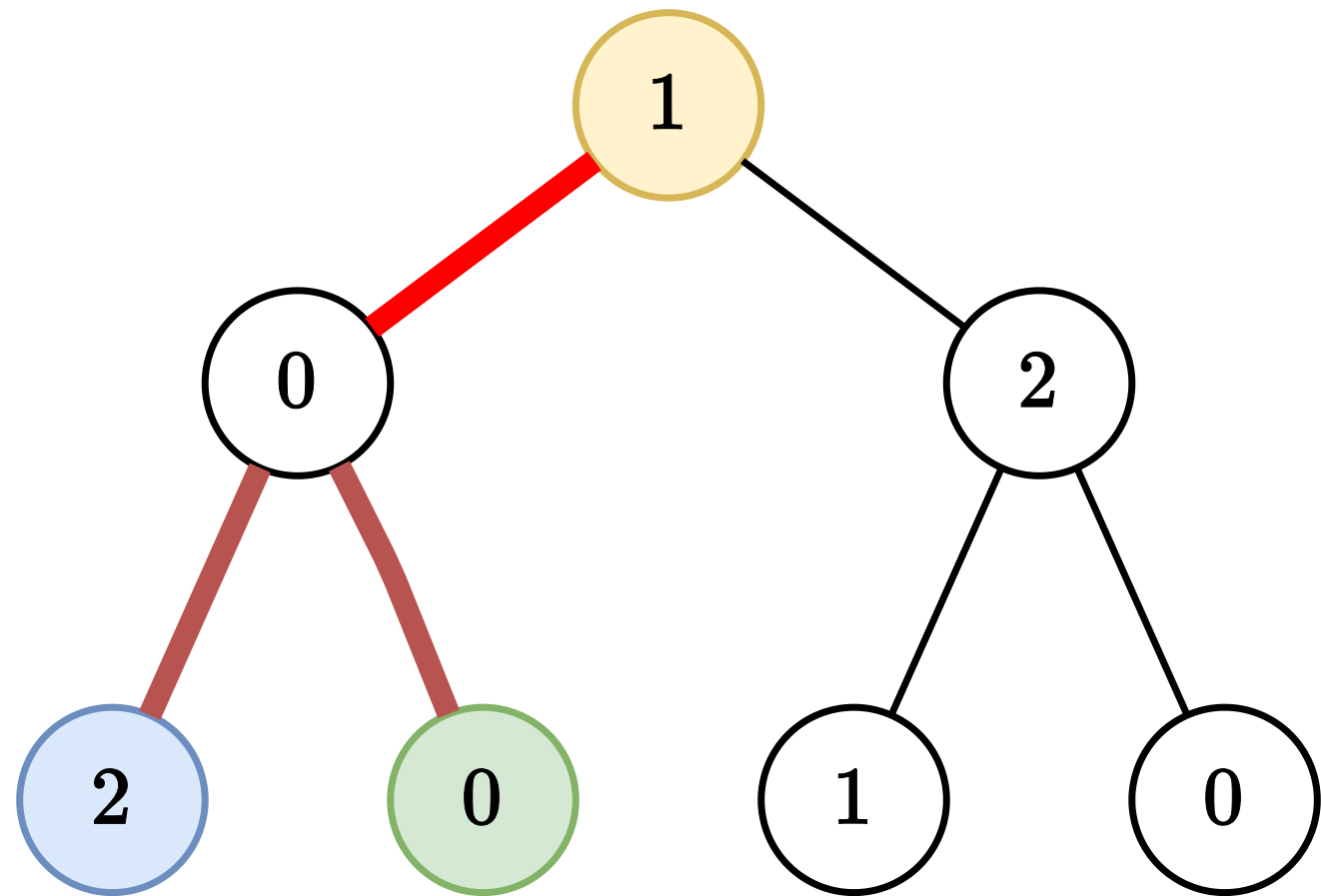
- 1 から見て 0 と 2 が
別の方向にあれば良い



1 を指定されたときの
0 — 1 — 2 パスの個数

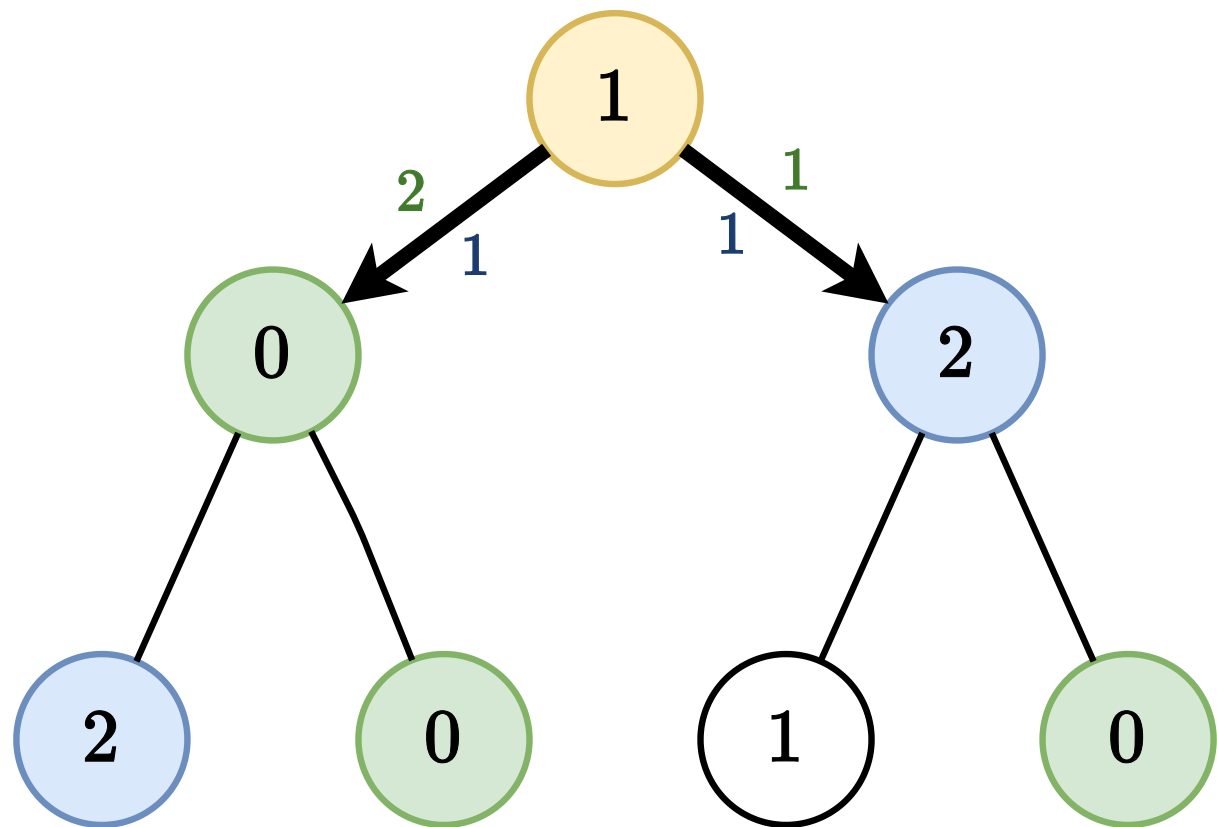
- 1 から見て 0 と 2 が
別の方向にあれば良い

(同じ方向にあると、同じ辺を
複数回通ってしまうため)



1 を指定されたときの 0 — 1 — 2 パスの個数

- 1 から見て 0 と 2 が別の方向にあれば良い
- 1 から各方向への 0 の個数, 2 の個数を数えて, $(0 \text{ の個数}) \times (2 \text{ の個数})$ から同じ方向にあるものを引く

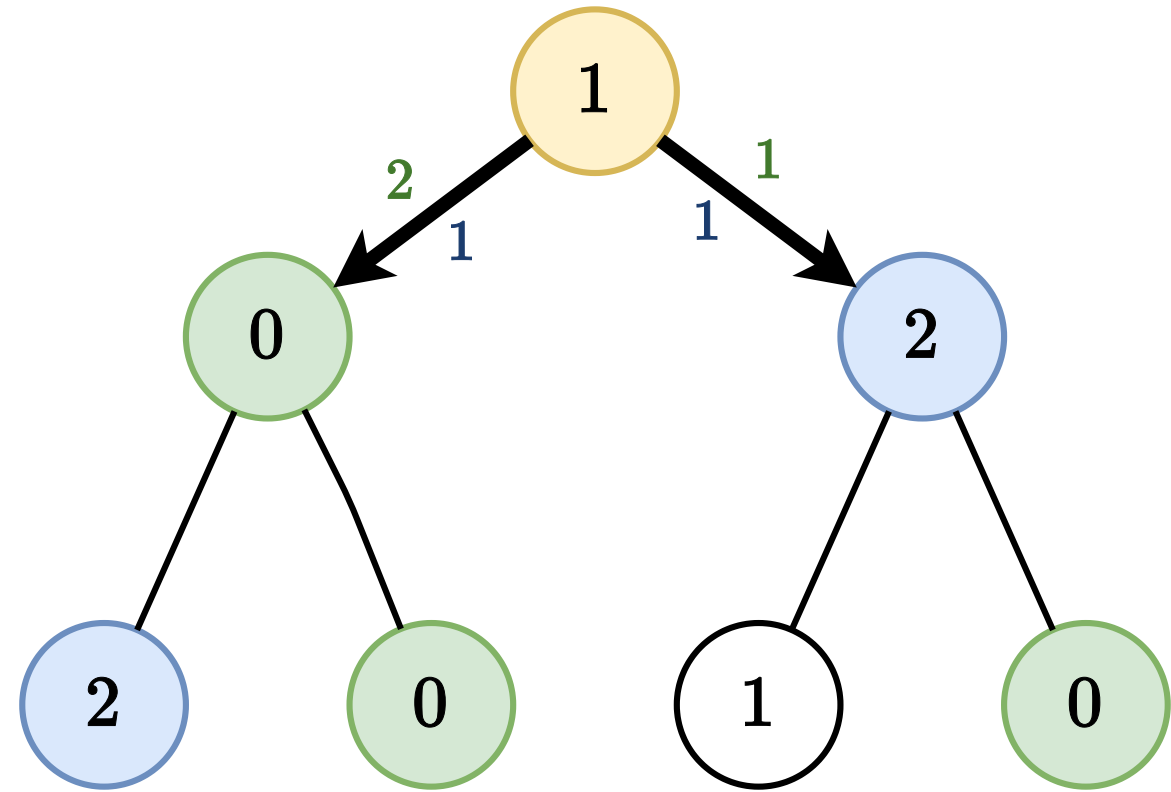


小課題 3 : $Q = 0$ (6 点)

- $0 - 1 - 2$ パスの個数を $O(N)$ 時間で数えれば良い
- 先ほどの「1 を指定されたときの $0 - 1 - 2$ パスの個数」に注目！

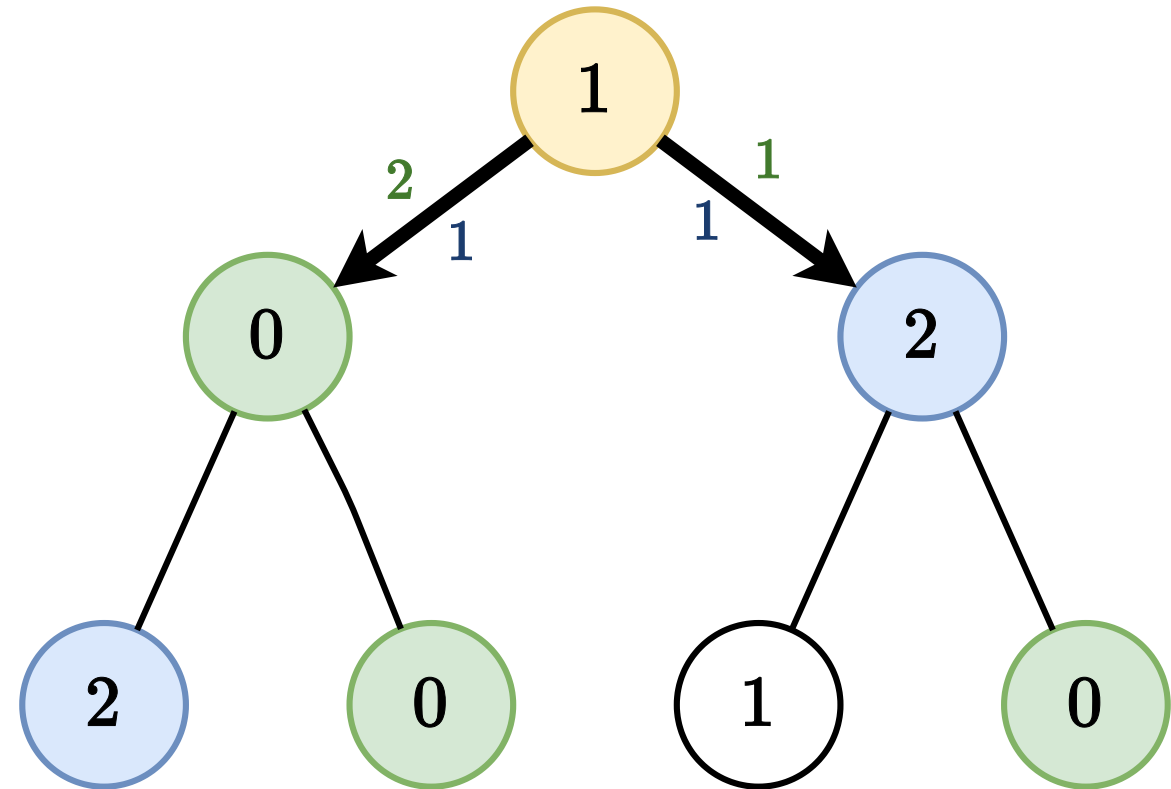
1 を指定されたときの
0 – 1 – 2 パスの個数

- 1 から各方向への
0 の個数, 2 の個数を数える



1 を指定されたときの
0 — 1 — 2 パスの個数

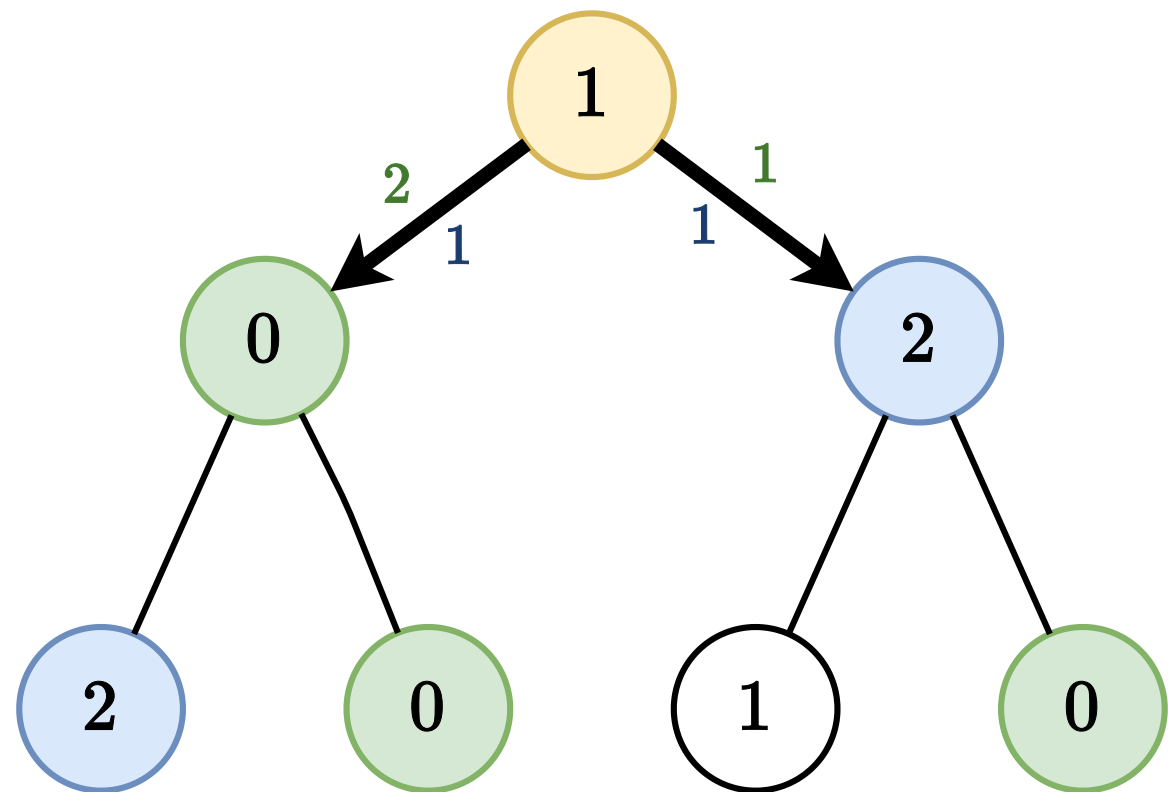
- **全頂点**から**全方向**への
0 の個数, 2 の個数を数える



1 を指定されたときの
0 — 1 — 2 パスの個数

- **全頂点**から**全方向**への
0 の個数, 2 の個数を数える

→ すべての部分木について
求めれば良く, $O(N)$ 時間



小課題 4 : パス (16 点)

- 変更がないとき, $dp[i \text{ まで見た}][j \text{ 文字一致した}]$ の DP
- 1 点変更がある

小課題 4 : パス (16 点)

- 変更がないとき, $dp[i \text{ まで見た}][j \text{ 文字一致した}]$ の DP
- 1 点変更がある

→ 列に対する DP をセグメント木に乗せよう！

列に対する DP をセグメント木に乗せる

- **区間**の $0 - 1 - 2$ パスの個数を求めたい

各区間は以下のデータを管理する

- $0 - 1 - 2$ の個数
- $0 - 1, 1 - 2, 1 - 0, 2 - 1$ の個数
- $0, 1, 2$ の個数

小課題 5 : 完全二分木 (16 点)

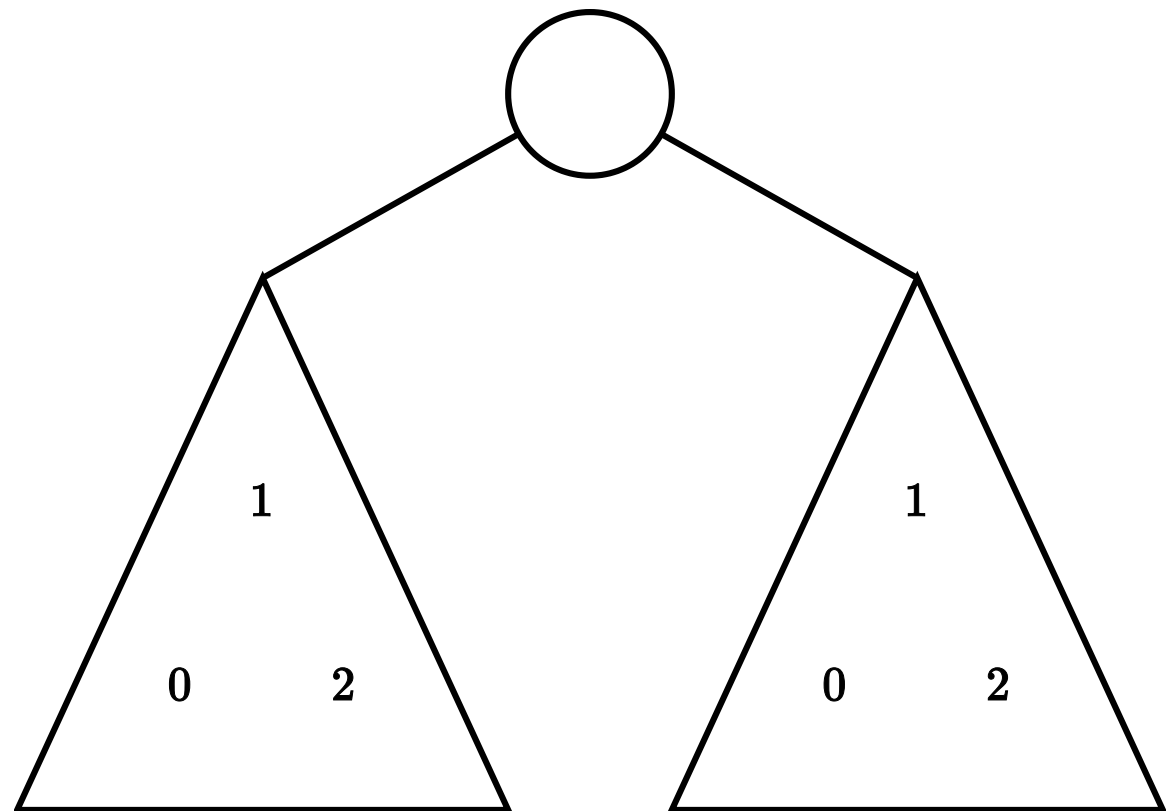
- 構造がほとんどセグメント木

→ 同様に解けないか？

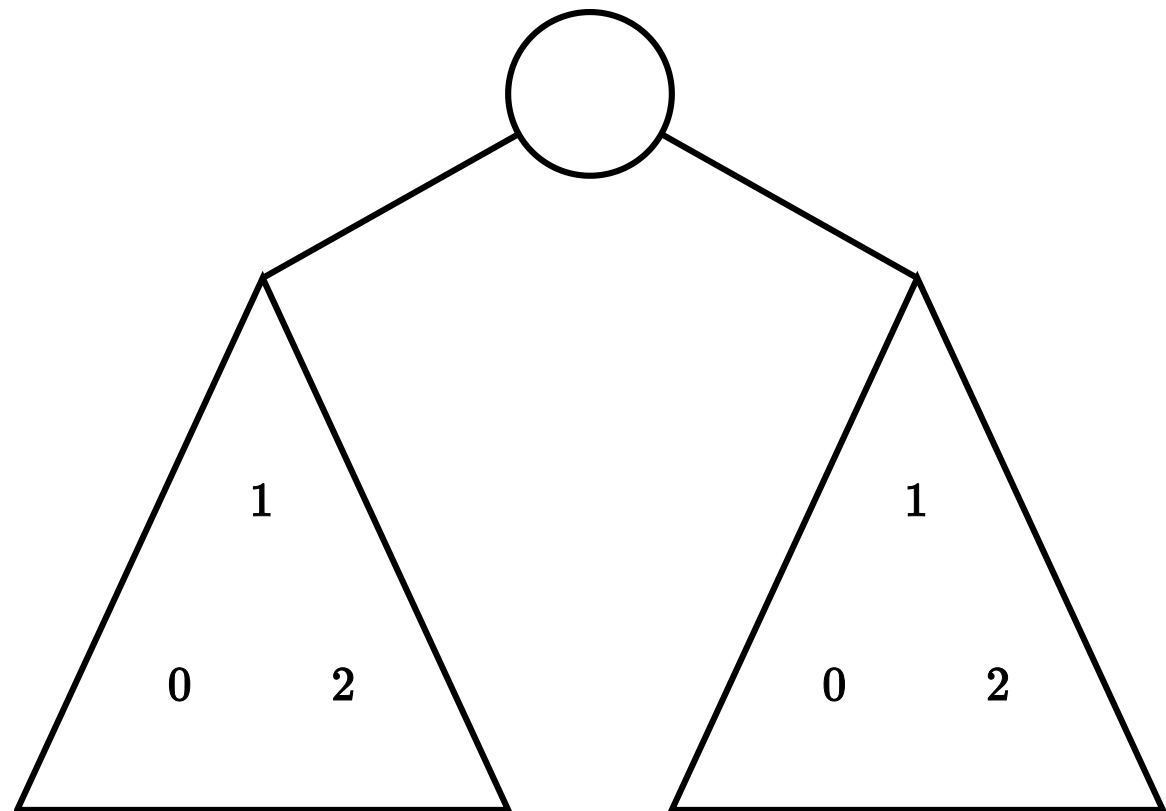
小課題 5 : 完全二分木 (16 点)

- **部分木**の 0 – 1 – 2 パスの
個数を求めたい

部分木のマージをするには... ?

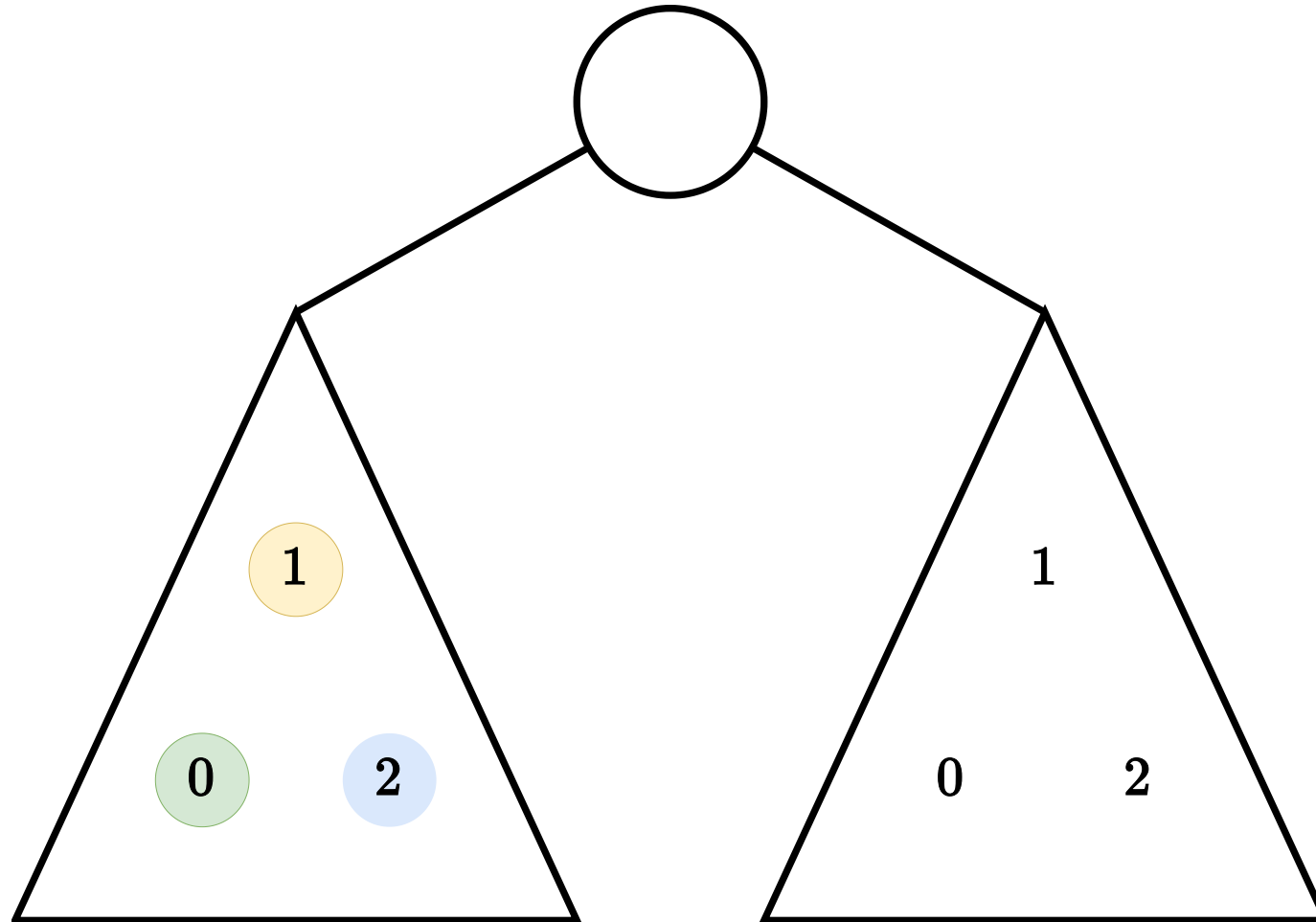


0 – 1 – 2 パスを
全パターン列挙しよう！



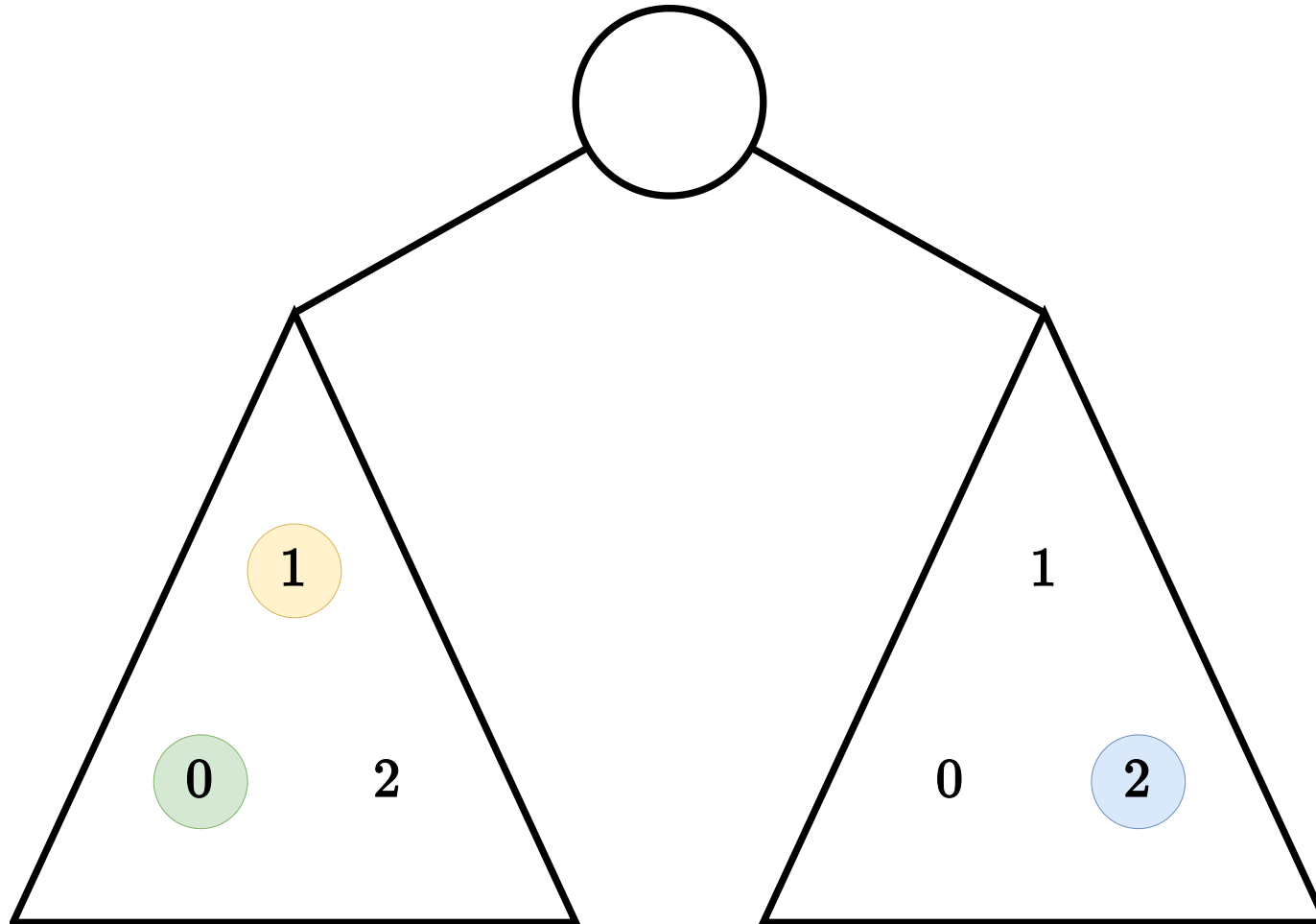
0 – 1 – 2 パスを全パターン列挙しよう！

- 中央の頂点を使わない場合



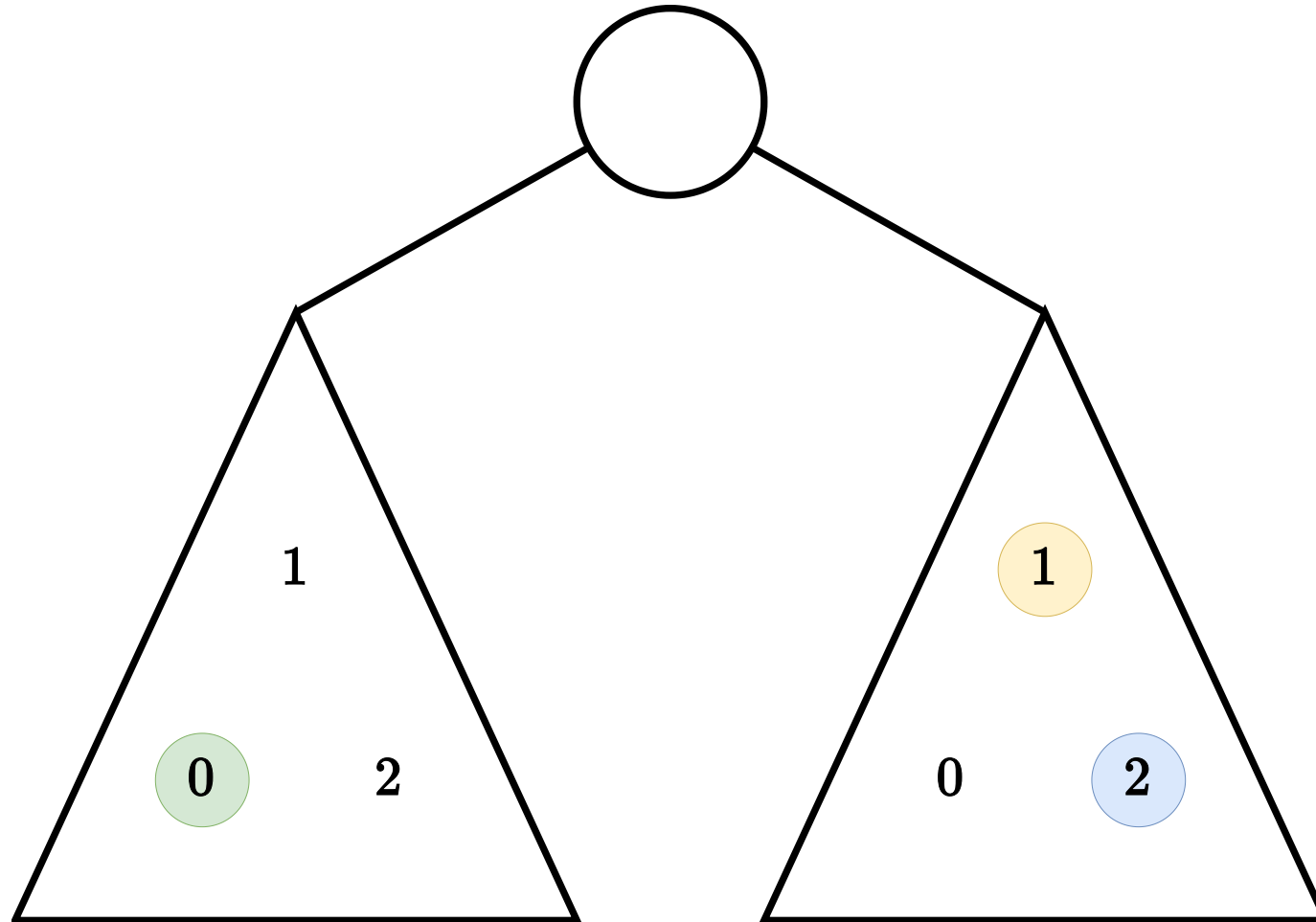
0 – 1 – 2 パスを全パターン列挙しよう！

- 中央の頂点を使わない場合



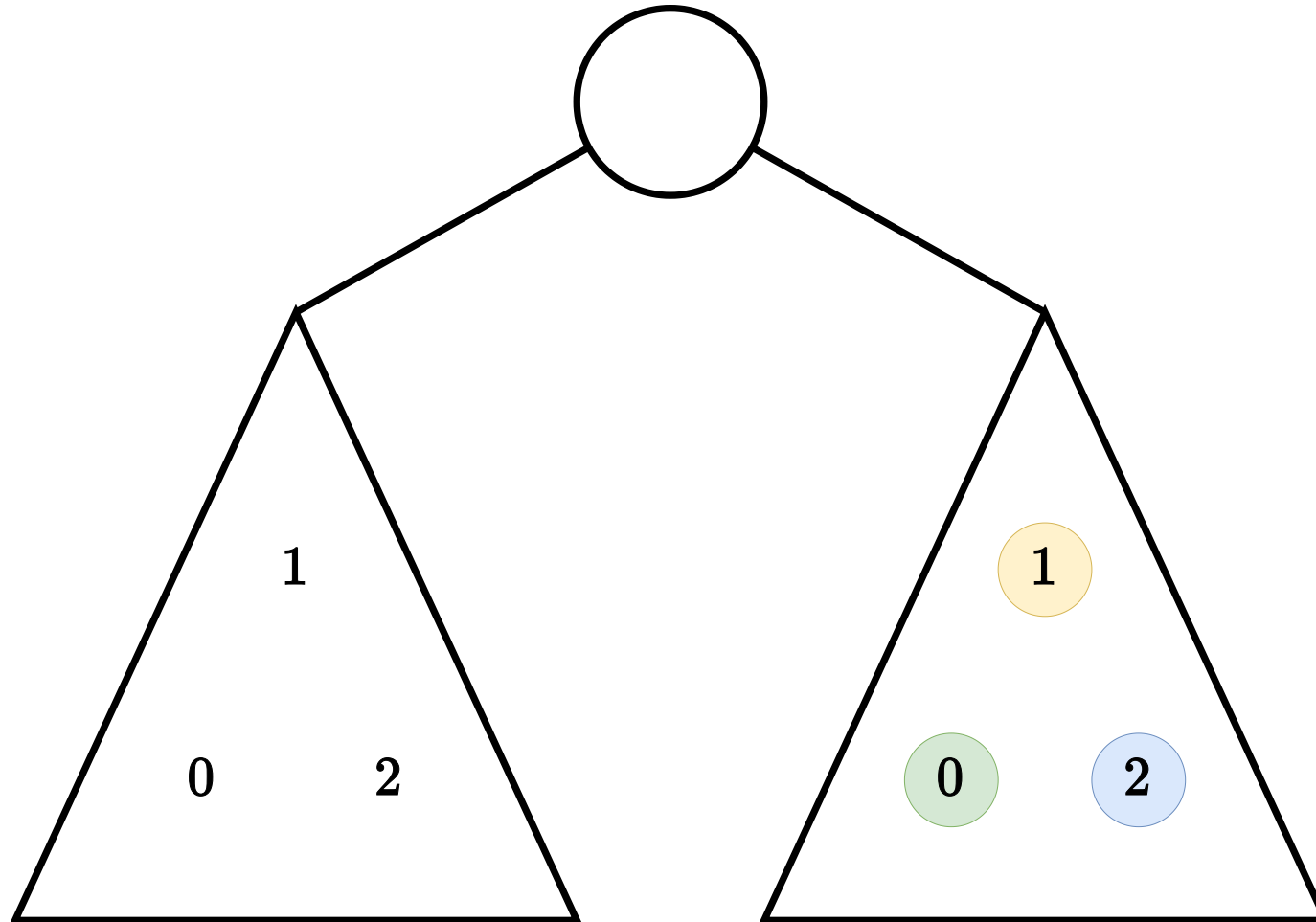
0 – 1 – 2 パスを全パターン列挙しよう！

- 中央の頂点を使わない場合



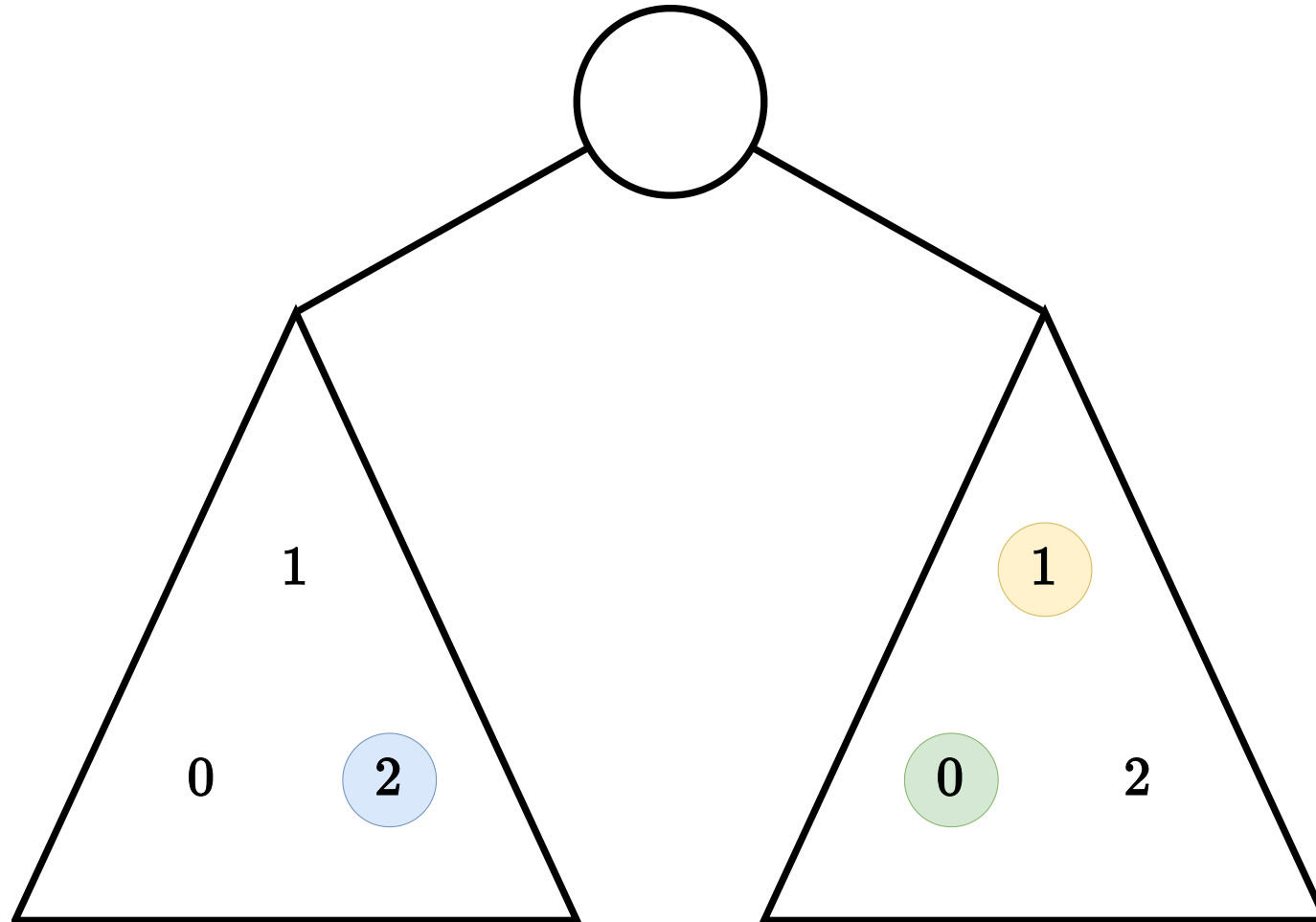
0 – 1 – 2 パスを全パターン列挙しよう！

- 中央の頂点を使わない場合



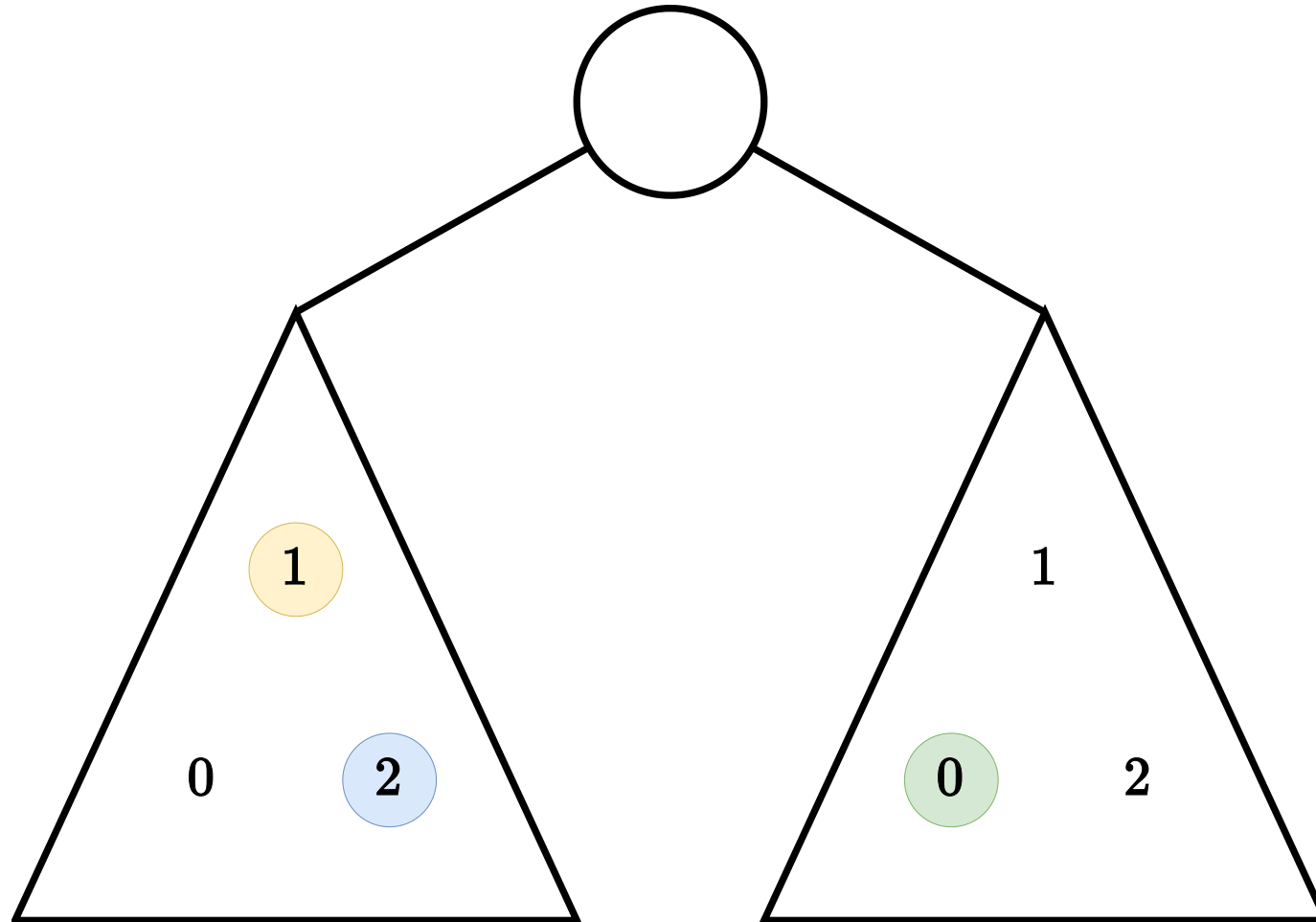
0 – 1 – 2 パスを全パターン列挙しよう！

- 中央の頂点を使わない場合



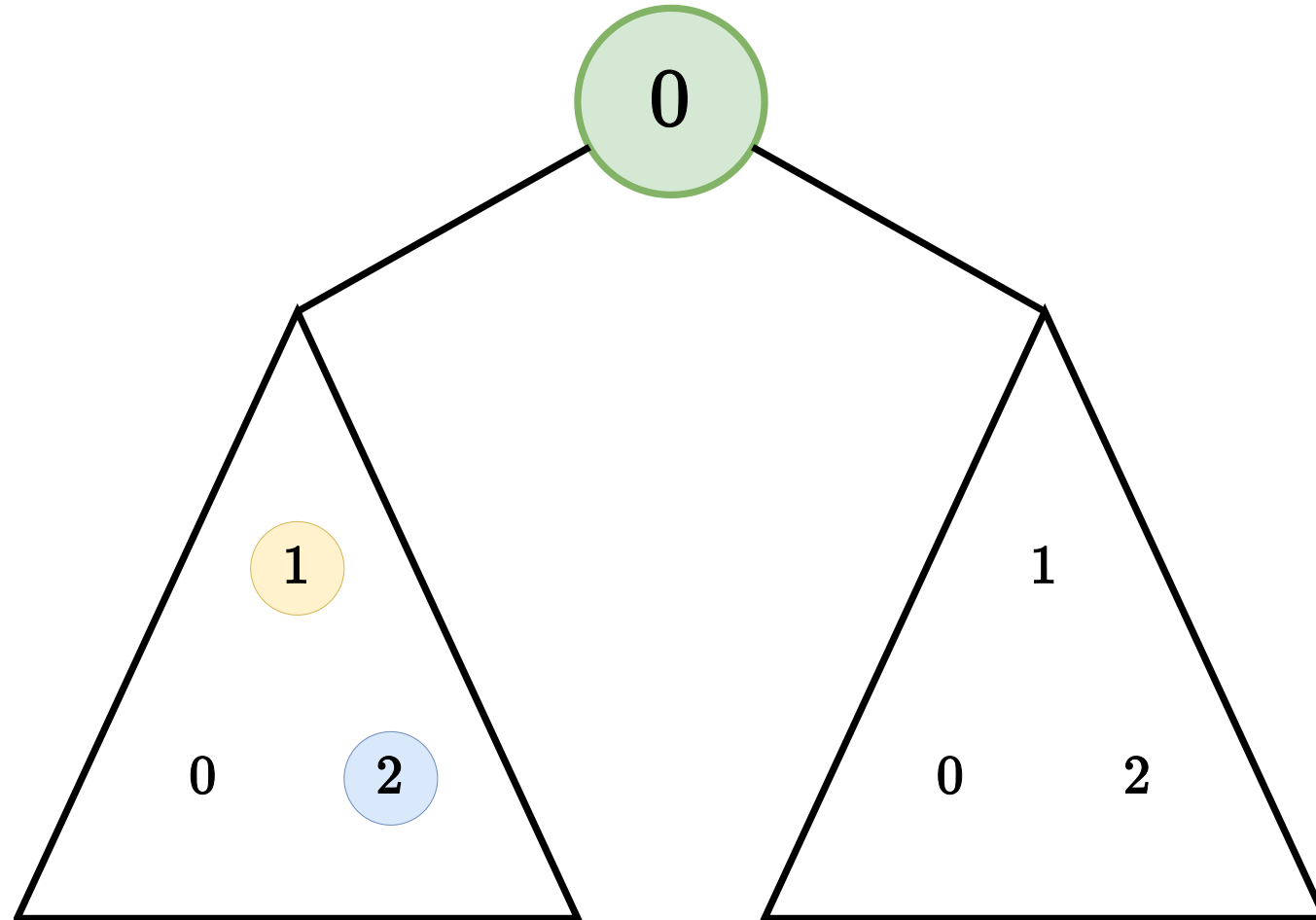
0 – 1 – 2 パスを全パターン列挙しよう！

- 中央の頂点を使わない場合



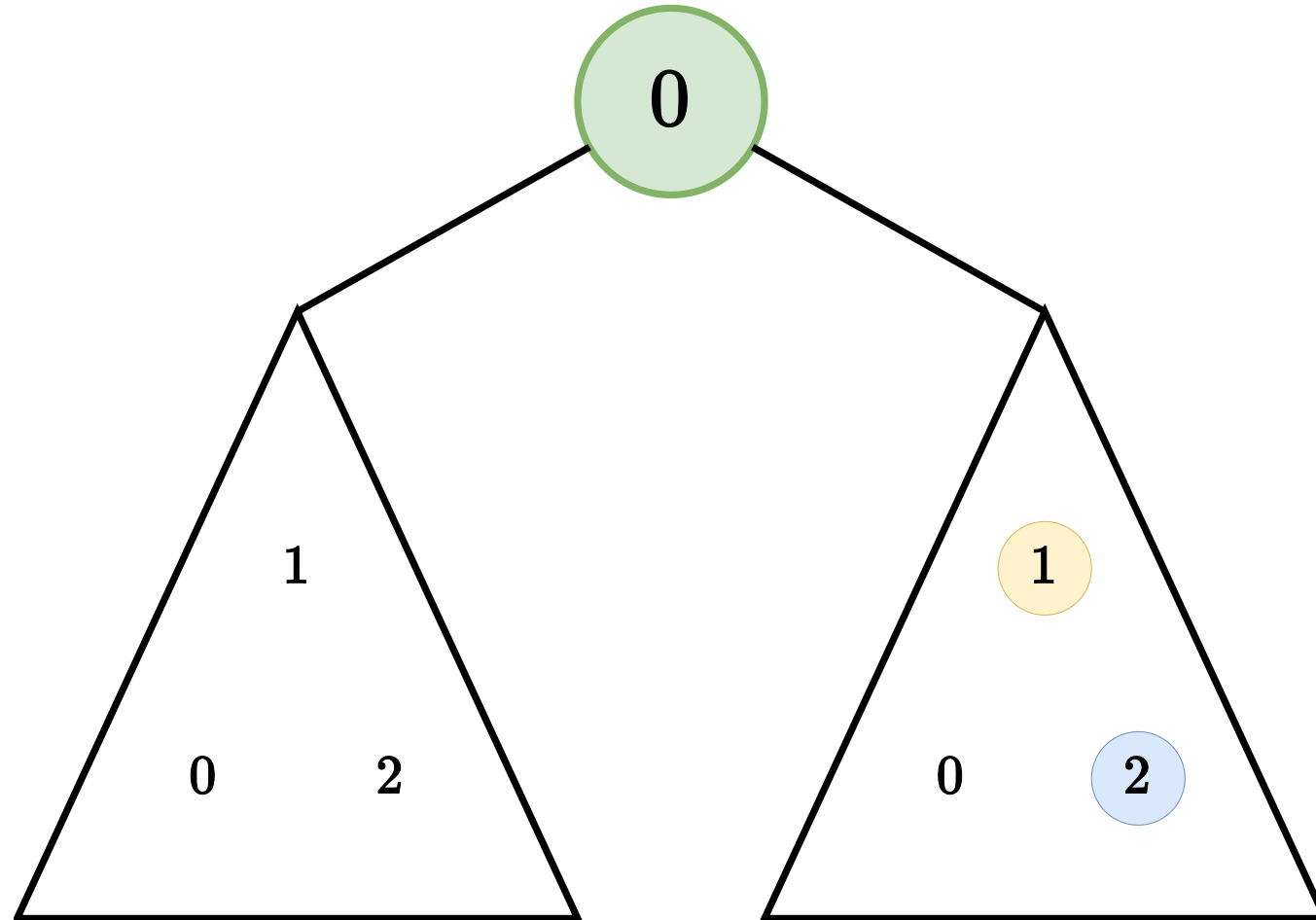
0 – 1 – 2 パスを全パターン列挙しよう！

- 中央の頂点を使う場合



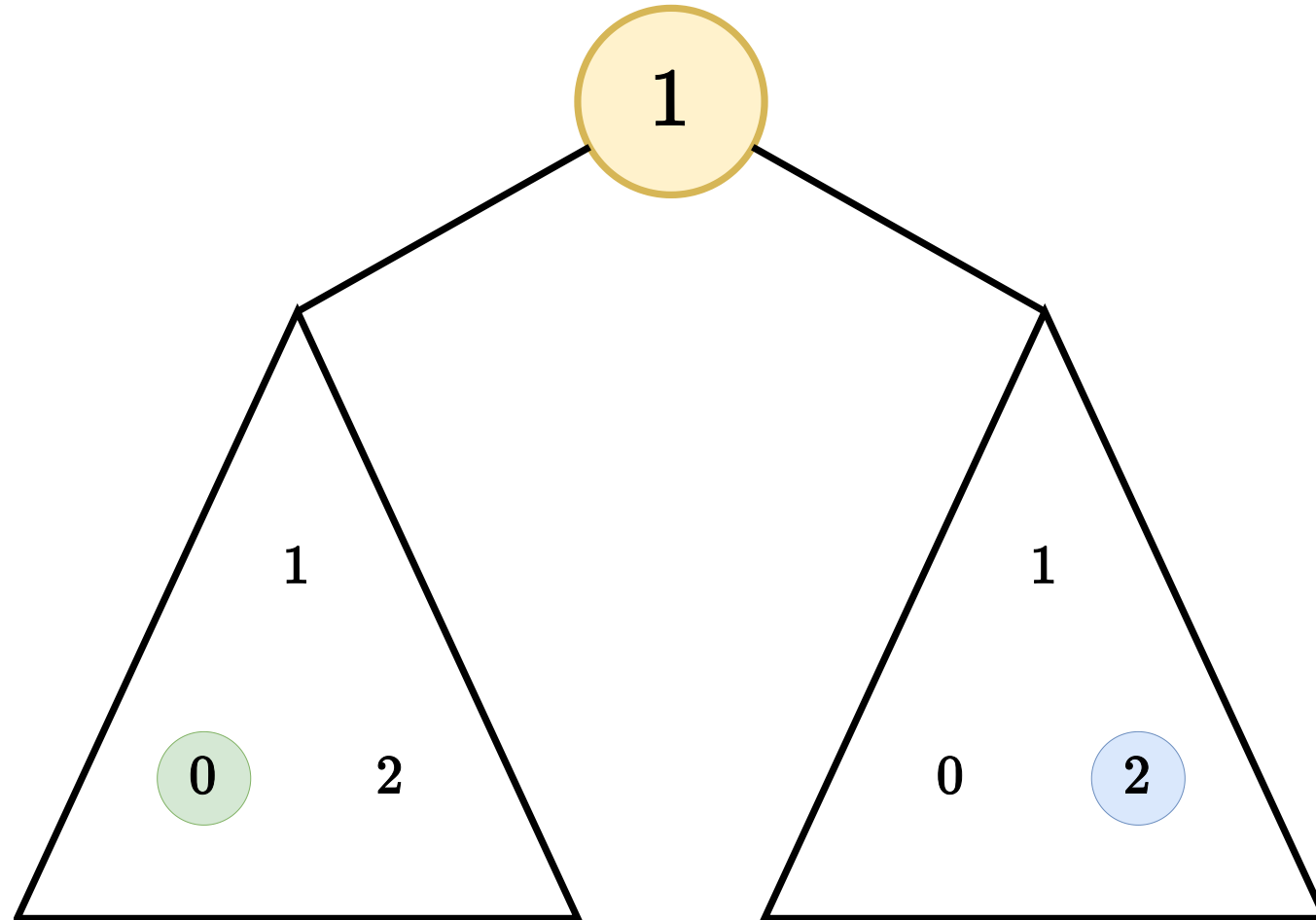
0 – 1 – 2 パスを全パターン列挙しよう！

- 中央の頂点を使う場合



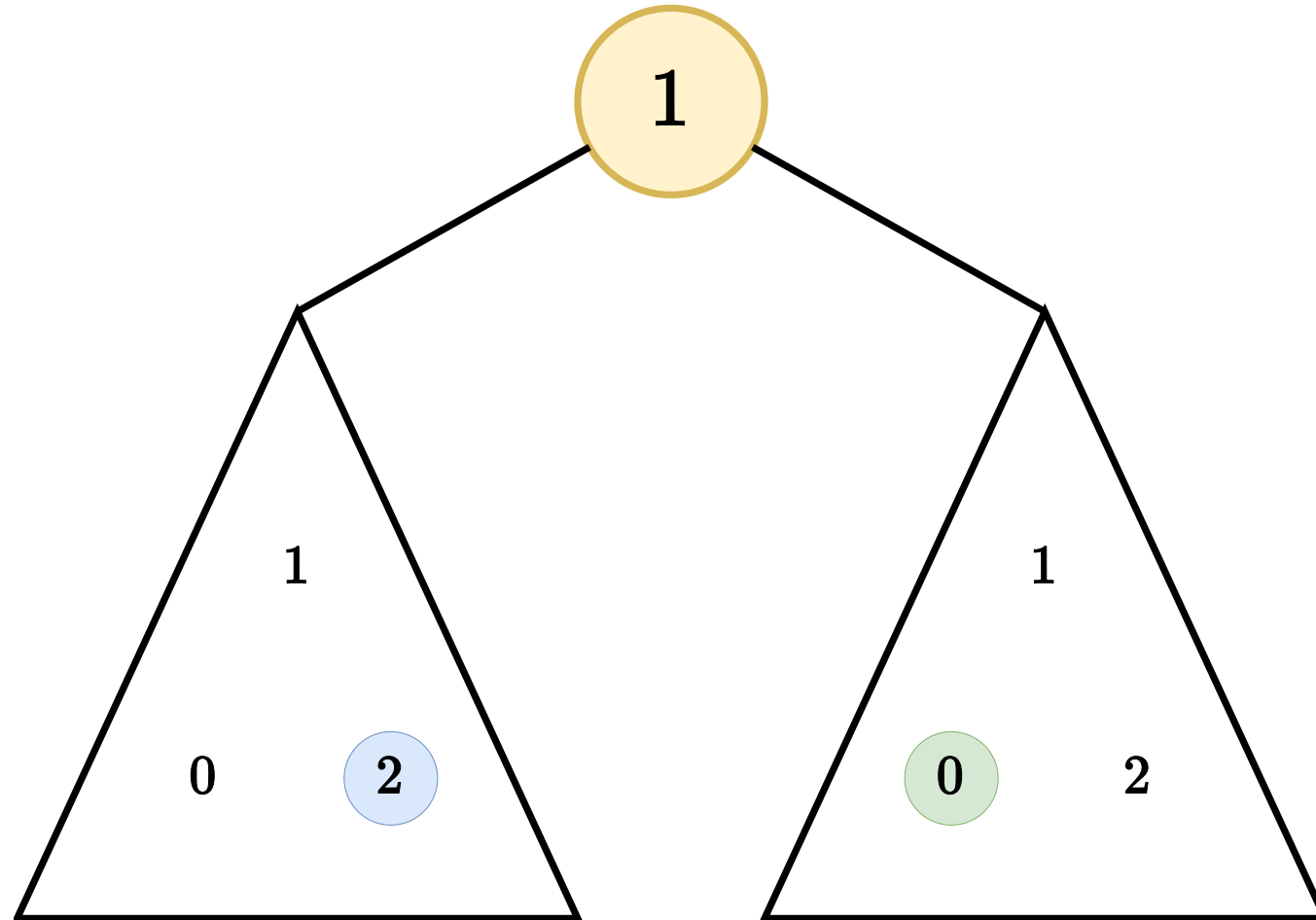
0 – 1 – 2 パスを全パターン列挙しよう！

- 中央の頂点を使う場合



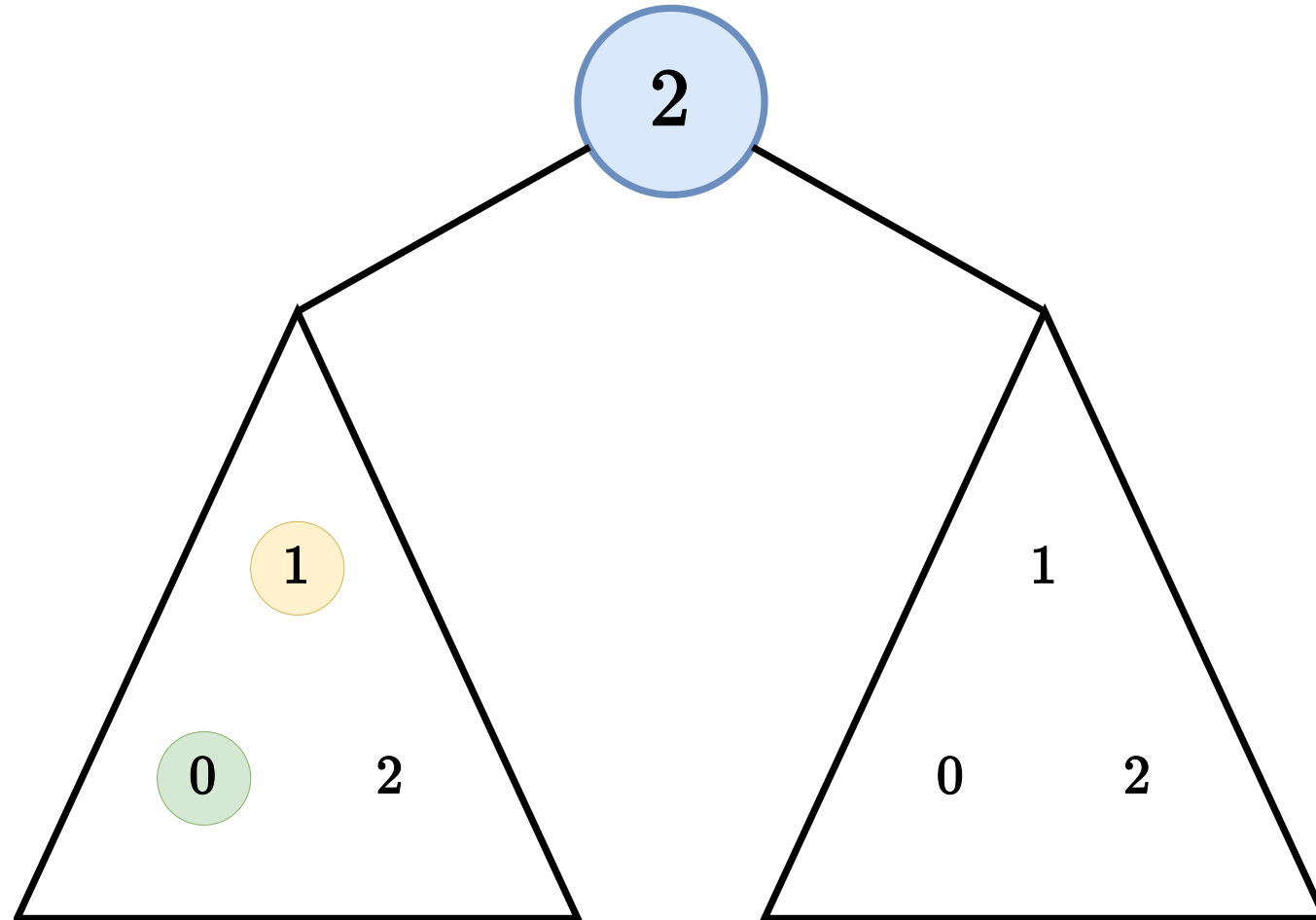
0 – 1 – 2 パスを全パターン列挙しよう！

- 中央の頂点を使う場合



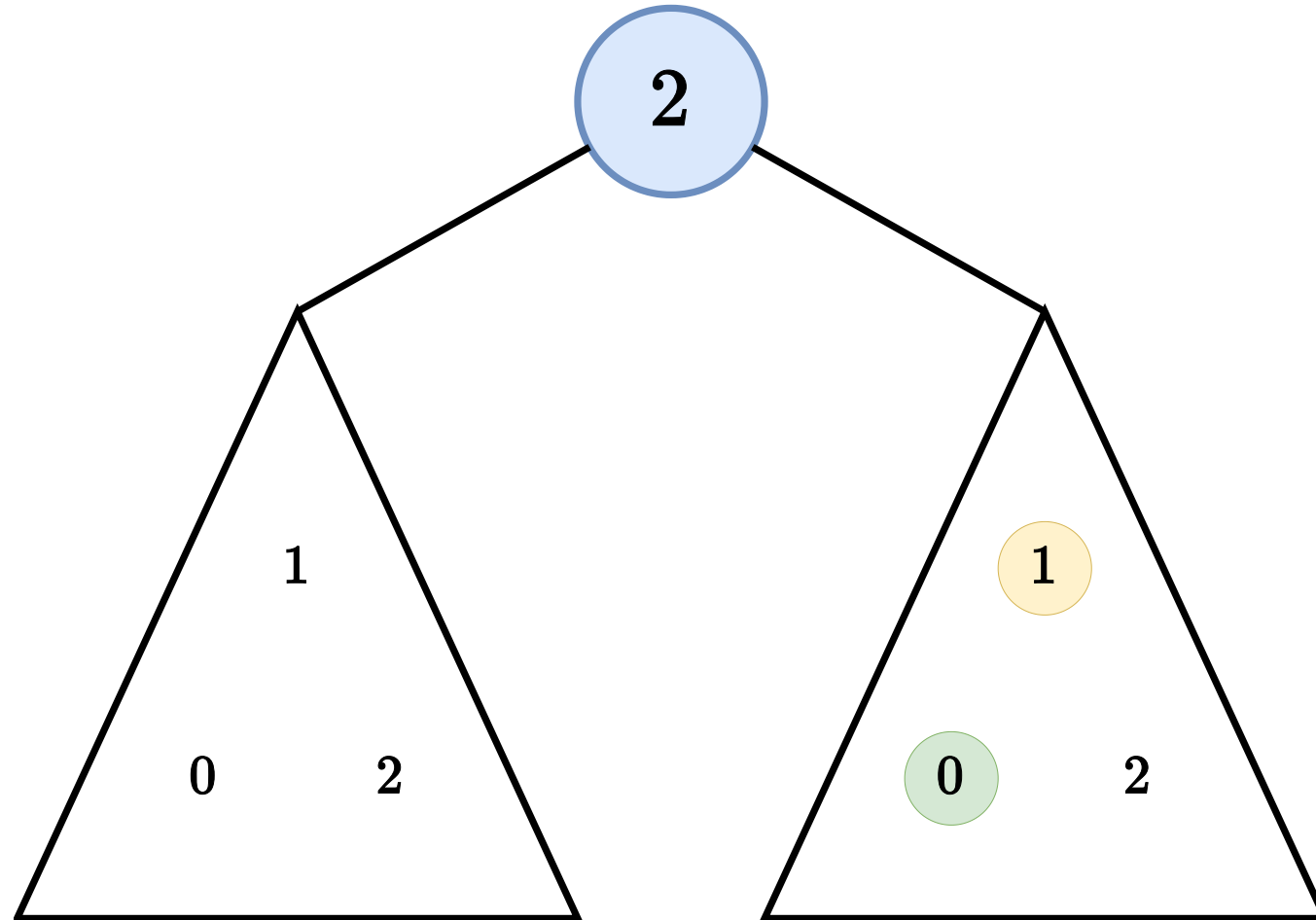
0 – 1 – 2 パスを全パターン列挙しよう！

- 中央の頂点を使う場合



0 – 1 – 2 パスを全パターン列挙しよう！

- 中央の頂点を使う場合

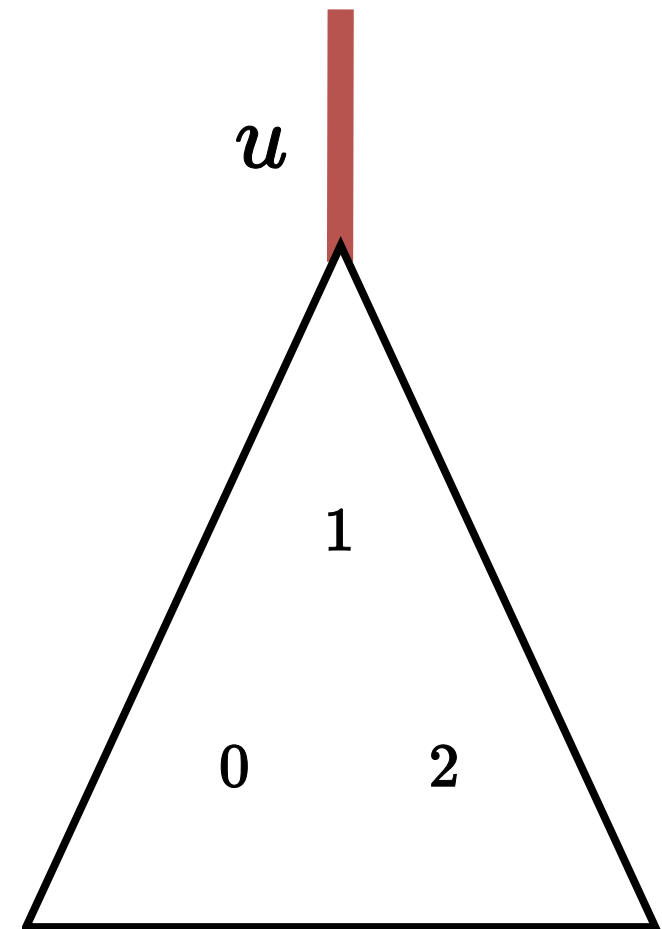


小課題 5 : 完全二分木 (16 点)

- **部分木**の $0 - 1 - 2$ パスの個数を求めたい

辺 u の部分木は以下の情報を持つ

- $0, 1, 2$ の個数
- $u - 1 - 0, u - 1 - 2$ パスの個数
- $0 - 1 - 2$ パスの個数



小課題 6 : $N \leq 100000, Q \leq 25000$ (34 点)

小課題 7 : $N \leq 200000, Q \leq 50000$ (14 点)

これを一般化したい！

小課題 7 : $N \leq 200000, Q \leq 50000$ (14 点)

これを一般化したい！

1. 木をセグメント木のように高さの低い構造に分解
2. それぞれの構造の中で, $0, 1, 2$ の個数, $0 - 1, 1 - 2$ の個数などを数える

解法 A: 重心分解

- ある頂点 X を使う $0 - 1 - 2$ パスの個数は, 以下が分かれば良い
 - $F_X = 0$: $X - 1 - 2$ パスの個数
 - $F_X = 1$: X から各方向への $0, 2$ の個数
 - $F_X = 2$: $X - 1 - 0$ パスの個数

解法 A : 重心分解

- ある頂点 X を**通過する** $0 - 1 - 2$ パスの個数は、以下が分かれば良い
 - X から各方向への $0, 2$ の個数
 - X から各方向への $X - 1 - (0/2)$ パスの個数

解法 A : 重心分解

重心分解で生じるそれぞれの木について, これらの情報を持つ
(重心を X とする)

- 全体の $0, 2$ の個数
- X から各方向への $0, 2$ の個数
- 全体の $X - 1 - 0, X - 1 - 2$ パスの個数
- X から各方向への $X - 1 - 0, X - 1 - 2$ パスの個数

解法 A : 重心分解

重心分解で生じるそれぞれの木について, これらの情報を持つ
(重心を X とする)

- 全体の $0, 2$ の個数
- X から各方向への $0, 2$ の個数
- 全体の $X - 1 - 0, X - 1 - 2$ パスの個数
- X から各方向への $X - 1 - 0, X - 1 - 2$ パスの個数

1 点変更をするとき, 変更される重心分解上の木は $O(\log N)$ 個

解法 A : 重心分解

1 点変更をしたときの

- X から各方向への $X - 1 - 0$, $X - 1 - 2$ パスの個数

の差分を $O(\log N)$ 時間で求めたい

「 X から各方向への $X - 1 - 2$ パスの個数」の差分

- ある頂点の 2 を消したとき:
 - $X - 2$ のパス上の 1 の数だけ, その方向のパスの個数が減る
- ある頂点の 1 を消したとき:
 - 1 以下の部分木内の 2 の数だけ, その方向のパスの個数が減る

「 X から各方向への $X - 1 - 2$ パスの個数」の差分

- ある頂点の 2 を消したとき:
 - $X - 2$ のパス上の 1 の数だけ, その方向のパスの個数が減る

→ オイラーツアー + BIT で 1 の数を管理すれば $O(\log N)$ 時間
(行きが重み 1, 帰りが重み -1 の辺属性オイラーツアー)

「 X から各方向への $X - 1 - 2$ パスの個数」の差分

- ある頂点の 1 を消したとき:
 - 1 以下の部分木内の 2 の数だけ, その方向のパスの個数が減る

→ オイラーツアー + BIT で 2 の数を管理すれば $O(\log N)$ 時間

解法 A : 重心分解

1. 重心分解

2. それぞれの木の各方向について, 以下を構築

- 根からのパス上の 1 の個数を管理する,
辺属性オイラーツアー + BIT
- 部分木内の 0, 2 の個数を管理する,
頂点属性オイラーツアー + BIT

ひたすらがんばると, $O(N \log N + Q(\log N)^2)$ 時間

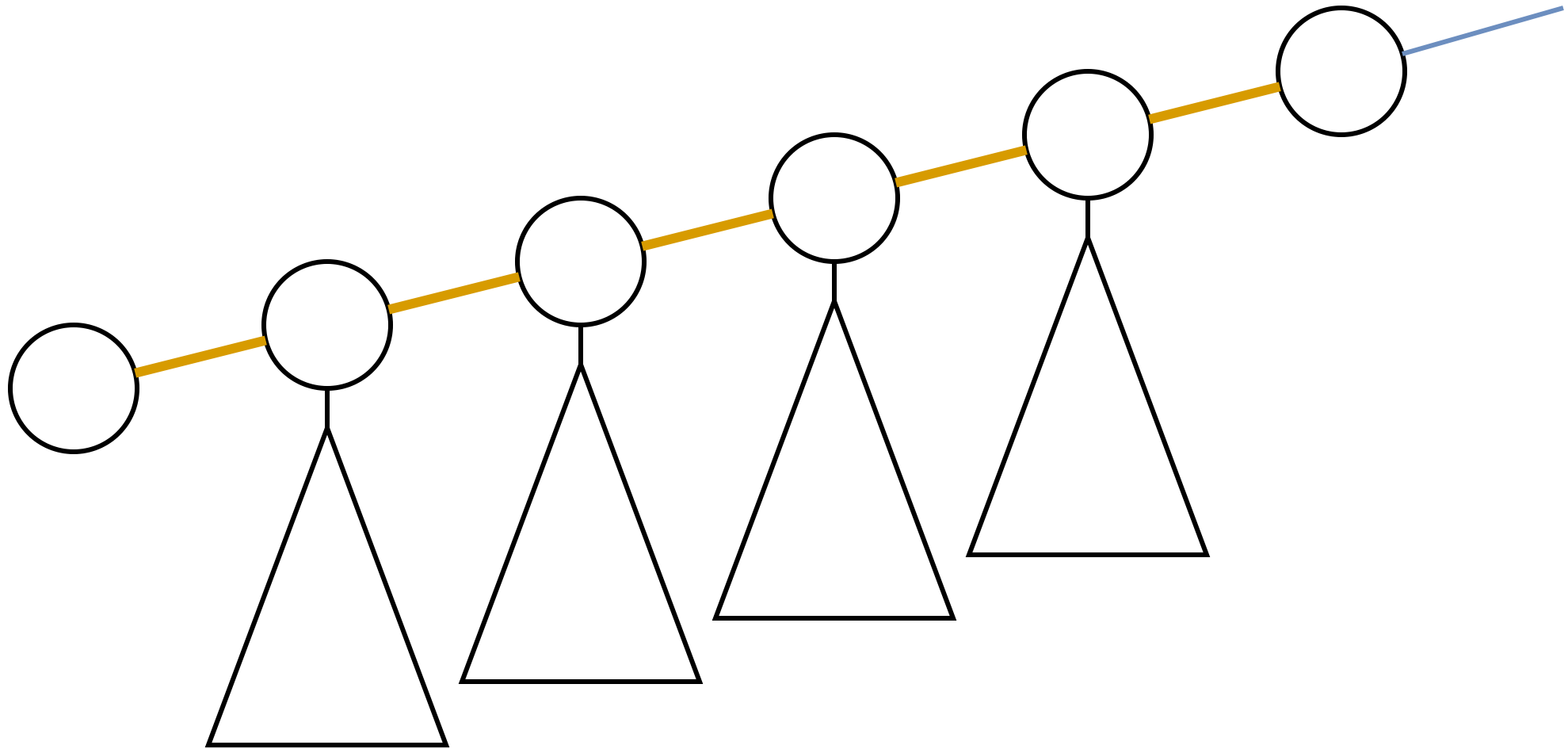
解法 B : HL 分解

- パスの場合は, セグメント木で解ける

→ HL 分解で, 高さ $O(\log N)$ のパスに分解

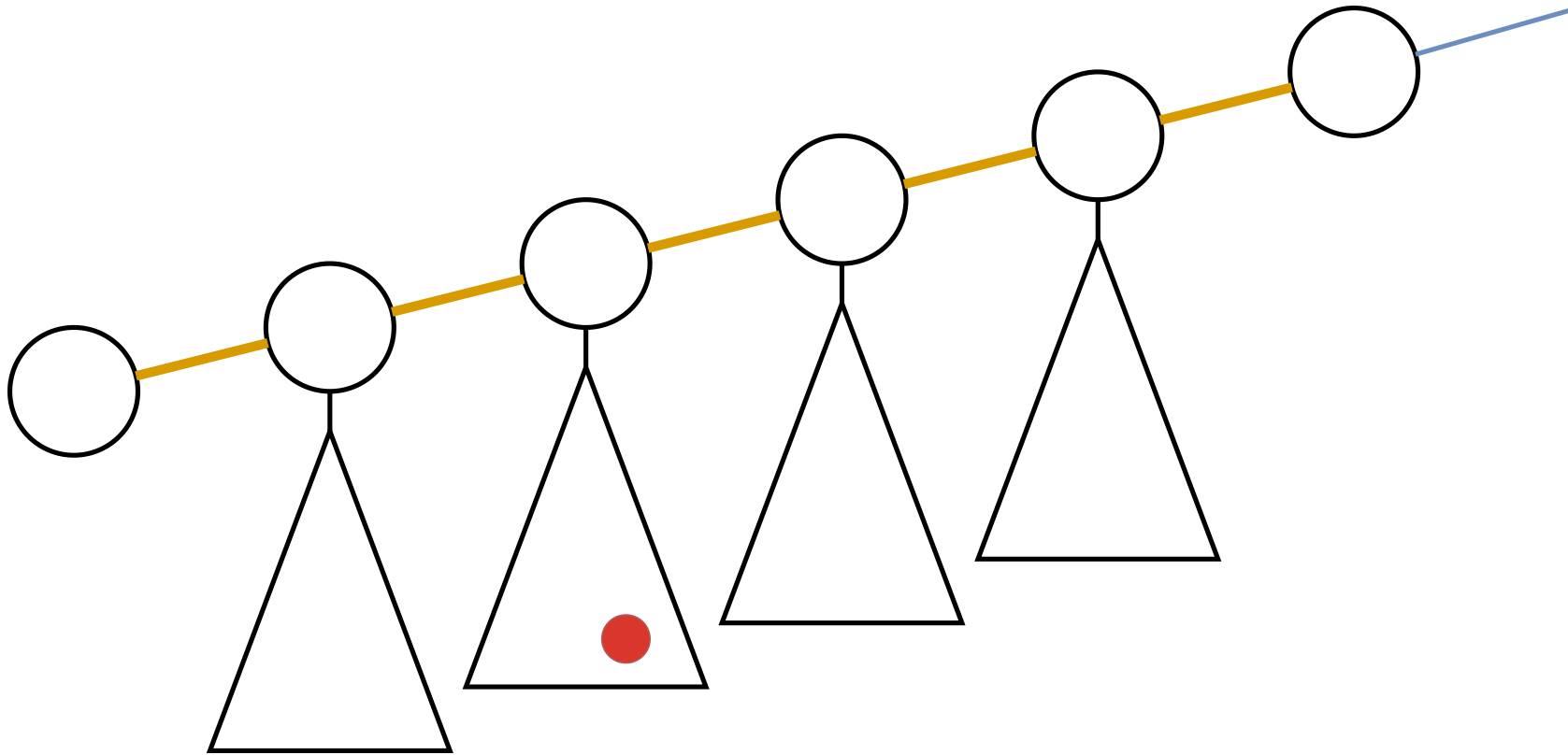
解法 B: HL 分解

- 高さ $O(\log N)$ のパスに分解, パスごとにセグ木を構築



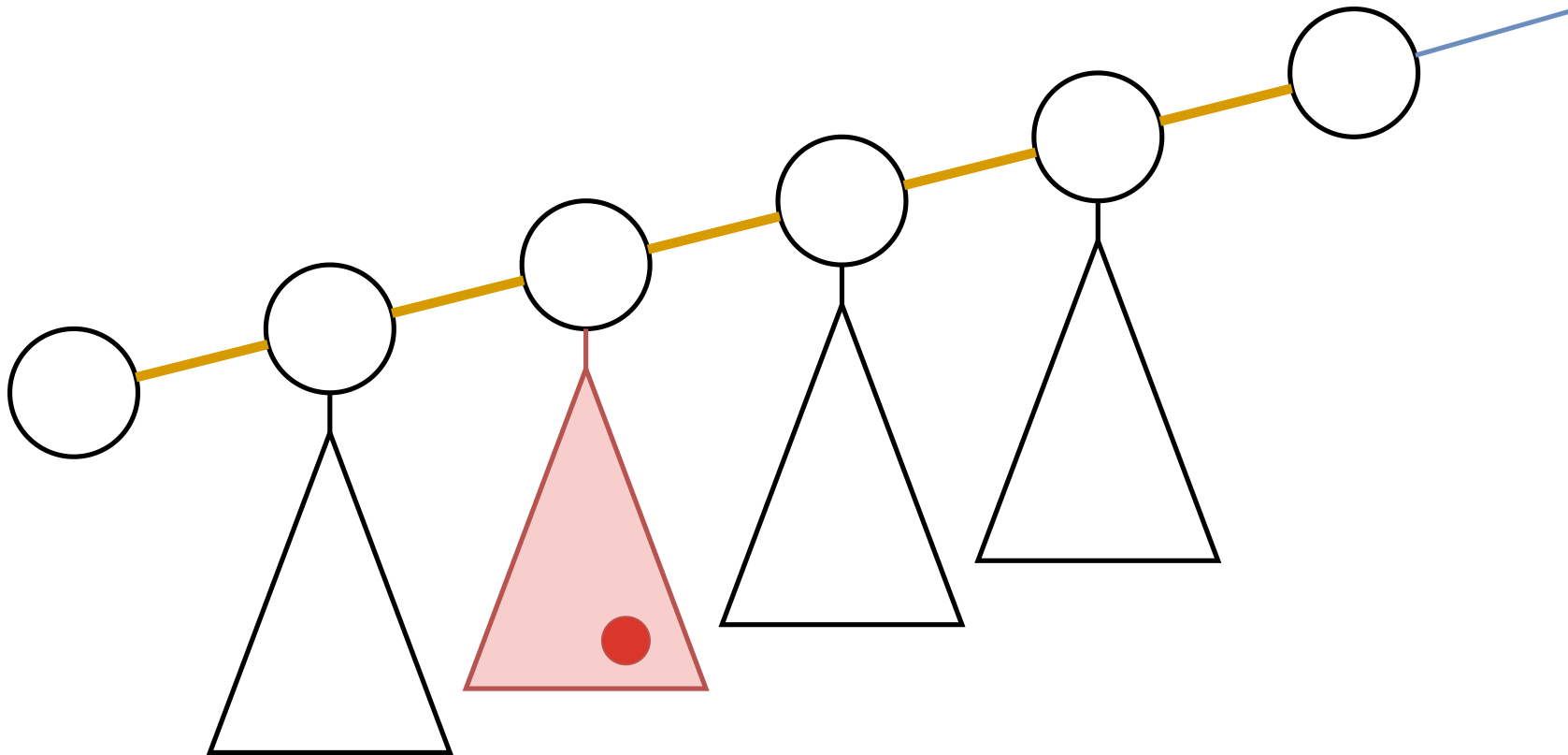
解法 B: HL 分解

- 高さ $O(\log N)$ のパスに分解, パスごとにセグ木を構築
- 1 点変更があったら,



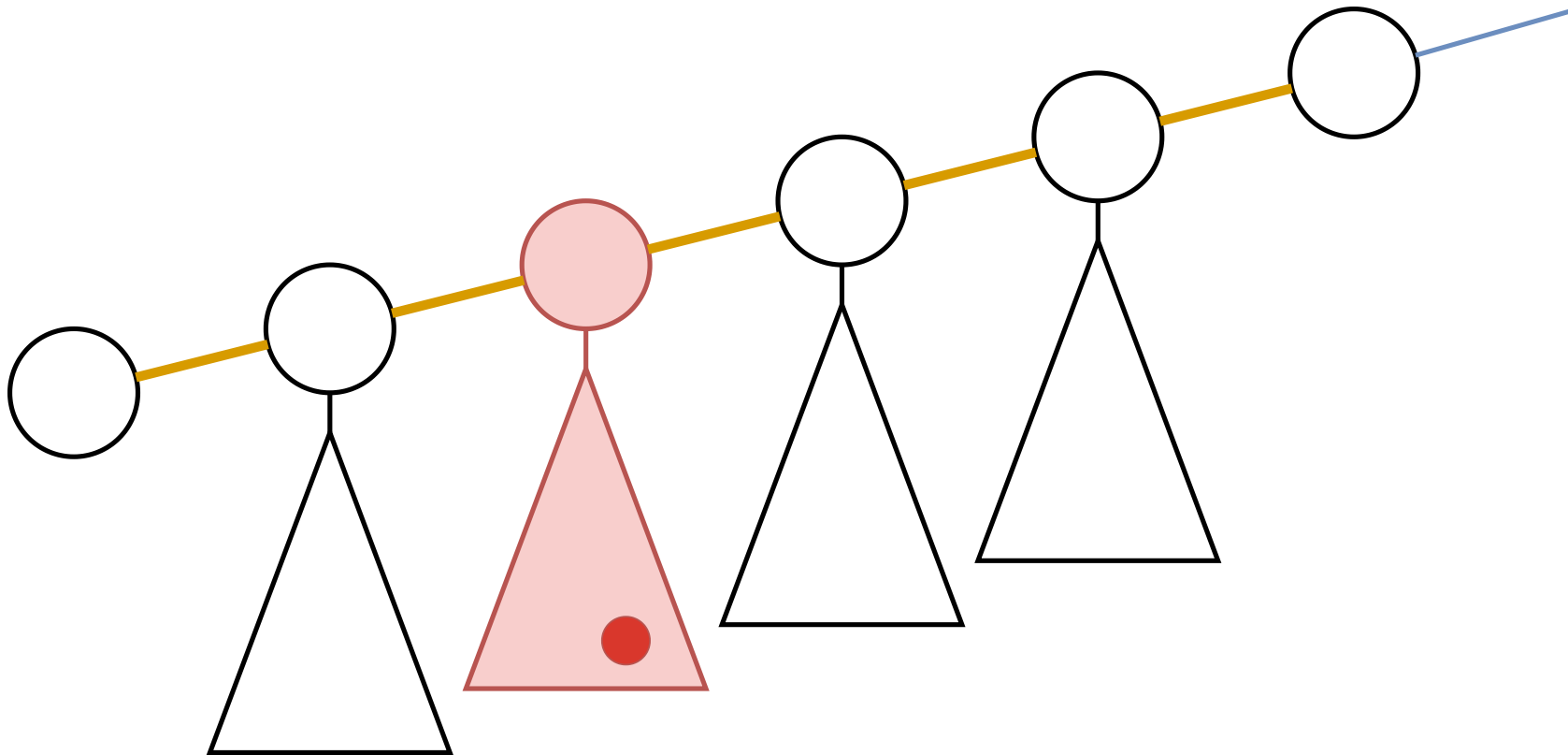
解法 B: HL 分解

- 高さ $O(\log N)$ のパスに分解, パスごとにセグ木を構築
- 1 点変更があったらその部分木を更新して,



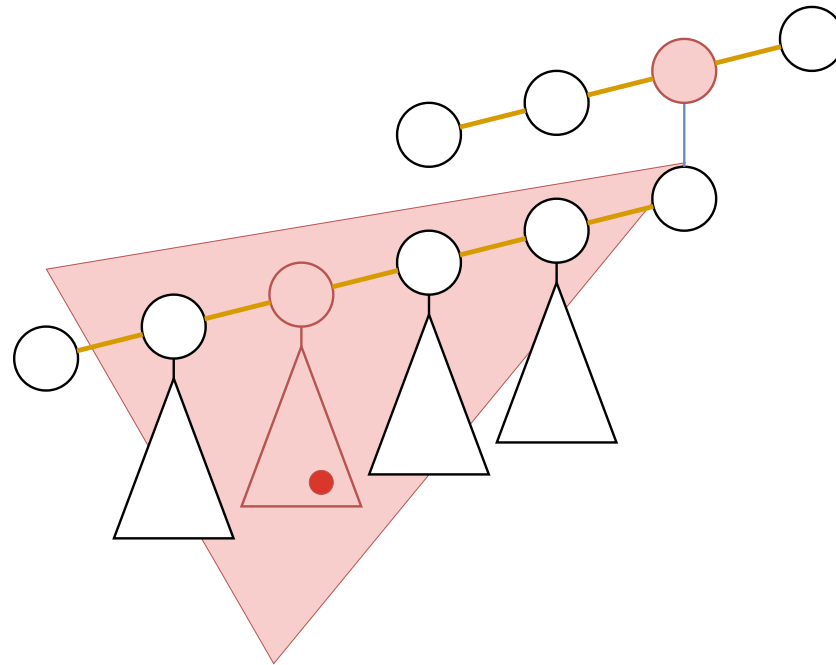
解法 B: HL 分解

- 1 点変更があったらその部分木を更新して, 部分木全体の結果でパス上のセグ木を 1 点更新



解法 B: HL 分解

- 1 点変更があったらその部分木を更新して, 部分木全体の結果でパス上のセグ木を 1 点更新
- さらにその結果で上のパス上のセグ木を 1 点更新...



解法 B : HL 分解

セグ木の各ノードは以下の情報を持つ

- $0, 2$ の個数
- パス上の 1 の個数
- (区間の左端 / 右端) $- 1 - (0/2)$ パスの個数
- $0 - 1 - 2$ パスの個数

ひたすらがんばると, $O(N \log N + Q(\log N)^2)$ 時間

$O((N + Q) \log N)$ 時間で解けます！

解法 C : static top tree

木に対する static top tree は, 列に対するセグ木を一般化したようなもの

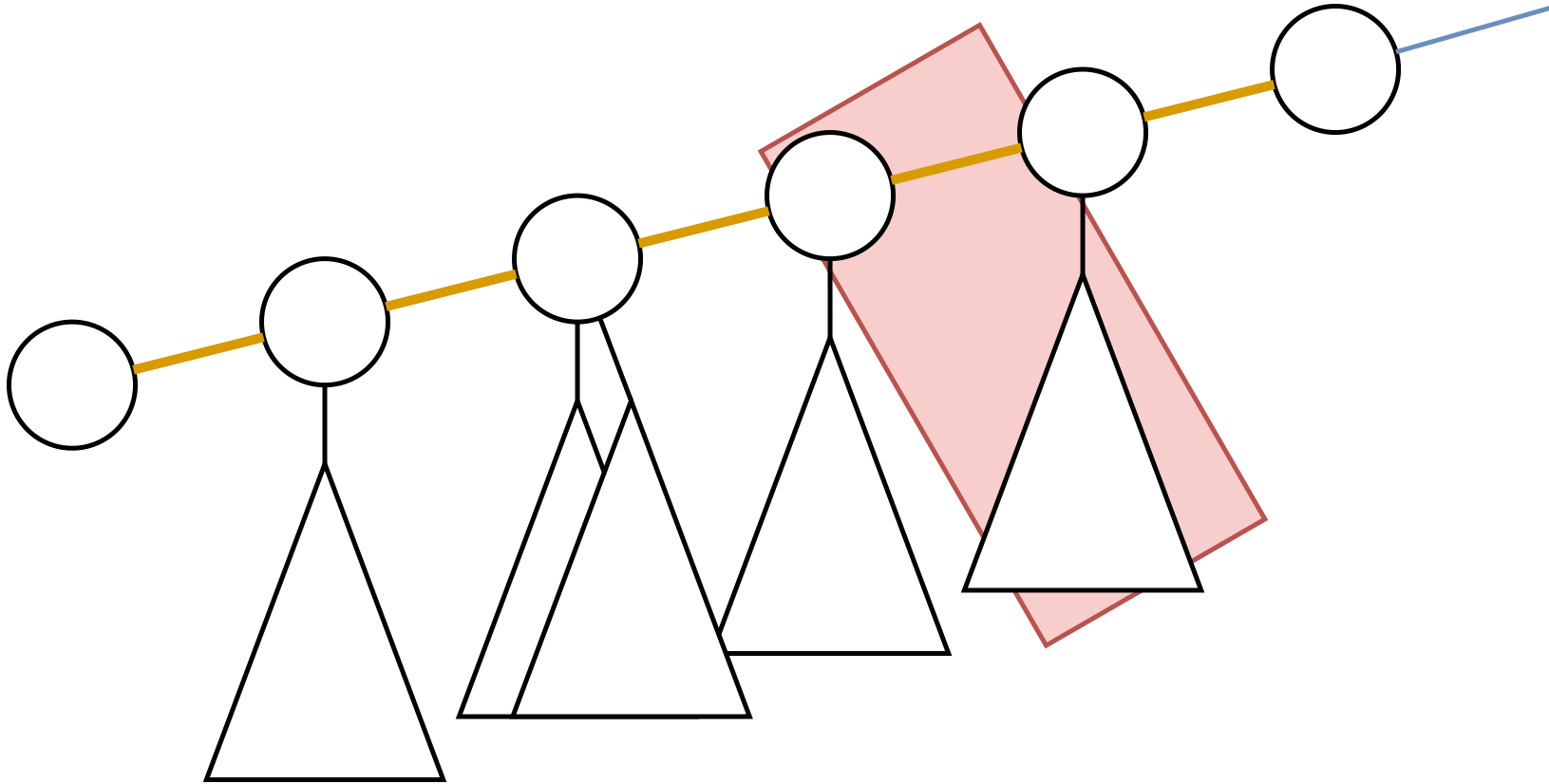
実は実装はこの方針の方がちょっと簡単かも？

解法 C : static top tree

- top tree は辺属性の方が扱いやすい
 - 適当に根を取り, 頂点の情報を親への辺に載せる

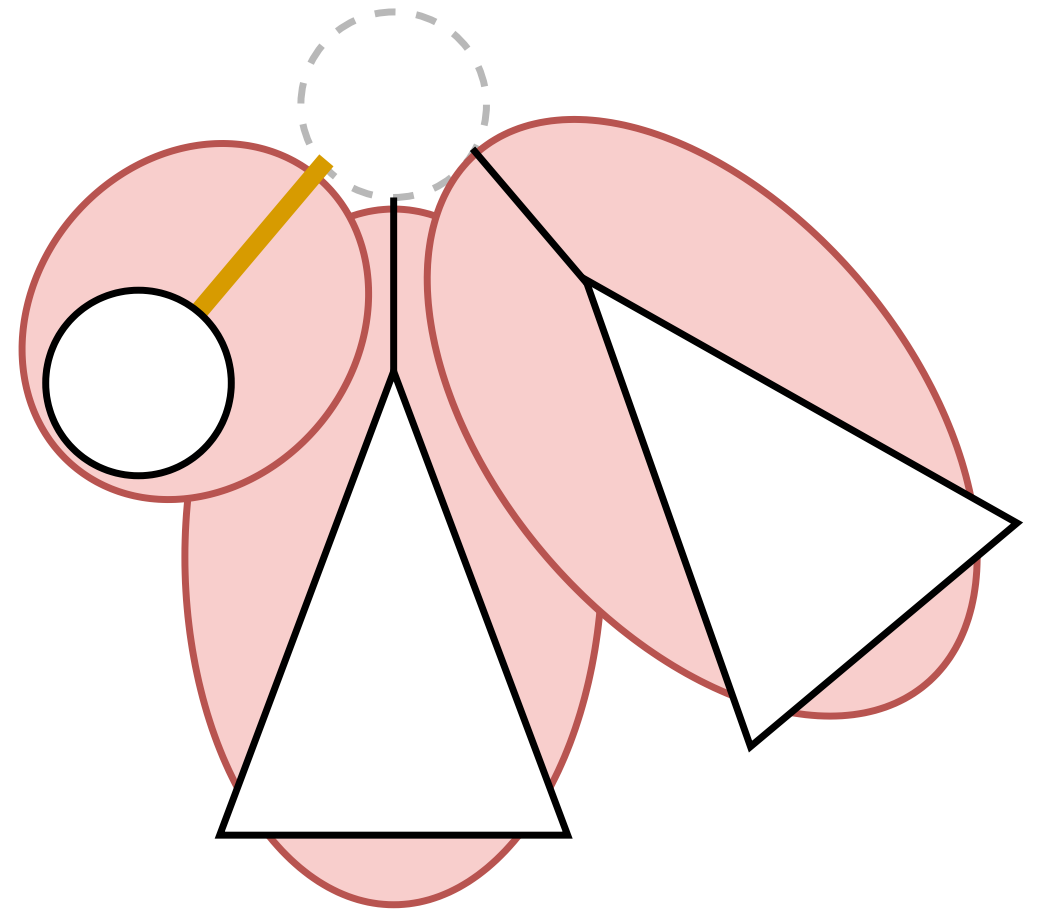
static top tree の構築

- heavy-path 上: この部分を 1 つのノードとして, 高さの最大値を最小化するようにマージ



static top tree の構築

さっきの部分は, 部分木ごとに
ノードを作って, マージ過程の
高さの最大値を最小化するように
マージして作る



解法 C : static top tree

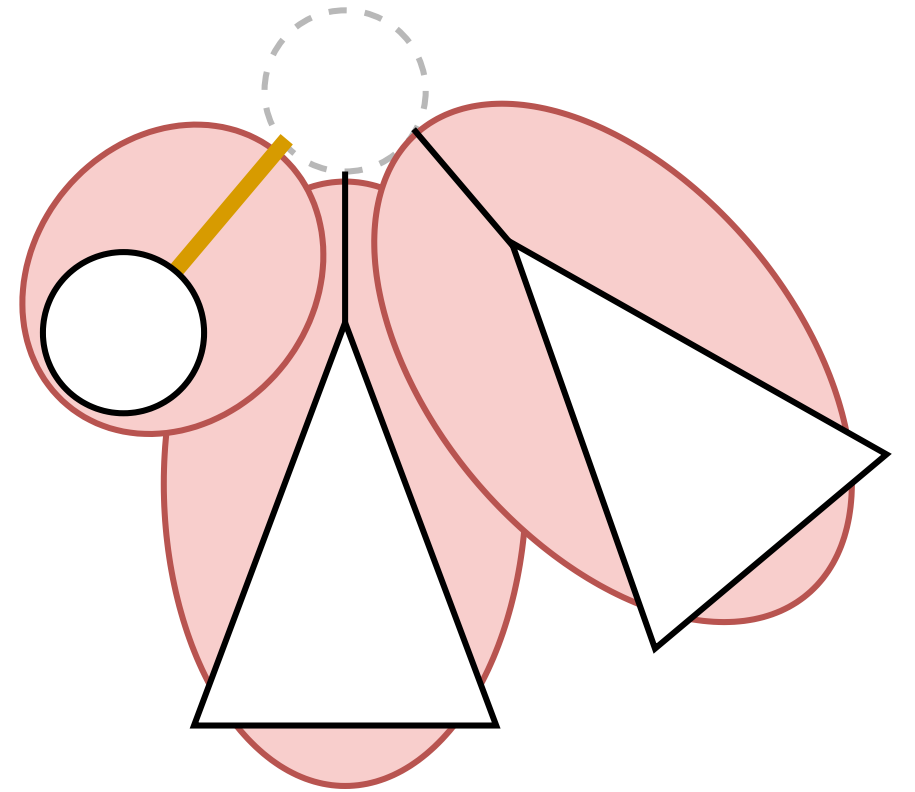
- 持つデータは HL 分解のときとだいたい同じ
- マージ過程の高さの最大値を最小化するようにマージすれば,
高さは $O(\log N)$
 - 所属する頂点数が均等になるように分割していく再帰

→ 更新が $O(\log N)$ 時間でできる！

解法 C : static top tree

マージの計算は, 以下の 2 種類が
できれば良い

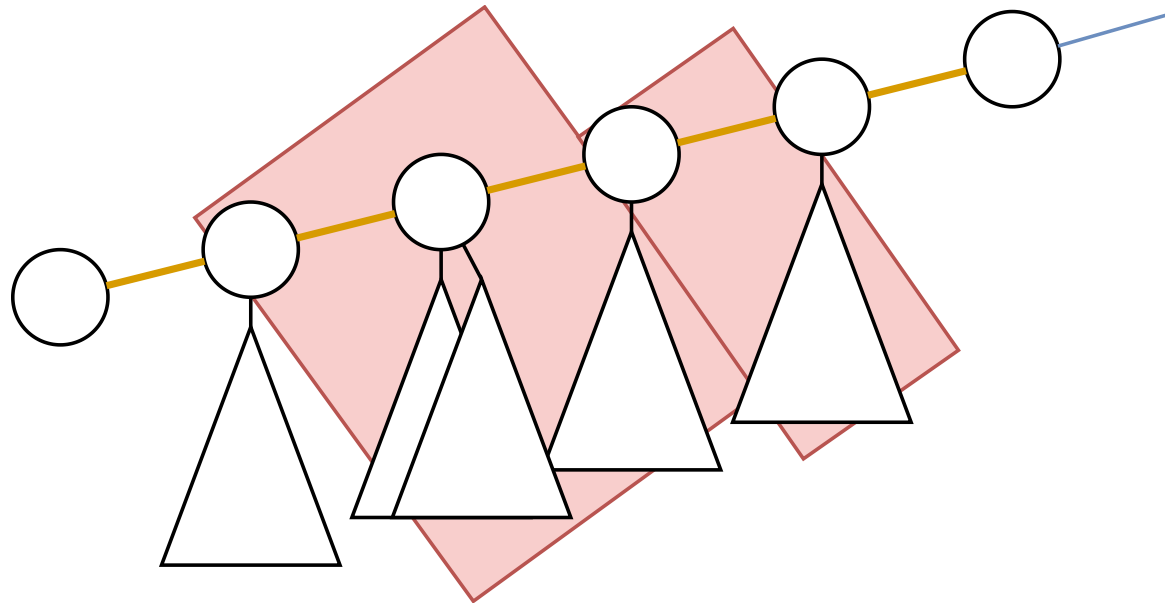
- rake
 - 根が共通である 2 つの
部分木のマージ
 - 根が共通である,
「heavy-path 上の区間」と
「部分木」のマージ



解法 C : static top tree

マージの計算は, 以下の 2 種類ができれば良い

- compress
 - heavy-path 上の連続する区間のマージ



解法 C: static top tree

マージの計算は、以下の 2 種類ができれば良い

- rake
 - 根が共通である 2 つの部分木のマージ
 - 根が共通である, 「heavy-path 上の区間」と「部分木」のマージ
- compress
 - heavy-path 上の連続する区間のマージ

→ これができる木上のなにかの計算は $O(\log N)$ / query

これを Link-Cut 木 (を発展させたもの) で管理したものが **top tree** で、
辺の追加・削除ができる

統計情報 (JOI)

点数	1	2	3	4	5	6	7	人数	累積人数
52	○	○	○	○	○	-	-	3	3
36	○	○	○	○	-	-	-	1	-
36	○	○	○	-	○	-	-	3	7
20	○	○	○	-	-	-	-	7	14
6	○	-	-	-	-	-	-	2	16
0	-	-	-	-	-	-	-	9	25

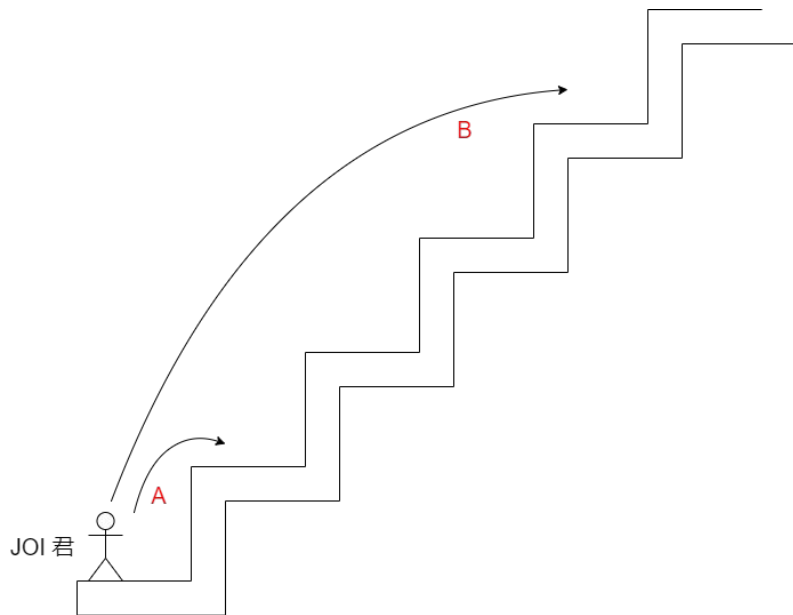
JOI 2023/2024 春季トレーニング Day 3

塔 (Tower) 解説

解説担当: 菅井

問題概要

- 階段を上る
- 「A 秒かけて 1 段上る」or「B 秒かけて D 段上る」ができる
- N 個の区間で工事が行われており、入ることができない
- Q クエリ: X 段目にたどり着けるか？たどり着けるなら何秒かかるか？
- 制約: (工事の個数) $\leq 2 \cdot 10^5$, (クエリの個数) $\leq 2 \cdot 10^5$, (段数) $\leq 10^{12}$



小課題

小課題 1 (5 点) 段数が 10^6 以下

小課題 2 (38 点) $N, Q \leq 2000$

小課題 3 (25 点) $A=1, B=D$

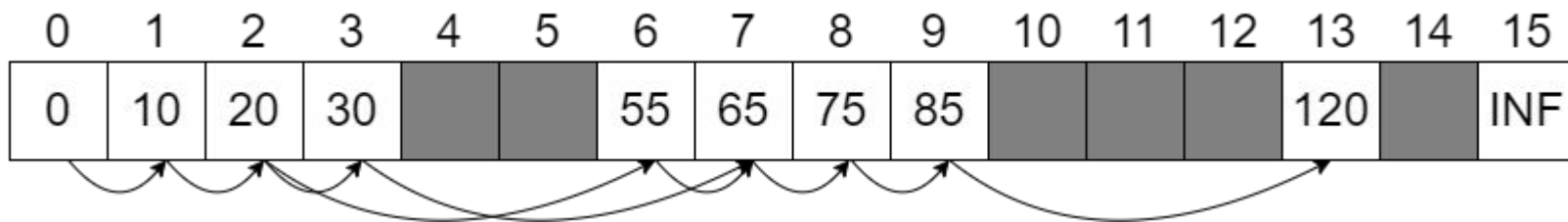
小課題 4 (32 点) 追加の制約はない

小課題 1 (5 点)

- 段数が 10^6 以下
- DP ができる
- $dp[i] = (0 \text{ 段目から } i \text{ 段目まで行くのにかかる最短時間 (不可能なら INF)})$
- 5 点獲得

小課題 2 ($N, Q \leq 2000$)

サンプル 1 の DP を眺めよう



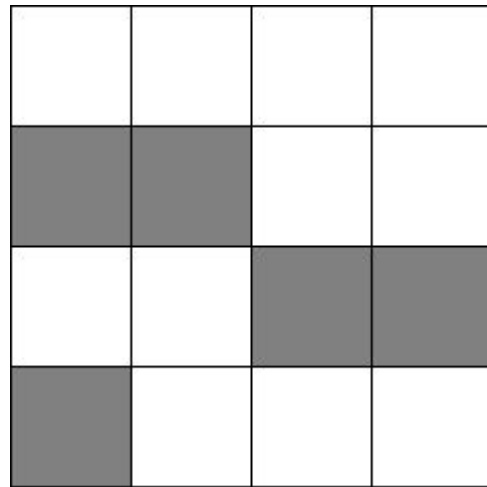
この問題の大きな特徴:「D 段上に上る」

D が定数 $\rightarrow \text{mod } D$ の値が重要になりそう

小課題 2 ($N, Q \leq 2000$)

D で割った余りと商に注目

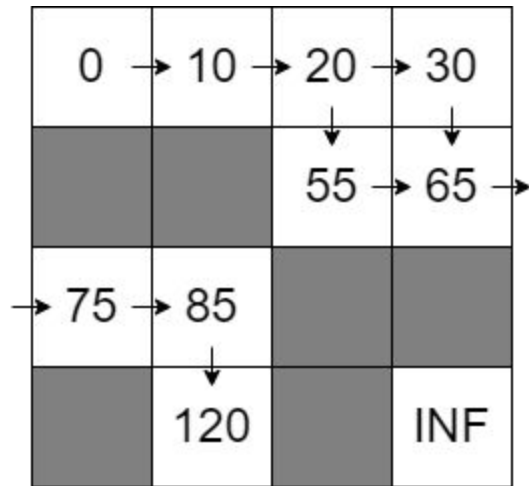
マスを手横 D 列のグリッド上に配置
できる



小課題 2 ($N, Q \leq 2000$)

グリッド上でできる移動

- 右に 1 進む
- 下に 1 進む
- 右端から次の行の左端へワープする

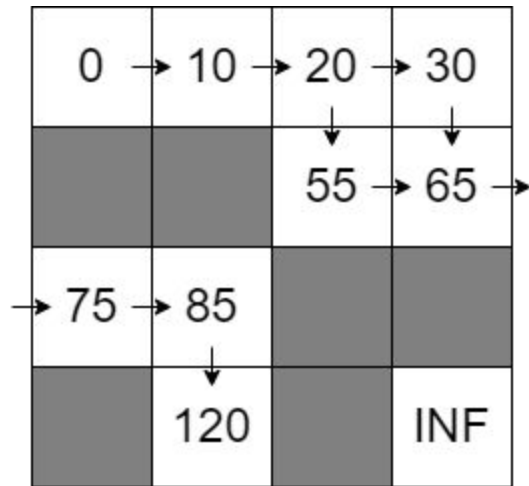


小課題 2 ($N, Q \leq 2000$)

グリッドの上に障害物とクエリを配置して、DP を行いたい

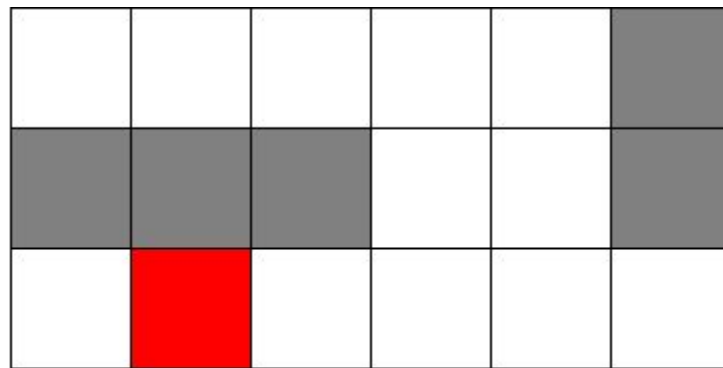
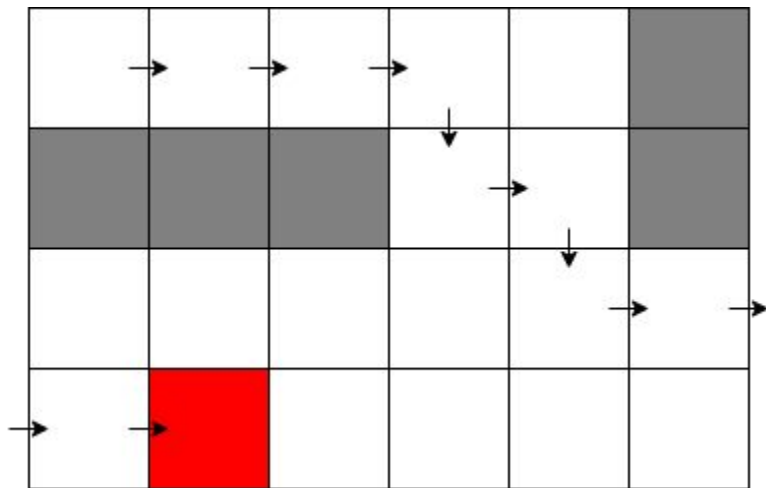
座標圧縮を行うと、縦横 $O(N+Q)$ マスにできる

→ 全体で $O((N+Q)^2)$



小課題 2 ($N, Q \leq 2000$)

何もない行を適当に消すと答えがずれるので、注意しましょう



小課題 3 ($A=1$, $B=D$)

どちらの移動方法でも、1 秒あたり 1 段上っている

→ X_j 段目にたどり着けるなら、どのようにしても X_j 秒かかる

→ X_j 段目にたどり着けるかどうかだけ判定できればよい！

小課題 3 ($A=1, B=D$)

小課題 2 の座標圧縮を行った後、DP を行ごとに見ていく

i-1 行目の DP から、i 行目の DP はどのように得られるか？

0	0	1	0	0	1	0	1	1	1
?	?	?	?	?	?	?	?	?	?

小課題 3 (A=1, B=D)

① とりあえずそのままコピーする

0	0	1	0	0	1	0	1	1	1
0	0	1	0	0	1	0	1	1	1

小課題 3 (A=1, B=D)

② 障害物があるところは0にする

0	0	1	0	0	1	0	1	1	1
0	0	1	0	0	1	0	0	1	1

小課題 3 (A=1, B=D)

③ i-1 行目の右端のマスが 1 なら、左端のマスを 1 にする

0	0	1	0	0	1	0	1	1	1
1	0	1	0	0	1	0	0	1	1

小課題 3 (A=1, B=D)

④ 障害物がない区間を列挙し、最も左にある1を見つけ、それ以降をすべて1にする

0	0	1	0	0	1	0	1	1	1
1	1	1	0	0	1	1	0	1	1

小課題 3 ($A=1$, $B=D$)

① とりあえずそのままコピーする

これは、 $i-1$ 行目の DP 配列を使いませばよい

小課題 3 ($A=1$, $B=D$)

② 障害物があるところは 0 にする

③ $i-1$ 行目の右端のマスが 1 なら、左端のマスを 1 にする

④ 障害物がない区間を列挙し、最も左にある 1 を見つけ、それ以降をすべて 1 にする

これらは、遅延セグメント木で処理できる (区間の最も左の 1 の位置を持つ)

満点

DP の遷移は以下ようになる:

- ① 全体に定数を加算
- ② 障害物があるところは INF にする
- ③ $\text{chmin}(i \text{ 行目の左端}, i-1 \text{ 行目の右端} + A)$
- ④ 障害物がない区間を列挙し、順に $\text{chmin}(\text{dp}[j + 1], \text{dp}[j] + A)$

満点

定数だけずらして、以下のようにできる:

- ① 障害物があるところは INF にする
- ② $\text{chmin}(i \text{ 行目の左端}, i-1 \text{ 行目の右端} + x)$
- ③ 障害物がない区間を列挙し、順に $\text{chmin}(\text{dp}[j + 1], \text{dp}[j])$

満点

実は、各行の DP 配列は、必ずある整数 k を用いて
 $((k+1)x$ と INF からなる列) + $(kx$ と INF からなる列) と表せる

証明: 帰納法

前半と後半の境界を考えることができる

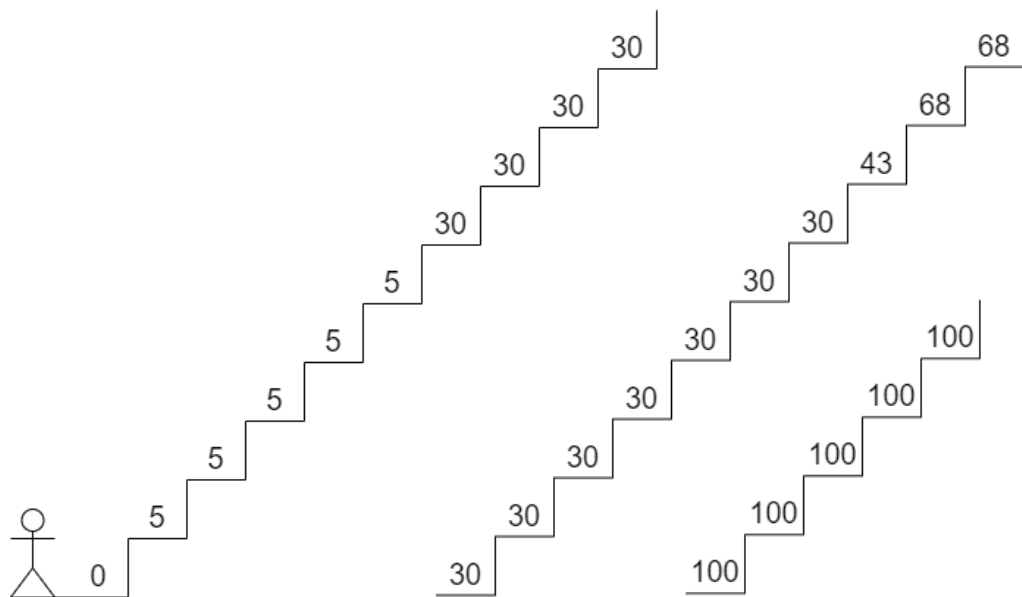
満点

「各要素が INF かどうか」は、小課題 3 と同様の遅延セグメント木で管理できる

DP を更新するとき、前半と後半の境界を適切に更新すればよい

計算量は $O((N+Q)\log(N+Q))$ など

得点分布



0	5	5	5	5
5	30	30	30	30
30	30	30	30	30
30	30	43	68	68
100	100	100	100	100

脱出経路 2 (Escape Route 2) 解説

問題概要

- 都市 i から都市 $i + 1$ への航空便が M_i 個あり, それぞれの出発時刻 $A_{i,j}$ と到着時刻 $B_{i,j}$ が分かっている
 - 各航空便を $E_{i,j} = (A_{i,j}, B_{i,j})$ と表すことにする
 - $N \leq 100\,000$, $M = \sum M_i \leq 100\,000$
- 時刻は T カウントで 1 周し, 航空便は毎ループ同じ時刻に出る
- Q 個の質問に答えよ:
都市 L_q から都市 R_q へ行くのにかかる,
乗り換え時間を含めた時間の最小値は?
 - $Q \leq 300\,000$

考察: 乗り継ぎにかかる時間

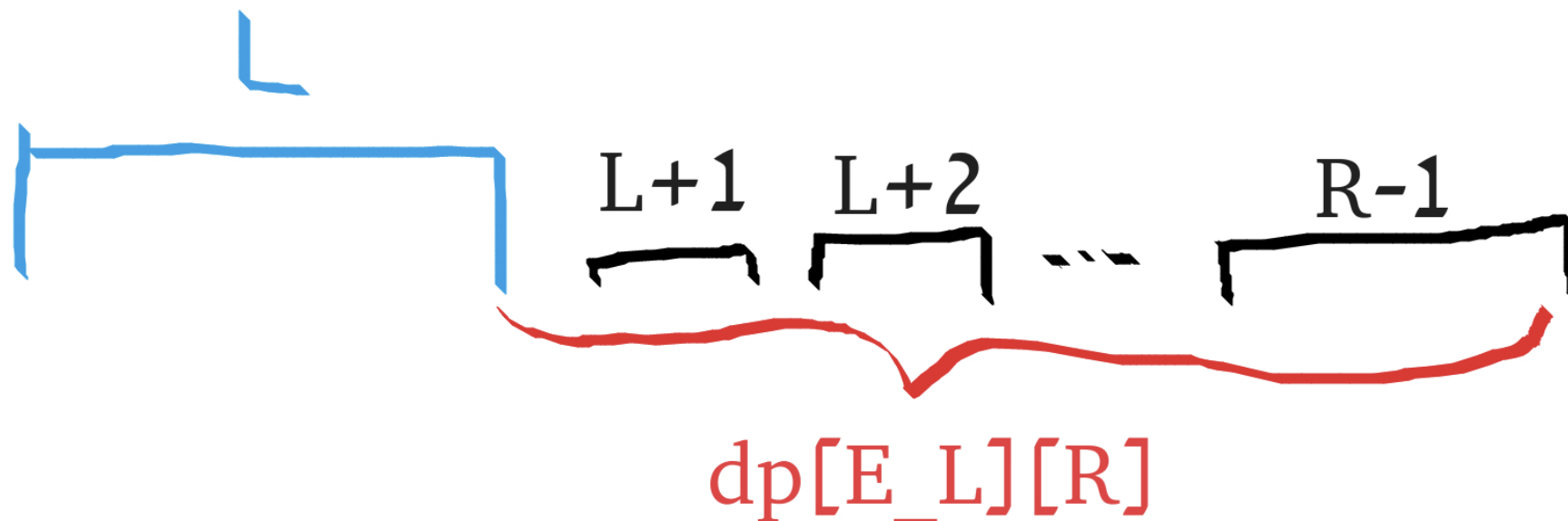
都市 x 発の航空便として $e_x = (a_x, b_x)$ を,
都市 $x + 1$ 発の航空便として $e_{x+1} = (a_{x+1}, b_{x+1})$ を選んだとき,
都市 $x + 1$ に到着してから都市 $x + 2$ に到着するまでの時間
 $d(e_x, e_{x+1})$ は以下のように求まる

- $b_{x+1} \leq a_{x+2}$ の場合, 求める時間は $b_{x+2} - b_{x+1}$
- $b_{x+1} > a_{x+2}$ の場合, 求める時間は $b_{x+2} - b_{x+1} + T$

小課題 1 ($N \leq 2\,000, M_i = 1$)

以下の値 $dp[E_L][R] = X_{L,R}$ を求めることを考える

- 航空便 E_L に乗って都市 $L+1$ に到着してから、都市 R に到着するまでの最短時間 $X_{L,R}$



小課題 1 ($N \leq 2\,000, M_i = 1$)

以下の値 $dp[E_L][R] = X_{L,R}$ を求めることを考える

- 航空便 E_L に乗って都市 $L + 1$ に到着してから,
都市 R に到着するまでの最短時間 $X_{L,R}$

各始点 L について, $R = L + 1, \dots, N$ について dp の値が昇順に求まる(L ごとに $O(N)$ 時間, 全体で $O(N^2)$ 時間)

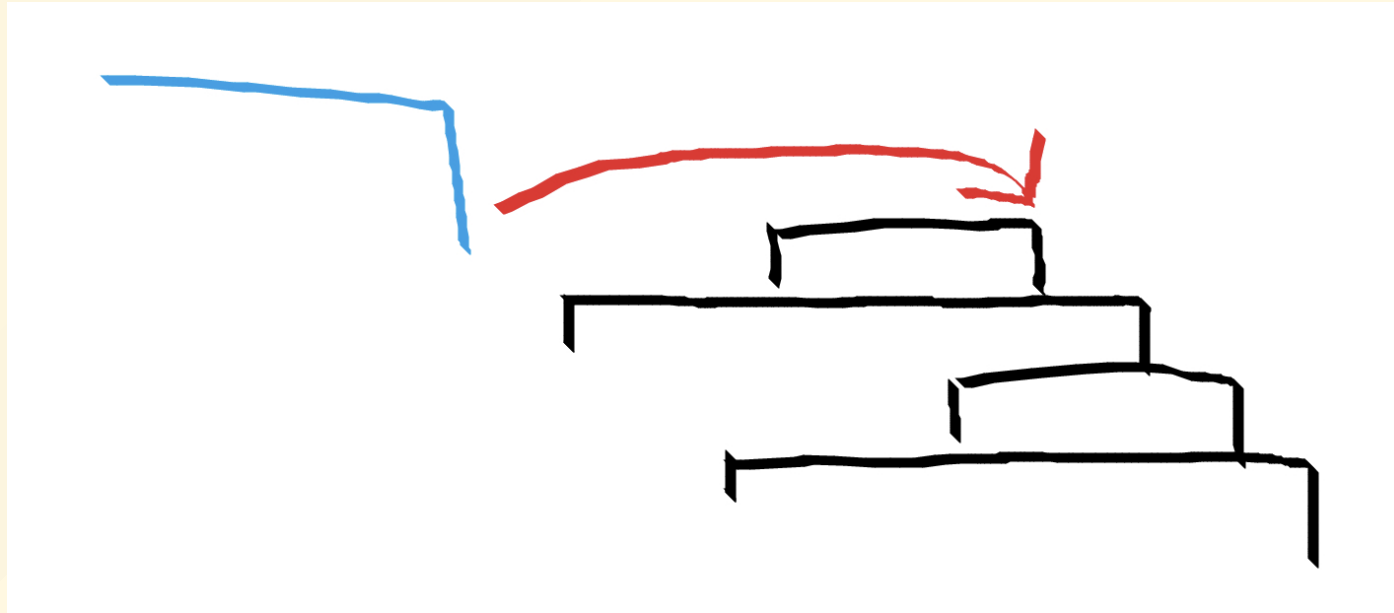
- $dp[E_L][L + 1] = 0, dp[E_L][R] = dp[E_L][R - 1] + d(E_{R-2}, E_{R-1})$

求める答えは $(B_L - A_L) + dp[E_L][R]$ なので, これで解けた

考察: 乗る航空便の選び方

都市 x 発の航空便として $e_x = (a_x, b_x)$ を選んだとき,
都市 $x + 1$ 発の航空便として選ぶものは一意に定まる

- $b_x \leq A_{x+1,j}$ となる j が存在するならば,
それを満たす中で $B_{x+1,j}$ が最も早いものに当日中に乗る



考察: 乗る航空便の選び方

都市 x 発の航空便として $e_x = (a_x, b_x)$ を選んだとき,
都市 $x + 1$ 発の航空便として選ぶものは一意に定まる

- $b_x \leq A_{x+1,j}$ となる j が存在するならば,
それを満たす中で $B_{x+1,j}$ が最も早いものに当日中に乗る
- $b_x \leq A_{x+1,j}$ となる j が存在しないならば,
 $B_{x+1,j}$ が最も早いものに翌日に乗る

(cf. 区間スケジューリング問題)

小課題 2 ($N \leq 2\,000, M_i \leq 5$)

都市 x 発の航空便として $e_x = (a_x, b_x)$ を選んだとき,
都市 $x + 1$ 発の航空便として選ぶものは一意に定まる

以下の値の組 $dp[e = E_{L,j}][R] = (X[e][R], E'[e][R])$ を考える

- 航空便 e に乗って都市 $L + 1$ に到着してから, 都市 R に到着するまでの最短時間 $X_{e,R}$
- その最短時間を達成する, 都市 R 着の航空便 $E'_{e,R}$

各航空便 $E_{L,j}$ について, $R = L + 1, \dots, N$ について $dp[E_{L,j}][R]$ の値が昇順に求まる(小課題 1 と同様, 全体 $O(N \sum M_i) = O(NM)$ 時間)

小課題 2 ($N \leq 2\,000, M_i \leq 5$)

始点 L を出発する航空便 $E_{L,j}$ をすべて試し,
 $(B_{L,j} - A_{L,j}) + dp[E_{L,j}][R]$ の最小値をとれば答え

$M_i \leq 5$ なので間に合う

小課題 3 ($M_i = 1$)

$M_i = 1$ のとき $dp[E_L][R] = \sum_{k=L}^{R-2} d(E_k, E_{k+1})$ であり, これは $\sum_{k=\text{左端}}^{\text{右端}} d_k$ の形になっている

Segtree なり累積和なり

小課題 4 ($M_i \leq 5$)

Segtree で通るということは, 区間 $[l, m)$ の情報と区間 $[m, r)$ の情報から区間 $[l, r)$ の情報が高速に求まっているということ

必ずしも $M_i = 1$ でなくてもこれは成り立つだろうか?

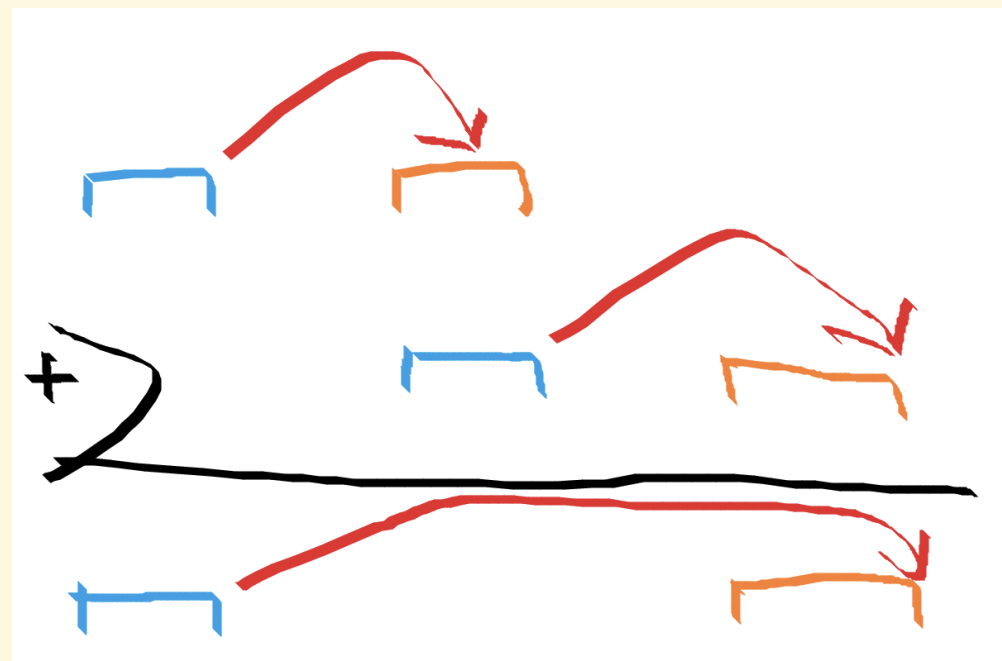
小課題 4 ($M_i \leq 5$)

区間 $[l, m)$ に対して $dp[E_{l,j}][m]$ ($1 \leq j \leq M_l$) が,
区間 $[m, r)$ に対して $dp[E_{m,j}][r]$ ($1 \leq j \leq M_m$) が分かっているとする

このとき, $e_l = E_{l,j}$, $e_m = E'[e_l][m]$ とすると,

- $X[e_l][r] = X[e_l][m] + X[e_m][r]$
- $E'[e_l][r] = E'[e_m][r]$

よって $dp[E_{l,j}][r]$ が $O(1)$ 時間で求まる.



小課題 4 ($M_i \leq 5$)

区間 $[l, m), [m, r)$ の情報から $O(M_l)$ 時間で区間 $[l, r)$ の情報が求まるので、Segtree に乗せたりダブリングをしたりすることで高速に計算できる。

- 前計算は $O(\sum M_i \log N) = O(M \log N)$ 時間
- クエリあたり $O(M_L \log N)$ 時間

$M_i \leq 5$ なので間に合う

小課題 4 ($M_i \leq 5$)

平方分割をしてもよい

一般に, $B \simeq N^{1/K}$ として K ステップで求めることにすると

- 前計算は $O(\sum M_i BK) = O(MN^{1/K}K)$ 時間
- クエリあたり $O(M_L K)$ 時間

メモリを多く使ううえ前計算が重い, クエリは速くなる

小課題 5,6

現状本問題のクエリが $\sum_q M_{L_q}$ 回のデータ構造クエリ処理に変換される

左のほうに巨大な M_i があると, それを最悪 $O(N)$ 回見せられることになって計算量がとっ散らかる

今のところ計算量が始点側の M_L に依存しているが, 終点側の M_{R-1} に依存を分散できないか?

考察(再掲): 乗る航空便の選び方

都市 x 発の航空便として $e_x = (a_x, b_x)$ を選んだとき,
都市 $x + 1$ 発の航空便として選ぶものは一意に定まる

時間軸を逆転させることで, 都市 $x + 2$ 着の航空便から都市 $x + 1$ 着
の航空便を一意に選べることが分かる!

小課題 5,6

左からと右からで 2 つ Segtree (or ダブリング or 平方分割) を用意し, 発着便が少ないほうを総当たりしてクエリを処理することで, クエリ計算量の M_L が $\min(M_L, M_{R-1})$ に置き換わる

これは本質的改善か？

小課題 5,6

クエリが被らないとし, M_i を大きい順に並べたものを $M'_1 \geq \dots \geq M'_{N-1}$ とすると, $\sum_{q=1}^Q \min(M_L, M_{R-1})$ の上界は

$$\sum_{k=1}^{\lfloor \sqrt{2Q} \rfloor} k M'_k \leq \sqrt{2Q} M$$

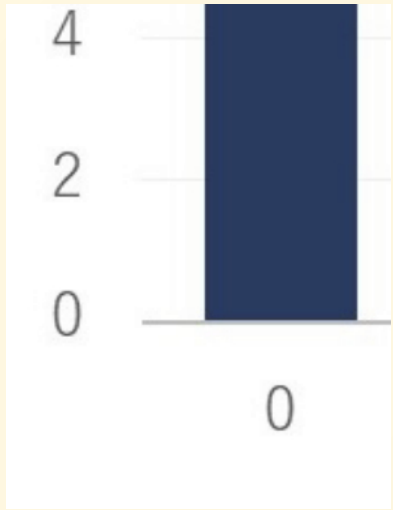
よって, クエリ処理が $O(M\sqrt{Q})$ 回に減らせる!

小課題 5,6

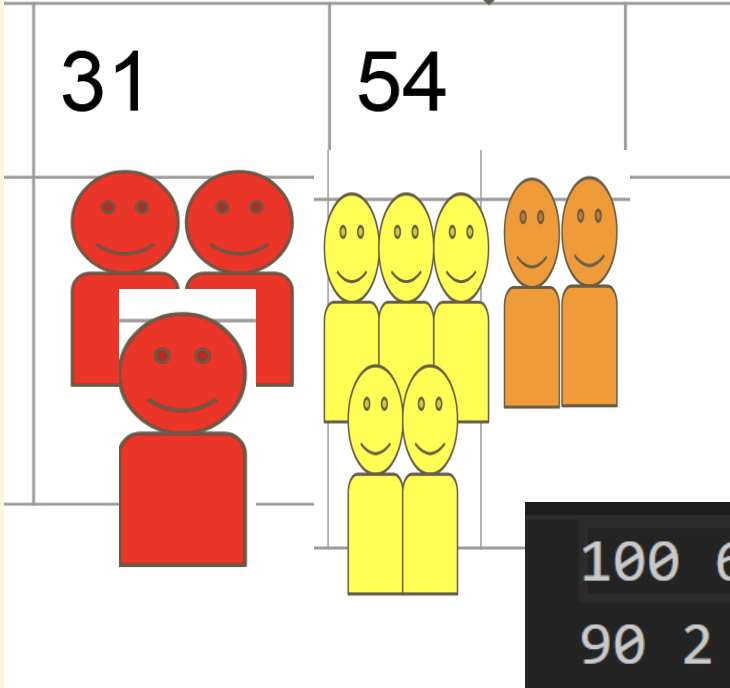
クエリ処理が $O(M\sqrt{Q})$ 回に減らせたので,

- Segtree やダブリングなら全体 $O(M \log N + Q\sqrt{M} \log N)$ 時間
- 平方分割系なら全体 $O(MN^{1/K}K + M\sqrt{Q}K)$ 時間
 - 特に $K = 2$ とすれば $O(MK(\sqrt{N} + \sqrt{Q}))$ 時間

いずれにせよ, 十分高速

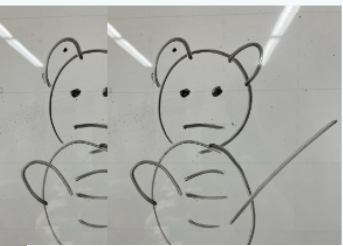


小課題1~4



得点分布

(90点)

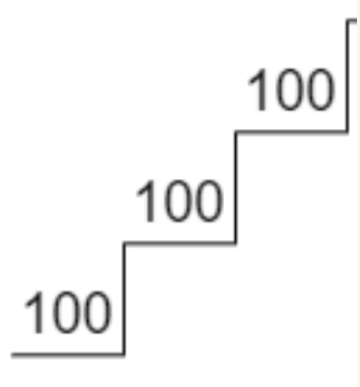
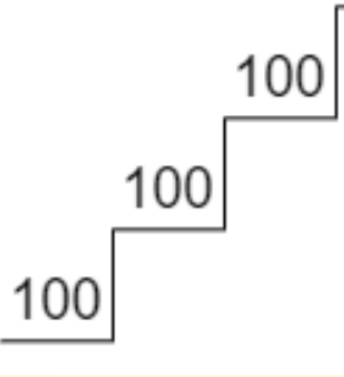


6 0 - - - -

23点



100	6
90	2
54	7
31	3
23	2
6	1
0	4



まともな見た目の得点分布

An underwater photograph showing a large, dense school of small, silvery fish swimming in clear blue water. A larger, dark-colored stingray is visible in the lower-left foreground, swimming towards the left. The background is filled with the movement of the fish, creating a sense of depth and activity.

Islands

Editorial : blackyuki



問題概要

問題概要

- N 頂点の木がある
- 木の情報は与えられていない
- 木の形を求めたい
- 「頂点 i から k 番目に近い頂点の番号は何か？」を質問できる
 - • • 同じ距離の頂点は番号の大小で決まる

問題の性質

インタラ
クティブ

木

インタラクティブにおける典型考察

- 色んな解法を試して上手くいくものを選ぶ
- 一つの解法に固執するのはもったいない
- 普通のBatch課題における典型アルゴリズムのシミュレーション
 - • • BFS, DFS, Dijkstra, など

木を特定する問題における典型考察

- A. 頂点 $1, 2, \dots, N$ の順に追加していく
- B. 頂点 1 から近い順に探索していく
- C. 葉から削っていく
- D. 直径の利用

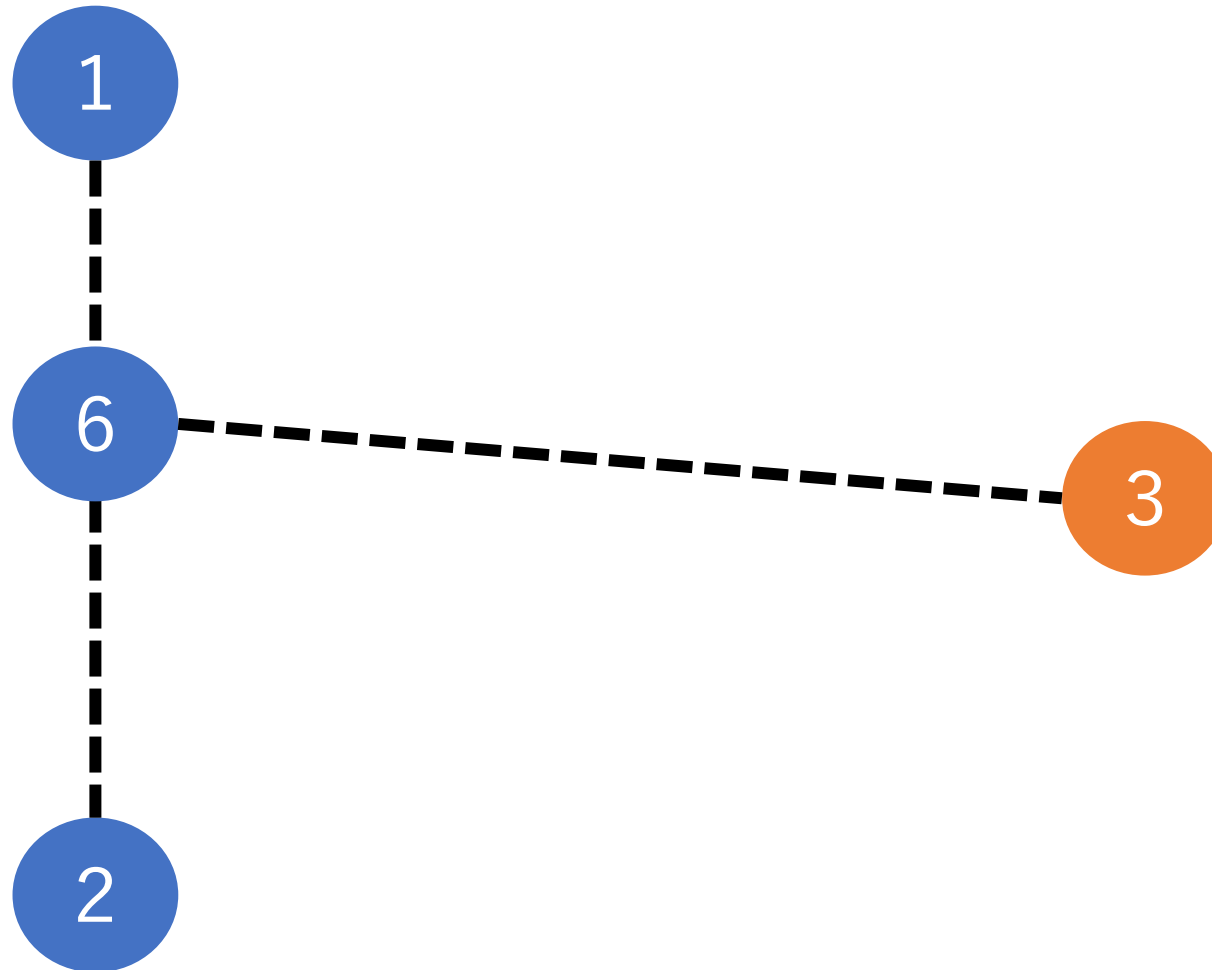
A. 頂点 $1, 2, \dots, N$ の順に追加していく

1

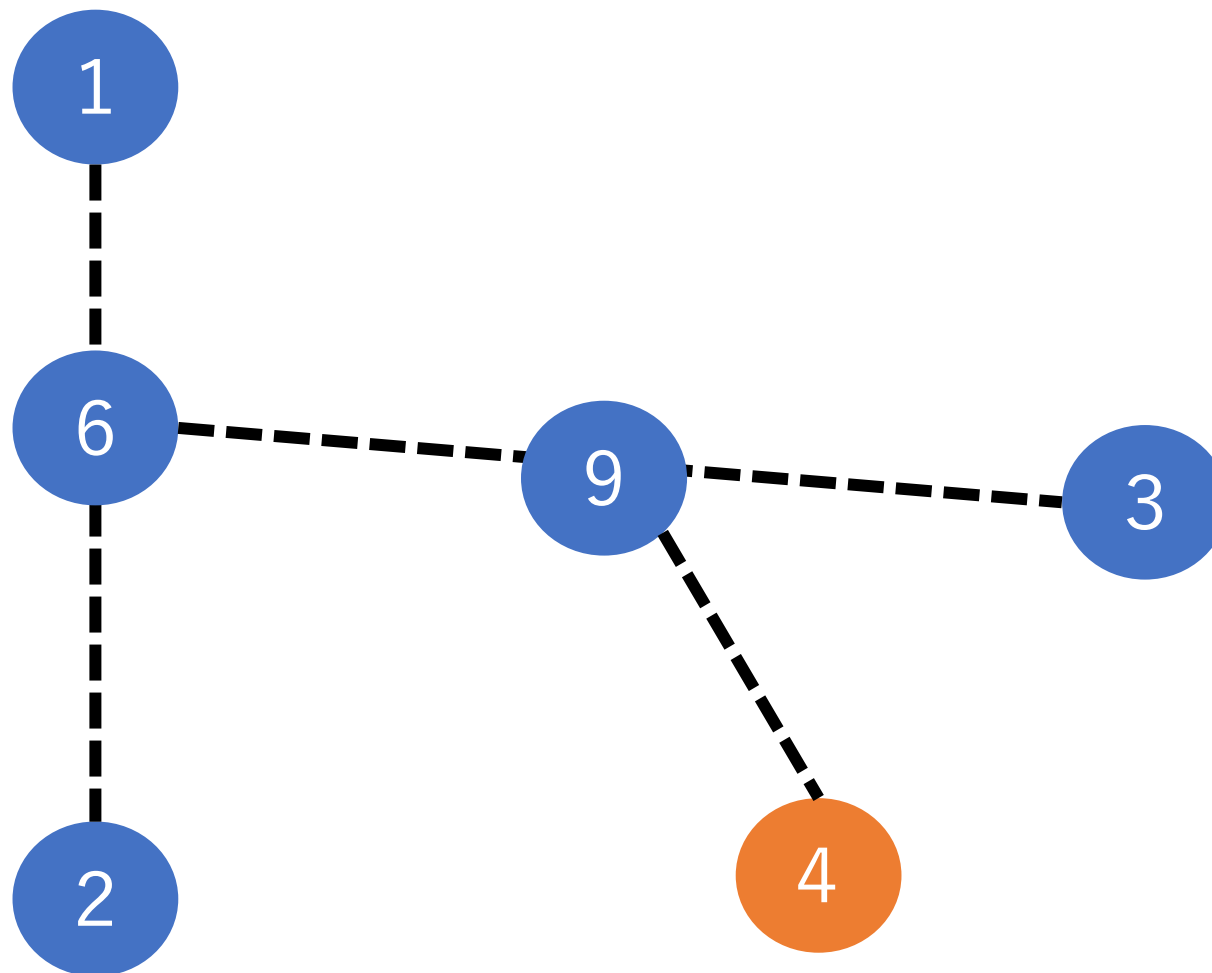
A. 頂点 $1, 2, \dots, N$ の順に追加していく



A. 頂点 $1, 2, \dots, N$ の順に追加していく



A. 頂点 $1, 2, \dots, N$ の順に追加していく



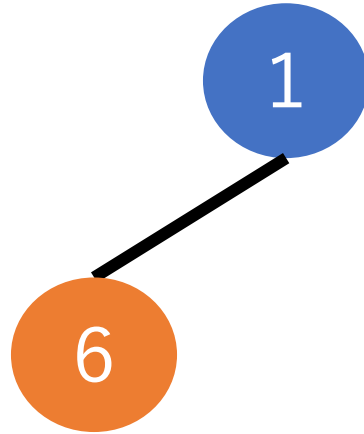
B. 頂点 1 から近い順に探索していく

BFS（幅優先探索）



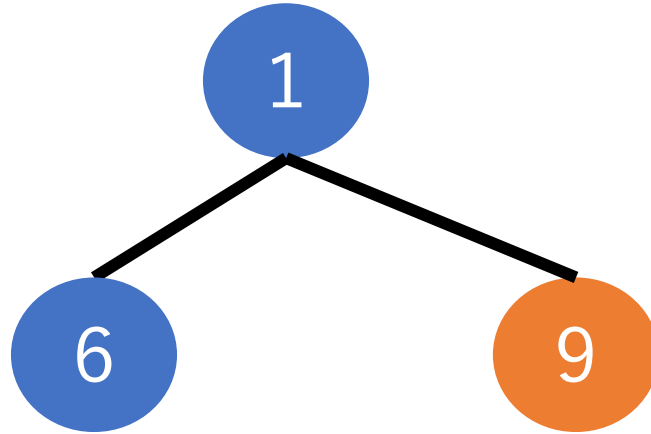
B. 頂点 1 から近い順に探索していく

BFS (幅優先探索)



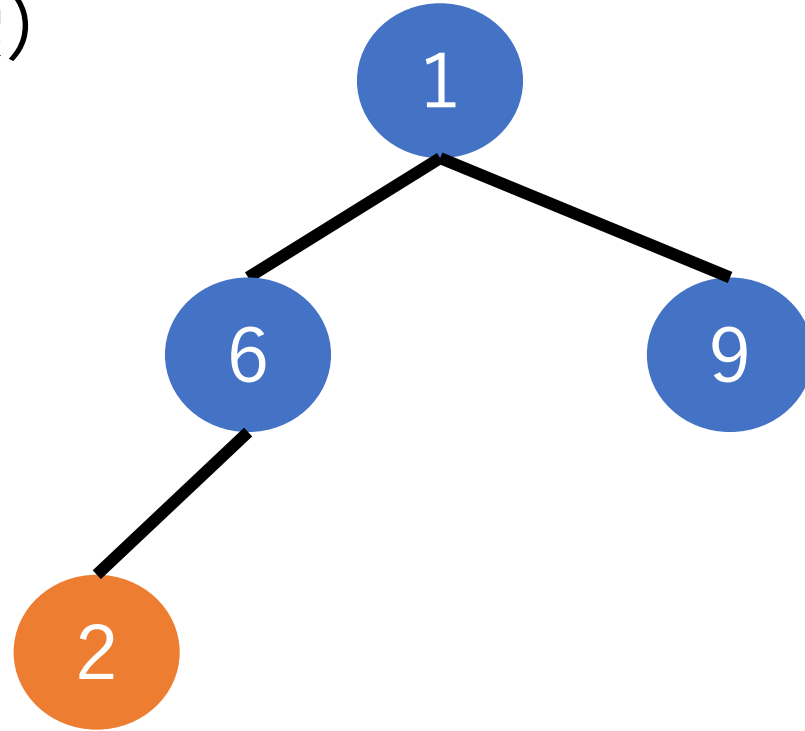
B. 頂点 1 から近い順に探索していく

BFS (幅優先探索)



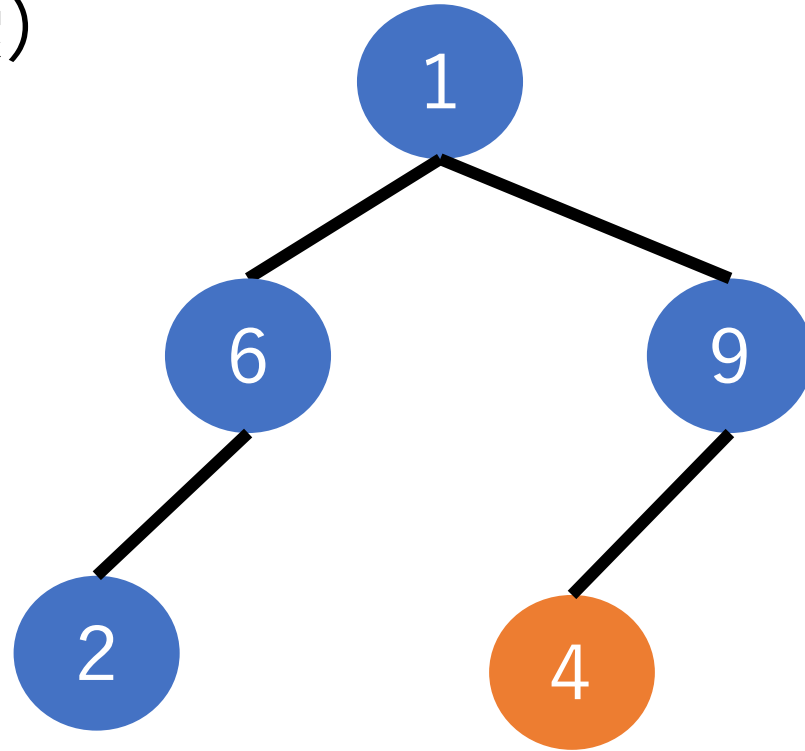
B. 頂点 1 から近い順に探索していく

BFS (幅優先探索)



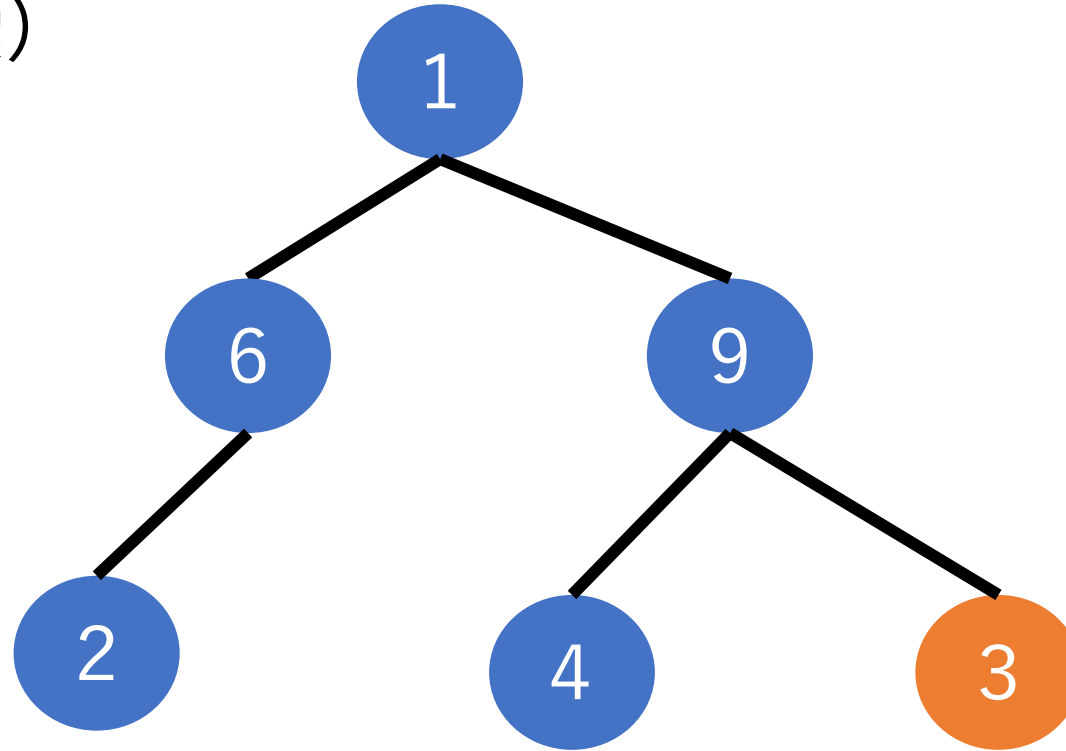
B. 頂点 1 から近い順に探索していく

BFS (幅優先探索)



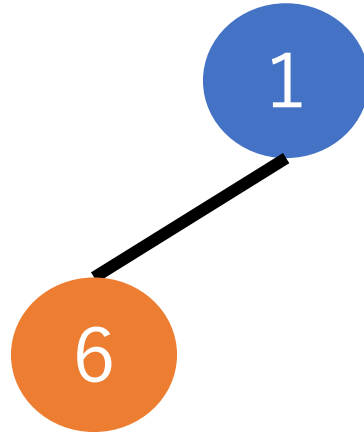
B. 頂点 1 から近い順に探索していく

BFS (幅優先探索)



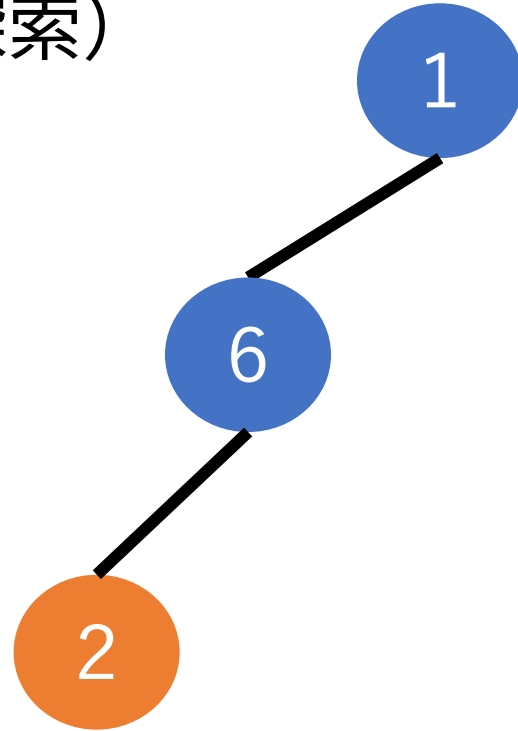
B. 頂点 1 から近い順に探索していく

DFS（深さ優先探索）



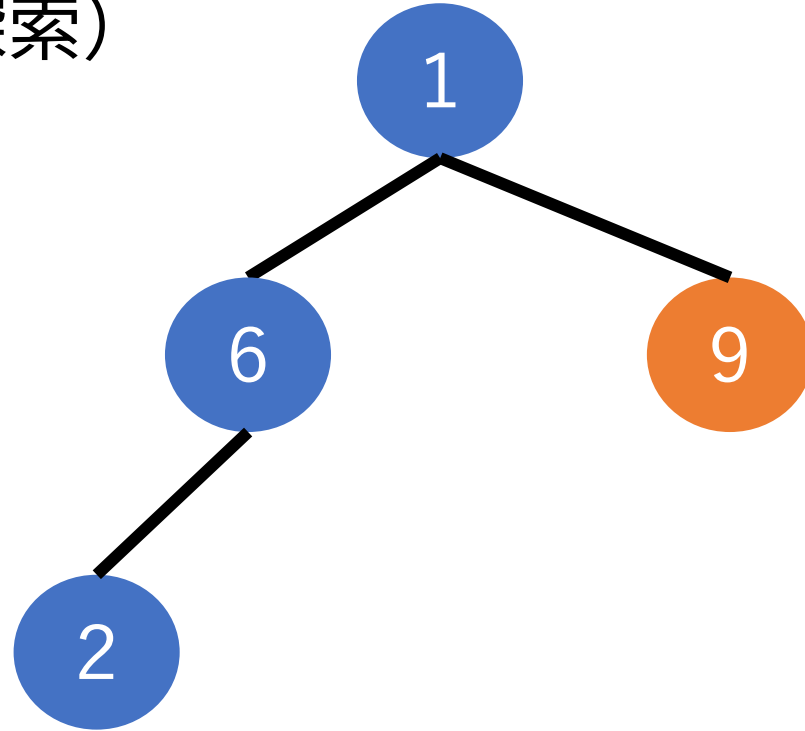
B. 頂点 1 から近い順に探索していく

DFS（深さ優先探索）



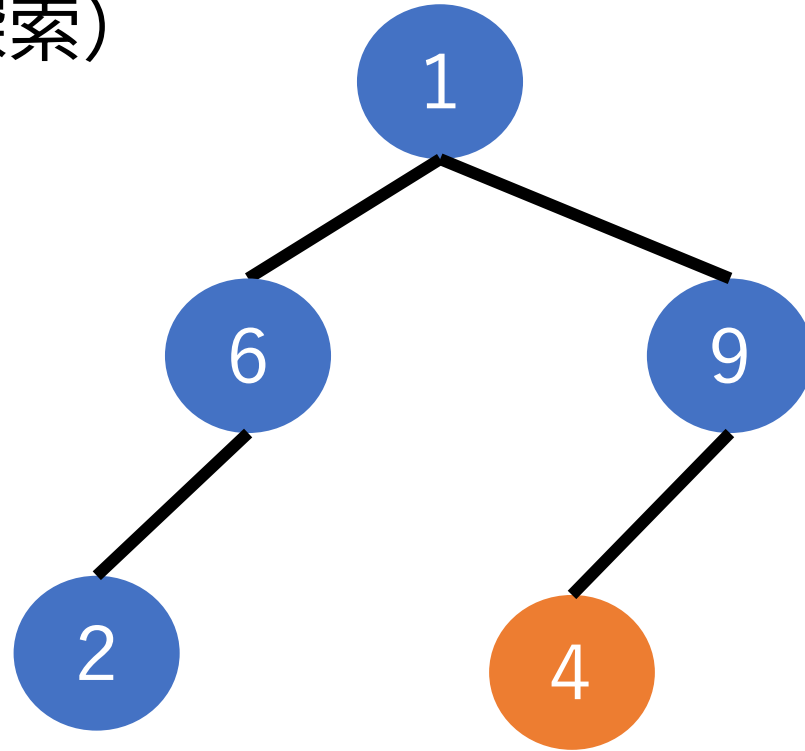
B. 頂点 1 から近い順に探索していく

DFS (深さ優先探索)



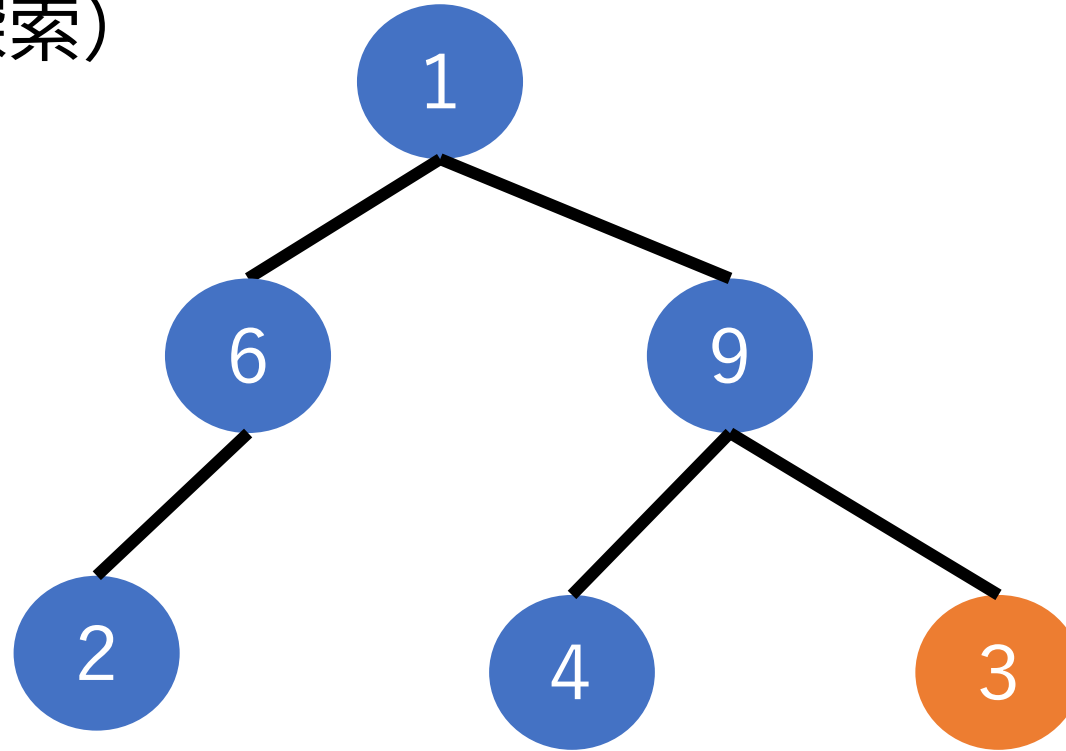
B. 頂点 1 から近い順に探索していく

DFS (深さ優先探索)



B. 頂点 1 から近い順に探索していく

DFS (深さ優先探索)

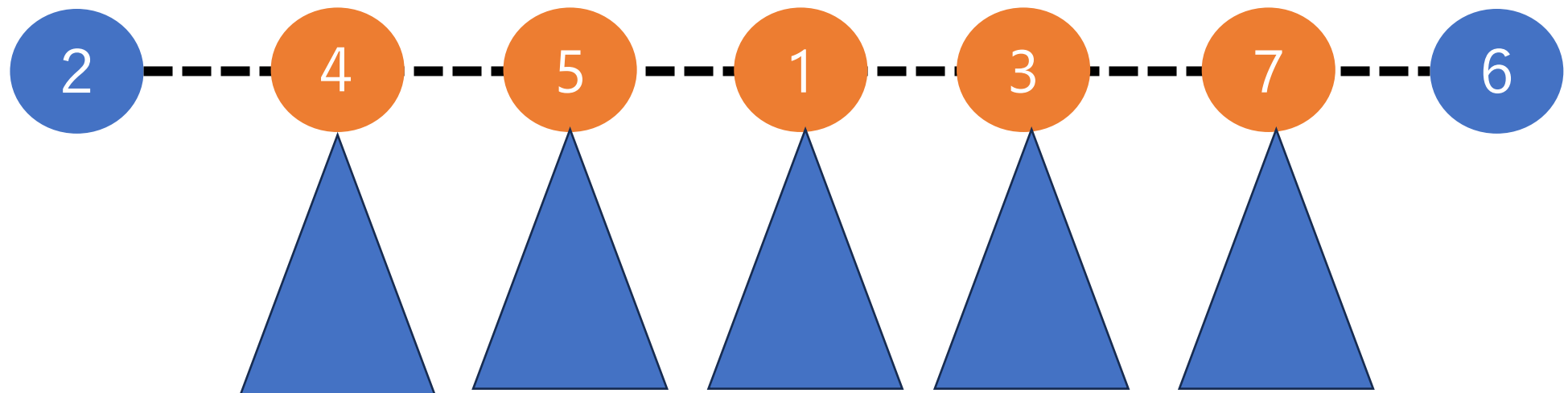


C. 葉から削っていく

まず何らかの方法で1つの葉とその接続先を見つける

→その頂点を取り除いて、頂点数が1つ少ない問題に帰着させる

D. 直径の利用

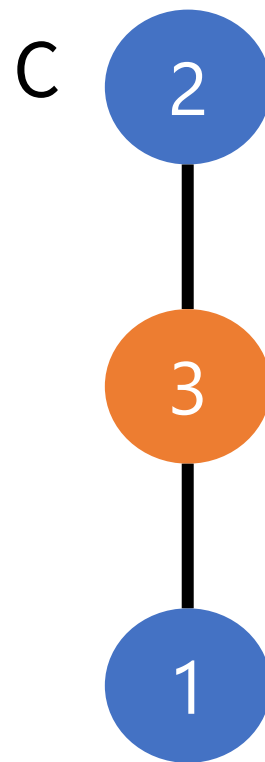
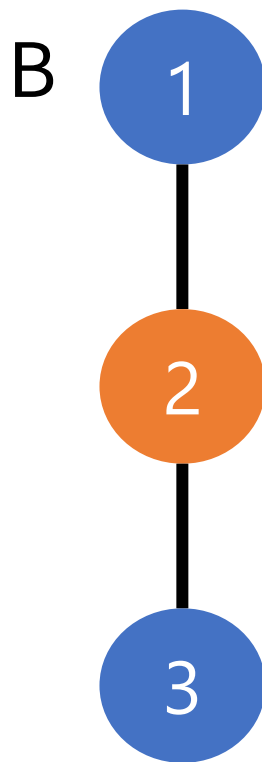
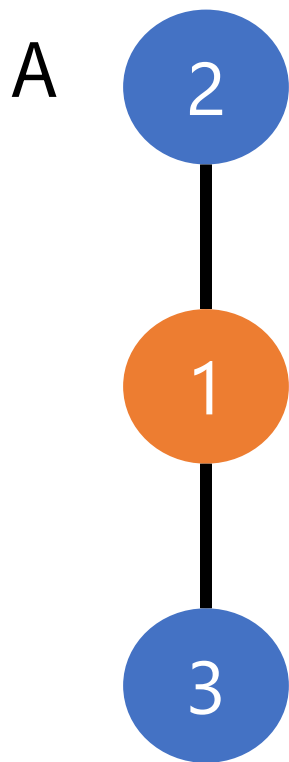


小課題 1

N=3

気づき

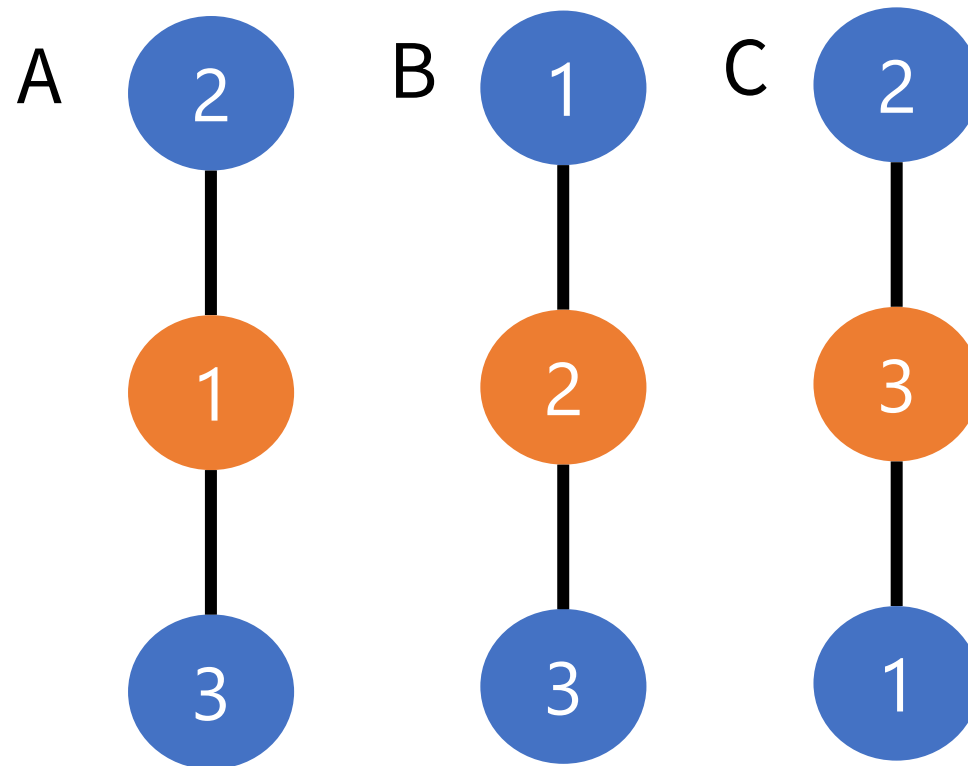
3頂点の木は種類が少なく、簡単に列挙できる



気づき

頂点 i から最も近い距離の頂点を聞く

	$i=1$	$i=2$	$i=3$
A	2	1	1
B	2	1	2
C	3	3	1



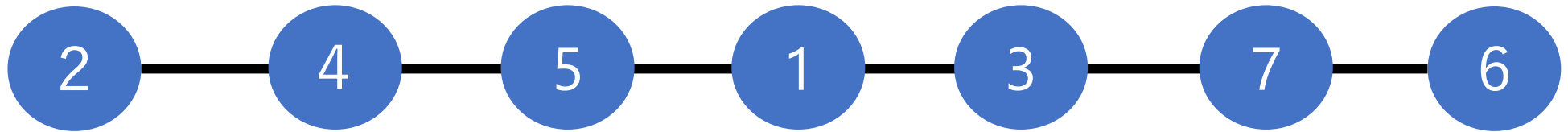
```
void solve(int N, int L) {  
    // N = 3, L = N^2  
    if (query(1, 1) == 3) {  
        answer(1, 3);  
        answer(2, 3);  
    }  
    else {  
        answer(1, 2);  
        answer(query(3, 1), 3);  
    }  
}
```

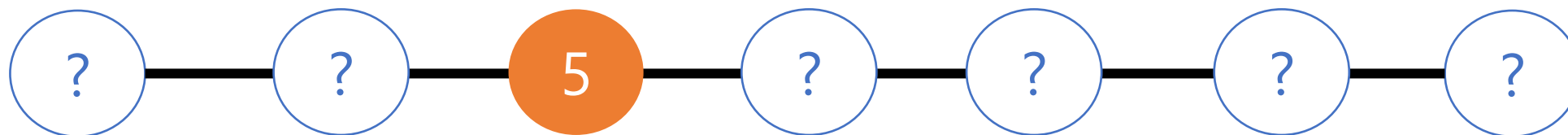
→小課題1 AC!

小課題2,3

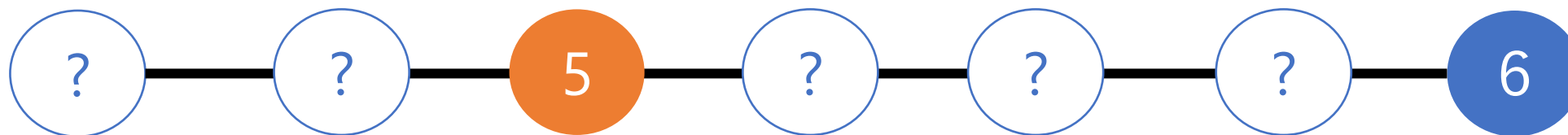
パスグラフ

パスグラフ

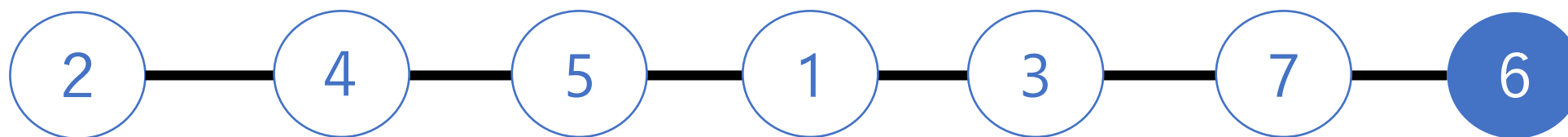




適当な頂点から最も遠い頂点を聞く



適当な頂点から最も遠い頂点を聞く



その頂点から近い頂点を順に聞く

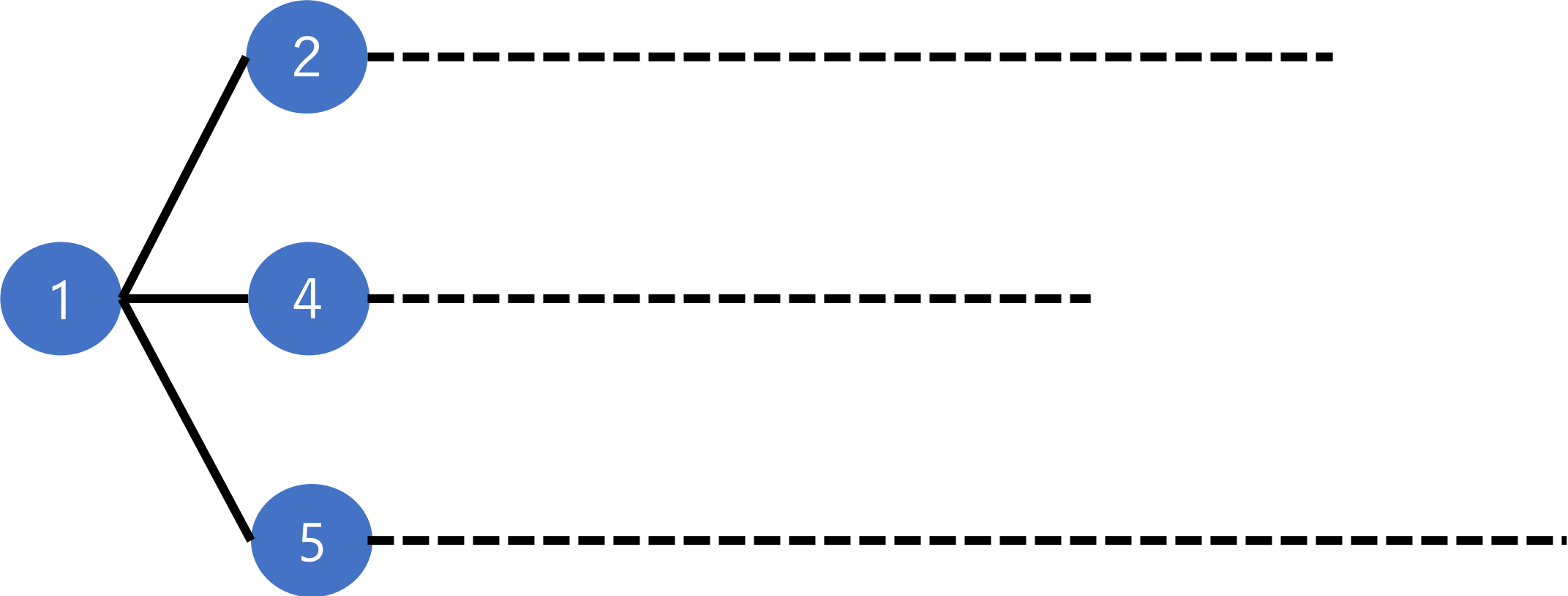
```
void solve(int N, int L){  
    // path, L = 2N  
    int a = query(1, N - 1), b;  
    b = a;  
    for(int i = 0; i < N - 1; i++){  
        int c = query(a, i + 1);  
        answer(min(b, c), max(b, c));  
        b = c;  
    }  
}
```

→小課題2,3 AC!

小課題4,5

特殊な形のグラフ

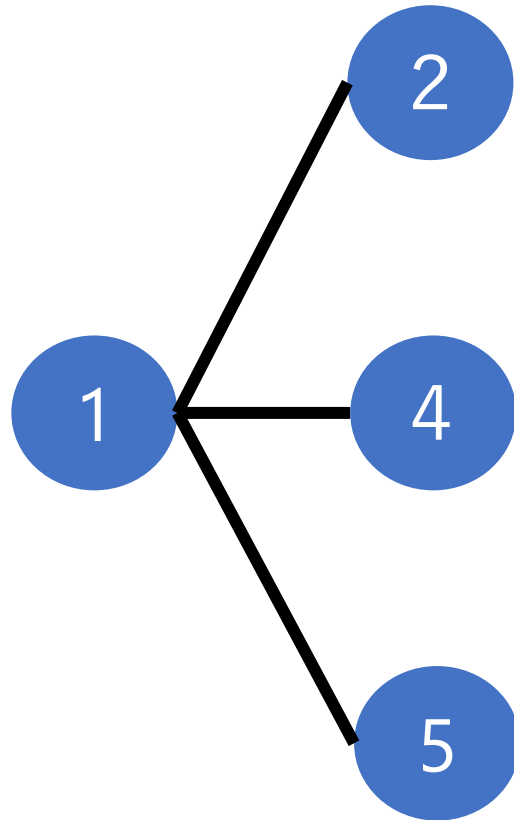
特殊な形のグラフ



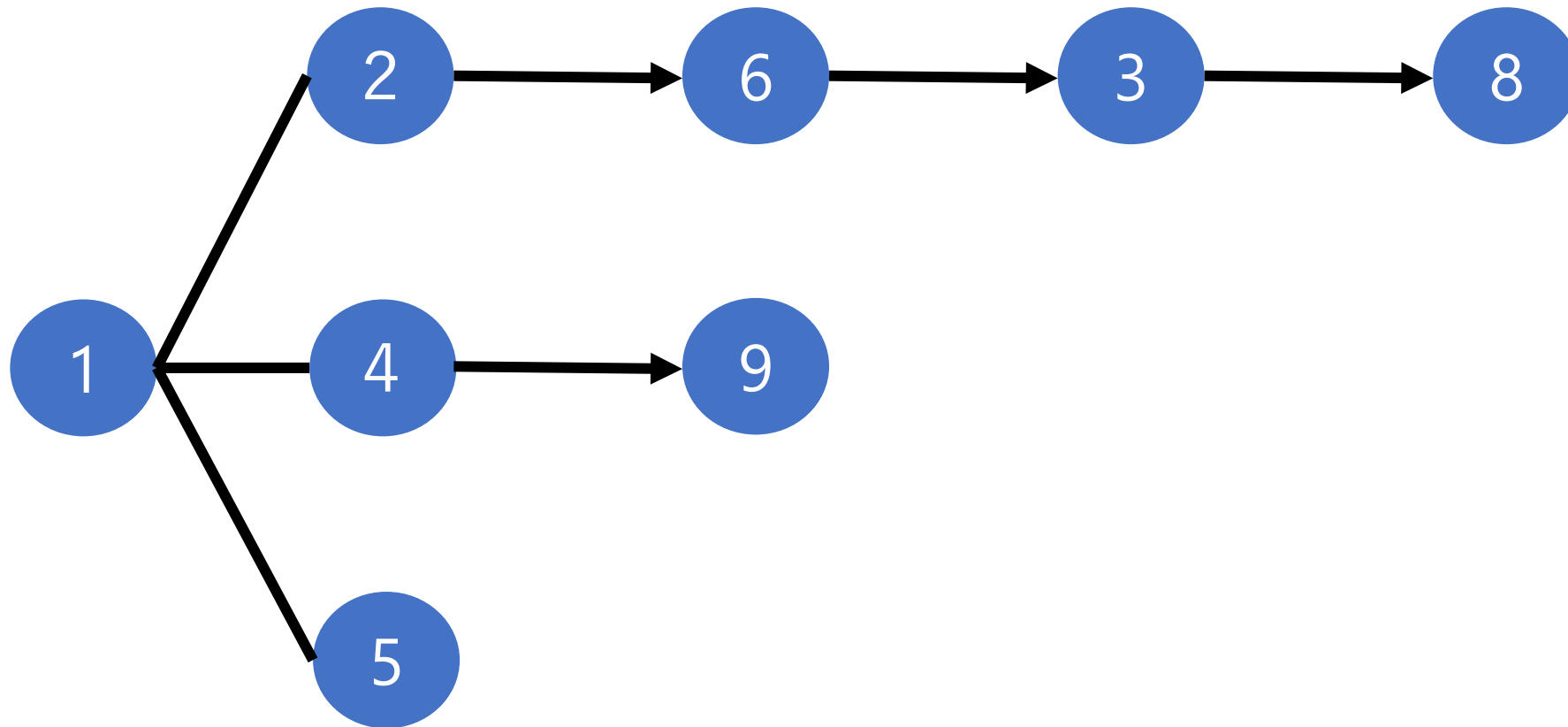
木を特定する問題における典型考察

- A. 頂点 $1, 2, \dots, N$ の順に追加していく
- B. 頂点 1 から近い順に探索していく
- C. 葉から削っていく
- D. 直径の利用

最初に頂点1に接続する3頂点の番号を求める



DFSする（今いる頂点から近い2頂点を聞く）



```

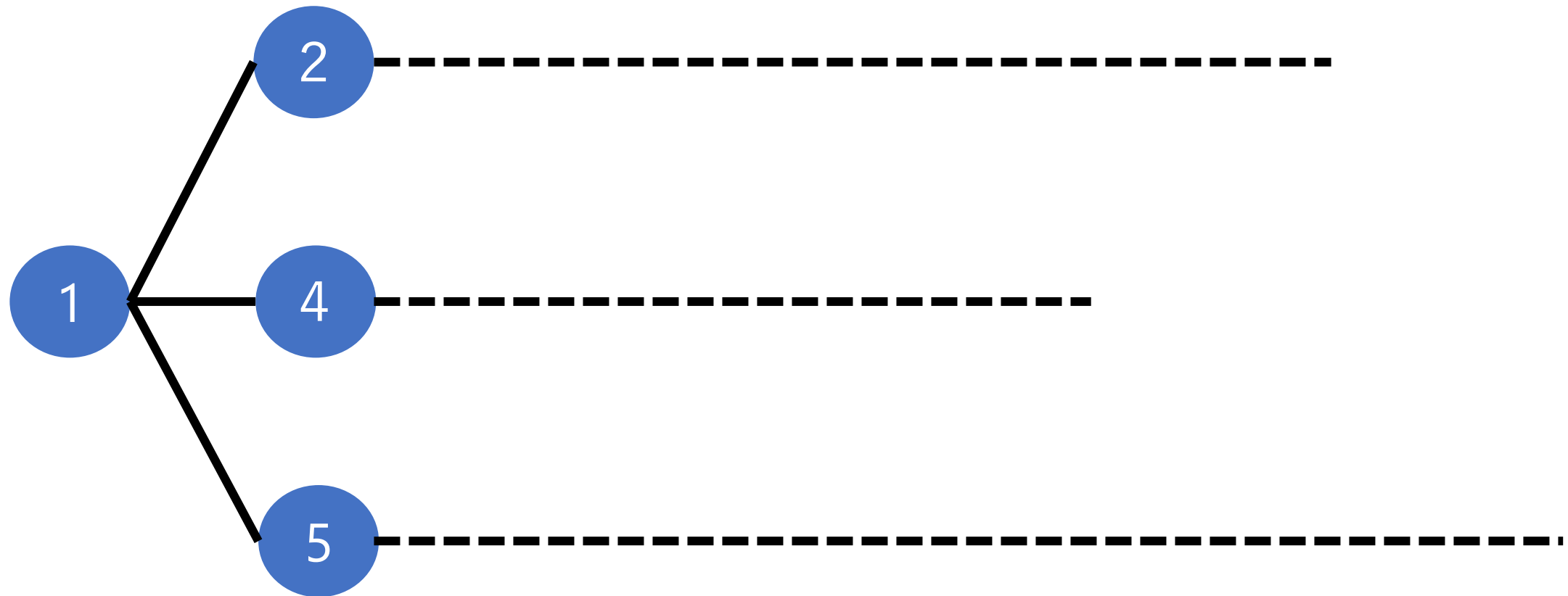
void solve(int N, int L){
    vector<int> ok(N + 1, 0);
    vector<int> v = {query(1, 1), query(1, 2), query(1, 3)};
    for(int i = 0; i < 3; i++){
        answer(1, v[i]);
    }
    ok[1] = 1;

    for(int i = 0; i < 3; i++){
        while(true){
            int b = query(v[i], 1);
            int c = query(v[i], 2);
            if(ok[b] == 0){
                ok[v[i]] = 1;
                answer(v[i], b);
                v[i] = b;
            }
            else if(ok[c] == 0){
                ok[v[i]] = 1;
                answer(v[i], c);
                v[i] = c;
            }
            else{
                break;
            }
        }
    }
}

```

→小課題4,5 AC!

(別解)



木を特定する問題における典型考察

- A. 頂点 $1, 2, \dots, N$ の順に追加していく
- B. 頂点 1 から近い順に探索していく
- C. 葉から削っていく
- D. 直径の利用

頂点1から最も距離の遠い頂点を聞く

→その頂点から近い順に決めていく



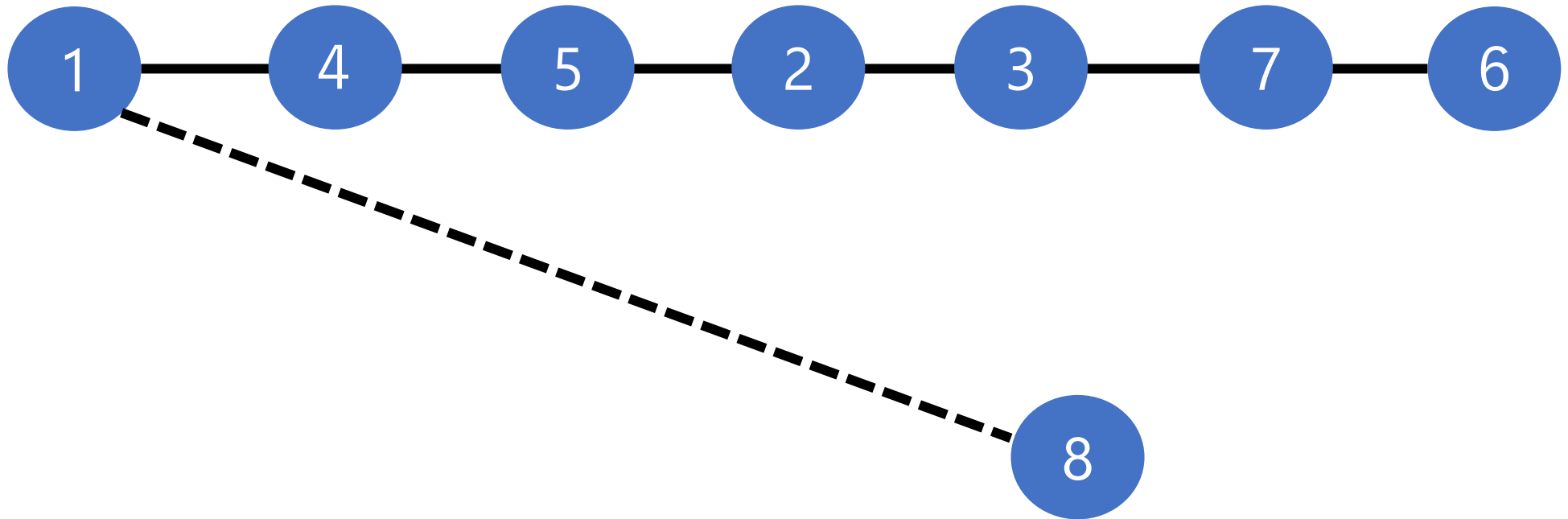
頂点1から最も距離の遠い頂点を聞く

→その頂点から近い順に決めていく

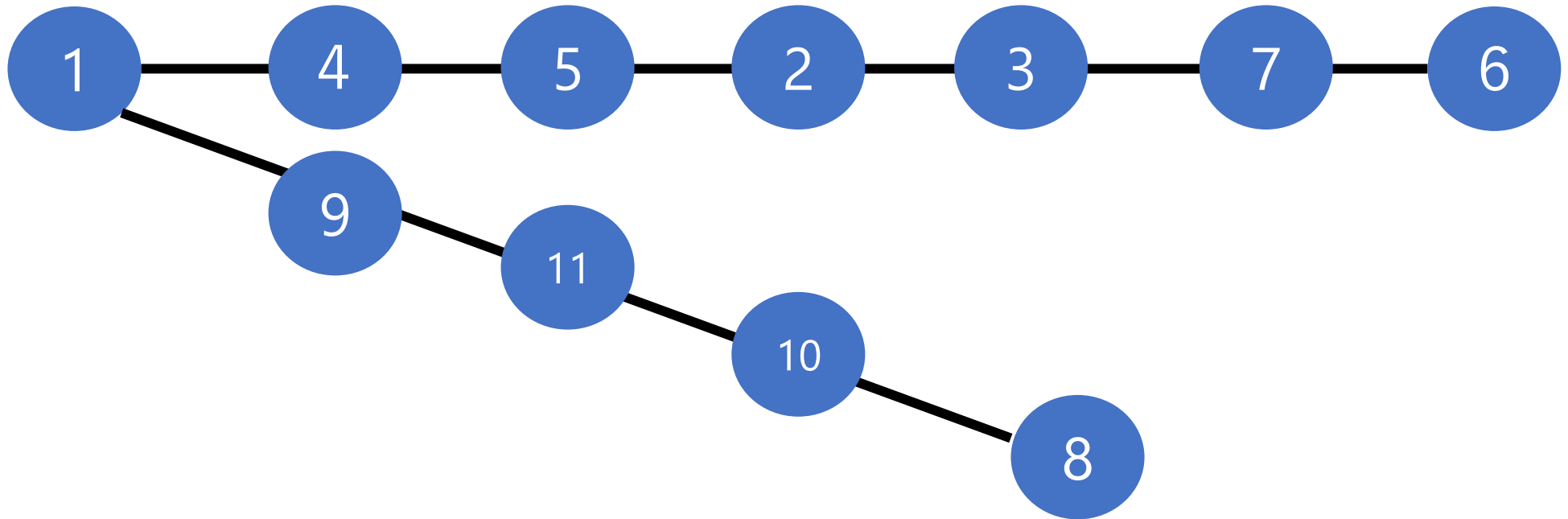
→頂点1が来たら打ち切り



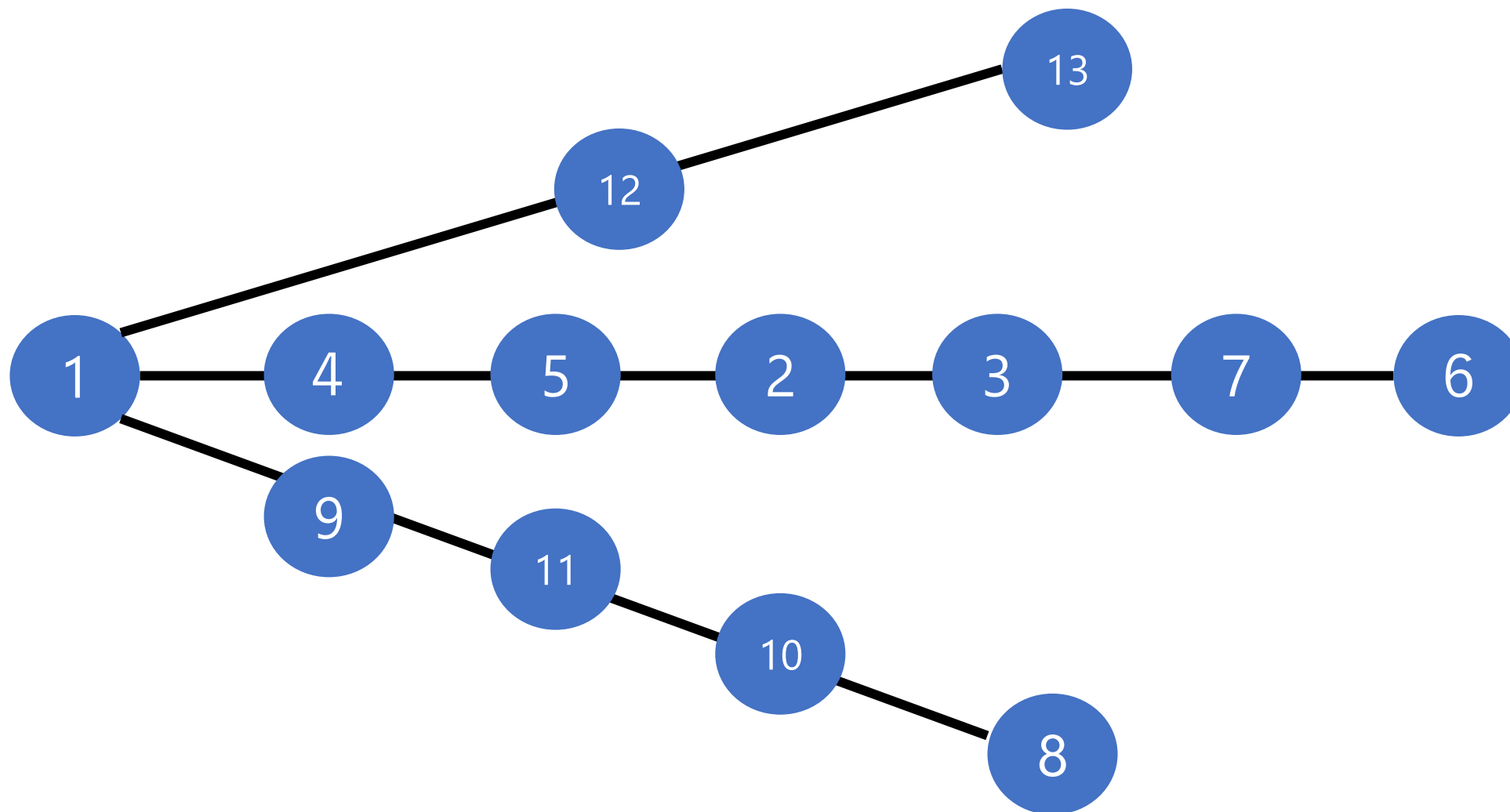
頂点6から最も遠い頂点を聞く



頂点8からも同様に聞いていく(頂点1で打ち切り)



残りの頂点は 1 からの近い順につなげる



小課題6

$$L=N^2$$

木を特定する問題における典型考察

- A. 頂点 $1, 2, \dots, N$ の順に追加していく
- B. 頂点 1 から近い順に探索していく
- C. 葉から削っていく
- D. 直径の利用

頂点1から最も距離の遠い頂点を聞く

→その頂点は葉であり、その接続先も分かる

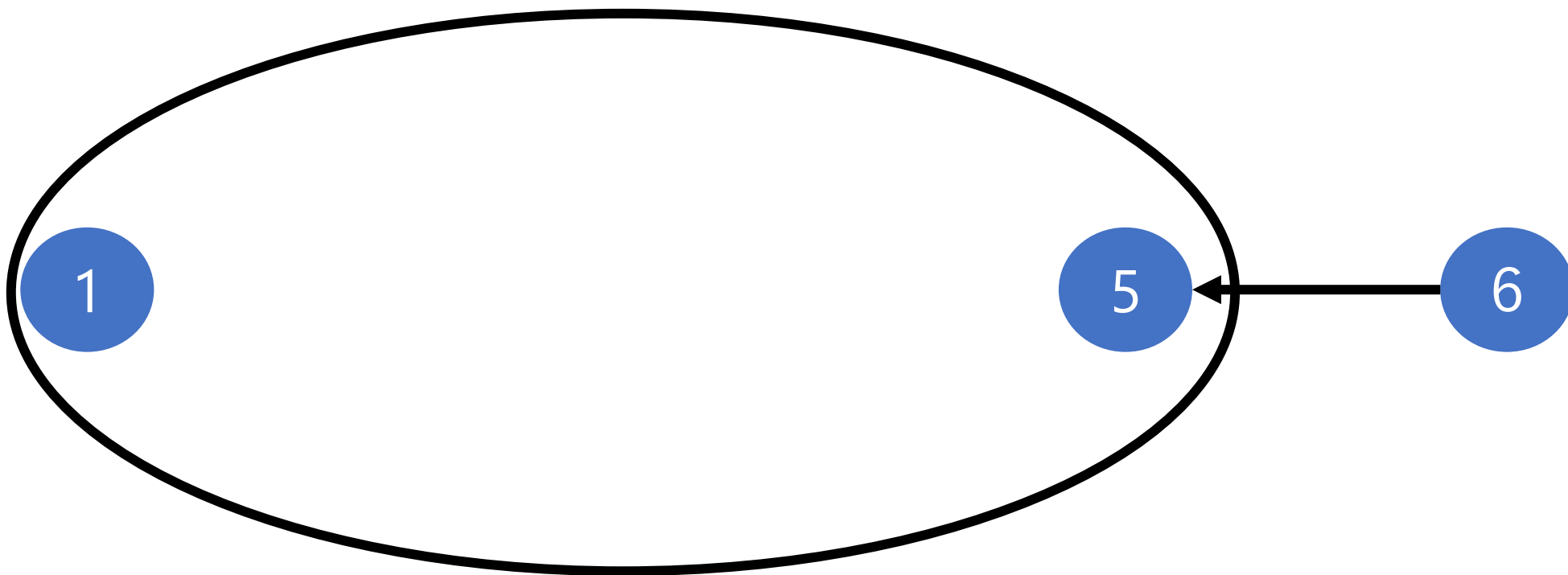
→その葉を無視して残りの木を再帰的に求めればよい



頂点1から最も距離の遠い頂点を聞く

→その頂点は葉であり、その接続先も分かる

→その葉を無視して残りの木を再帰的に求めればよい



```

vector<int> used(N, 0);
int a = Q[0][N - 1];

for(int i = 0; i < N - 1; i++){
    for(int j = 1; j < N; j++){
        int b = Q[a][j];
        if(!used[b]){
            used[a] = 1;
            answer(a + 1, b + 1);
            for(int k = N - 1; k >= 0; k--){
                int c = Q[b][k];
                if(!used[c]){
                    a = c;
                    break;
                }
            }
            break;
        }
    }
}

```

→小課題6 AC!

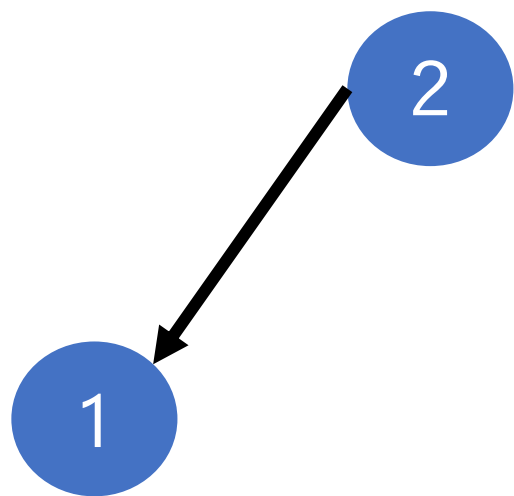
小課題7,8

$L=3N, 2N$

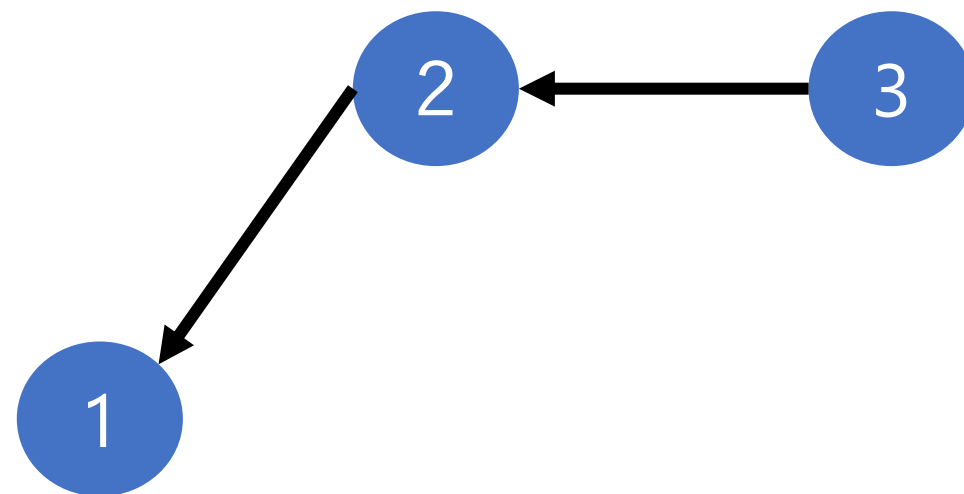
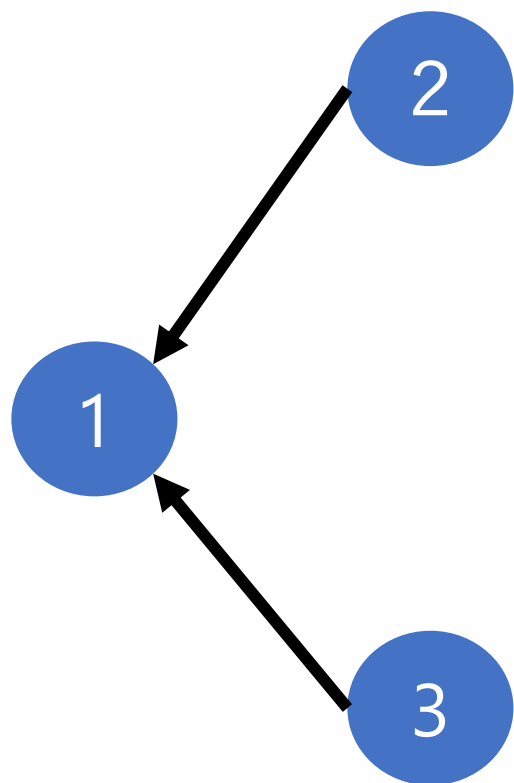
木を特定する問題における典型考察

- A. 頂点 $1, 2, \dots, N$ の順に追加していく
- B. 頂点 1 から近い順に探索していく
- C. 葉から削っていく
- D. 直径の利用

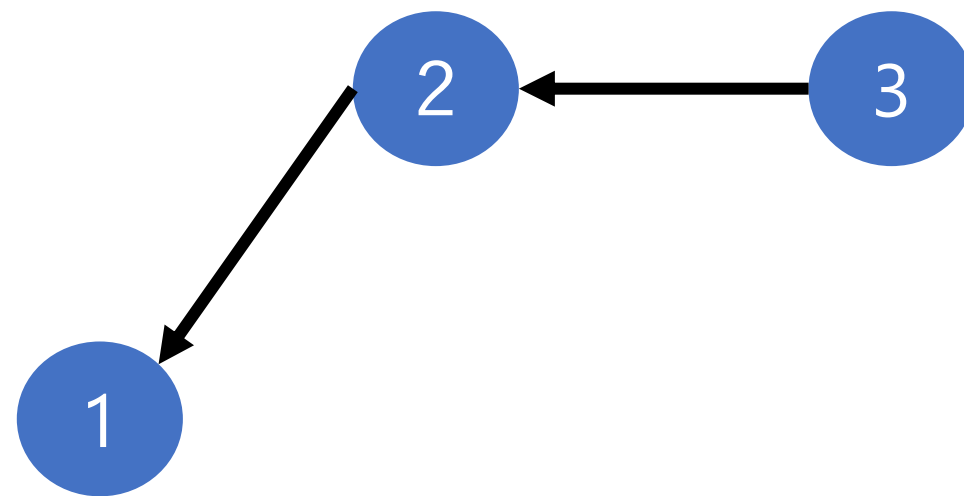
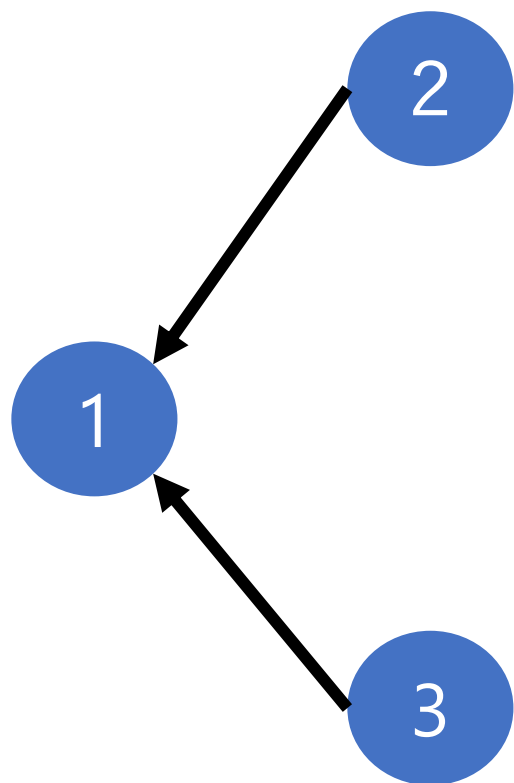
頂点1から最も近い頂点をつなげる



頂点1から次に近い頂点をつなげる



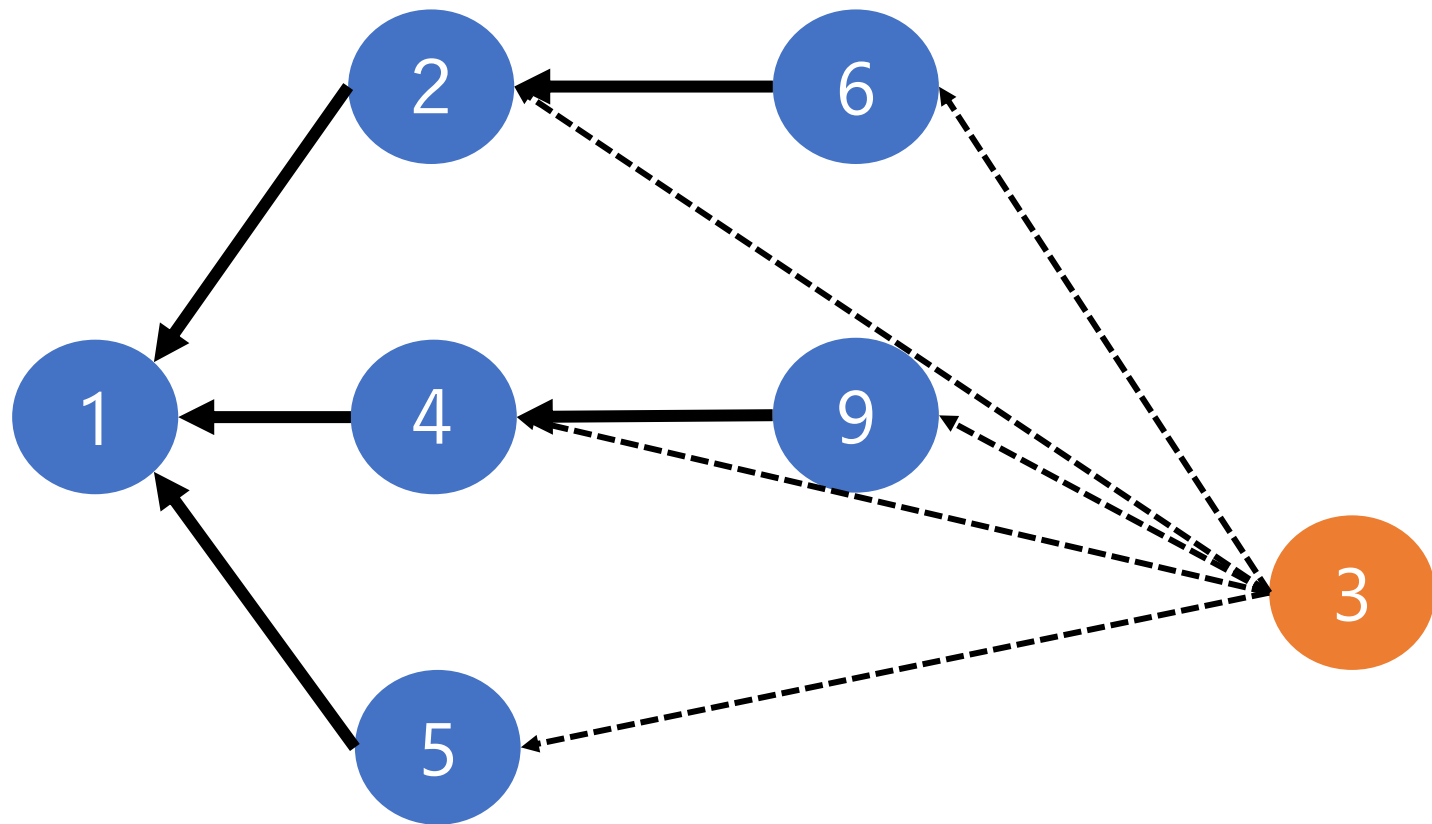
(どっちか分からない)



頂点3から近い頂点を順に聞いていき、
頂点1と頂点2のどちらが先に現れるか
で判定

まとめると・・・

- 1． 頂点1から近い順に頂点を追加していく
- 2． 追加したい頂点から近い頂点を順に質問していく
- 3． 既に見た頂点があればそれが親に確定する

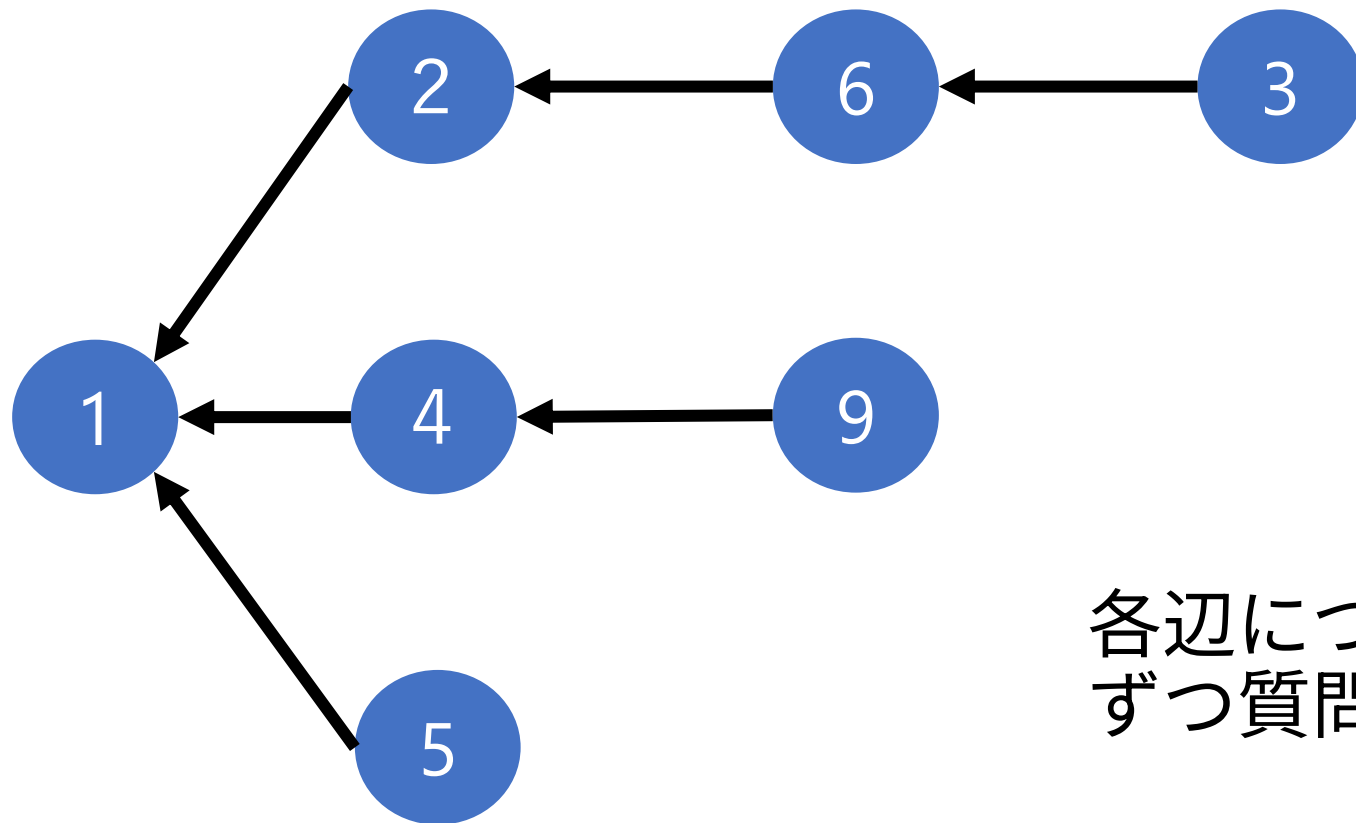


まとめると・・・

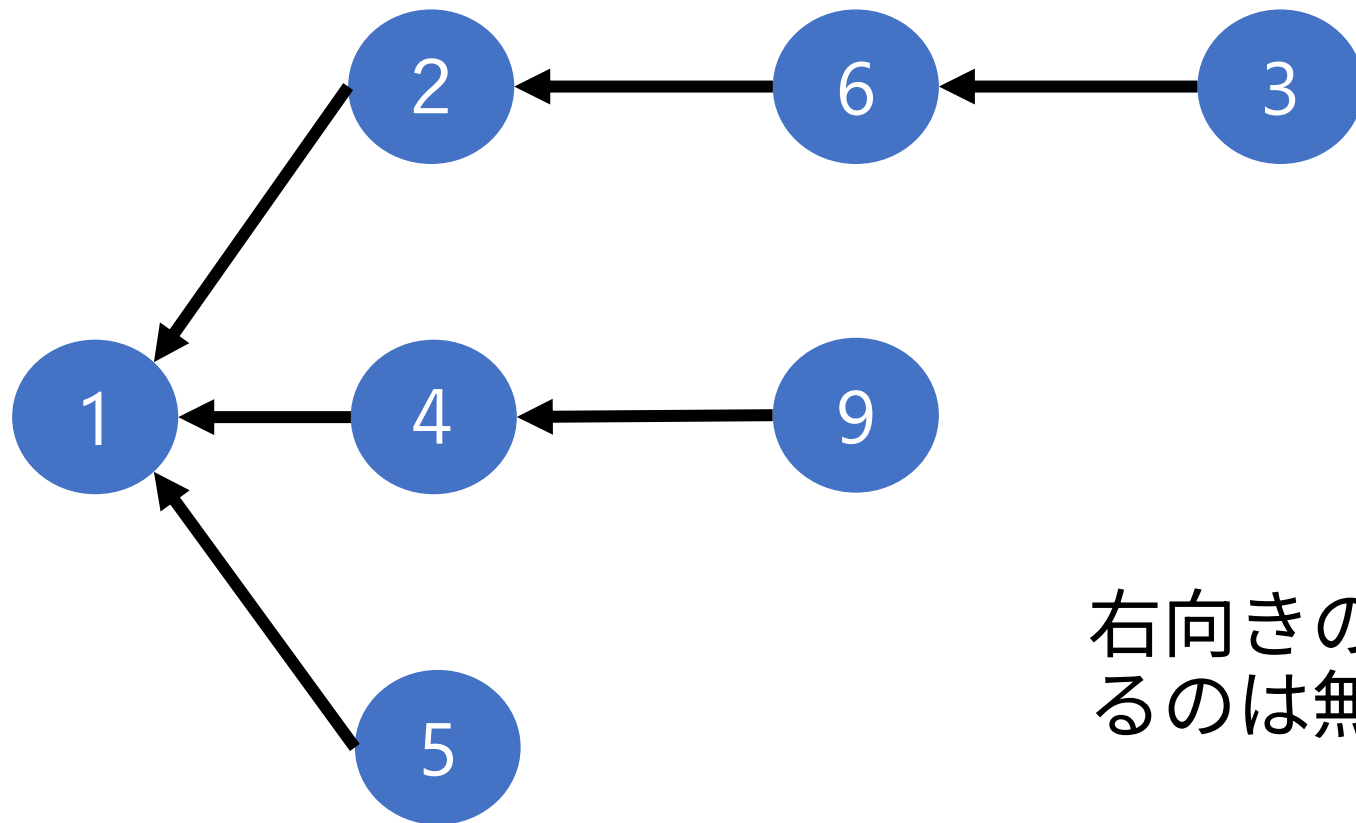
1. 頂点1から近い順に頂点を見ていく
2. 追加したい頂点から近い頂点を順に質問していく
3. 既に見た頂点があればそれが親に確定する

最初の N 回 + $2N$ 回 → 合計 $3N$ 回

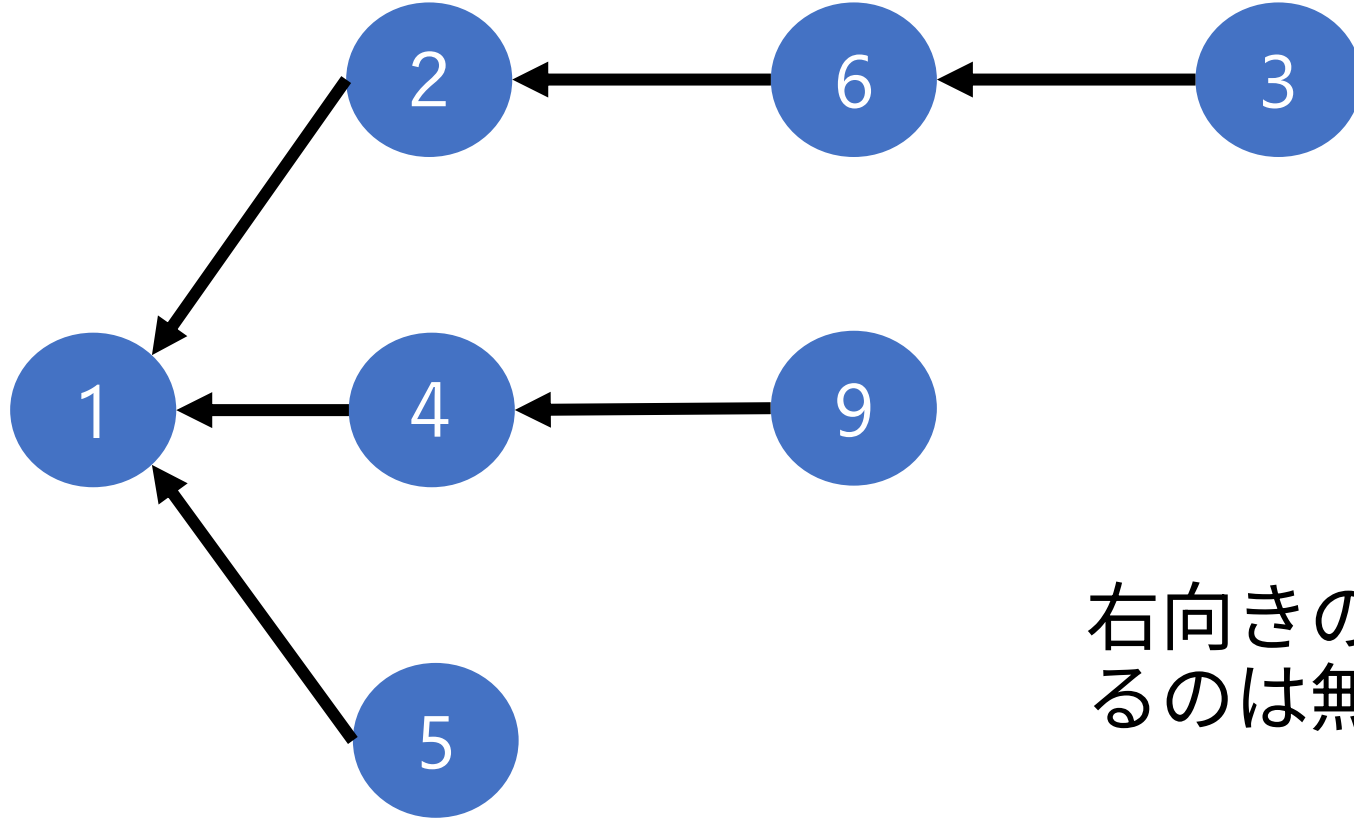
なぜか？



各辺について左向きと右向きの2回
ずつ質問することになる



右向きの辺を聞いた時の情報を捨てるのは無駄



右向きの辺を聞いた時の情報を捨てるのは無駄

→再利用すれば合計 $2N$ で満点が得られる

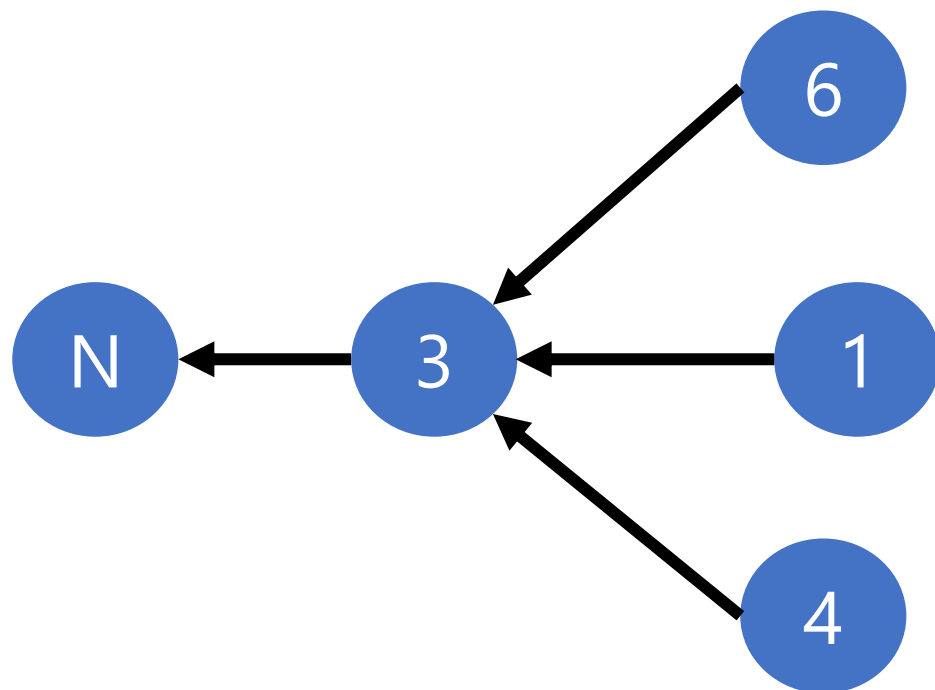
```
for(int i = 1; i < N; i++){
    int a = query(1, i) - 1;
    dist[a] = i;
    near[i] = a;
}

for(int t = 1; t < N; t++){
    int i = near[t];
    if(ok[i]){
        continue;
    }
    for(int j = 1; j < N; j++){
        int a = query(i + 1, j) - 1;
        if(dist[a] < dist[i]){
            answer(i + 1, a + 1);
            break;
        }
        else{
            answer(i + 1, a + 1);
            ok[a] = 1;
        }
    }
}
```

→100点!

頂点番号の大小をうまく利用する解法

・・・ 頂点 N の隣の頂点を聞く



→ $L=2.5N$ で満点は厳しい

得点分布

JOI

100点：5人

26点：7人

72点：9人

4点：4人

50点：3人

0点：0人

JOIG

66点：1人

20点：1人

52点：1人

4点：4人

41点：1人

0点：1人



JOI/JOIG 2023/2024 春季トレーニング 卓球 (Table Tennis) 解説

解説: 蜂矢 倫久 (Mitsubachi)

Step 0

問題概要

0

問題概要

- N 人の総当たり戦の結果を作る
- 条件「3 すくみの数が M 個」を満たしたいです
- 可能か不可能か判定、可能なら 1 つ作ってください
- 1 つの入力ファイルで Q 個解いてください

- $1 \leq Q$
- $N \leq 5\,000$
- $M \leq {}_N C_3$
- N の総和は 5 000 以下

0

小課題

小課題番号	内容	配点 (JOI)	配点 (JOIG)
1 (JOIG のみ)	$M = 0$	0 点	5 点
2	$M \leq N - 2$	5 点	12 点
3 (JOIG のみ)	$M \leq (N-1)^2/4$	0 点	15 点
4	N の総和 ≤ 7	4 点	10 点
5	N の総和 ≤ 20	23 点	23 点
6	N の総和 ≤ 150	30 点	19 点
7	N の総和 ≤ 600	15 点	8 点
8	追加の制約なし	23 点	8 点

0

小課題

- 以降の小課題の番号は JOIG に準拠します

0 マルチテストケースの実装

- 1つの入力ファイルでテストケースが複数ある問題はたまにみえます(マルチテストケースとか言ったりする)
- 以下みたいに書くと楽です

```
void solve(){  
    // ここに処理  
}  
int main(){  
    int t; cin >> t;  
    while(t--){  
        solve()  
    }  
}
```

Subtask 1

$$M = 0$$

1

考察

- 3 すくみが 0 個
- 強い人がとても勝って、弱い人がとても負けてそう

1

考察

- つまり各人に強さを決めて、その強さの大小で勝敗を決めれば OK

Subtask 2

$$M \leq N - 2$$

2 考察

- 強さの設定はそのままにしたい
- 1 個逆転が起こったとする
- 強さが A 番目に小さい人と B 番目に小さい人の結果を逆転すると 3 すくみが $B - A - 1$ 個できる

2

B を探す

- $A = 1$ とすると $B - 2 = M$ になるような B を見つけたい
- $M \leq N - 2$ なのでそのような B は N 以下
- つまり必ず存在！

Subtask 3

$$M \leq (N-1)^2/4$$

3 考察

- $O(N^2)$ の式ということは N ごとに何か足しそう
- 係数が $1/4$ なので 2 人ずつ何かをしそう？
- 2 人以外と 2 人の間の関係を考える

3

考察

- 2 人を A, B として A は B に勝ったとする
- 後の人は全員 A に勝てた or 負けたとしよう
- B に対して同様
- 全員 A に勝てた & B に負けたとすると 3 すくみは $N - 2$ 個
- $N - 2$ 人の場合の問題になる

3

考察

- $(N - 1)^2/4 - (N - 2) \leq (N - 3)^2/4$ なので
- N を 2 小さくした小課題 3 の問題に帰着される！
- 実装上の注意として、 N が 6 以下ぐらいで場合分けが必要
- これをどうするかは次の小課題 4 で

Subtask 4

$$N \leq 7$$

4

考察

- 試合の数は $_NC_2 \leq 21$
- bit 全探索ができそう
- 3 すくみの数は愚直に数えて $O(N^3)$
- $O(2^{(N/2)} N^3)$ で解けました

4

埋め込み

- この場合ありうる (N, M) の数が少ない
- 全通り手で解いて答えをソースコード中に埋め込む
- 出力 $O(N^2)$ で OK !
- 10 ケースを超えるのを手で解くなら多分他の小課題に手をつけた方がいいと思います

Subtask 5

$$N \leq 20$$

5

3 すくみの数を数える

- 制約に $N \leq 5\,000$ とある
- 3 すくみの数を $O(N^3)$ より速くジャッジしてそう
- $O(N^2)$ で数える方法を考える
- 3 すくみでない数を数えられないか

5

3 すくみの数を数える

- A, B, C が 3 すくみでないときその 3 人の中の試合結果を考える
- A が B, C に勝って、B が C に勝ってる感じ
- つまり A が 2 勝している
- $N - 1$ 人のうち A が勝った人のうち 2 人選ぶと、A が 2 勝してるタイプの 3 すくみになる
- これで $O(N^2)$ で数えられる

5

問題の言い換え

- つまり人 i の勝利数 $d(i)$ を適切に決めて $d(i)C_2$ の総和を ${}_NC_3 - M$ にしたい
- まず $d(i)$ の条件を考える $d(1) \leq d(2) \leq \dots$ としよう
- 必要条件を考える
- $0 \leq d(i) \leq N - 1$
- $d(1)$ から $d(i)$ までの総和が ${}_iC_2$ 以上
- $d(1)$ から $d(N)$ までの総和が ${}_NC_2$

- 逆にこれが十分条件になっている(ランダウの定理)
- 証明: 略
- とにかくこれで DP をできそう
- DP[d(i)まで決めた][ここまでの d(i) の総和][ここまでの $\sum_{d(i)} C_2$ の総和]
- 計算量 $O(N^8)$ で d(i) を求められる

5

結果を作る

- $d(i)$ から試合結果を作りたい
- 人 N に着目 この人は $d(N)$ 勝っている
- 直感: 人 1 から人 $d(N)$ までには勝ってそう
- これでやると実はうまくいく
- 証明: ランダウの定理の正当性を認めるとできます

5

結果を作る

- 人 N は人 1 から人 $d(N)$ までには勝って人 $d(N) + 1$ から人 $N - 1$ までには負けたとする
- $d(d(N) + 1)$ から $d(N - 1)$ までを 1 減らす(人 N に勝ったので)
- $d(1)$ から $d(N - 1)$ を sort する(これで人の順番が変わるので最初どの人だったかを管理する)
- $O(N \log N)$ で人を 1 人減らせるので $O(N^2 \log N)$
- $N - 1$ 人の結果をもらった後 N 人の場合の番号に直すと $O(N^3)$

5

解法

- DP で $d(i)$ を求める $O(N^8)$
- 再帰的に試合結果を作る $O(N^2 \log N)$ or $O(N^3)$
- 全体で $O(N^8)$ で解けました

Subtask 6

$$N \leq 150$$

6

判定問題を考える

- $d(i)$ を作ることを速くしたい
- ところで Yes か No の判定は？
- $d(i)C_2$ の総和を ${}_NC_3 - M$ にできるか
- 最大値は $M = 0$ のとき これは小課題 1 で達成してる
- 最小値を求めよう

6

最小値を求める

- 一旦条件「 $d(1)$ から $d(i)$ までの総和が ${}_iC_2$ 以上」を無視して「 $0 \leq d(i) \leq N - 1$ 」「 $d(1)$ から $d(N)$ までの総和が ${}_NC_2$ 」だけ考える
- このとき $i < j$ について $d(i) + 2 \leq d(j)$ なら $d(i)$ に 1 を足して $d(j)$ に 1 を引くと ${}_{d(i)}C_2$ の総和が小さくなる
- $({}_{d(j)}C_2 + {}_{d(i)}C_2) - ({}_{d(j)-1}C_2 + {}_{d(i)+1}C_2) = d(j) - d(i) - 1 \geq 1$
- そのような i, j がない状態が最小
- つまり $d(1) + 2 > d(N)$

6

最小値を求める

- このような $d(i)$ は N の偶奇で異なる
- N が偶数のとき $d(i)$ は $N / 2$ と $N / 2 - 1$ が $N / 2$ 個ずつ
- N が奇数のとき $d(i)$ は全部 $(N - 1) / 2$
- またこのとき「 $d(1)$ から $d(i)$ までの総和が ${}_iC_2$ 以上」が成り立っている
- ${}_{d(i)}C_2$ の総和の最小値が求まりました

6

間はどうか

- $d(i)C_2$ の総和が最小となる実際の試合結果は人 i が $i+1$ から $i+d(i)$ までに勝って、残り負けてるとすれば実現可能
- $d(i)C_2$ の総和の最小値と最大値の間にしたいときどうしよう
- $(d(j)C_2 + d(i)C_2) - (d(j)-1C_2 + d(i)+1C_2) = d(j) - d(i) - 1$ を思い出すと、 $d(i) = d(j) (= x)$ ならこの 2 人の勝利数を $x+1$ と $x-1$ にすると 1 増やせる
- これを ${}_NC_3 - M - (d(i)C_2 \text{ の総和の最小値})$ 回すればいい

6

間はどうか

- 実はこれが可能
- i と j の間の試合結果を入れ替えてみる
- i が j に勝ってたのを入れ替えると i は $x - 1$ 勝、 j は $x + 1$ 勝になる 元々 i が j に負けてたら i は $x + 1$ 勝、 j は $x - 1$ 勝に
- いずれにせよ $d(i) = x$ となる 2 人を $x + 1$ と $x - 1$ にできる

6

解法

- 1 回あたりの操作は i, j を見つけるのに $O(N)$
- 操作回数は $O(N^3)$
- なので $O(N^4)$ でできる
- さっきの方法で直接試合結果を作ってもいいし小課題 5 で述べた方法で d を作ってから試合結果を作ってもいい
- どれでも $O(N^4)$ とかで解けます

Subtask 7

$$N \leq 600$$

7

高速化

- d を変えるところを高速化しよう
- 案 1: 1 回あたりの操作を $O(N)$ より速くしたい
- x 勝った人の番号とかを管理して、 x 勝った人が 2 人以上いるような x の一覧を set とかで管理すれば $O(\log N)$ などでも OK
- もうちょっと頑張ると $O(1)$ とかにもできます

7

高速化

- 案 2: 操作回数を $O(N^3)$ より小さくしたい
- $d(N) = N - 1$ とする このとき人 N がいなくとも他の d は変わらない
- ${}_NC_3 - M - {}_{N-1}C_2$ が $N - 1$ のときの ${}_{d(i)}C_2$ の総和の範囲なら N が小さい問題になる
- 逆にこれができないとき操作回数は $O(N^2)$ になってる
- 案 1 と案 2 のどちらを採用しても $O(N^3)$ とか $O(N^3 \log N)$ で解けます

Subtask 8

$N \leq 5\,000$

- 案 1 と案 2 の両方を採用すれば $O(N^2)$ とか $O(N^2 \log N)$ で解けます
- d から試合結果を作るのを $O(N^3)$ だと難しいかも
- 実は $O(N^2)$ で作るアルゴリズムがあったりします

Step 9

補足

9

違う視点で見る

- 乱択や山登りができそう
- 実際書いてみると $N \leq 150$ ぐらいまでは行けたりする
- 何も思いつかなかったら乱択や山登りを書いてみるのもあり



Step 10

得点分布

10 得点分布(JOI)

- 100点 : ?人
- 77点: ?人
- 62点: ?人
- 27点: ?人
- 9点: ?人
- 5点: ?人
- 4点: ?人

10 得点分布(JOI)

- 100点 : 1人
- 77点: 1人
- 62点: 3人
- 27点: 1人
- 9点: 11人
- 5点: 3人
- 4点: 1人

10 得点分布(JOIG)

- 42点 : ?人
- 17点: ?人
- 15点: ?人
- 5点: ?人

10 得点分布(JOIG)

- 42点 : 1人
- 17点: 2人
- 15点: 1人
- 5点: 4人